



שם - עומר יפת שטרן

בית ספר - תיכון הרצוג

תעודת זהות - 326552056

שם המורה - אופיר שביט

עבודת גמר

תאריך ההגשה - 02.05.2026

תוכן עניינים

2.....	תוכן עניינים
5.....	מבוא
5.....	הגדרת המוצר
6.....	מה הפרויקט פותר?
7.....	סקירת שוק
7.....	פייבר
8.....	חסרונות של פייבר
9.....	ת'אמבטק
10.....	חסרונות של ת'אמבטק
11.....	השוואה של המוצר שלי עם פתרונות קיימים
13.....	סקירת הטכנולוגיה בפרויקט
13.....	מגבלות של המערכת
14.....	תחומי הפרויקט
14.....	תחומים בהם המערכת לא מטפלת
15.....	תיאור המערכת
16.....	פירוט הבדיקות
18.....	לוח זמנים לבניית פרויקט
19.....	תכנון סיכונים
20.....	תיאור יכולות
24.....	ארכיטקטורה של הפרויקט
24.....	תיאור החומרה
24.....	תיאור הטכנולוגיה הרלוונטית
26.....	תיאור זרימת המערכת
35.....	האלגוריתמים המרכזיים בפרויקט
39.....	תיאור סביבת הפיתוח
41.....	פרוטוקול תקשורת
42.....	סוגי ההודעות במערכת
45.....	מסכי המערכת
45.....	התחברות
46.....	הרשמה למערכת
50.....	שכחתי סיסמה
52.....	מסך בית (משתמש)
53.....	מסך בית (בעל מקצוע)
57.....	סרגל ניווט
58.....	חיפוש בעלי מקצוע
60.....	עמוד הבית של בעל מקצוע - תצוגת לקוח
61.....	קביעת תור
62.....	היררכיית המסכים
67.....	תיאור מבני נתונים

67.....	מבני הנתונים
67.....	מאגרי המידע של המערכת
68.....	מסד הנתונים
72.....	חולשות ואיומים
75.....	מימוש הפרויקט
75.....	סקירת מודולים מיובאים בשרת
77.....	סקירת מודולים מובאים ללקוח
83.....	מחלקות ופעולות ממומשות בשרת
97.....	מחלקות ופעולות ממומשות צד לקוח
103.....	סקירת בעיות אלגוריתמיות
103.....	חיפוש בעל מקצוע
106.....	קביעת תור
116.....	בדיקות
123.....	מדריך למשתמש
123.....	מערכת קבצים
124.....	פירוט הסביבה הנדרשת
130.....	ארכיטקטורה מינימלית הנדרשת
130.....	נתונים התחלתיים
130.....	רשת (Network)
130.....	ארכיטקטורה מינימלית נדרשת
131.....	רפלקציה
133.....	ביליוגרפיה
134.....	נספחים
135.....	נספח קוד

מבוא

הגדרת המוצר

תאם-לי היא פלטפורמה דיגיטלית שנועדה לייעל את הקשר בין לקוחות לבעלי מקצוע. בעלי מקצוע רבים היום מנהלים את לוח הזמנים שלהם דרך שיחות טלפון והודעות בוואטסאפ, מה שמוביל לעומס, טעויות בתיאום ותורים שמתפספסים. המערכת מחליפה את התקשורת המסורבלת הזאת במערכת אוטומטית ואמינה שמבטיחה סנכרון מלא. המערכת כוללת בתוכה מנוע חיפוש מתקדם המחבר בין לקוחות לבעלי מקצוע מהאזור.

המערכת תכלול בתוכה מספר חלקים עיקריים

איתור בעלי מקצוע – איתור בעלי המקצוע על ידי סוג מקצוע ועיר עבודה

דפים אישיים – דפים הכוללים מידע בשביל בעל המקצוע/לקוח

קביעת תורים – מערכת לקביעת תורים בצורה מסודרת

למה בחרתי בפרויקט?

בחרתי בפרויקט הזה מכיוון והוא מאוד שימושי ליום יום ויכול למנוע טעויות אי-הבנה בין בעלי מקצוע ללקוחות שקורות לעיתים קרובות. האתר אמור להקל על כל התהליך קביעת התורים וגם לתת לך מגוון הרבה יותר גדול של בעלי מקצוע. הפרויקט הזה יכול מאוד לסייע לכל אדם שחשוב לו למצוא בעלי מקצוע מקומיים ולהבטיח את השירותים שלהם. ובגלל זה לדעתי המוצר שלי כל כך חיוני.

מי הלקוחות?

בעלי מקצוע שמנהלים את זמנם על ידי יומן ורוצים להרחיב את מספר הלקוחות שלהם

לקוחות שמחפשים שירות אמין ומקצועי באזור שלהם שניתן לתאם בצורה מהירה וקלה

האתגרים שצפיתי

ממשק וחוויית משתמש - עבודה עם כל הטכנולוגיה שחדשה עבורי (React) כדי ליצור אתר שיהיה פשוט ונוח לשימוש היה אתגר מרכזי שהתחלתי לעבוד.

ניהול נתונים - סנכרון של הבסיס נתונים בזמן אמת כדי למנוע מצבי Race Condition וכל הסנכרון בין התהליכים היה משהו שהייתי צריך להבין איך עושים.

אבטחת המידע - הגנה על פרטיות הלקוחות ופרטי הקשר שלהם בתוך המערכת בזמן שאני חושף כל כך הרבה פרטים לגביהם גם היה בעיה שצפיתי שתהיה לי.

מה הפרויקט פותר?

בישראל, תקשורת בין לקוחות לבין בעלי מקצוע מתבצעת ברוב המקרים בצורה לא מסודרת – דרך שיחות טלפון, הודעות ב-WhatsApp, פייסבוק או אינסטגרם. השיטה הזו יוצרת כמה בעיות עיקריות:

1. חוסר סדר בניהול הזמנים של בעלי מקצוע

רוב בעלי המקצוע מנהלים את היומן שלהם ידנית או דרך הודעות מפוזרות. זה מוביל לטעויות, כפילויות, שכחה של לקוחות או בלבול על מועדי העבודה.

2. קושי ללקוחות למצוא בעל מקצוע זמין ואמין

לקוח שרוצה שירות צריך לחפש בעצמו בקבוצות, בפייסבוק, בהמלצות מחברים – בלי פילטרים ברורים, בלי דירוגים ובלי דרך לראות זמינות אמיתית.

3. אי-הבנות שגורמות לעיכובים ותסכול

שיחה בעל-פה על המחיר, על הכתובת או על זמן ההגעה בקלות יכולה להתפספס. אין תיעוד מסודר, וזה גורם לעיתים לטענות, ויכוחים והפסד זמן לשני הצדדים.

4. שירות לא אחיד ולא מקצועי

לקוחות מצפים לניהול מתקדם ודיגיטלי, אבל הרבה בעלי מקצוע עדיין עובדים בצורה מיושנת – מה שמוביל לחוויית משתמש פחות טובה ויכולת שירות מוגבלת.

המערכת תאם-לי פותרת את הבעיות האלה על ידי זה שהיא מרכזת את כל התהליך במקום אחד ומשנה את הדרך שבה לקוחות ובעלי מקצוע מתקשרים:

1. ניהול זמנים חכם לבעלי מקצוע

במקום לארגן תורים בהודעות, בעל המקצוע מקבל יומן מאורגן שבו הלקוחות יכולים לראות מתי הוא פנוי ולהגיש בקשה לתור. זה מצמצם טעויות ומייעל את יום העבודה.

2. איתור בעלי מקצוע לפי אזור ושירות

המערכת מציגה למשתמש רק בעלי מקצוע שרלוונטיים עבורו – לפי המיקום שלו ותחום המקצוע שהלקוח צריך. במקום לחפש לבד, הכול מגיע בצורה מסודרת.

3. קביעת תורים מסודרת ומאושרת

לקוח בוחר מתי הוא רוצה, בעל המקצוע מאשר – ולכן אין אי-הבנות. הכול מתועד אוטומטית בשני הצדדים.

5. דף אישי ופרופיל ללקוחות

הלקוח מנהל את הפרטים שלו, רואה תורים ושומר על קשר מסודר עם בעלי המקצוע.

בשורה התחתונה

המערכת מייצרת אמון בתחום שעדיין עובד בצורה מיושנת בישראל שלוקה באמון. היא מאפשרת יעילות לבעלי מקצוע, נוחות ללקוחות, ומקטינה בצורה משמעותית טעויות, בלבול ותסכול מה שהופך את כל תהליך העבודה להרבה יותר מקצועי וברור.

סקירת שוק

פייבר

פייבר היא פלטפורמה בינלאומית שמחברת בין פריילנסרים (נותני שירות) לבין לקוחות שמחפשים שירותים דיגיטליים במחיר נגיש. האתר נוסד בישראל, ונחשב לאחד המובילים בעולם בתחום. באתר מספר פיצ'רים שגורמים לו להיות ייחודי ומוביל בתחום.

שירותים - להתחלה, יש מאות קטגוריות של שירות שבהם אפשר למצוא כמעט כל בעל מקצוע.

מערכת דירוג - שנית, המערכת מציעה מערכת דירוג לכל פריילנסר. כל קונה יכול לראות:

- ציון ממוצע של הפריילנסר
- חוות דעת אמיתיות
- מספר מכירות
- רמת הפריילנסר (ביחס למתחרים)

מערכת תשלומים

התשלום עובר ישיר מהלקוח ברגע שהוא מאשר את העבודה ונשלח לפייבר ומשם נשלח ללקוח. כלומר פלטפורמה מאוד אמינה למניעת רמאות

מערכת תקשורת

צ'אט נוח בין הלקוח לפריילנסר, בקשות מותאמות אישית, ושליחת קבצים.

בינה מלאכותית

יש לפייבר עוזר AI למציאת פריילנסרים לפי בקשה, כלי בינה מלאכותית להפקת תוכן, שירות AI שיכול להבין מה הצרכים שלך לפי הפרויקט ולעזור לך לבחור בעל מקצוע.

חסרונות של פייבר

עיצוב ומבנה האתר

למרות שהאתר נראה מודרני, האתר מאוד עמוס מבחינת קטגוריות, תתי קטגוריות, פילטרים ואלפי תוצאות. כל העומס הזה גורם לבלבול אצל הלקוח, קושי למוצא שירות מתאים וחיפוש ארוך ומתיש של המשתמש.

מנוע חיפוש שאינו תמיד מדויק

- החיפוש לא תמיד מציג תוצאות מדויקות ורלוונטיות מה שגורם לתלונות רבות מלקוחות כגון
- נותן עדיפות לפריילנסרים מפורסמים על פני חיפוש לפי הבקשה של המשתמש
- לא תמיד מפריד בין המילים ולכן קיימות בעיות בחיפוש
- מביא פריילנסרים מתחומים שונים שרק מבזבז את הזמן של לקוחות

מערכת דירוג מטעה

רוב הפריילנסרים עם דירוג דומה ולכן קשה למצוא בעלי מקצוע שהדירוג שלהם משקף את העבודה שלהם.

אין מערכת פילטור נוחה כדי למצוא בעל מקצוע עם דירוג מקצועי שמראה על מידת שירותו

דף השירות

יש מגבלות תצוגה לדף השירות.

- כמות תמונות מוגבלת
- הדגמות וידאו קצרות מאוד
- עיצוב מאוד סטטי

מערכת הודעות בדוקה יתר על המידה

באתר מנגנון שסורק הודעות כדי לבדוק שאין החלפה של פרטים אישיים. מערכת זו דבר ראשון מעוד פולשנית מכיוון שבאה להגן על הפרטיות של הפרט אבל בסוף חודרת מאוד כי קוראת כל הודעה ובדוקת את תוכנה.

המערכת הזאת של הבדיקה גורמת למספר בעיות

- חוסם מילים בטעות
- מטשטש קישורים לגיטימיים
- מוציא אזהרות בלי קשר להודעות

תמיכה טכנית

בדרך כלל לא נעשה על ידי אנשים אלא מתחיל מבוססים ורק אחרי תהליך ארוך עובר לאנשים. גם למצוא את המקום עצמו שבו אפשר לפתוח פנייה לשירות לקוחות גם קשה למציאה.

פייבר היא פלטפורמה גדולה למציאת שירותים דיגיטליים, אבל בפועל היא רחוקה מלהיות מושלמת. האתר עמוס, מבולגן, ומנוע החיפוש שלו לעיתים מציג תוצאות לא רלוונטיות. מערכת הדירוגים נראית מרשימה, אבל בפועל כמעט כולם מקבלים ציונים גבוהים, כך שקשה להעריך איכות אמיתית. גם דפי השירות מוגבלים מבחינת תצוגה, מה שמקשה להבין מה השירות באמת כולל.

ת'אמבטק

ת'אמבטק הוא שוק מקוון שמחבר בין לקוחות לבין אנשי מקצוע מקומיים (פרילנסרים) בתחומים מאוד "שטחיים", כלומר שירותים פיזיים ותיקונים, כמו בנייה, שיפוצים, מתקני חשמל, ניקיון, הדרכה, צילום, ועוד.

באתר מספר פיצ'רים שעוזרים לו להיות מאוד מוביל בתחום

הגשת בקשה לפרויקט

לקוחות מתארים בדיוק את מה שהם רוצים לפרויקט שלהם, סוג שירות, מיקום, לוח זמנים, ואז הבקשה נשלחת לפרילנסרים מתאימים.

קבלת הצעות

אנשי המקצוע שולחים הצעות שמותאמות לפרויקט שבהם כלול פרטים כגון עלות, זמן ביצוע, ומה השירות כולל

אלגוריתם התאמה חכם

יש אלגוריתם חכם שמנסה להתאים בין הצרכים של הלקוחות אל האנשי המקצוע המתאימים על בסיס הפרויקט שהמשתמש הגיש

ניהול לוח שנה

אנשי מקצוע יכולים לסנכרן את זמינותם עם לוח שנה, לקבוע תורים מדויקים לפי המערכת שעות שלהם.

מערכת דירוגים

יש מערכת דירוגים של ביקורות של לקוחות כלפי הפרילנסר ותמונות של עבודות של בעל המקצוע

חסרונות של ת'אמבטק

תשלומים

בעלי מקצוע משלמים על לידים בכל פעם שמישהו פונה אליהם ולא כאשר הם מסיימים עבודה. כלומר אם לקוח החליט בסוף לא להמשיך עם הפרילנסר הזה הפרילנסר עדיין מחויב מה שגורם על הוצאות גדולות מהצפוי.

מנוע חיפוש לא מדויק

החיפוש לא תמיד מחזיר תוצאות שמתאימות בדיוק לשירות שחופש. התוצאות בדרך כלל רחבות מדי לא מסוננות לפי סוג שירות ספציפי, או מציגות פרילנסרים שלא באמת זמינים

דפי השירות

הדפי שירות של הפרילנסרים מאוד מוגבל מבחינת

- תצוגת תמונות
- סדר מידע
- פורמטים

מערכת התאמה לא ברורה

האלגוריתם קובע איזה מקצוען מוצג ללקוחות לפי מספר קריטריונים שמשפיעים על איכות החיפוש

- מעדיף מקצוענים שמוציאים יותר על כל ליד שהם מקבלים
- מציג התאמות רחוקות גאוגרפית מאיפה שהעבודה

כלומר יכול לקחת המון זמן למצוא פרילנסר שמתאים לצרכים בגלל מנגנון החיפוש המוטעה של המערכת

מערכת התראות מוגזמת

האתר שולח המון התראות גם כאשר הן לא רלוונטיות או שהן חוזרות על עצמן. אין באתר מספיק אפשרות להתאמה אישית בהתראות שהלקוח רוצה לקבל.

בקשת שירות

כשלקוח ממלא בקשה:

- לא תמיד יש אפשרות להוסיף פירוט אמיתי
 - קטגוריות מוגבלות
 - לא ניתן לצרף מספיק קבצים / תמונות
- זה יכול להפוך פרויקטים מורכבים ל"שטחיים" מדי מבחינת תיאור.

בסופו של דבר, למרות ש-Thumbtack מציע פלטפורמה נוחה לכאורה למציאת אנשי מקצוע, החוויה בפועל רחוקה מלהיות חלקה: מנוע החיפוש לא מדויק, עמודי הפרופיל מוגבלים, הפלטפורמה איטית לעיתים, הפילטרים בסיסיים מדי וההתאמות שהאתר מציג לא באמת שקופות או עקביות. התוצאה היא מערכת שנראית מבטיחה מבחוץ, אבל בפועל מרגישה מגבילה, לא מדויקת ולעיתים מתסכלת לשימוש.

השוואה של המוצר שלי עם פתרונות קיימים

כמו שהוצג, בשוק יש מספר רב של אפליקציות שדומות לשלי ברעיון אך החזקות ביותר בתחום הן פייבר, ות'אמבטק. האפליקציות מציעות פלטפורמה נוחה למציאת בעלי מקצוע אך קיימים בשתייהן חסרונות מובהקים. תאם לי אינה חדשה לשוק הזה אך מציעה שדרוגים שלא נראו עוד בתחום שעונים על החסרונות של האתרים.

חוויית משתמש

חוויית המשתמש באתר תאם-לי לעומת המתחרות שלה מאוד טובה מכיוון ואינה מלאה בדברים שאינם קשורים למטרת האתר; חיפוש בעלי מקצוע מאזור מסוים וקביעת תור איתם.

מנגנון החיפוש

מנגנון החיפוש בתאם-לי הוא הכי טוב בשוק מכיוון ומונע על ידי דבר אחד, סוג בעל המקצוע שמחפשים והמיקום שלו. בסקירה שעשיתי על המוצרים המובילים בשוק בתחום הזה היה ניתן לראות שמנגנוני החיפוש בדרך כלל לא מדויקים ואף מוטעים מכיוון ומושפעים מכמה שבעל המקצוע משלם לאתר.

ניהול תורים

תאם-לי ות'אמבטק משתמשות בשיטה דומה לקביעת תורים שמאוד נוחה למשתמשים. האתרים מראים את השעות הפנויות של בעלי המקצוע ונותנים לך לתת פירוט וכתובת אך לעומת זאת המנגנון של פייבר לקבוע תורים מאוד לא נוחה.

תקשורת צ'אטים

התקשורת בתאם-לי הכי נוחה מבין האתרים מכיוון ואין בה מערכת לתקשר עם בעל המקצוע חוץ מהתיאור שיש בקביעת התור. ת'אמבטק מובילה בתחום הזה מכיוון ויש להם צ'אט מובנה שאתה יכולה לדבר עם בעל המקצוע בחופשיות לעומת פייבר שיש לה מערכת פורשנית שמוודאה שאתה ובעל המקצוע לא משתפים מידע פרטי בהודעות

דירוגים

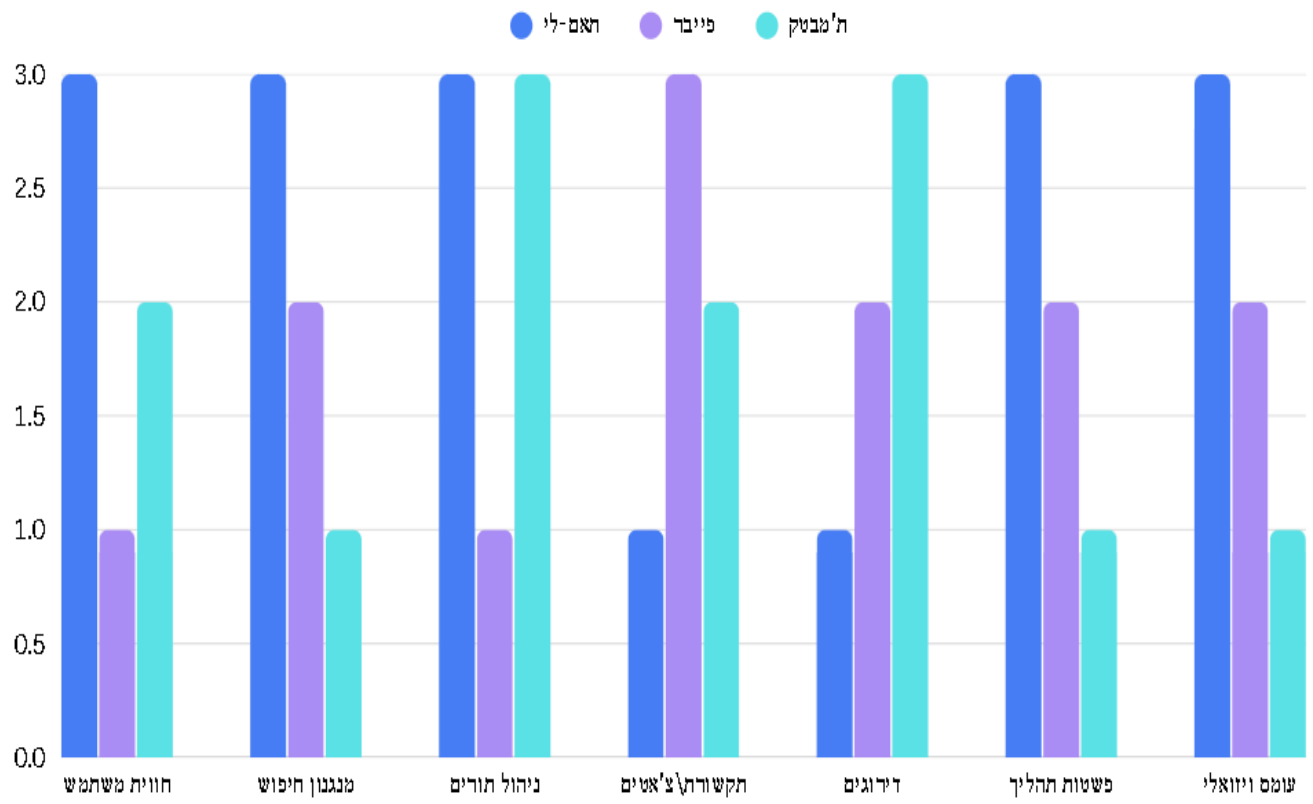
במוצר תאם-לי אין מערכת דירוגים. לכן היא נחשבת מאחורה בתחום הזה מבחינת המוצרים מהסוג הזה. פייבר גם בתחום הזה לא צולחת מאוד מכיוון וכל הדירוגים באתר זהים כמעט לחלוטים לכן לא עוזרת למשתמשים בקבלת החלטות. ת'אמבטק מממשת פלטפורמת דירוגים נוחה לשימוש.

פשטות התהליך

במוצר תאם-לי מציאת בעל מקצוע מתאים וקביעת התורים איתו יכול להיעשות בפחות מ-2 דקות. האתר מאוד נוח וקל לשימוש מכיוון ולא נותן מודעות ואינו מתבסס על אלגוריתמים מסובכים. מנגד בת'אמבטק יש אלגוריתם התאמה שאינו מדויק ולא נותן לך את האופציה לחיפוש ולכן קשה יותר למצוא את בעל המקצוע הראוי. בפייבר האלגוריתם של החיפוש גם מוטעה הרבה מהפעמים מכיוון ומושפע מכמה בעלי המקצוע משלמים לאתר

עומס ויזואלי

תאם-לי מאוד קצרה ולעניין, יש בו גרפיקה מאוד נקייה ולא עמוסה כדי שהמשתמש לא ישקע בעיצוב. האתר מעוצב מאוד אך בו זמנית אין בו עמודים עמוסים. מנגד פייבר ות'אמבטק מאוד עמוסים ויזואלית ויותר קשים להתמצאות.



סקירת הטכנולוגיה בפרויקט

המערכת משתמשת בשילוב של טכנולוגיות מוכרות:

- **React** צד הלקוח
- **Python** צד השרת

אך היא מיישמת אותן באמצעות פרוטוקול WebSocket. לשרת ש-WebSocket הוא פרוטוקול מוכר, השימוש בו לצורך תקשורת סינכרונית במבנה של Request-Response אינה המימוש הקלאסי שלו. הבחירה בטכנולוגיה זו נועדה לייעל משמעותית את התקשורת על ידי ביטול ה-Overhead של לחיצות יד חוזרות המאפיינות את פרוטוקול HTTP. ברגע שהחיבור הראשוני נוצר, המידע עובר בחיבור פתוח וקבוע, מה שמצמצם את נפח הנתונים המועברים בכל בקשה, מקטין את זמן התגובה ומאפשר לשרת לבצע אימות תקינות על כל הודעת JSON נכנסת לפני טיפול בה.

מגבלות של המערכת

הגדרת המערכת כוללת מספר סייגים הנובעים מהארכיטקטורה שבחרתי. מאחר שהמערכת פועלת במודל של עדכון נתונים המותנה בטעינה מחדש של הדף, המשתמש אינו רואה שינויים שבוצעו על ידי משתמשים אחרים בזמן אמת עד שהלקוח מבצע פעולה שמעדכנת את המידע על ידי בקשה חוזרת מהשרת. בנוסף קיים תלות: המערכת מחייבת דפדפן מודרני שתומך ב-JavaScript וגם חיבור אינטרנט יציב ורציף מכיוון ובמקרה של ניתוק זמני, המשתמש יידרש לטעון את האתר מחדש כדי לאתחל את ה-Socket מול השרת. לבסוף, קיים מגבלת משאבים בצד השרת, שכן החזקת חיבור קבוע ופתוח לכל משתמש (**Stateful Connection**) דורש ניהול זיכרון קפדני יותר בהשוואה למערכת שמבוססת על HTTP, במיוחד במצבי עומס.

תחומי הפרויקט

פיתוח Full-Stack וחווית משתמש - הפרויקט מתמקד בבניית ממשק לקוח אינטראקטיבי המעניק חוויה חלקה ונוחה, אך גם פיתוח צד שרת המנהל את כל הלוגיקה כגון אימות הנתונים וניהול המשתמשים.

רשתות ותקשורת - המערכת מבוססת על מודל שרת-לקוח המשתמש בפרוטוקול WebSocket להעברת נתונים בפורמט JSON. התחום כולל ניהול חיבורים רציפים וטיפול בעומסי תקשורת על השרת.

ניהול מסדי נתונים - הפרויקט עוסק בתכנון וארגון מסד נתונים. שמתי דגש רב לפני שהתחלתי לתכנת על לנרמל את הטבלאות כדי למנוע כפילויות, מה שמאפשר שליפה מהירה ויעילה של מידע מורכב.

אבטחת מידע - הגנה על פרטיות המשתמשים בפרויקט כזה הוא חיוני. התיחום כולל הגנה מפני SQL Injections, מניעת התקפות XSS על ממשק המשתמש, סינון קלט ברמת השרת כדי למנוע זליגת מידע אישי, ועוד.

מערכות הפעלה: הפרויקט מתייחס למערכת ההפעלה כסביבת ההרצה של השרת והלקוח. התיחום כולל ניהול משאבי מערכת (זיכרון ועיבוד) בזמן החזקת חיבורי Socket פתוחים.

תחומים בהם המערכת לא מטפלת

סליקת תשלומים בפועל - המערכת מתמקדת בתיאום התור בלבד ואינה כוללת חיבור למערכות סליקה.

אימות זהות ממשלתי - המערכת מסתמכת על ניהול משתמשים פנימי ואינה מבצעת אימות מול מאגרי מידע חיצוניים.

הצפנת נתונים בסד הנתונים - הנתונים במסד הנתונים נשמרים ללא הצפנה ברמת הקובץ. כלומר לא מוצפנים בתוך הקובץ

שימוש במערכות חיצוניות - המערכת אינה מסתנכרנת עם יומנים חיצוניים או שירותי מפות חיצוניים.

תיאור המערכת

מערכת "תאם-לי" היא פלטפורמה אינטראקטיבית לניהול ותיאום תורים. המערכת מחליפה את התקשורת הלא-מסודרת בממשק דיגיטלי המאפשר סדר ואמינות בתהליך קביעת השירות.

בעל מקצוע

ניהול יומן עבודה דיגיטלי - הגדרת שעות פעילות וזמינות בצורה גרפית נוחה.

בקרה ואישור תורים - יכולת לקבל בקשות תור מלקוחות, לאשר או לדחות אותן, ובכך למנוע כפילויות

דף פרופיל אישי - הצגת מידע על העסק ואזור השירות למשיכת לקוחות חדשים.

ריכוז נתונים - צפייה בכל התורים העתידיים במקום אחד מסודר.

לקוח

איתור בעלי מקצוע - מנוע חיפוש המאפשר סינון בעלי מקצוע לפי קטגוריות ולפי אזורים בארץ.

קביעת תור עצמאית - צפייה ביומן הזמינות של בעל המקצוע ובחירת תור מתאים ללא צורך בשיחה טלפונית.

פירוט העבודה - אפשרות להוספת פירוט על סוג הבעיה או השירות הנדרש כבר בשלב קביעת התור, מה שמונע אי-הבנות.

ארגון אישי: צפייה בכל התורים שנקבעו.

פירוט הבדיקות

במערכת גדולה כמו שיש לי שיש הרבה סוגים של פעולות שהלקוח יכול לעשות יש הרבה פרצות שצריך לחסום. לכן על כל הפריצות שאני מודע שקיימות ניהלתי בדיקות לראות אם אני חוסם אותן. אז אני אסקור כל חלק בפרויקט שעליו ביצעתי את הבדיקות.

מערכת ההרשמה

אימות ייחודיות משתמש - ניסיון הרשמה עם כתובת דואר אלקטרוני שכבר קיימת במערכת.

תוצאה מצופה: חסימת הרישום והצגת הודעה שהמשתמש כבר קיים.

התאמת סיסמאות - הזנת סיסמאות שונות בשדות "סיסמה" ו"אימות סיסמה".

תוצאה מצופה: מניעת שליחת הטופס והצגת התראה על חוסר התאמה.

בדיקה של סוג מקצוע ועיר - שליחת בקשת הרשמה המכילה סוג מקצוע או עיר שאינם קיימים ברשימות המוגדרות בשרת.

תוצאה מצופה: דחיית הבקשה על ידי השרת ומניעת כניסת נתונים לא תקינים ל-DB.

בדיקת ערכי קצה

שנות ניסיון ומחיר - הזנת מספרים קיצוניים (למשל: 999 שנות ניסיון או מחיר שעתי של מיליוני שקלים).

תוצאה מצופה: חסימת הקלט על ידי מגבלות שדה בשרת.

לוגיקת זמני עבודה - הזנת שעת התחלה מאוחרת משעת סיום, או קביעת משך עבודה הארוך מהחלון שבין שעת ההתחלה לסיום.

תוצאה מצופה: השרת יזהה סתירה לוגית ויחזיר שגיאה.

בדיקת אורך טקסטואלי - "הפצצת" שדה הפירוט במחרוזת ענקית כדי לבדוק קריסת מסד נתונים או חריגת זיכרון.

תוצאה מצופה: חסימה לפי מגבלת תווים.

אבטחת שדות - ניסיון להזריק פקודות SQL לכל אחד משדות ההרשמה.

תוצאה מצופה: ניקוי הקלט ומניעת הרצת הפקודה.

אימות דוא"ל ושכחתי סיסמה

תקינות קוד האימות: הזנת קוד המכיל תווים שאינם ספרות או קוד שגוי שאינו תואם לזה שנשלח.

תוצאה מצופה: חסימת הגישה והודעה על קוד לא תקין.

שכחתי סיסמה -בדיקת קיום - הזנת אימייל שאינו רשום במערכת.

תוצאה מצופה: המערכת לא תשלח קוד ותודיע שהמשתמש לא נמצא (או הודעה כללית לשמירה על פרטיות).

מניעת הספמה : ביצוע פעולת "שכחתי סיסמה" פעמים רבות ברצף עבור אותו משתמש.

תוצאה מצופה: הפעלת מנגנון הגבלת קצב וחסימה זמנית.

מערכת התחברות

פרטי גישה שגויים: הזנת אימייל לא קיים או סיסמה שאינה תואמת לאימייל.

תוצאה מצופה: דחיית הגישה והצגת הודעת שגיאה גנרית.

הגנה מפני Brute Force: ניסיונות התחברות חוזרים ונשנים בפרק זמן קצר לאותו חשבון.

תוצאה מצופה: חסימת ה-IP או נעילת החשבון לזמן מוגדר.

עמוד הבית ומנגנון החיפוש

בדיקת עומסים : שליחת כמות עצומה של בקשות לעמוד הבית בזמן קצר.

תוצאה מצופה: השרת נשאר יציב ומפעיל הגנות נגד הצפה.

חיפוש ערכים לא קיימים: הזנת סוג בעל מקצוע או עיר שאינם קיימים במסד הנתונים.

תוצאה מצופה: הצגת הודעה "לא נמצאו תוצאות" ללא קריסת הממשק.

ניהול וקביעת תורים

מניעת מתקפת XSS: ניסיון להכניס תגיות HTML בתוך שדה פירוט העבודה בעת קביעת תור.

תוצאה מצופה: המערכת תציג את הטקסט כפי שהוא מבלי להריץ את הקוד בדפדפן של בעל המקצוע.

מניעת כפילות תורים : ניסיון לקבוע תור שכבר נתפס על ידי משתמש אחר (או קביעת תור תפוס על ידי שליחת בקשה ישירה לשרת).

תוצאה מצופה: השרת יבדוק את זמינות התור ב-DB רגע לפני הרישום ויחזיר שגיאה במידה והוא תפוס.

סנכרון יומן בעל המקצוע: ניסיון לאשר שני תורים שונים שנקבעו על אותה משבצת זמן בדיוק.

תוצאה מצופה: המערכת תאפשר אישור של תור אחד בלבד ותחסום/תבטל אוטומטית את השני כדי למנוע התנגשות.

כלל הבדיקות שכתבתי למעלה ביצעתי, בזמן שהמטרה היא לשבור את לוגיקת המערכת. השילוב בין אימות נתונים קפדני בצד השרת (Python) לבין מנגנון התקשורת של ה-WebSocket מבטיח שהמערכת לא רק מספקת חוויית משתמש מהירה, אלא גם עמידה בפני התקפות זריקת קוד, כפילויות נתונים וניסיונות עקיפת הרשאות.

לוח זמנים לבניית פרויקט

זה היה לוח הזמנים שתכננתי בפועל בשביל איך לכתוב את הפרויקט בזמן שאני מנהל את הזמן שלי בצורה הכי יעילה

1. בניית עמוד ההתחברות
 - a. קודם כל לבנות "יצירת משתמש"
 - i. אממש לקוח
 - ii. אממש בעל מקצוע
 - b. אבנה התחברות
2. אבנה את עמוד הבית
 - a. אתכנת קודם כל את עמוד הבית בשביל לקוח
 - b. אבנה את עמוד הבית בשביל בעל מקצוע
 - i. אבנה את הפניות הנכנסות לגבי עבודות
3. אבנה את מנגנון החיפוש
4. לוח זמני תורים של לקוח (בעל מקצוע ומשתמש)

בפועל ניהול הזמן שלי השתנה לפי מה שראיתי ליותר חשוב באותו הרגע וככה בסוף בניתי את הפרויקט:

1. עמוד ההתחברות
2. עמוד ההרשמה
 - a. הרשמת לקוח
 - b. הרשמת בעל מקצוע
 - c. אימות אימייל
3. עמוד שכחתי סיסמה
4. עמוד הבית
 - a. עמוד הלקוח
 - b. עמוד בעל המקצוע
5. מנגנון החיפוש
6. הצגת העמודים לאחר מציאת כל בעלי המקצוע
7. עמוד של המקצוען בפני הלקוח
8. קביעת התורים

תכנון סיכונים

בזמן פיתוח המערכת זיהיתי כמה סיכונים במערכת שכזאת שעלולים לפגוע בחווית המשתמש, בביצועי המערכת, בשלמות הנתונים ובפרטיות השתמש. מתחת אני מפרט את כל הסיכונים שעלולים לפגוע במערכת ודרכי ההתמודדות שלי איתן.

סיכון	רמת הסיכון	איך הסיכון ממומש	איך תכננתי לטפל	איך טיפלתי בפועל
נפילה של השרת (עומס יתר)	גבוה	הרבה לקוחות מתחברים במקביל או התקפת DDOS	להשתמש בשרתים יציבים ושימוש בתקשורת בצורה הכי יעילה	הגבלתי את כמות הלקוחות שהאתר יכול לטפל בהם ברגע נתון (Rate Limiting), בדקתי שאף IP בודד לא פותח הרבה חיבורים בפעם אחת (מתקפת DDOS), בדקתי שאין IP אחד שמספיק בקשות לשרת
שימוש לרעה במערכת (לקבוע מלא תורים לבעל מקצוע יחיד)	נמוכה	לקוח אחד בוחר את כל התורים האפשריים ומבקש לקבוע עם בעל המקצוע	לא תוכנן	קביעת התורים הוא תהליך שלא הלקוח מנהל ולכן כשלקוח א' קובע גם לקוח ב' יכול לקבוע לאותו יום אבל בסוף בעל המקצוע בוחר את מי לאשר ומי לבטל (ידנית) ובכך התורים לא נתפסים סתם
התנתקות הלקוח באמצע	בינונית	לקוח פשוט סוגר את החלונות באמצע שהשרת מטפל בו	בדיקת התנתקות בכל הודעה שהשרת מקבל	טיפלתי כמו בתכנון אבל גם הוספתי מנגנוני - TRY EXCEPT במקרים מסוימים כדי להיות בטוח
לקוחות מתחזים	גבוה	לקוח גובב Cookie של לקוח אחר לקוח עושה Brute Force ומתחבר כלקוח אחר	לשים מספר מוגבל של ניסיונות התחברות	בפועל שמתי כמות הודעות מוגבלת שכל לקוח יכול לשלוח לשרת בזמן נתון לכן התקפת Brute Force לא תעבוד
SQL Injection	גבוה	הלקוח שם באחת מהשדות הפתוחים SQL Query	להשתמש בפעולות בPython שמוודאות שהמידע אינו "רעיל" למערכת	טיפלתי כמו בתכנון באמצעות Parameterized Queries המפרידות בין הקלט לקוד.
גניבת מידע	בינוני	התקפת MITM	להשתמש בהצפנות לצורך שמירה על פרטיות המשתמש	השתמשתי בהצפנת TLS שקיים בה בדיקות שנועדו למנוע התקפות MITM

תיאור יכולות

היכולת - הרשמה למערכת

מהות - רישום משתמש חדש למערכת תוך בדיקת תקינות נתוני ההרשמה שלו

אוסף יכולות הנדרשות -

- מסך הרשמה
- קלט של נתונים
- בדיקת קלט של משתמש
- אבטוח מידע (הצפנה)
- העברת מידע מהלקוח לשרת
- בדיקה מול מסד הנתונים
- קבלת תגובה מהשרת
- פענוח תשובה\בקשה
- הצגת התשובה למשתמש

היכולת - אימות משתמש

מהות - לאמת שאימייל מסוים שייך למשתמש מסוים

אוסף יכולות הנדרשות -

- שליחת אימייל על ידי השרת עם קוד לאימייל
- מסך אימות
- קלט מהמשתמש
- אימות קלט
- אבטוח מידע
- שליחת קלט אל השרת
- אימות של השרת מול מסד הנתונים שהקלט מתאים
- קבלת תגובה מהשרת
- פענוח תשובה\בקשה
- הצגת תשובה למשתמש

היכולת - התחברות

מהות - חיבור משתמש למערכת וטיפול בו

אוסף יכולות הנדרשות -

- מסך התחברות
- קלט משתמש
- בדיקת קלט של משתמש
- אבטוח מידע
- העברת מידע אל השרת
- אימות מידע מול מסד נתונים
- קבלת תגובה מהשרת
- פענוח תשובה\בקשה
- הצגת תשובה ללקוח

היכולת - שכחתי סיסמה

מהות - לאפס סיסמה של משתמש קיים

אוסף יכולות הנדרשות -

- מסך שכחתי סיסמה
- קלט משתמש
- בדיקת קלט משתמש
- הצפנה
- שליחת אל השרת
- אימות מול מסד הנתונים שהמשתמש קיים
- יצירת סיסמה זמנית לאימות אימייל
- שליחת הודעה ללקוח
- פענוח תשובה\בקשה
- הצגת מסכים שונים
- אימות סיסמה
- עדכון מסד הנתונים

היכולת - עמוד בית

מהות - הצגת מידע אישי למשתמש על חשבון

אוסף יכולות הנדרשות -

- שליחת בקשה לשרת
- הצפנה של מידע
- פענוח TOKEN בצד השרת
- מציאת מידע של המשתמש במסד הנתונים
- העברת מידע רלוונטי אל המשתמש
- פענוח תשובה/בקשה
- פענוח סוג מסך להציג
- הצגת תשובה מתאימה ללקוח

היכולת - אישור/דחיית בקשות תורים

מהות - לתת לבעל המקצוע אפשרות לקבל/לדחות תורים

אוסף יכולות הנדרשות -

- שליחת מידע לשרת
- הצפנה
- פענוח של הבקשה בשרת
- פענוח TOKEN בצד שרת
- חיפוש במסד הנתונים תורים הרלוונטיים
- שליחת הודעה אל בעל המקצוע
- פענוח הודעה של השרת
- הצגת המידע אצל הלקוח
- מסך אינטראקטיבי לקבל/לדחות תורים
- עדכון מסד נתונים על פי החלטות בעל המקצוע

היכולת - חיפוש בעל מקצוע

מהות - מציאת בעל מקצוע מתאים באזור שלך

אוסף יכולות הנדרשות -

- בקשה מהשרת למידע
- הצפנה
- פיענוח TOKEN בצד שרת
- להוציא פרטים על פי בקשת הלקוח ממסד הנתונים
- שליחת מידע אל הלקוח
- פיענוח תשובה\בקשה
- הצגת מסך מתאים
- החלפה בין מסכים

היכולת - עמוד בית של בעל מקצוע

מהות - להציג מידע לגבי בעל המקצוע

אוסף יכולות הנדרשות -

- בקשה מהשרת למידע
- הצפנה
- פיענוח TOKEN בצד שרת
- להוציא מידע רלוונטי ממסד הנתונים
- לפלטר נתונים אישיים לגבי בעל מקצוע
- שליחת מידע אל לקוח
- הצגת מסך מתאים
- פיענוח בקשה\תשובה

היכולת - קביעת תורים

מהות - לקבוע תור עם בעל המקצוע המתאים

אוסף יכולות הנדרשות -

- בקשה מהשרת למידע
- הצפנה
- פיענוח TOKEN בצד שרת
- להוציא מידע רלוונטי ממסד נתונים
- לפלטר פרטים אישיים
- שליחת תשובה אל לקוח
- לבנות עמוד מתאים
- קבלת קלט
- פיענוח תשובה\בקשה

ארכיטקטורה של הפרויקט

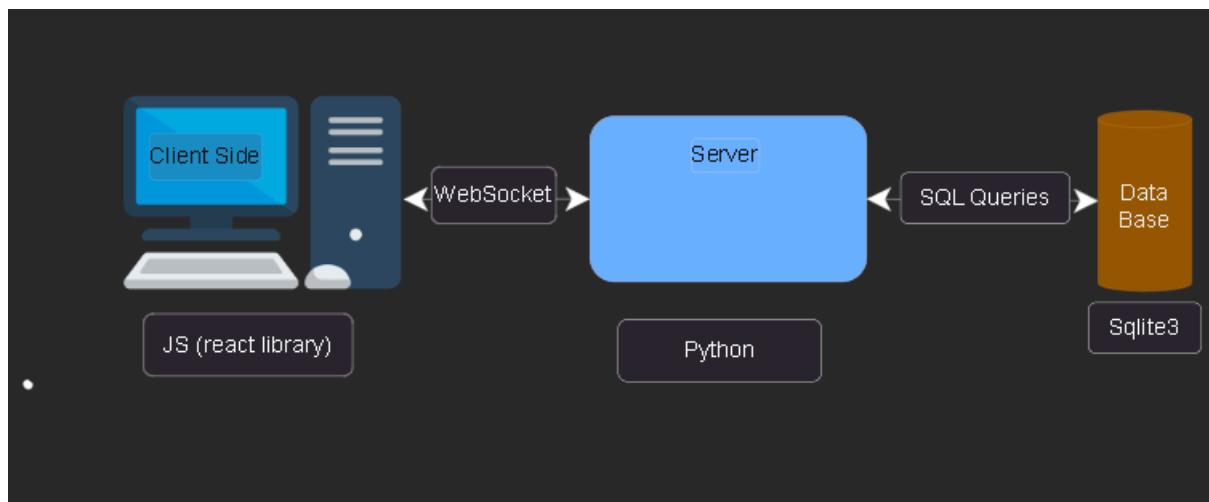
תיאור החומרה

המוצר שלי דורש 3 כלים חומרתיים כדי לרוץ.

מחשב לקוח - מחשב שמריץ דפדפן. זה הרכיב שדרכו המשתמשים ניגשים למערכת "תאם-לי". המחשב זה מה שאחראי על עיבוד ממשק המשתמש ושליחת בקשות לשרת.

שרת המערכת - השרת המרכזי עליו רץ קוד ה-Backend. השרת מקבל פניות המגיעות מלקוחות, מעבד אותן ומחזיר תשובות מתאימות.

מסד נתונים - רכיב האחסון של המערכת. כאן נשמר כל המידע. ההפרדה לשרת שמיועד לנתונים מאפשר יכולת גדילה ושמירה על שלמות הנתונים.



תיאור הטכנולוגיה הרלוונטית

שפות תכנות

צד לקוח - פותחה ב-Java Script עם ספריית React. בחרתי ב-React מכיוון ונותנת לבנות ממשק שגם נראה טוב וגם אינטראקטיבי כך שלמשתמש יהיה את החוויה הטובה ביותר. בנוסף, בחרתי ב-React מכיוון והופכת את האתר ל-Single Page Application.

צד שרת - כתבתי ב-Python. זו שפה עוצמתית שנותנת אפשרות לנהל בקלות את כל הלוגיקה של המערכת בשילוב עם ספריות מאוד טובות שיש לה.

מערכת הפעלה

המערכת פותחה בסביבת Windows 11 אבל מכיוון והיא פותחה ב-Python ו-Java Script (React) אז המערכת היא Cross Platform. לכן ניתן להריץ אותה גם על מערכות הפעלה אחרות אם יש להם את הכלים הנכונים להריץ את הקודים האלו.

תקשרות

התקשרות בפרויקט נעשתה על ידי WebSockets. זהו הפרוטוקול שהשתמשתי בפרויקט. בניגוד ל-HTTP, פרוטוקול WebSocket מאפשר תקשורת דו-כיוונית רציפה. בשביל העברת המידע השתמשתי ב-JSON. זה מבנה נתונים קל להעברת מידע בעיקר כי Java Script תומכים בזה וניתן להעביר ככה כל מיני סוגים של אובייקטים.

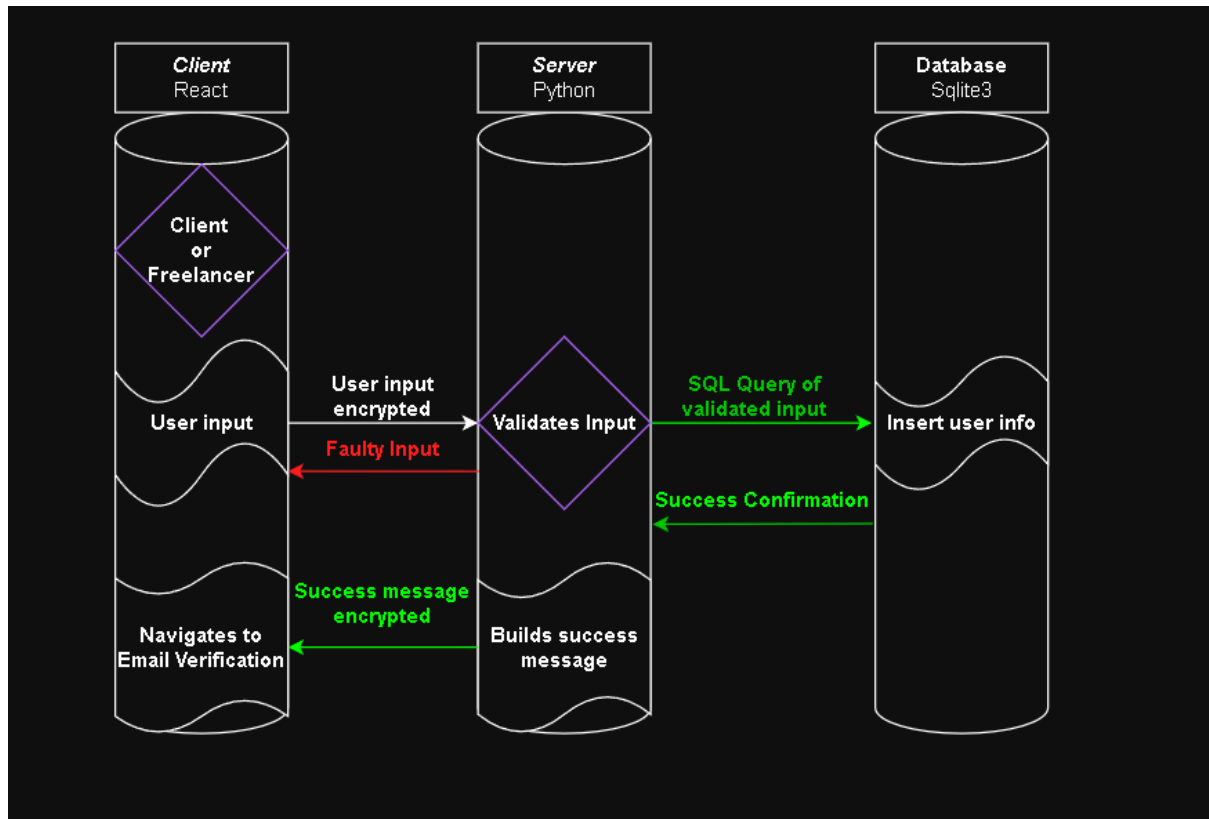
תחומי עניין

הפרויקט משלב מספר תחומים עיקריים שהם:

- פיתוח Full-Stack
- ניהול מסדי נתונים
- והגנה נגד התקפות (Cyber Security)
 - התקפת DDOS
 - התקפת SQL Injection
 - התקפת XSS
 - ועוד

תיאור זרימת המערכת

מערכת ההרשמה

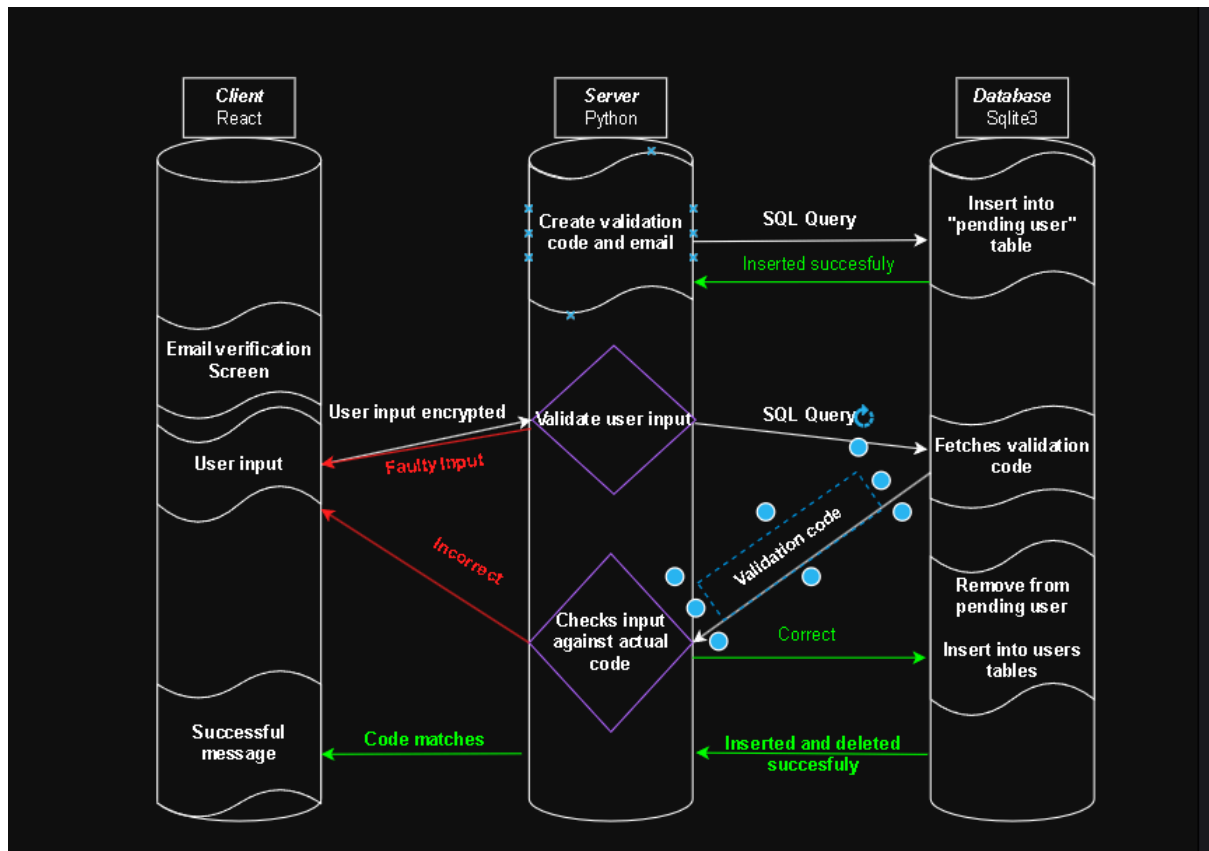


מטרת התהליך - לרשום משתמשים חדשים למערכת תאם-לי

היבט אבטחה - המידע המועבר מהלקוח לשרת מוצפן על ידי פרוטוקול TLS. בנוסף השרת מבצע בדיקות בקפדנות כדי למנוע סוגי התקפות שונות ולשמור על שלמות הנתונים.

לוגיקת השרת - השרת בעצם משמש כאן כ-Proxy בין הלקוח למסד הנתונים ומטרתו היא לוודא שכל הנתונים עומדים בתנאים כדי להיכנס למסד הנתונים ורק אחרי שווידא שולח שאילתה למסד הנתונים להכניס את הנתונים ומחזיר תשובה ללקוח כשהכל עבר.

אימות אימייל

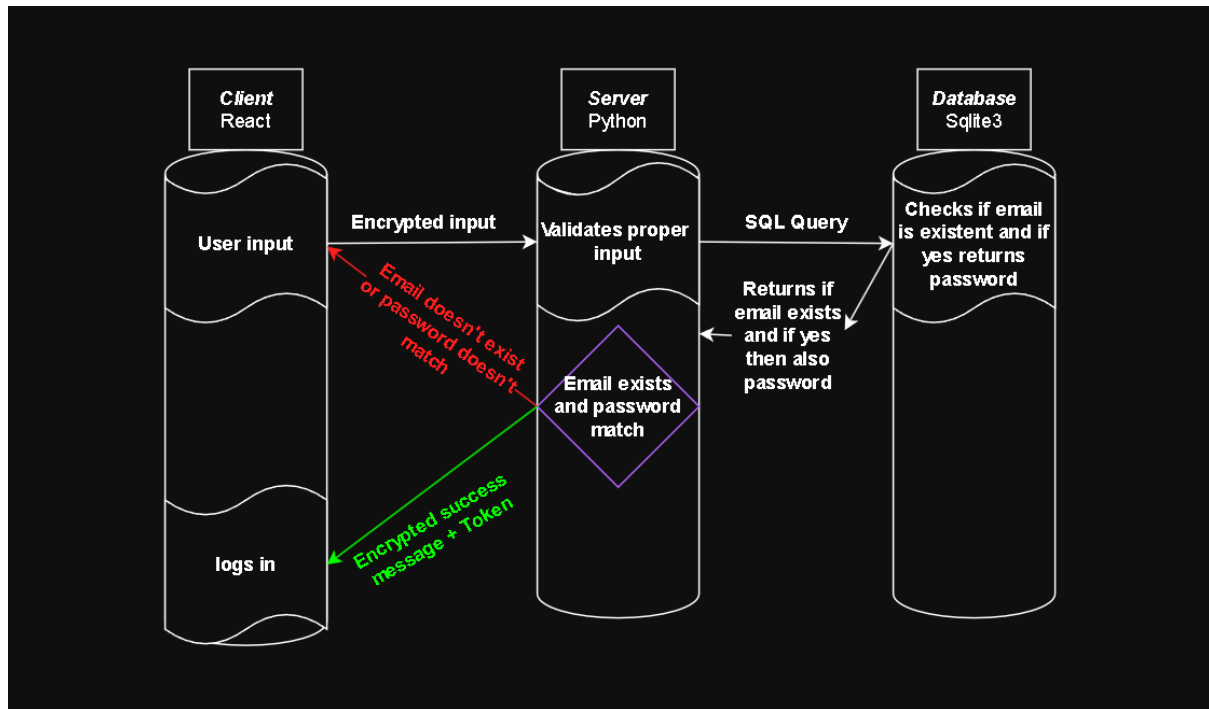


מטרת התהליך - אימות האימייל באמצעות קוד שנשלח לאימייל והעברת משתמש מאומת מטבלת המתנה לטבלת משתמשים.

היבט אבטחה - המערכת משתמש בטבלה זמנית כדי למנוע רישום של משתמשים לא מאומתים למסד הנתונים הראשי. אימות הקוד מתבצע בצד השרת.

לוגיקת השרת - השרת יוצר את הקוד למשתמש (עוד בחלק ההרשמה). השרת אז שומר את הקוד עם פרטי המשתמש בטבלת המתנה. השרת אחר כך מאמת את הקוד שהמשתמש שלח עם הקוד ששומר למשתמש בטבלת ההמתנה ואם מתאים מוציא אותו מטבלת ההמתנה ומוסיף אותו לטבלת המשתמשים.

התחברות

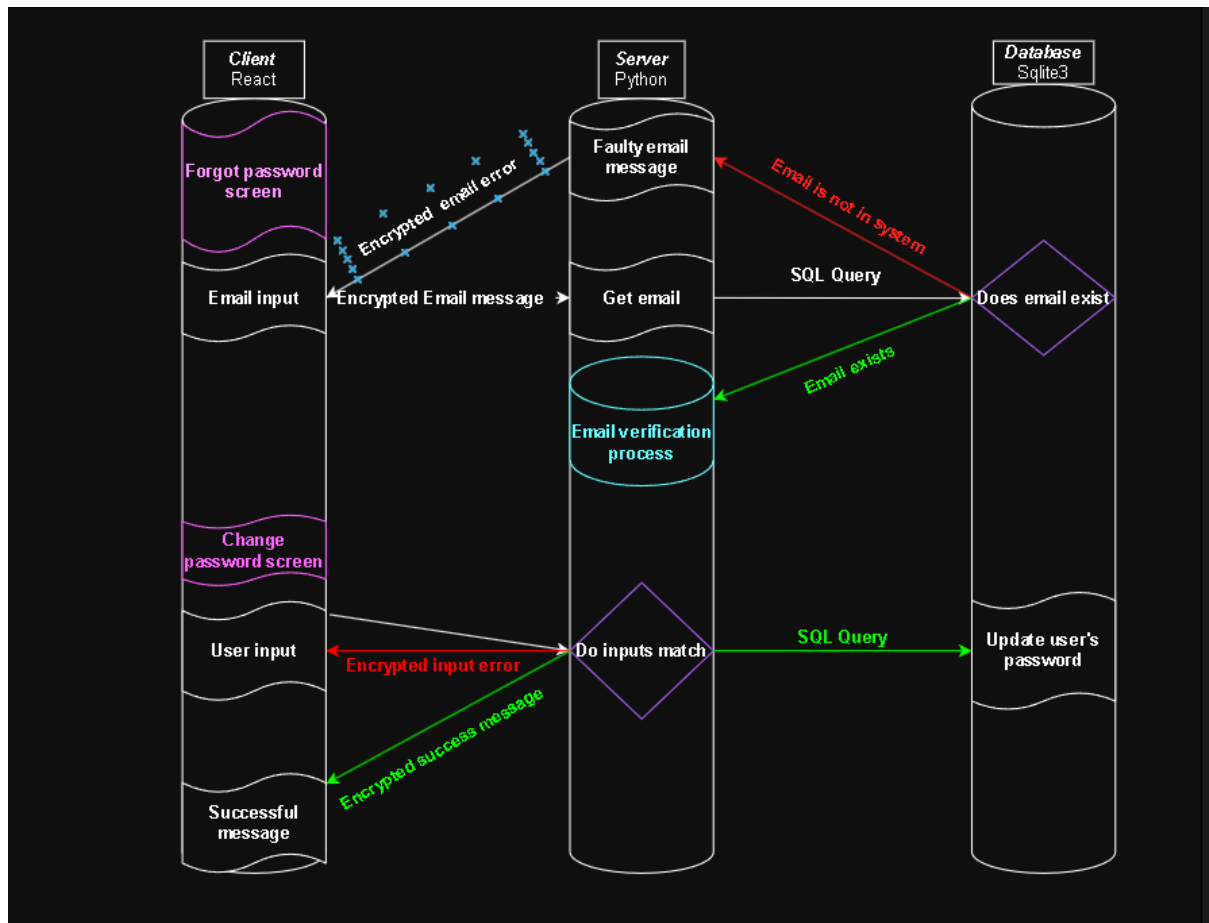


מטרת התהליך - אימות משתמש קיים כדי לתת לו גישה למערכת

היבט אבטחה - הנתונים מועברים על ידי פרוטוקול TLS ולכן מוצפנים. השרת מחזיר הודעות שגיאה כלליות במקרה של כשל כדי לשמור על פרטיות משתמשים רשומים

לוגיקת השרת - השרת מבצע שאילתה כדי להשיג את האימייל והסיסמה של המשתמש ומשווה אותם מול קלט המשתמש. אם הנתונים מתאימים אז השרת מחבר את המשתמש למערכת בכך ששולח לו Token.

שכחתי סיסמה

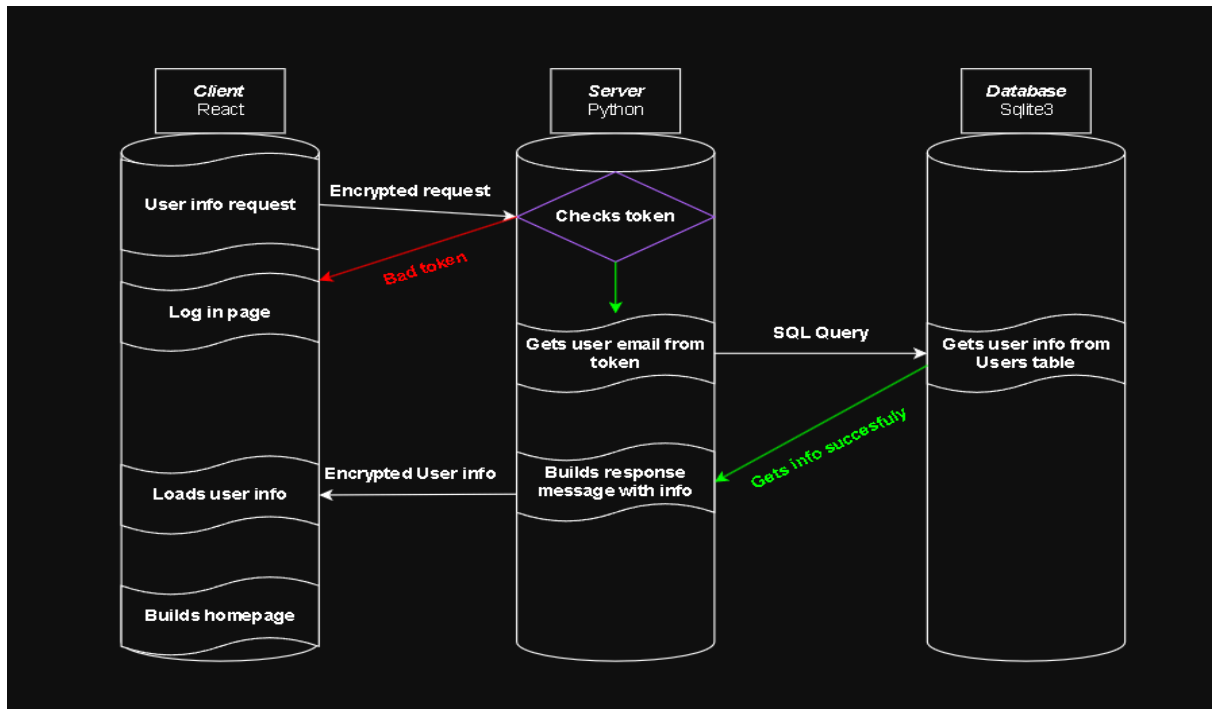


מטרת התהליך - לתת למשתמש ששכח את הסיסמה שלו אפשרות להגדיר סיסמה חדשה

היבט אבטחה - התהליך מוודא שהמשתמש באמת בעל האימייל שהזין בהחלפת סיסמה, בנוסף המערכת בודקת שהאימייל אכן נמצא במערכת כדי שלא ייוצר סיסמה ללא משתמש. בנוסף כל השיחה הזאת בין הלקוח לשרת מוצפנת על ידי פרוטוקול TLS.

לוגיקת השרת - השרת קודם בודק שהמייל קיים במערכת, אם זה המצב אז הוא קורה ללוגיקת אימות המייל. לאחר שהשרת מאמת את המשתמש השרת מקבל את הסיסמה החדשה ומעדכן אותה במסד הנתונים.

עמוד הבית

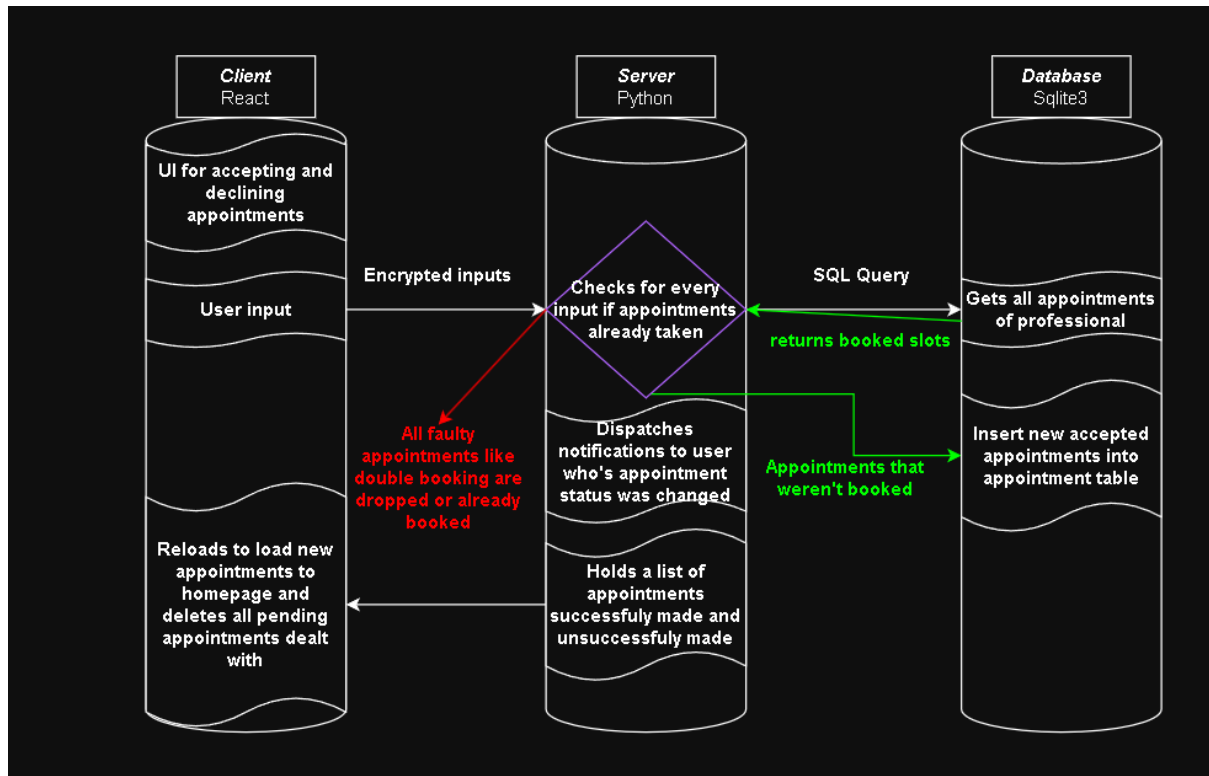


מטרת התהליך - הצגת הפרטים האישיים של המשתמש בדף הבית לאחר התחברות

היבט אבטחה - המערכת אינה מסתמכת על זיהוי שנשלח ללקוח אלא דורשת Token מוצפן וחתום על ידי השרת. השרת מאמת את Token. אם ה-Token תקין רק אז שולחת את הנתונים האישיים של המשתמש.

לוגיקת השרת - השרת מפענח את ה-Token כדי להשיג את האימייל ועם האימייל מוציא ממסד הנתונים את נתוני המשתמש. לאחר שמחלץ את נתוני המשתמש בונה איתם הודעה מסודרת שאותה הוא שולח ללקוח.

אישור/דחיית בקשות תורים

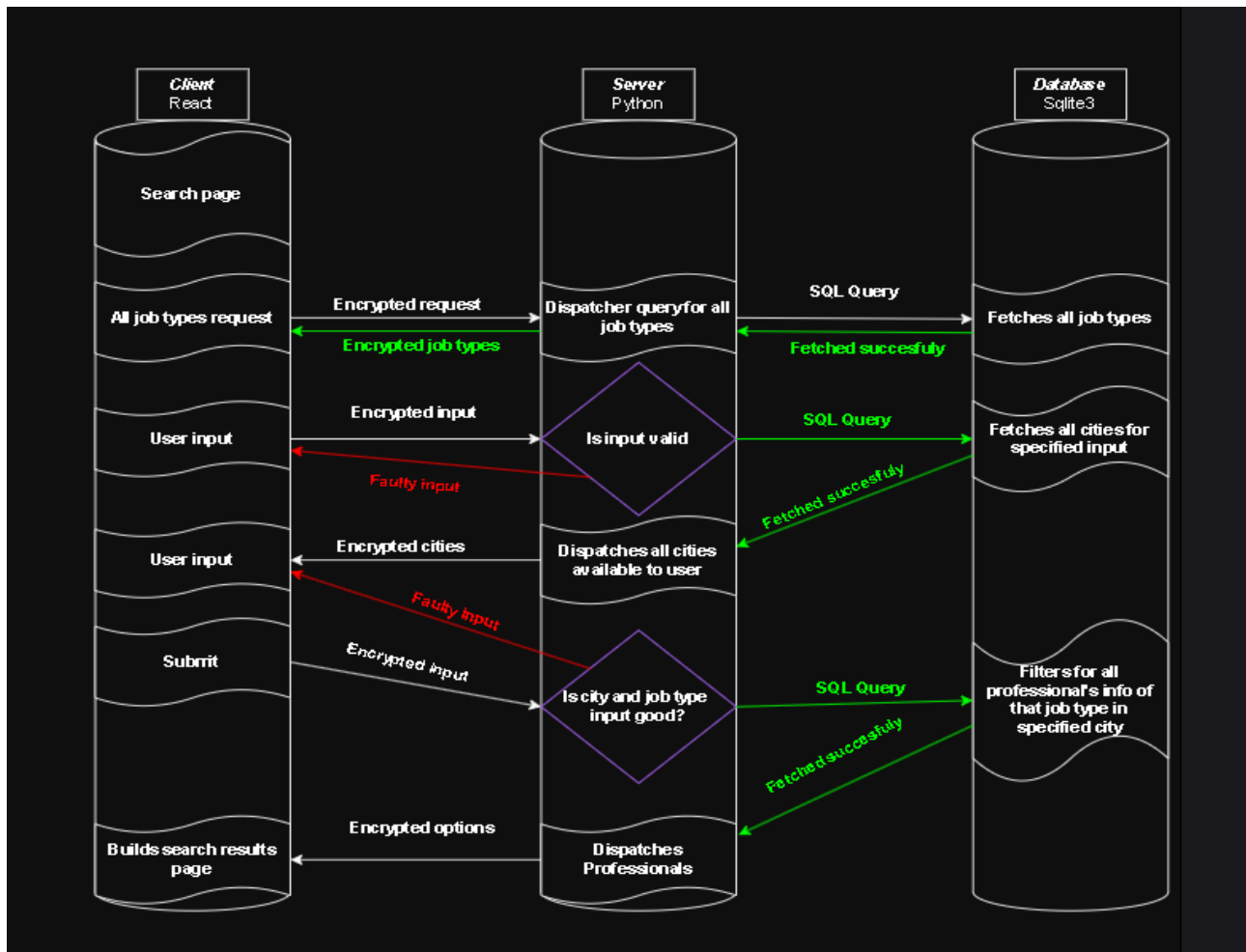


מטרת התהליך - ניהול הבקשות לתורים על ידי איש המקצוע

היבט אבטחה - השרת מוודא שאין ניסיון לביצוע Double Booking בכך שמשווה את התורים שעכשיו אושרו לתורים שקיימים כבר במערכת

לוגיקת השרת - השרת מקבל רשימת תורים עם הסטטוס שלהם, שולף את התורים הקיימים ממסד הנתונים ומסנן לפי תורים שכבר נלקחו כדי למנוע Double Booking רק תורים שפנויים ואושרו מוכנסים לתורים של בעל המקצוע. השרת מעביר הודעות אל המשתמשים שהתורים שלהם אושרו/נדחו לגבי שינוי הסטטוס.

חיפוש בעל מקצוע

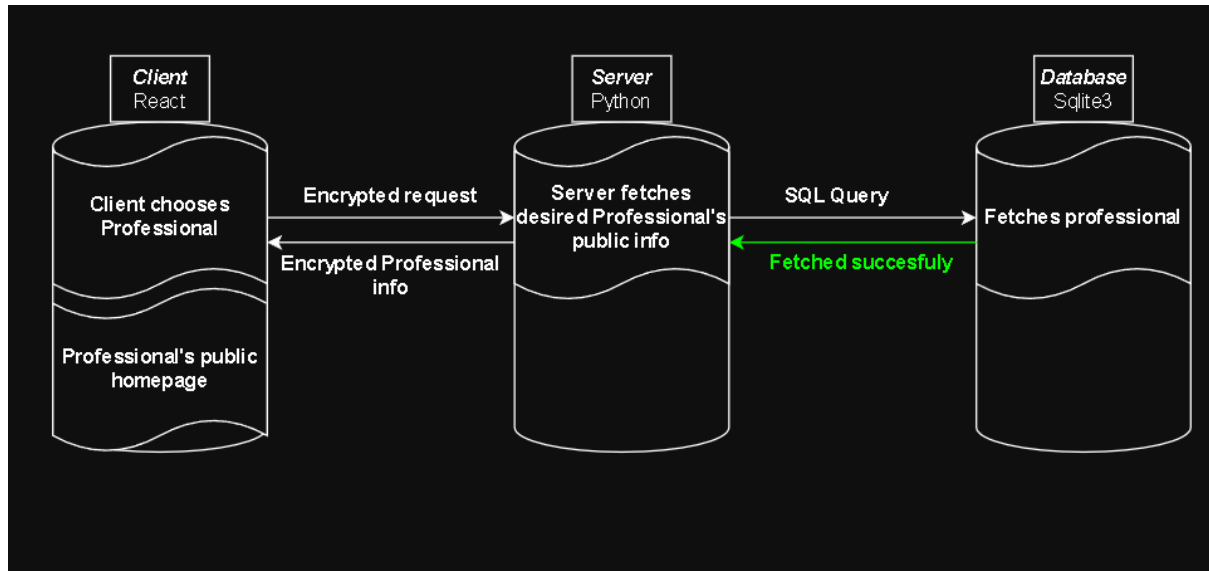


מטרת התהליך - מאפשר למשתמש לסנן ולמצוא אנשי מקצוע לפי סוג המקצוע והמיקום הגאוגרפי

היבט אבטחה - כל הבקשות והמידע שנשלח מוצפנות. השרת מבצע על כל קלט אימות כדי לוודא שאין שקלטים זדונים מהמשתמש ובמטרה להחזיר רק מידע נחוץ ממסד הנתונים.

לוגיקת השרת - השרת משמש כמתווך הוא קודם שולף את כל בעלי המקצוע ואז לפי המקצוע שנבחר שולף את כל הערים האפשריות ואז אחרי שהשלקוח שלח בקשה מושלמת השרת מוצא את כל בעלי המקצוע הרלוונטים ומחזיר רק נתונים פרקטיים לגביהם.

עמוד בית של בעל מקצוע

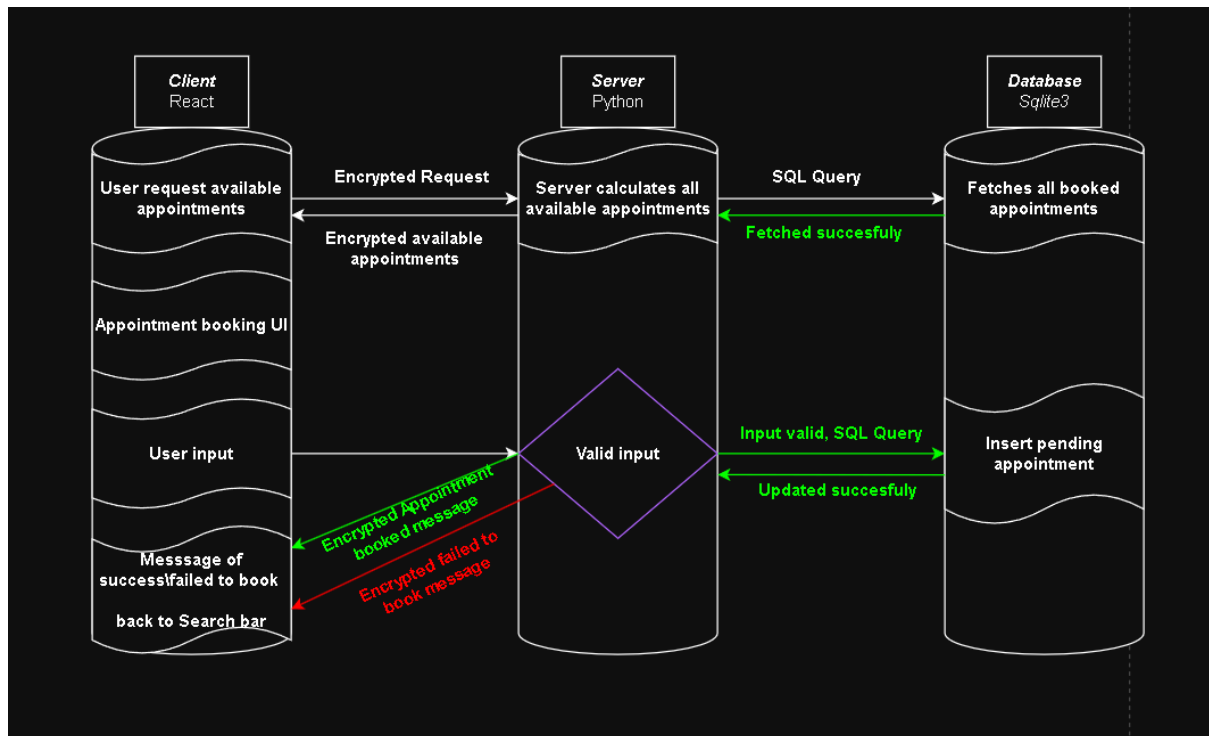


מטרת התהליך - הצגת עמוד הבית של בעל המקצוע הרצוי

היבט אבטחה - השרת מוודא שרק מידע שהוגדר כציבורי יועבר אל הלקוח. בקשת המידע והמידע מוצפנות בעזרת TLS.

לוגיקת השרת - השרת מקבל את ה-ID של בעל המקצוע ומחזיר את הנתונים הציבוריים של בעל המקצוע הרצוי.

קביעת תור



מטרת התהליך - לאפשר למשתמש לצפות בתורים הפנויים של איש מקצוע ולבקש לקבוע תור

היבט אבטחה - כל התקשורת מוצפנת בעזרת TLS. השרת לא מסתמך על על הקלט של המשתמש אלא מוודא כל פעם שהקלט הגיוני

לוגיקת השרת - השרת מבצע בדיקה של כל התורים הפנויים של בעל המקצוע לחודש הקרוב. לאחר מכן השרת מקבל את התור הרצוי בודק שכל הקלט של התור עומד בדרישות ומכניס אותו לתורות שמחכים אישור על ידי בעל המקצוע

האלגוריתמים המרכזיים בפרויקט

ניתוח של הבעיה האלגוריתמית

הגדרת הבעיה - כיצד להציג למשתמשים רק את חלונות הזמן הפנויים של בעל המקצוע ולוודא שברגע האישור של התור לא נוצרת התנגשות עם תור שכבר אושר באותו הזמן.

האתגר - התורים נקבעים לא באופן רציף. עלי לחשב דינמית את התורים הפנויים בכך שאני מתבסס על אורך העבודה המשתנה של כל בעל מקצוע ועל התורים שכבר נקבעו

תיאור אלגוריתמים קיימים

אלגוריתם חלונות זמן קבועים - הגישה מאוד פשוטה, בה היום של כל בעל מקצוע מחולק לבלוקים שווים. הפתרון הוא הפשטות של הניהול של זה אבל החיסרון הוא חוסר הגמישות בנגנון כזה

אלגוריתם Interval Tree - מבנה נתונים שנחשב אופטימלי לחיפוש טווחים אבל מורכב למימוש ותחזוקה במערכת Web פשוטה.

שאליתת חפיפה במסד הנתונים - שימוש בלוגיקה בתוך ה-SQL כדי לסנן התנגשויות עוד לפני שהנתונים מגיעים לקוד. זהו הפתרון הכי יעיל למערכת כמו שלי כי הוא מנצל את המהירות של מסד הנתונים

הפנייה למקורות רלוונטים

[Date and Time Functions](#) - במהלך הפיתוח הסתמכתי על התיעוד הרשמי בנושא ניהול זמנים במסד הנתונים. למדתי איך להשתמש בפונקציות מובנות לחישוב טווחי זמן והשוואת תאריכים בשרת.

[Shceduling in Greedy Algorithms](#) - חקרתי את בעיית ה-Interval Scheduling שהיא בעיה קלאסית באלגוריתמיקה.

[ISO 8601](#) - קראתי על התקן הבינלאומי לייצוג תאריכים וזמנים. הבנתי ששמירת זמנים בפורמט סטנדרטי זה היא קריטית כדי לאפשר למסד הנתונים לבצע מיון וחיפוש בצורה היעילה ביותר.

סקירת הפתרון הנבחר**הפתרון:**

האלגוריתם מחשב בחכמה לכל בעל מקצוע את התורים הפנויים שלו בתהליך הבא

א.

1. השרת מקבל ממסד הנתונים את הזמן הממוצע לתור ואת שעות העבודה של בעל המקצו
2. השרת מבצע מבקשה ממסד הנתונים את כל התורים שכבר נקבעו בטווח של חודש קדימה
3. השרת מייצר רשימה של חלונות זמינים בכך שמריץ לולאה ובודק אם לתאריך המסוים ולשעה המסוימת יש תור שקיים

נימוק - שימוש בשיטה הזאתי מבטיחה לנו בסיבוכיות קבועה של $O(1)$ מכיוון וכמות ההרצות של הלולאה קבועה ורק כמות התורים שכבר קיימים משתנה אבל זה לא משפיע על הריצה של הלולאה

האלגוריתם מוסיף את התור המאושר למסד הנתונים

ב.

כדי לוודא שלא נוצרת התנגשות עם תור קיים האלגוריתם משתמש בלוגיקה בשאלת ה SQL שבודקת את החפיפה בין השעות. במקום לבדוק רק אם שעת ההתחלה זהה האלגוריתם בודק אם יש כל נקודת זמן משותפת בין תור קיים לתור הרצוי לקבוע

נימוק:

דינמיות - השיטה מאפשרת לי בעתיד להוסיף דרך לקבוע תורים באורכים שונים מבלי לבלבל את המערכת בניגוד לשיטה של "חלונות קבועים".

אמינות - הבדיקה מתבצעת ברגע האחרון לפני עדכון מסד הנתונים במטרה שלא יקרה מקרה קצה שבו יואשרו שני תורים של משתמשים שאושרו באותו הזמן.

ניתוח של הבעיה האלגוריתמית

הגדרת הבעיה - כיצד לתת למשתמש למצוא את בעל המקצוע המתאים ביותר מתוך מאגר נתונים גדול תוך התחשבות במספר משתנים.

האתגר - במאגר נתונים גדול חיפוש ידני או לא יעיל יכול להוביל לזמני טעינה ארוכים ולתוצאות לא מתאימות לקלט שהמשתמש הכניס. עלי לייצר מנגנון שיודע לשלב פילטרים (יותר מאחד) שיעזרו לי להציג למשתמש את בעל המקצוע הכי מתאים.

תיאור אלגוריתמים קיימים

Linear Search - מעבר על כל בעלי המקצוע ברשימה ובדיקה אחד אחד אם הם תואמים את הסינון, החיסרון שזה בסיבוכיות של $O(n)$.

Full Text Search - אלגוריתם מורכב שסורק מילים בתוך טקסט חופשי. יתרון - אלגוריתם מאוד גמיש בחיפוש, חיסרון - דורש משאבי מערכת רבים וכבדים עבור מערכת בגודל שלי.

Indexed Database Filtering - שימוש באינדקסים של מסד נתונים כדי לצמצם את מרחב החיפוש באופן מהיר.

הפנייה למקורות הרלוונטים

[SQL Where Clause Optimization](#) - מחקר על הדרך שבה מנועי מסדי נתונים מייעלים שאילתות עם תנאים מרובים.

[Relational Algebra](#) - הבסיס התאורטי של מסדי נתונים

[Indexing Strategies](#) - לימוד על הדרכים לייעל חיפושים במסדי נתונים

סקירת הפתרון הנבחר**בחרתי לבצע את החיפוש שלי על ידי שימוש בשאילתה דינמית שמבוססת על פרמטרים**

1. השרת בונה שאילתת SQL שיש בה פרמטרים של העיר ושל סוג המקצוע
2. השאילתה נשלחת אל מסד הנתונים פעם אחת בלבד עם כל הפרמטרים שנבחרו
3. התוצאות חוזרות ואני יכול גם להוסיף בשאילתה הגדרה של איך למיין אותן

נימוק:

יעילות - במקום להשיג את כל בעלי המקצוע ולסנן אותם בצד הלקוח הסינון מתבצע אצל השרת.

גמישות - המבנה הזה של השאילתת SQL נותן אפשרות להוסיף עוד פרמטרים בעתיד לשאילתה (כמו סינון לפי דירוג או לפי שם) מבלי לשנות את ארכיטקטורת הקוד.

תיאור סביבת הפיתוח

כלי הפיתוח הדרושים לפיתוח

לצורך פיתוח המערכת השתמשתי בכלים שמאפשרים הפרדה בין צד הלקוח לצד השרת.

סביבת העבודה - Visual Studio Code

זו בעצם הסביבה בה כתבתי את הקוד שלי, היא איפשרה לי עבודה נוחה ומקבילה בין צד הלקוח ב-React וצד השרת ב-Python בלי לעבור בין תוכנות שונות.

צד לקוח - React & Redux

React זו ספרייה של JavaScript לבניית ממשק משתמש ריאקטיבי ומעוצב יפה. ה-Syntax בשפה הזאת נקרא JSX שזה בעצם שילוב של JavaScript בשביל ליצור גרפיקה יפה שמשלבת בה לוגיקה שבנויה על החלטות המשתמש ו-HTML שמוכרת בתור שפה שמאוד קל לפתח בה דברים יפים ב-Web.

Redux היא ספרייה לניהול מצב האפליקציה. היא עוזרת לשמור מידע של משתנים כשעוברים בין עמודים ומבטיחה מקור מידע אמין לכל המערכת, מה שמקל על הדרך שבה React מצפה שתעביר משתנים ותשמור אותם.

Node.js

סביבת הרצה שמאפשרת להריץ קוד JavaScript ישר על המחשב ולא להגביל את השפה רק לדפדפן.

שימושים של זה בפרויקט שלי הם:

1. התקנת ספריות - הדרך שבה הורדתי את React ו-Redux
2. שרת פיתוח - Node.js גם הופך את הקוד שכתבתי לאתר שאפשר לראות בדפדפן

צד שרת - Python

Python זו שפה עליה הלוגיקה של השרת מתבססת. היא שפה מאוד נוחה לבנות עליה לוגיקה מסובכת שלא מצריכה הרבה זמן ריצה ולכן האיטיות של Python בשילוב עם האופטימיזציה של השרת מבחינת הלוגיקה זמן התגובה למשתמש נשאר מהיר.

מסד הנתונים - Sqlite3

בחרתי במסד נתונים SQLite3 מכיוון שהוא מסד נתונים קטן יותר ששמור בתיקייה מקומית.

ניידות - בפרויקט כמו שלי, שאני מציג אותו ומעביר אותו בין מחשבים, זה הרבה יותר קל כשמסד הנתונים הוא קובץ מקומי.

תאימות - Python תומכת בפיתוח במסד נתונים מסוג SQLite3, באמצעות ספרייה שמקלה על התקשורת עם מסד הנתונים

ביצועים - עבור אתרים קטנים (כמו "תאם-לי"), SQLite3 מהיר מאוד. הוא מבצע פעולות של קריאה וכתובה ישירות מהדיסק, מה שחוסך את ה-Overhead של בקשה משרת נתונים מרוחק.

יציבות - בזכות השמירה כקובץ מקומי, אין חשש ששרת מסד הנתונים יפסיק לעבוד בגלל תקלות תקשורת עם השרת של מסד הנתונים

סביבה וכלים הנדרשים לבדיקות

כדי לוודא שהמערכת פועלת בצורה תקינה ומסונכרנת השתמשתי בכלים ובשיטות הבאות:

בדיקת תקשורת בזמן אמת (Websocket)

פתחתי חיבור Websocket בין הלקוח לשרת ושלחתי הודעות. את ביצוע מה שנשלח ומה התקבל השתמשתי בהדפסות במסך השחור בצד השרת ובלשונית ה-Inspect בדפדפן שם וידאתי שהמידע ששלחתי בין הלקוח לשרת זה מה שציפיתי שישלח.

ניתוח מצב המערכת (Redux Devtools)

השתמשתי בתוסף של Redux Devtools כדי לעקוב אחרי זרימת הנתונים באתר במהלך ריצה. בעזרת תוסף זה וידאתי של פעולה של המשתמש שאמור לעדכן את ה-State של ה-Store באמת מעדכנת אותה לאורך השימוש באתר.

בדיקת זיכרון הדפדפן ו-Cache

ביצעתי בדיקות במטמון של הדפדפן כדי לוודא שנתונים זמניים ששמרתי שם אכן נשמרים וכשאני מוחק אותם הם נמחקים כדי לוודא שהמערכת לא שומרת מידע ישן.

Backend Debugging

השתמשתי בהדפסות בנקודות מפתח בקוד בשני הצדדים

- בצד השרת - עקבתי אחרי ה-Terminal כדי לזהות שגיאות בלוגיקה של השרת
- בצד הלקוח - עקבתי אחרי ה-Console בלשונית ה-Inspect כדי למצוא בעיות לוגיות

Database Verification

כדי לוודא שהשאליות SQL שלי באמת נקלטו השתמשתי בפקודות Print מתוך הקוד שהדפיסו את מה שחיפשתי. כך וידאתי שהמידע שנשמר במסד הנתונים אכן תואם למידע שהשרת היה אמור להכניס למסד הנתונים.

פרוטוקול תקשורת

פרוטוקול התקשורת

בפרוטוקול התקשורת שלי אני משתמש ב-WebSocket עם תוספת לאיך אני שולח את ה-Payload. הפרוטוקול של WebSocket אני מימשתי בצד השרת אבל בעזרת Socket. ככה נראה פרוטוקול WebSocket:

FIN (1 bit)	RSV1,RSV2,RSV3 (1 bit each)	Opcode (4 bits)	Mask (1 bit)	Payload Length (7 bits)	Extended payload length (16/64 bits)	Masking Key (32 bits)	Payload
----------------	-----------------------------	--------------------	-----------------	----------------------------	---	--------------------------	---------

כל Frame נשלח באופן הבא עם השדות הבאים

- **FIN (ביט 1)** - קובע אם זו החבילה האחרונה של ההודעה
- **RSV1-3 (ביטים 3)** - תוספים אופצינאליים (לא רלוונטי אלי)
- **Opcode (ביטים 4)** - סוג ההודעה
- **Mask (ביטים 1)** - מציין אם המידע מוצפן.
- **Payload Length (ביטים 7+הרחבה)** - מגדיר את אורך ההודעה. אם האורך קטן מ-126 בתים הוא נכתב כאן. אם הוא גדול יותר נעשה שימוש בשדות ההרחבה (16 או 64 ביט) בהתאם לגודל המידע.
- **Masking Key (ביטים 32)** - המפתח המשמש לפענוח המידע אם ביט המסיכה דלוק
- **Payload** - המידע עצמו

לאחר פירוק חבילת ה-WebSocket, המידע מועבר במבנה של מילון שעובר סריאליזציה לפורמט JSON.

המילון בנוי בצורה הבאה:

1. **סוג הודעה:** מוגדר מראש בשני הצדדים
2. **מידע:** מילון עם המידע עצמו
3. **סוג המידע:** האם המידע טקסטואלי או בינארי.
4. **טוקן:** מזהה אבטחה ייחודי של הלקוח.

לפני השליחה, ההודעה עוברת דרך שכבת TLS המבצעת הצפנה של כל תעבורת הנתונים בין השרת ללקוח.

סוגי ההודעות במערכת

חשוב לפני - ה-Token נוצר רק לאחר ההתחברות לכן בחלק מההודעות הוא ריק ובחלק יש ערך

שם ההודעה	נשלח מ... אל...	מבנה השדות (Payload) במילון	תיאור השדות במילון שבשדה ה-Payload	מטרת ההודעה
LOGIN	לקוח ← שרת	Message type, payload, data type, token (ריק)	email, password	בקשה להתחבר למערכת
LOGIN	שרת ← לקוח	Message type, payload, data type, token (ריק)	status	להודיע ללקוח אם החיבור צלח (LOGGED/FAILED_LOG) ואם כן להביא לו את ה-Token שהוא צריך ואם לא להביא לו שגיאה גנרית
USERSIGNUP	לקוח ← שרת	Message type, payload, data type, token (ריק)	name, email, password	בקשה לרשום משתמש חדש למערכת
USERSIGNUP	שרת ← לקוח	Message type, payload, data type, token (ריק)	status	להודיע ללקוח אם השרת מעביר אותו לתהליך הורפיקציה או שהיה בעיה בהרשמה (SIGNED/FAILED_SIGNED) ומודיע מה הייתה הבעיה
FRLNCSIGNUP	לקוח ← שרת	Message type, payload, data type, token (ריק)	name, email, password, profession, cities, years, startWorking, finish_working, job_duration, description, hour_price	בקשה לרשום בעל מקצוע חדש למערכת
FRLNCSIGNUP	שרת ← לקוח	Message type, payload, data type, token (ריק)	status	להודיע ללקוח אם השרת מעביר אותו לתהליך הורפיקציה או שהיה בעיה בה (SIGNED/FAILED_SIGNED) רשמה ומודיע מה הייתה הבעיה
VERIFYING	לקוח ← שרת	Message type, payload, data type, token (ריק)	email, verification_code, role	להוכיח לשרת שהאימייל שהוזן באמת תחת בעלות הלקוח
VERIFYING	שרת ← לקוח	Message type, payload, data type, token (ריק)	status	להודיע ללקוח שההרשמה נגמרה בהצלחה (VERI_GOOD/VERI_BAD) והשרת הוסיף אותו למערכת
FRGT_PAS_RQT	לקוח ← שרת	Message type, payload, data type, token (ריק)	email	לבקש מהשרת להכניס את הלקוח למצב שינוי סיסמה

להודיע למשתמש אם הוא יכול להתחיל את תהליך ה-"שכחתי סיסמה" או אם אירעה תקלה (FRGT_GOOD/FRGT_BAD)	status	Message type, payload, data type, token (ריק)	שרת ← לקוח	FRGT_PAS_RQT
לאמת לשרת שהאימייל שוהזן באמת תחת בעלות הלקוח	email, veri_code	Message type, payload, data type, token (ריק)	לקוח ← שרת	FRGT_PAS_AUTH
להודיע ללקוח אם הוא יכול להתקדם לשינוי הסיסמה או אם האימות נכשל (VERI_GOOD/VERI_BAD)	status	Message type, payload, data type, token (ריק)	שרת ← לקוח	FRGT_PAS_AUTH
לשלוח לשרת את הסיסמה החדשה שאליה הלקוח רוצה להחליף	email, password	Message type, payload, data type, token (ריק)	לקוח ← שרת	CHNG_PAS
להודיע ללקוח אם הסיסמה שונתה בהצלחה ואם לא מה הייתה השגיאה (CHNG_GOOD/CHNG_BAD)	status	Message type, payload, data type, token (ריק)	שרת ← לקוח	CHNG_PAS
להודיע ללקוח שהמזהה סשן שלו לא טוב (BAD_TKN)	status	Message type, payload, data type, token	שרת ← לקוח	BRD
להשיג את המידע האישי של הלקוח מהשרת	email	Message type, payload, data type, token	לקוח ← שרת	USR_INFO
להודיע ללקוח אם הצליח להשיג את המידע ואם כן שולח את המזהה של המשתמש ואת כל המידע האישי שלו (GOT_USR_INFO/FLD_USR_INFO)	status, user_id	Message type, payload, data type, token	שרת ← לקוח	USR_INFO
להודיע לשרת שהמשתמש קרא את ההודעות שקיבל	user_id	Message type, payload, data type, token	לקוח ← שרת	RD_NTFC
להודיע ללקוח שהוא עידכן את המערכת ובמידה שנתקל בבעיה מודיע ללקוח (READ_NOTIFICATIONS/FAILD_READ_NOTIFICATIONS)	status	Message type, payload, data type, token	שרת ← לקוח	RD_NTFC
לעדכן את השרת עם ההחלטות שבעל המקצוע החליט לגבי התורים שחיכו למענה	dictionary: key: appointment_id value: decision	Message type, payload, data type, token	לקוח ← שרת	UPDT_APP_STAT
להודיע ללקוח אם הצליח לקבל את ההחלטות שלו ואם כן אז אם היו כשלים בחלקם או לא (UPDT_APP/FAILED_UPDT_APP)	status	Message type, payload, data type, token	שרת ← לקוח	UPDT_APP_STAT
לקבל מהשרת את כל השעות	freelancer_id	Message type,	לקוח ← שרת	(GET_APP_TIME

הפנויות של בעל המקצוע		payload, data type, token		
להודיע ללקוח אם הצליח להשיג את המידע ואם כן אז שם בערך את השעות הפנויות (GOT_APP_TIME/FLD_APP_TIME)	status	Message type, payload, data type, token	שרת ← לקוח	GET_APP_TIME
להכניס בקשה לקבוע תור עם בעל מקצוע	dictionary: date, start_time, end_time, address, details	Message type, payload, data type, token	לקוח ← שרת	MK_APP
להודיע ללקוח אם התור נקלט בהצלחה או שלא (BKD_APP/FLD_BKD_APP)	status	Message type, payload, data type, token	שרת ← לקוח	MK_APP
לקבל מהשרת את המידע הפומבי של בעל המקצוע	freelancer_id	Message type, payload, data type, token	לקוח ← שרת	PBL_INFO
לעדכן את הלקוח אם הצליח להשיג את המידע (GOT_FREE/FLD_FREE) ואם כן מצרף את המידע הפומבי על בעל המקצוע	status	Message type, payload, data type, token	שרת ← לקוח	PBL_INFO
לקבל מידע מינמלי לתצוגה יפה של כל בעלי המקצוע שמופיעים אחרי פילטור לפי הערכים בשדות שנשלחו	city, job	Message type, payload, data type, token	לקוח ← שרת	MNML_VIEW
מודיע ללקוח אם הצליח להשיג את המידע (GOT_MNML/FLD_MNML) ואם כן אז שולח מילון עם כל בעלי המקצוע שנמצאו עם מידע מינמלי עליהם	status	Message type, payload, data type, token	שרת ← לקוח	MNML_VIEW
שהשרת שיחזיר תשובה עם כל סוג העבודה שרשומים במערכת	ריק	Message type, payload, data type, token	לקוח ← שרת	GET_JBS
מודיע ללקוח אם הצליח להשיג (GOT_JBS/FLD_JBS) ואם כן מצרף רשימה של כל הסוגים של בעלי המקצוע	status	Message type, payload, data type, token	שרת ← לקוח	GET_JBS
מבקש מהשרת שישגי את כל הערים שבהם יש בעל מקצוע מהסוג שנשלח	job	Message type, payload, data type, token	לקוח ← שרת	GET_CITIS
מודיע ללקוח אם הצליח להשיג את הערים (GOT_CITIS/FLD_CITIS) במידה שכן שולח רשימה של כל הערים האפשריות	status	Message type, payload, data type, token	שרת ← לקוח	GET_CITIS

מסכי המערכת

התחברות

תפקיד המסך - המסך משמש כשער הכניסה למערכת. התפקיד שלו לאפשר למשתמשים שקיימים כבר במערכת להזדהות מול השרת ולקבל גישה לחשבונם בזמן שמירה על פרטיותם.

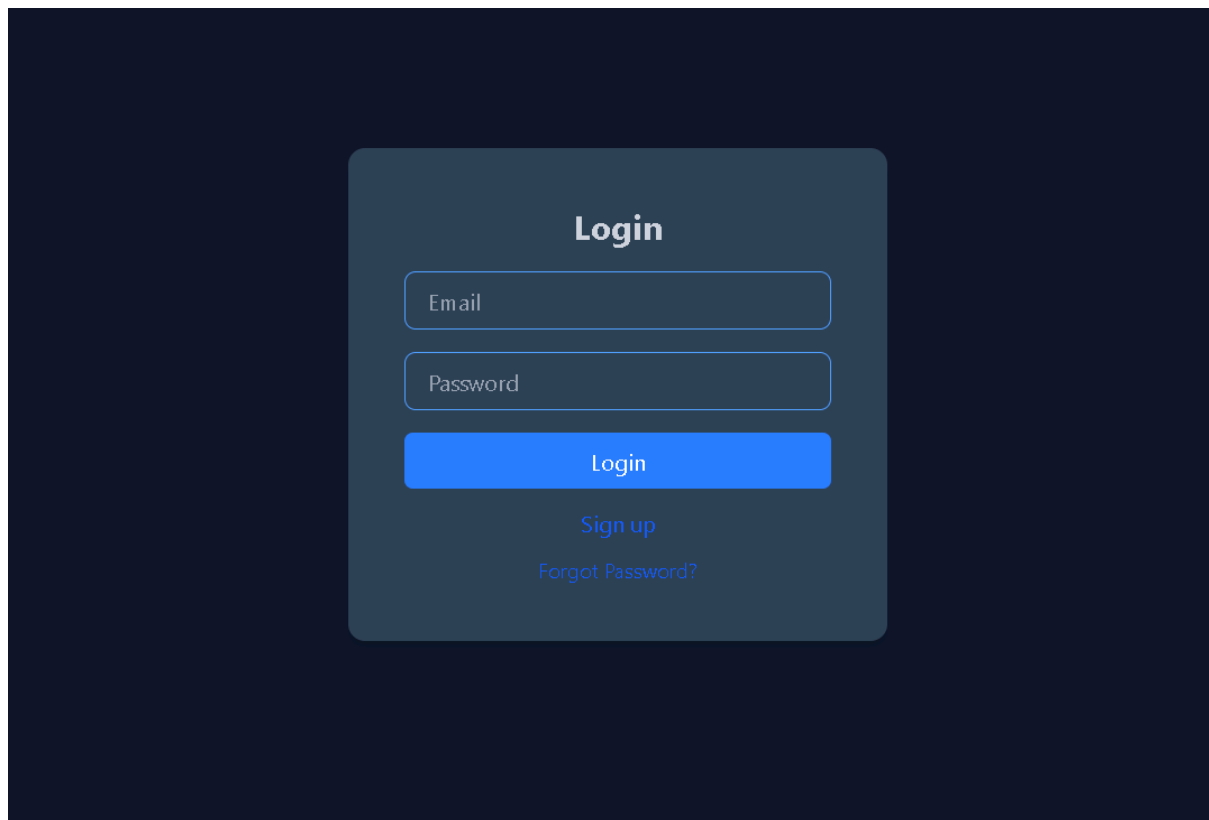
תיאור המסך ופונקציונליות

1. **הזנת פרטי הזדהות -** מקום שבו המשתמש יכול להכניס את האימייל שלו ואת הסיסמה שלו ובעזרתה יכול להתחבר למערכת בעזרת לחיצה על הכפתור **Login**

2. **משוב מהשרת -** במקרה של בעיה באחד מהנתונים שהוזנו הודעה גנרית מוצגת בפני המשתמש

3. **כפתור הרשמה -** כפתור שמתחיל ללקוח את תהליך יצירת המשתמש

4. **כפתור של שחזור סיסמה -** כפתור שמתחיל ללקוח את תהליך האיתחול סיסמה

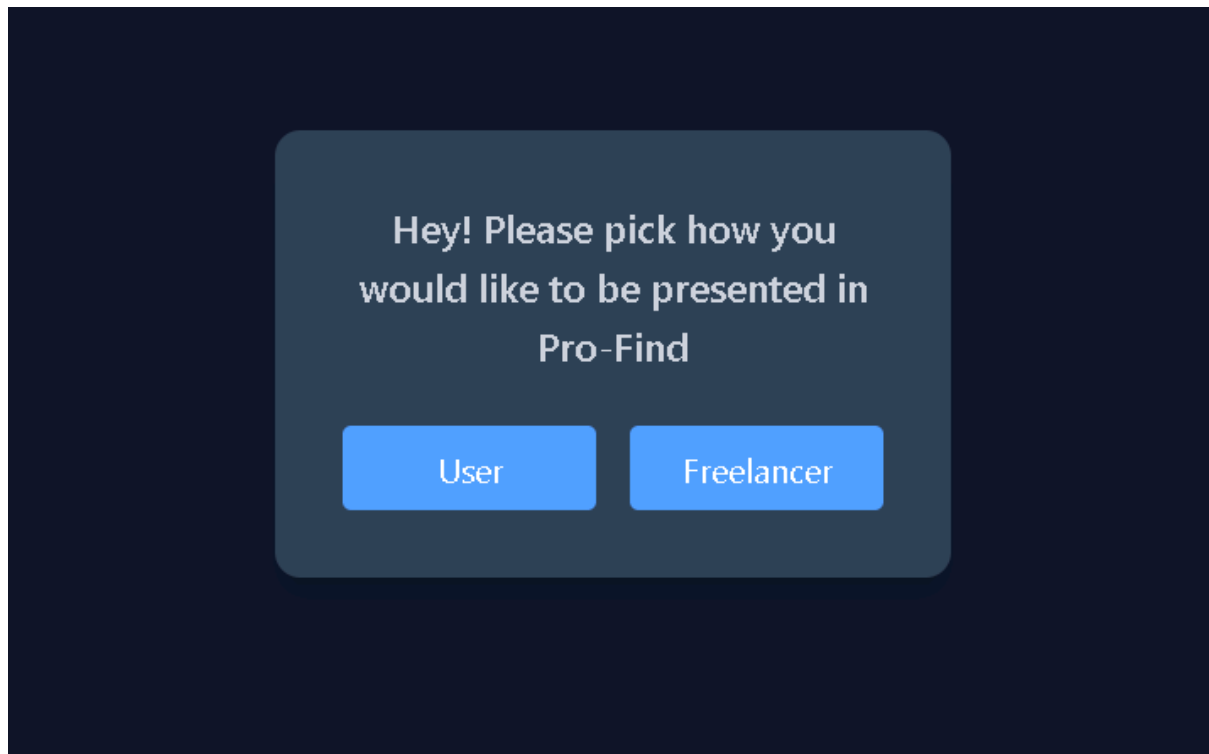
A dark-themed login screen with a central light blue rounded rectangle. Inside the rectangle, the word "Login" is at the top. Below it are two input fields: "Email" and "Password". Under the "Password" field is a blue "Login" button. Below the button are two links: "Sign up" and "Forgot Password?".

הרשמה למערכת**בחירת סוג משתמש**

תפקיד המסך - המסך משמש כצומת להכוונת המשתמש בתהליך הרישום. תפקידו לאפשר למשתמש לבחור את סוג החשבון הרלוונטי עבורו ולקחת אותו לטופס מילוי הפרטים המתאים לו.

תיאור המסך ופונקציונליות

1. **כפתור 'לקוח' -** ניתוב המשתמש למסך ההרשמה הרלוונטי ללקוח
2. **כפתור 'בעל מקצוע' -** ניתוב המשתמש למסך ההרשמה הרלוונטי לבעל מקצוע



הרשמת לקוח

תפקיד המסך - המסך מאפשר למשתמשים חדשים המעוניינים להשתמש בשירותי המערכת כלקוחות ליצור חשבון אישי. תפקידו של המסך הוא לאסוף את נתוני הלקוח ורישומם במערכת.

תיאור המסך והפונקציונליות - המסך מציג טופס רישום הכולל את השדות והאלמנטים הבאים:

1. שדות הזנת מידע

- a. *Username*: שדה חובה לבחירת שם משתמש.
 - b. *Email*: הזנת כתובת אימייל תקנית שתשמש לזיהוי ולשחזור סיסמה.
 - c. *Password & Confirm Password*: הזנת סיסמה מאובטחת ואימותה למניעת טעויות הקלדה.
2. **כפתור Sign up** - בעת לחיצה על הכפתור, המערכת בודקת את תקינות הנתונים ומוסיפה את הלקוח למערכת.
3. **קבלת משוב מהשרת** - אם קיים איזושהי בעיה באחת הנתוהים השרת מחזיר הודעת שגיאה מתאימה שמוצגת בתחתית העמוד

The image shows a registration form with a dark background. The form is a light blue rectangle with rounded corners. At the top, it says 'Please enter your information below' in bold. Below this are four input fields, each with a label: 'Username', 'Email', 'password', and 'Confirm password'. At the bottom of the form is a blue button with the text 'Sign Up' in white.

הרשמת בעל מקצוע

תפקיד המסך - המסך אוסף מידע מפורט על בעלי המקצוע המעוניינים להציע את שירותיהם במערכת. תפקידו להקים פרופיל עסקי מלא בעל נתונים חיונים לנהל את יומן במערכת.

תיאור המסך והפונקציונליות - המסך כולל טופס המחולק למספר קטגוריות:

1. **פרטי משתמש בסיסיים** - שם משתמש, אימייל וסיסמה (כולל אימות סיסמה).
2. **הגדרות מקצועיות**
 - a. *Profession*: בחירת תחום העיסוק.
 - b. *Select your areas*: בחירת אזורי השירות הגיאוגרפיים בהם יכול לעבוד.
 - c. *Years of expertise*: הגדרת הניסיון המקצועי.
3. **ניהול זמנים**
 - a. הגדרת שעות עבודה.
 - b. *Job duration time*: קביעת משך זמן ממוצע לטיפול/פגישה.
4. **פרסום ותמחור**
 - a. *Description*: שדה טקסט חופשי לתיאור העסק והשירותים.
 - b. *Price per hour*: קביעת עלות שעתית.
5. **כפתור Sign Up** - בעת לחיצה על הכפתור המערכת בודקת את תקינות הקלט ומוסיפה את הלקוח למערכת.
6. **קבלת משוב מהשרת** - במקרה של בעיה באחד השדות השרת יראה הודעת שגיאה מתאימה.

Please enter your information below

Username

Email

password

Confirm password

Profession | v

Select your areas | v

Select years of expertise | v

Start time | v : | v

End time | v : | v

Job duration time | v : | v

Description

Price per hour \$\$\$\$ | v

Sign Up

מסך אימות משתמש

תפקיד המסך - המסך משמש כשלב אימות בתהליך ההרשמה (וגם בתהליך שחזור הסיסמה). תפקידו להבטיח שכתובת האימייל שהוזנה אכן שייכת למשתמש.

תיאור המסך והפונקציונליות

1. **הזנת קוד אימות -** ממשק המכיל שדות ייעודיים להזנת קוד בן 6 ספרות שנשלח לתיבת האימייל של המשתמש.

2. **אימות מול השרת -** בעת לחיצה על כפתור השליחה, הקוד נשלח לשרת

3. **קבלת משוב -** במקרה של קוד שגוי, המערכת מחזירה הודעת שגיאה בתחתית העמוד.

Insert Verification Code

A verification code was sent to your email.

Submit

שכחתי סיסמה**הקלדת אימייל**

תפקיד המסך - המסך נותן למשתמשים אפשרות להתחיל את תהליך איפוס הסיסמה שלהם. תפקידו לאפשר הזדהות מחוץ למערכת כדי לאפשר הגדרת סיסמה חדשה בצורה מאובטחת.

תיאור המסך והפונקציונליות

1. **הזנת כתובת אימייל -** שדה טקסט שבו המשתמש מזין את הכתובת איתה נרשם למערכת.
2. **כפתור שליחת אימייל -** בלחיצה על הכפתור, נשלחת בקשה לאיפוס סיסמה לשרת.
3. **כפתור חזרה להתחברות -** כפתור ניווט המאפשר למשתמש לחזור למסך הכניסה הראשי.
4. **קבלת משוב -** אם קיימת בעיה עם האימייל שהוזן המערכת שולחת הודעת שגיאה מותאמת המוצגת בתחתית העמוד.

Forgot Password

Enter your email and we will send a verification code so you can reset your password.

Email

Send Reset Email

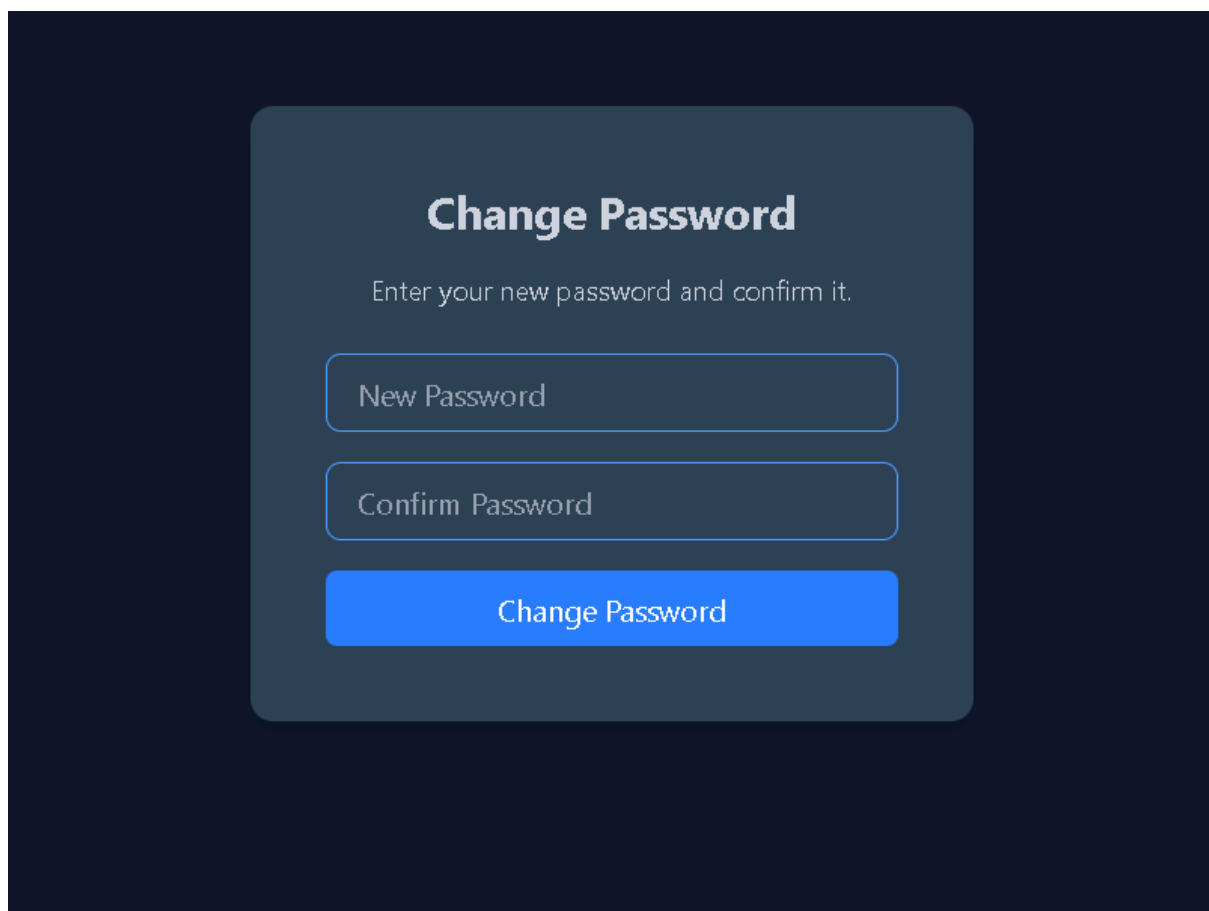
Back to login

שינוי סיסמה

- **תפקיד המסך - המסך מהווה את השלב האחרון בתהליך איפוס הסיסמה. תפקידו לשלוח לשרת את הסיסמה החדשה שבה המשתמש בחר**

תיאור המסך והפונקציונליות

1. **הזנת סיסמה חדשה ואימותה - שני שדות קלט של הסיסמה.**
2. **כפתור אישור - בלחיצה על הכפתור השרת מקבל את הסיסמה החדשה של הלקוח ומעדכן את זה במסד הנתונים.**
3. **קבלת משוב - המערכת מעדכנת את הסיסמה ואם הייתה בעיה כלשהי שולחת משוב שמוצג בתחתית העמוד**



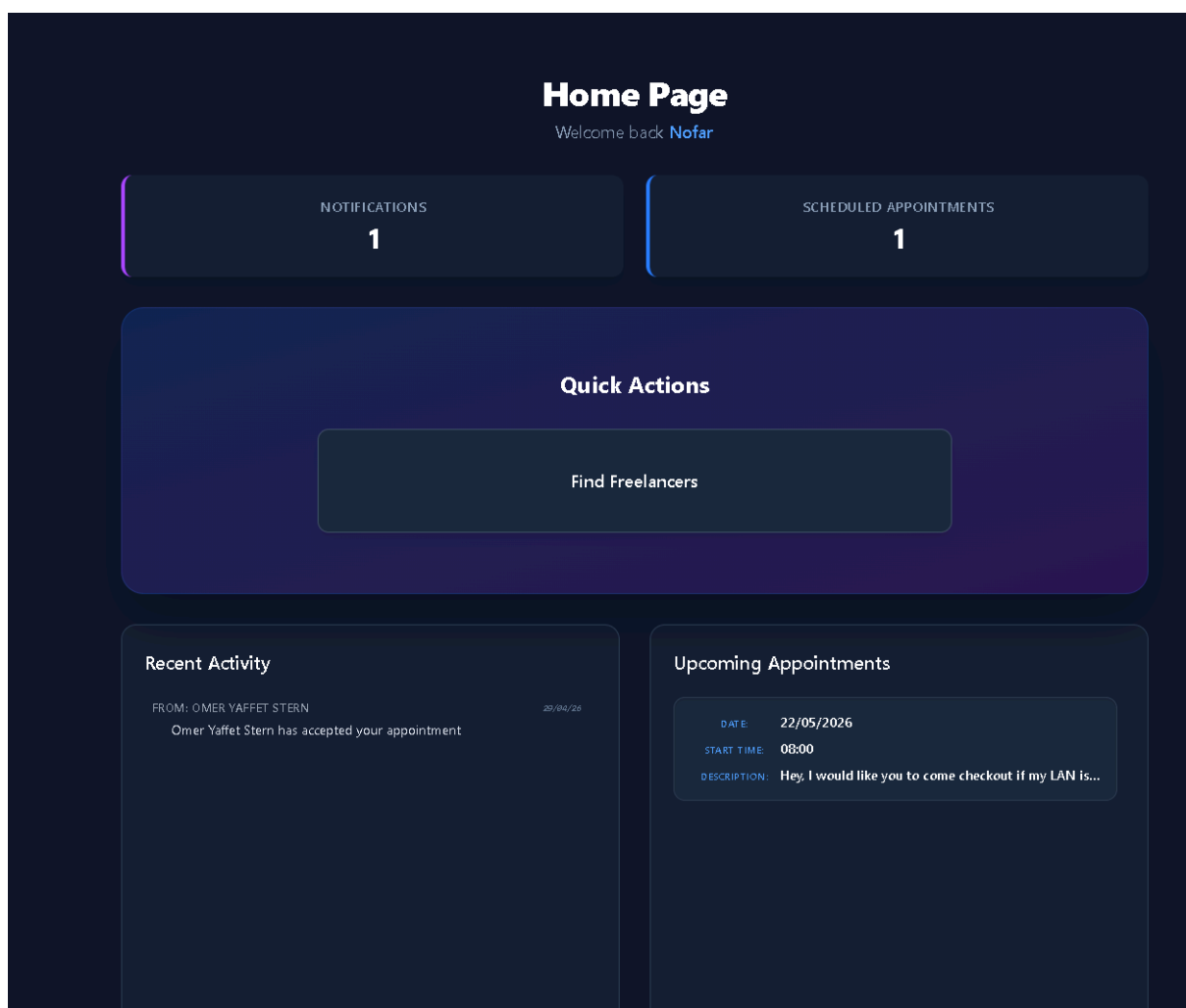
The image shows a 'Change Password' form on a dark blue background. The form is a lighter blue rounded rectangle containing the title 'Change Password', a subtitle 'Enter your new password and confirm it.', two input fields labeled 'New Password' and 'Confirm Password', and a blue 'Change Password' button.

מסך בית (משתמש)

תפקיד המסך - לרכז עבור המשתמש את הנתונים שלו במערכת. המסך משמש כמקום המרכזי לניווט מהיר ולצפייה בעדכונים חשובים בזמן אמת.

תיאור המסך והפונקציונליות

1. **סיכום נתונים מהיר** - הצגת מדדים מרכזיים בחלק העליון (כמות התראות חדשות והמות התורים שנקבעו).
2. **פעולות מהירות** - גישה לפעולות המרכזיות שהמשתמש יכול לעשות כמו חיפוש פריילנסרים, באמצעות כפתור בולט במרכז המסך (במידה ואוסיף עוד פיצ'רים גם יופיעו שם).
3. **התראות** - הצגה של כל ההתראות ששמורות למשתמש במערכת (חדשות מופיעות בכחול ישנות באפור)
4. **תורים קרבים** - רשימה של כל התורים שנקבעו עם תאריך, שעת התחלה, ופירוט שהשאר לבעל המקצוע.



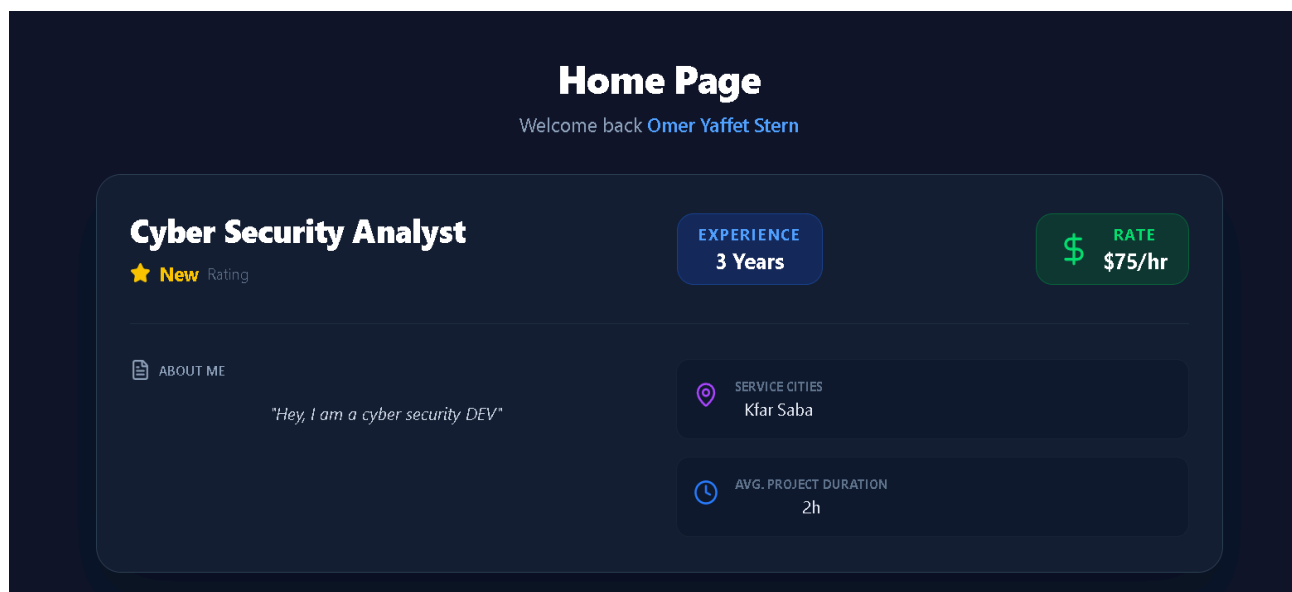
מסך בית (בעל מקצוע)

כרטיס ביקור

תפקיד המסך - להציג לבעל המקצוע איך לקוחות רואים את הכרטיס ביקור שלו

תיאור המסך והפונקציונליות

1. פרטי מומחיות ודירוג - הצגת התפקיד המקצועי, שנות הניסיון והדירוג הממוצע באתר.
2. מידע אישי - תיאור קצר ואישי של הפרילנסר על עצמו.
3. תנאי שירות ולוגיסטיקה - הצגת פרטים טכניים חשובים כמו תעריף שעותי, ערים בהן ניתן השירות וזמן ממוצע לעבודה.

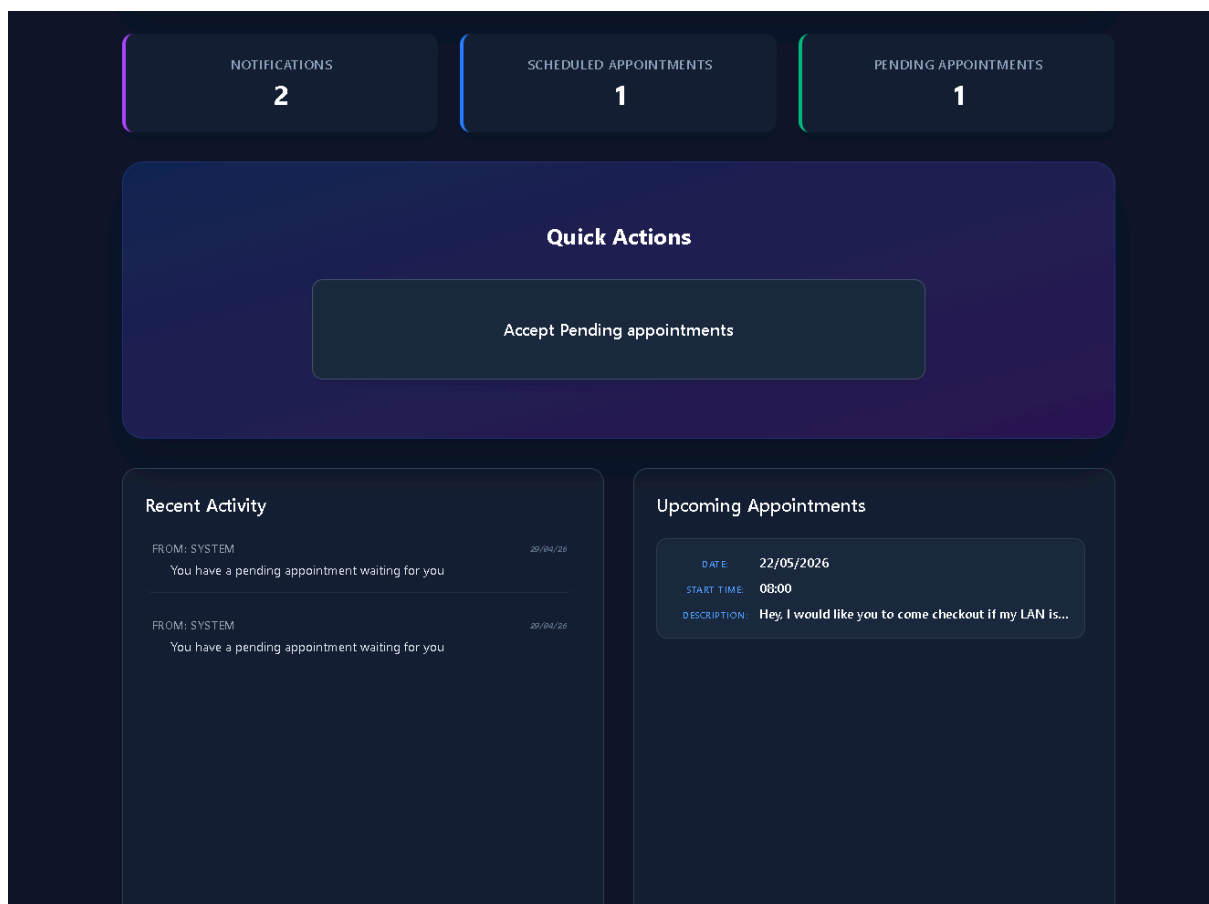


מסך הבית

תפקיד המסך - ריכוז של כל הפעילות המקצועית של בעל המקצוע במערכת.

תיאור המסך והפונקציונליות

1. **לוח מדדי פעילות** - תצוגה עליונה המסכמת את כמות ההתראות החדשות, התורים המתוכננים ובקשות לתורים הממתינות לאישור.
2. **פעולות ניהול** - מאפשר גישה מהירה לאישור בקשות תורים חדשות שהתקבלו מלקוחות.
3. **מרכז עדכונים** - פירוט של פעולות אחרונות שבוצעו במערכת, כגון קבלת בקשות תור חדשות מהמערכת (הודעה חדשה מסומנת בכחול הודעה ישנה באפור)
4. **לוח תורים קרובים** - תצוגה מפורטת של התורים שאושרו. התצוגה כוללת תאריך, שעה ותיאור קצר של העבודה מהלקוח.



אישור דחיית תורים

תפקיד המסך - לאפשר לבעל המקצוע לצפות בבקשות תור חדשות שהתקבלו מלקוחות ולקבל החלטה האם לאשר או לדחות אותן.

תיאור המסך והפונקציונליות

1. **תצוגת ריכוז בקשות -** הצגת כל התורים ואת פרטי התורים: תאריך, שם הלקוח, שעות התחלה וסיום, כתובת ופרטי הבקשה.
2. **אפשרויות בחירה -** לכל בקשה קיימת אפשרות לסמן אישור או דחייה של התור המוצע.
3. **מידע נוסף -** כפתור המאפשר צפייה בפרטים מורחבים בצורה מסודרת
4. **שמירת שינויים -** כפתור מרכזי המאפשר לעדכן את כל הבחירות שבוצעו ברשימה מול מסד הנתונים בפעולה אחת.
5. **קבלת משוב -**

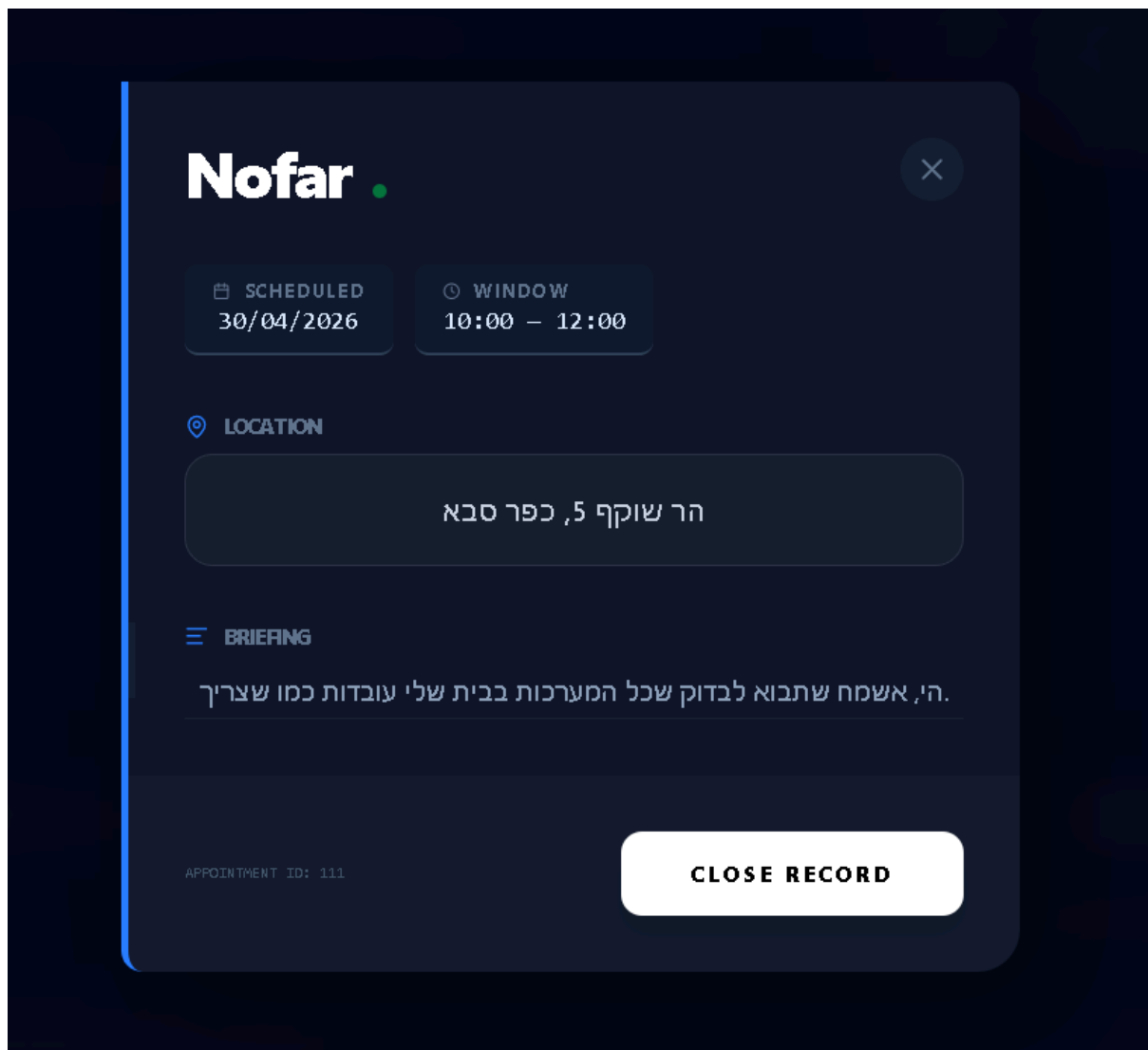
DATE	CLIENT	START	END	ADDRESS	DETAILS	INFO	ACTION
30/04/2026	Nofar	10:00	12:00	הר שוקף 5, כפר סבא	המערכת בבית שלי עובדת כמו שצריך		
Select appointments to accept or decline.							Confirm & Save Changes

פירוט תור

תפקיד המסך - להציג מידע על תור ספציפי המסך

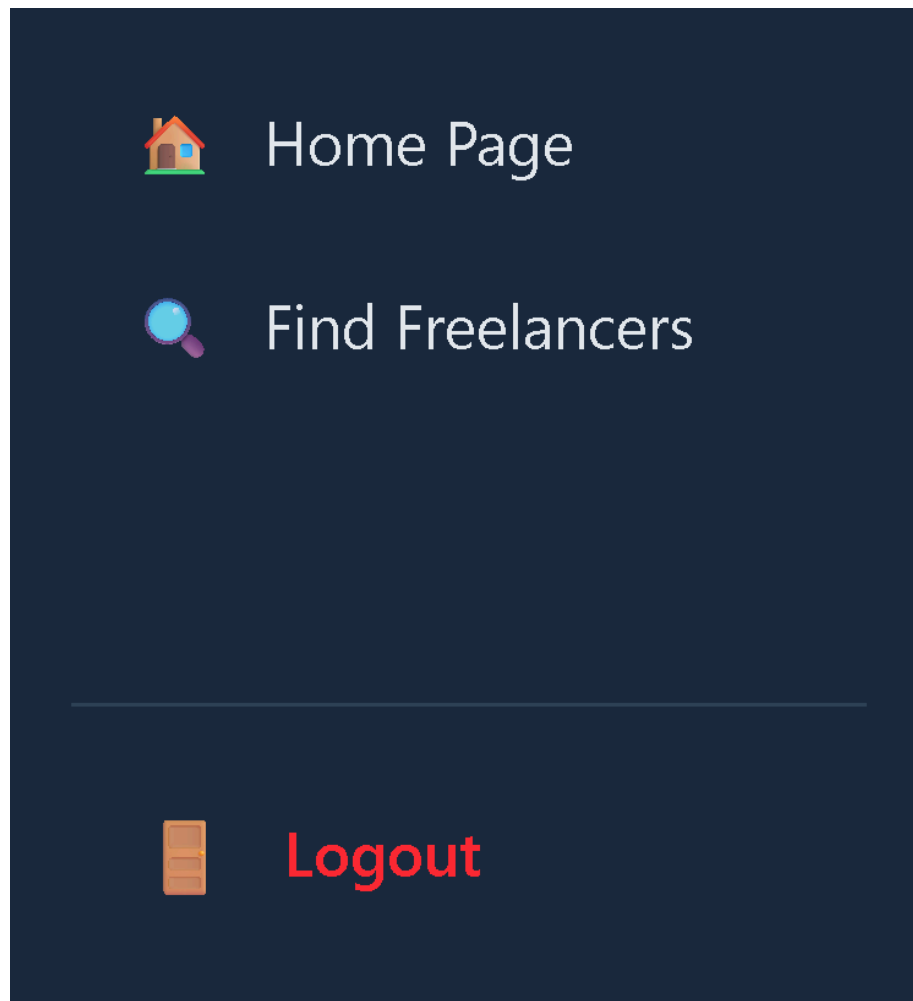
תיאור המסך והפונקציונליות

1. **פרטי לוגיסטיקה** - שם הלקוח, תאריך וחלון הזמן המדויק שנקבע.
2. **מיקום** - תצוגה ברורה של הכתובת בה יתבצע השירות.
3. **תיאור הבקשה** - מציג את ההודעה המלאה שכתב הלקוח בעת הזמנת התור
4. **זיהוי פנימי** - הצגת מספר מזהה ייחודי של התור לצורך מעקב במערכת ואסטיקה.
5. **יציאה** - אפשרות לסגירת חלונית המידע.



סרגל ניווט

1. תפקיד הרכיב - כלי הניווט המרכזי של המשתמש באפליקציה.
2. תיאור הרכיב והפונקציונליות
3. קישורי ניווט מהירים
- a. *Home Page*: חזרה ללוח הבקרה האישי.
- b. *Find Freelancers*: מעבר למסך החיפוש לאיתור בעלי מקצוע.
4. כפתור התנתקות - כפתור המאפשר סיום מאובטח ויציאה מהחשבון חזרה למסך הכניסה.
5. עיצוב עקבי - הסרגל מקובע לצד המסך ברגע ההתחברות בכל החלונות



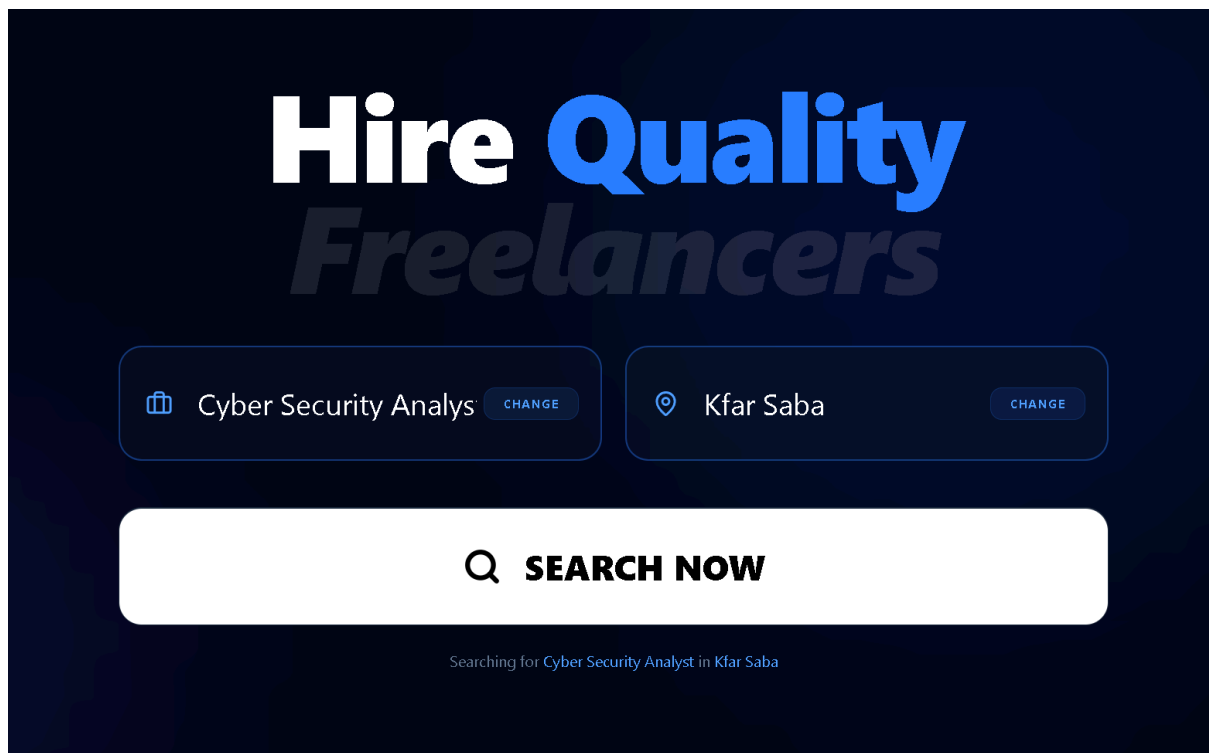
חיפוש בעלי מקצוע

חיפוש

- **תפקיד המסך** - לאפשר למשתמש לאתר בעלי מקצוע עבורו על ידי סינון.

תיאור המסך והפונקציונליות

1. **בחירת תחום עיסוק** - המשתמש יכול לבחור או לשנות את סוג בעל המקצוע שהוא מחפש.
2. **בחירת מיקום** - המשתמש יכול להגדיר את העיר או האזור בו הוא זקוק לשירות לפי מיקום העבודה של בעלי המקצוע מהתחום שנבחר.
3. **ביצוע חיפוש** - לחיצה על כפתור החיפוש המרכזי מפעיל את הסינון.

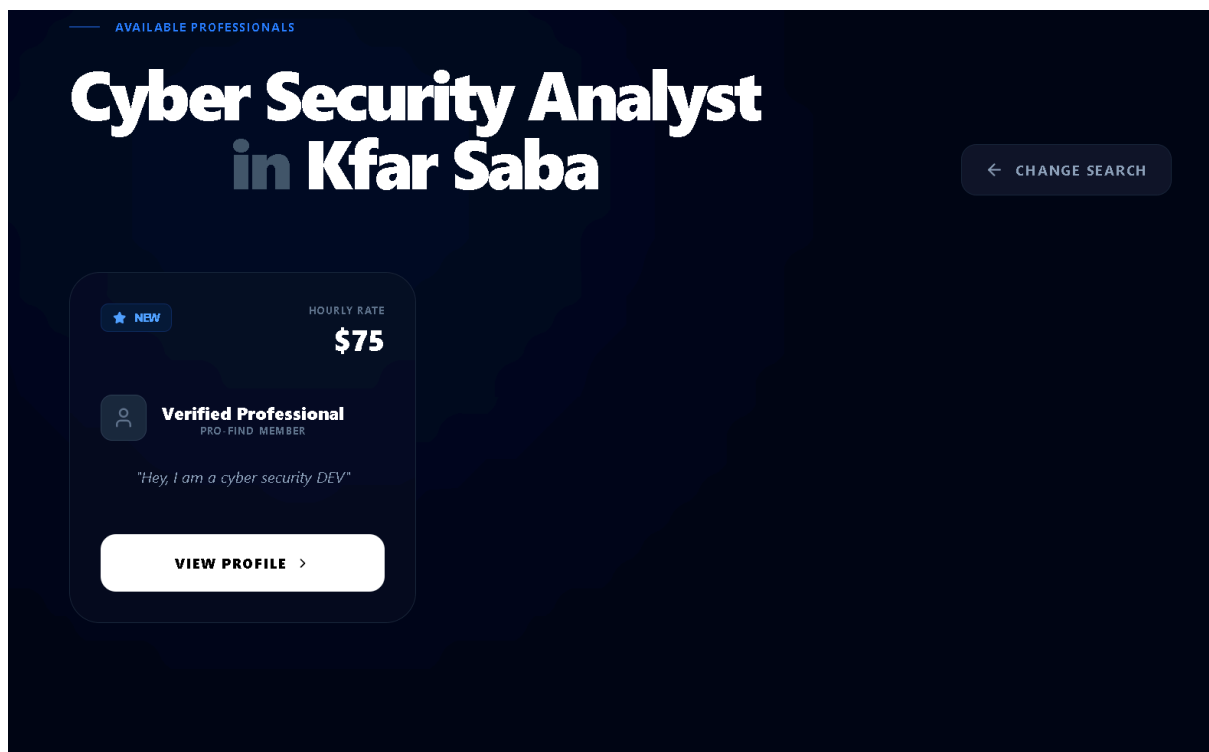


תוצאות חיפוש בעלי מקצוע

תפקיד המסך - מציג למשתמש רשימה של בעלי המקצוע הזמינים שעברו את הסינון.

תיאור המסך והפונקציונליות

1. **כרטיסי פרילנסרים** - הצגת כרטיס מידע תמציתי עבור כל בעל מקצוע, הכולל דירוג, תעריף שעות, ותיאור קצר.
2. **שינוי חיפוש** - מאפשר למשתמש לחזור אחורה כדי לעדכן את פרטי החיפוש.
3. **צפייה בפרופיל** - מאפשר מעבר למסך הפרופיל המלא של בעל המקצוע.
4. **כותרת דינמית** - הכותרת הראשית אשר שתנה בהתאם לחיפוש שבוצע.

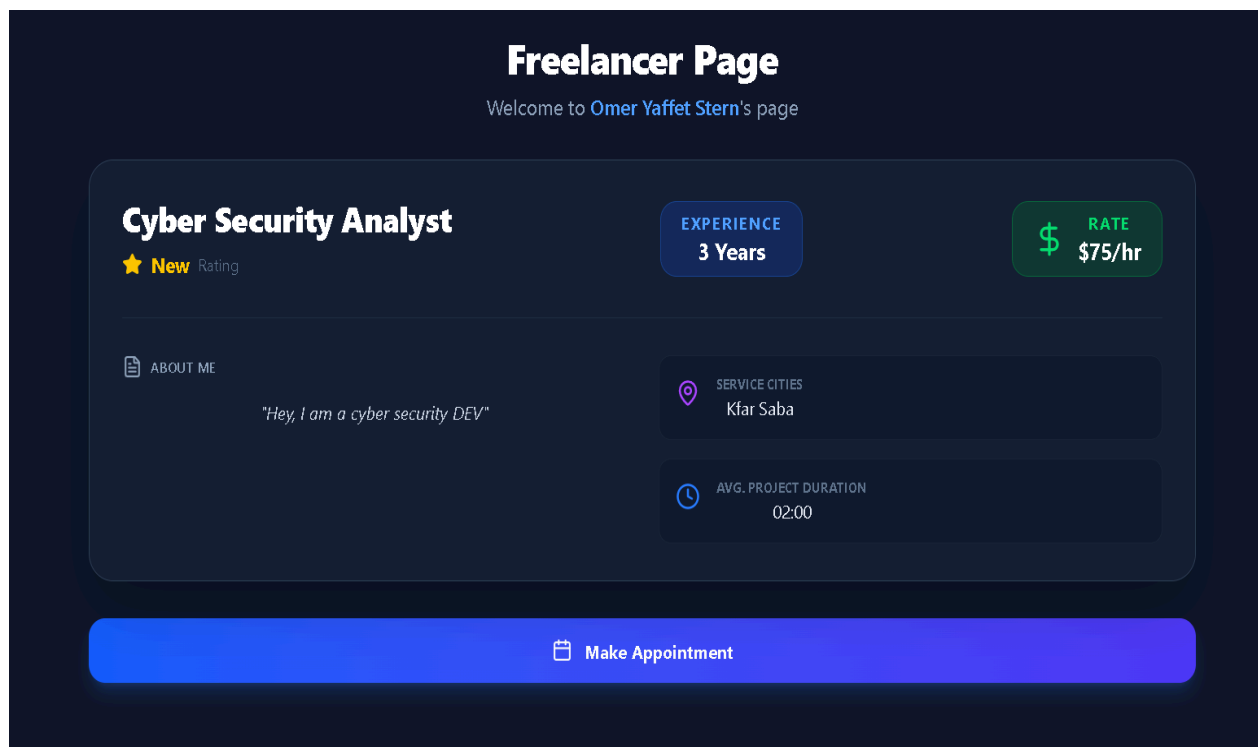


עמוד הבית של בעל מקצוע - תצוגת לקוח

תפקיד המסך - להציג למשתמש את המידע המלא והמפורט על בעל המקצוע הנבחר.

תיאור המסך והפונקציונליות

1. פרטי מומחיות ודירוג - הצגת התפקיד המקצועי, שנות הניסיון ודירוג של בעל המקצוע.
2. תמחור - הצגת התעריף השעתי של בעל המקצוע בצורה בולטת.
3. תיאור אישי - תיאור של בעל המקצוע בו הוא מציג את עצמו ואת שירותיו.
4. מידע לוגיסטי - פירוט הערים בהן ניתן השירות וזמן ממוצע לביצוע עבודה.
5. תיאום תור - כפתור שמאפשר למשתמש לעבור לשלב קביעת התור מול בעל המקצוע.



קביעת תור

תפקיד המסך - לאפשר למשתמש לקבוע תור מול בעל המקצוע עם הפרטים ההכרחיים.

תיאור המסך והפונקציונליות

1. **בחירת זמן** - לוח שנה אינטראקטיבי המציג ימים ושעות זמינות לבחירה.
2. **פרטי מיקום ושירות** - שדות להזנת כתובת השירות ופרטים נוספים עבור בעל המקצוע.
3. **סיכום הזמנה** - הצגת פרטי התור, כולל חישוב אוטומטי של העלות הכוללת.
4. **אישור והסכמה** - הצגת תנאי השירות ואישור העסקה באמצעות כפתור "Confirm Booking".
5. **קבלת משוב** - מקום בתחתית העמוד לקבל הודעות שגיאה מהשרת.

Book Appointment

SELECT DATE

APR 30	MAY 1	MAY 2	MAY 3	MAY 4	MAY 5	MAY 6
MAY 7	MAY 8	MAY 9	MAY 10	MAY 11	MAY 12	MAY 13
MAY 14	MAY 15	MAY 16	MAY 17	MAY 18	MAY 19	MAY 20
MAY 21	MAY 22	MAY 23	MAY 24	MAY 25	MAY 26	MAY 27
MAY 28	MAY 29	MAY 30	MAY 31	JUN 1	JUN 2	JUN 3

AVAILABLE TIMES duration: 02:00

08:00 **10:00** 12:00 14:00

SERVICE ADDRESS

דוכיפת 5

ADDITIONAL DETAILS

Gate codes, specific requirements...

APPOINTMENT SUMMARY 2 hour Service

10:00 — 12:00
Tuesday, May 5

דוכיפת 5
SERVICE LOCATION

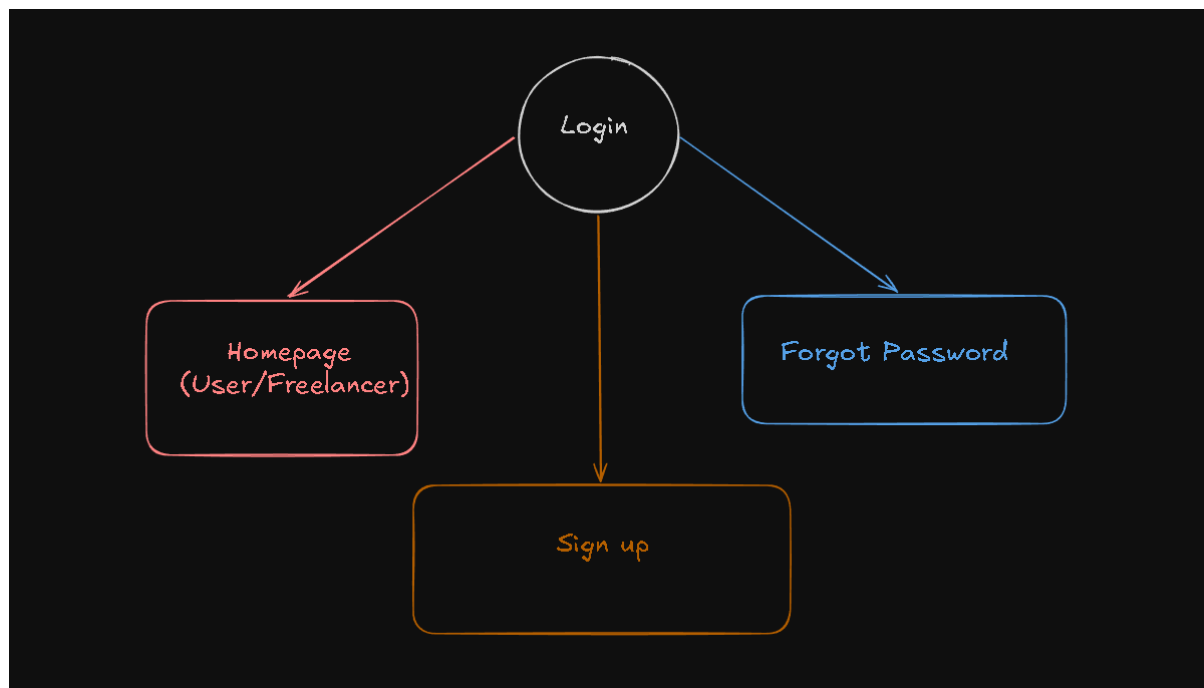
\$150.00
השכר שירותי מקצוע 2 שעות

By confirming, you agree to the hourly fee of **\$75**.
By confirming you also agree to knowledge that
Pro-Find is not accountable to any price changes the freelancer might impose

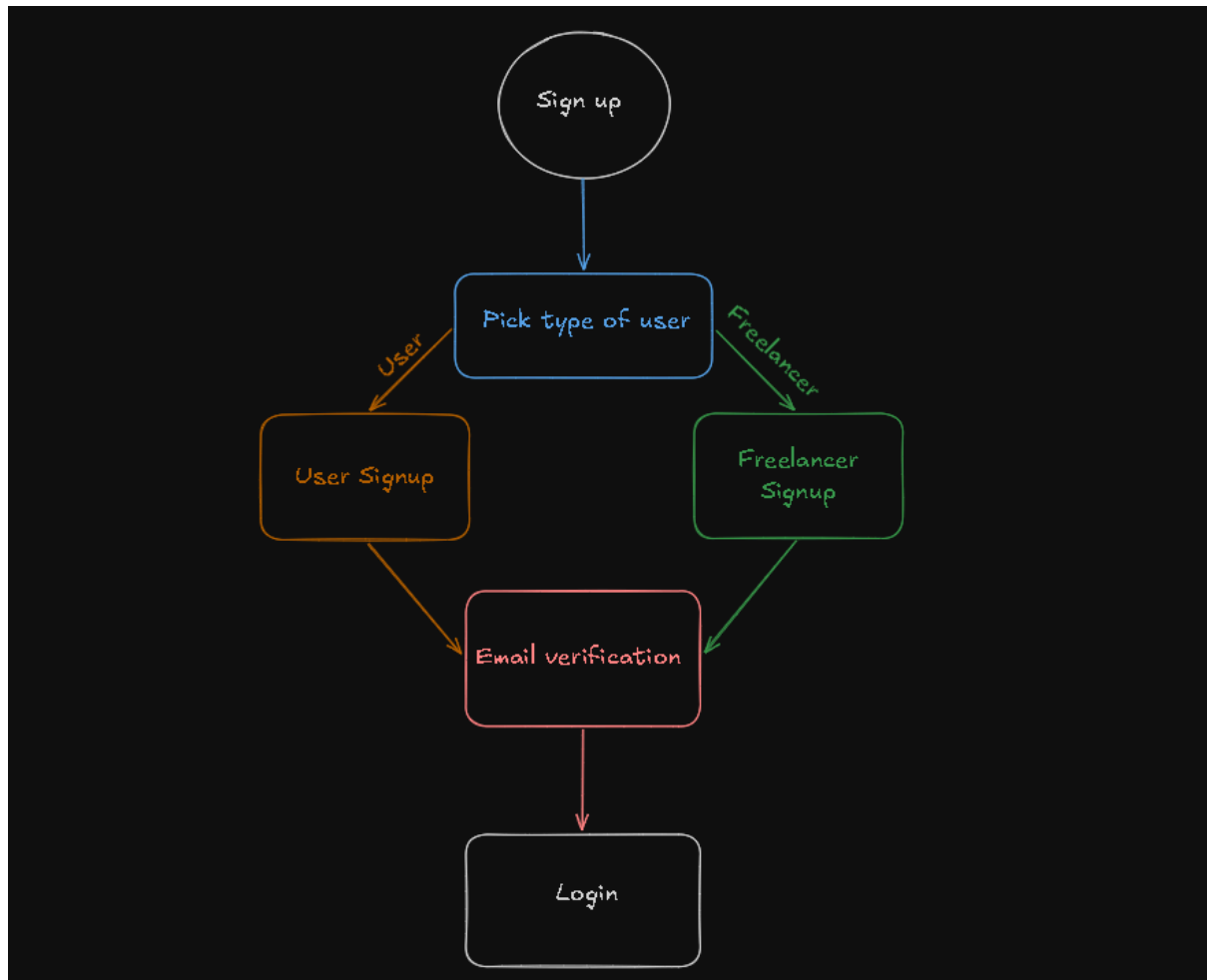
Cancel **Confirm Booking**

היררכיית המסכים

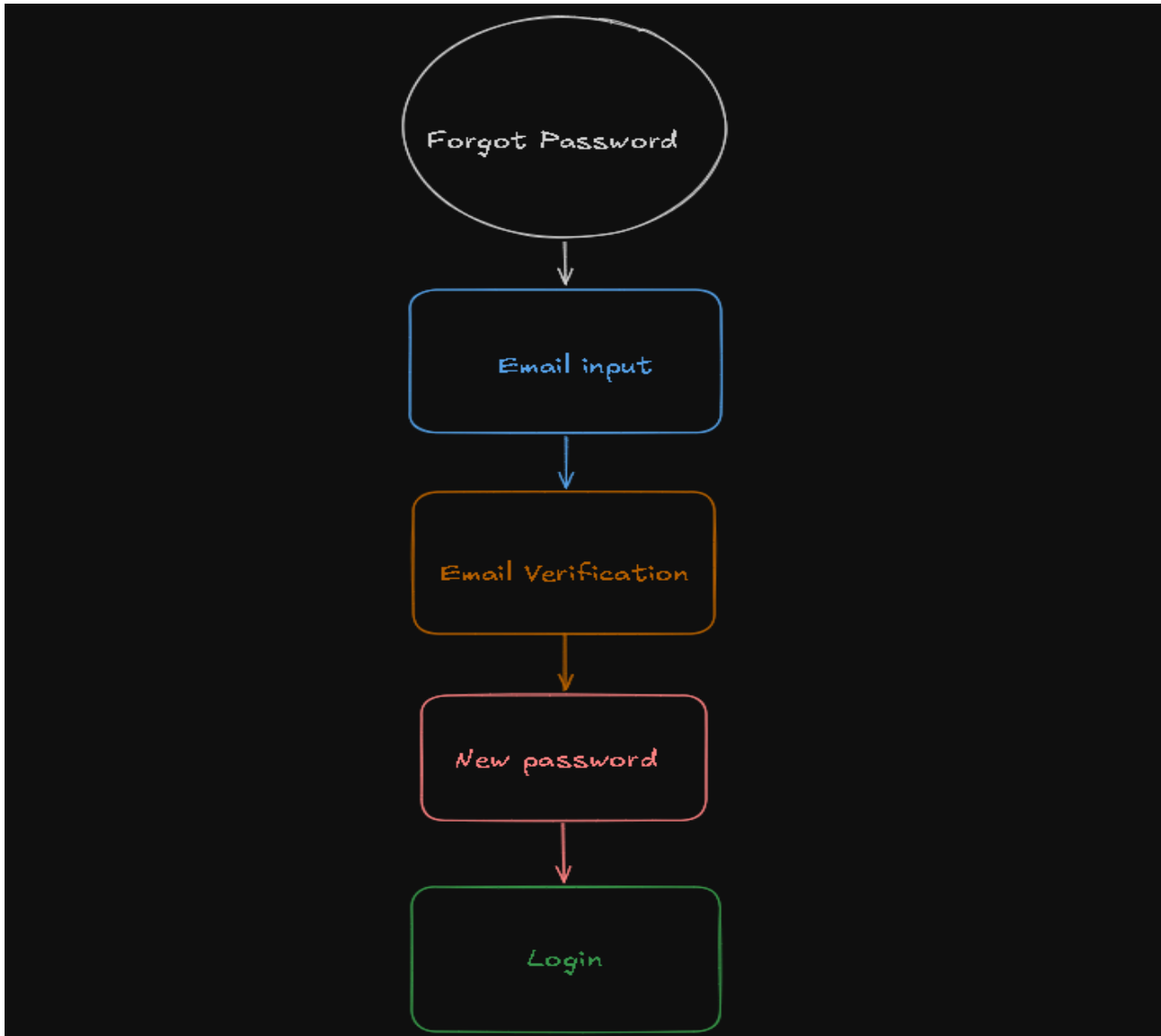
התחברות



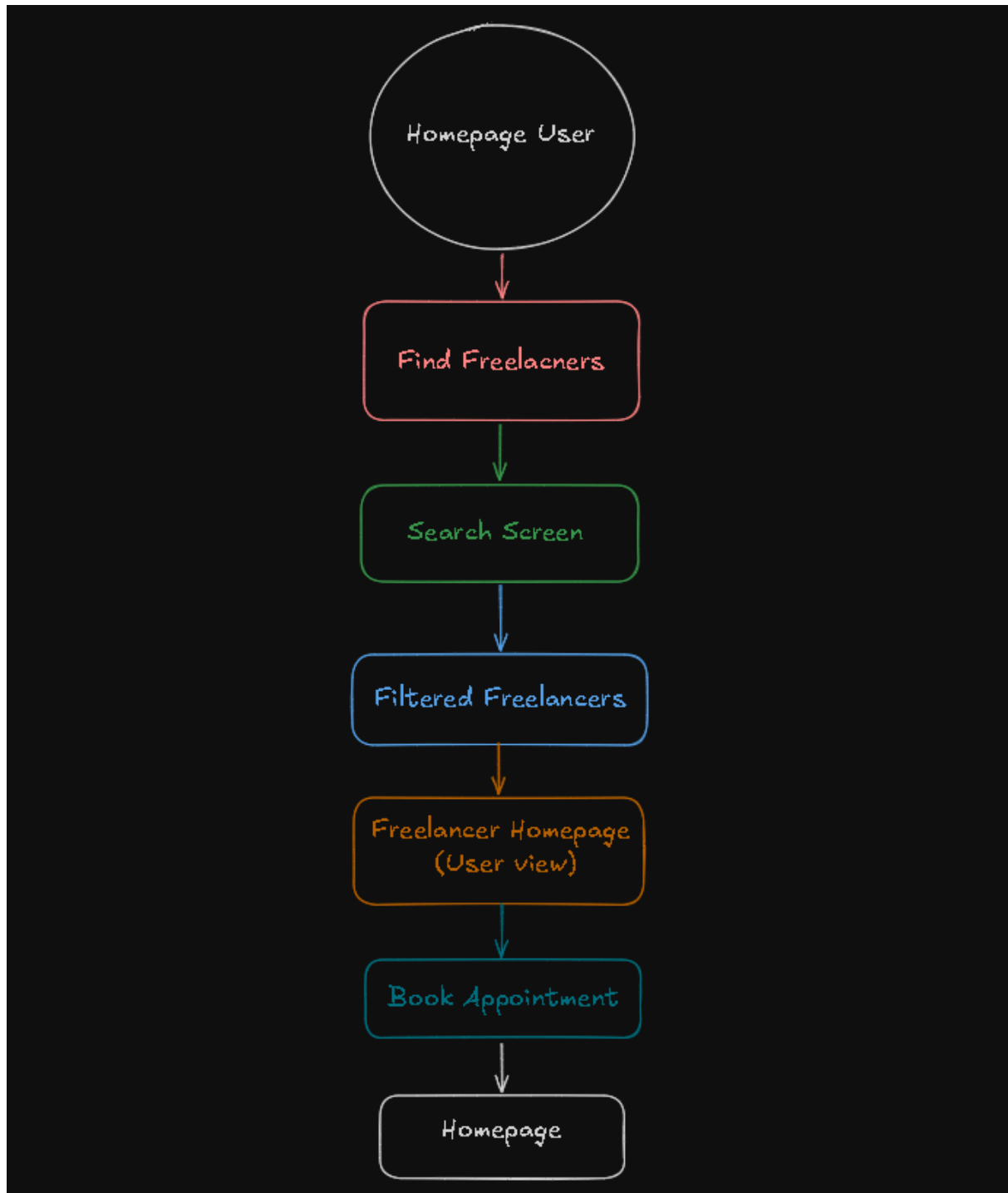
הרשמה



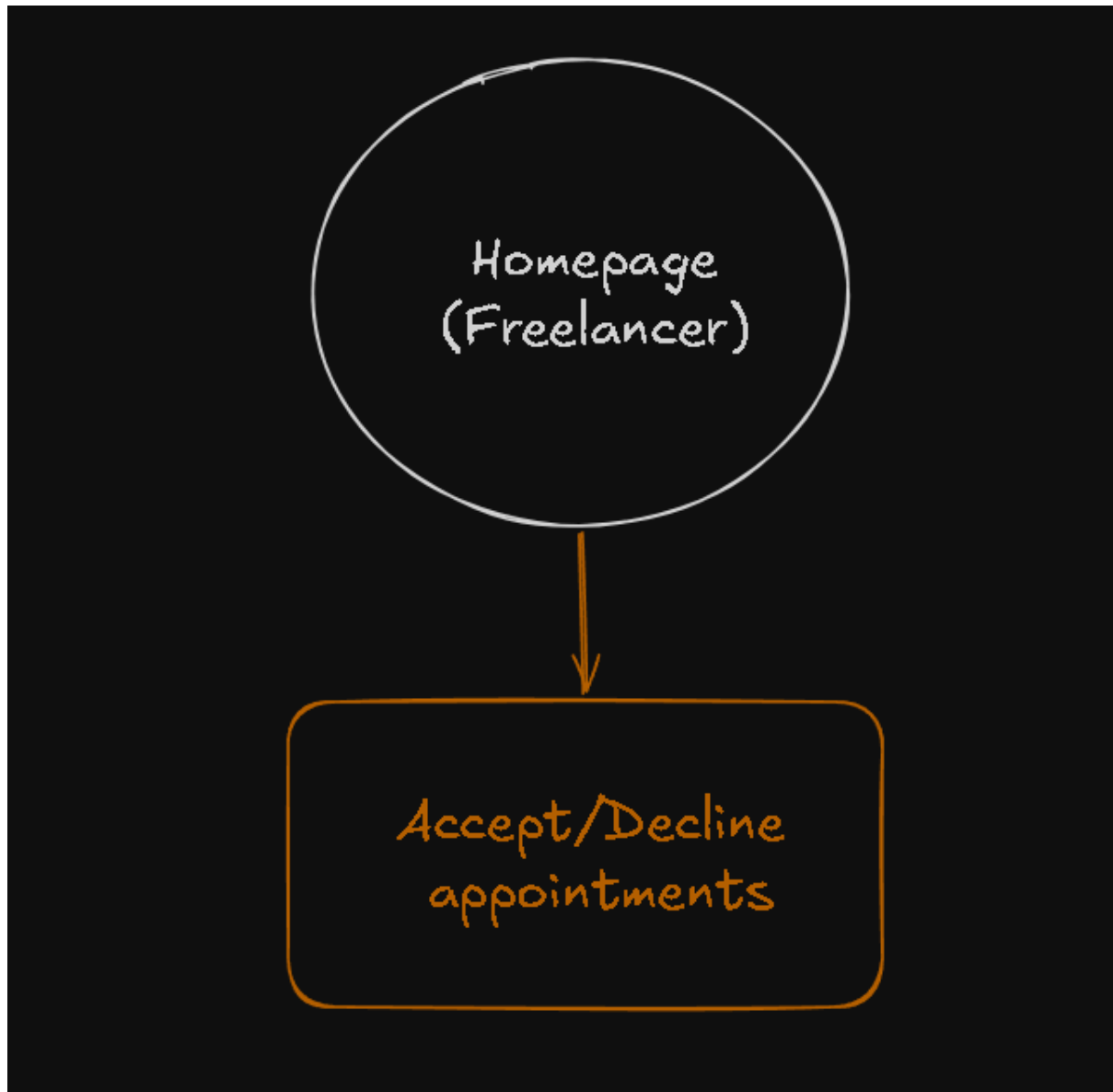
שכחתי סיסמה



עמוד הבית (משתמש)



עמוד הבית (בעל מקצוע)



תיאור מבני נתונים

מבני הנתונים

במערכת קיימים שלושה סוגים של מבני נתונים המשמשים לאחסון מידע

מסד נתונים - המערכת משתמשת בקובץ בשם my_app.db כמסד הנתונים. זה המקום שבו נשמר המידע של האפליקציה. המידע מאורגן בטבלאות מקושרות ביניהן ומאפשר ניהול מסודר של הפעולות באתר.

קבצי מידע סטטיים - המערכת נעזרת בקבצי טקסט חיצוניים לצורך הצגת אפשרויות בחירה למשתמש וללקוח

cities.txt - מכיל רשימה של ערים בישראל

professional.txt - מכיל רשימה של תחומי עיסוק

שני הקבצים משומשים בהרשמה של בעל המקצוע ובבדיקות שהשרת מבצע.

קובץ הגדרות מערכת - קובץ זה משמש לשמירת משתני סביבה ונתונים רגישים של המערכת. הוא מאפשר להפריד בין לוגיקת התוכנית לבין פרטים שצריכים להישאר מאובטחים.

מאגרי המידע של המערכת

שם המאגר	תיאור המידע הלוגי
מאגר משתמשים והרשאות	ריכוז כל נתוני המשתמשים במערכת כולל פרטי התחברות מאובטחים וסיווג סוג החשבון
מאגר מידע מקצועי וזמינות	שמירת כל המידע העסקי של בעלי המקצוע: תחומי עיסוק, עלות שעתית, אזורי שירות, זמינות וזמני עבודה.
מאגר תיאום פגישות ותקשורת	ניהול לוח הזמנים של המערכת. לקשר בין לקוח לנותן שירות ותיעוד תורים והסטטוס שלהם.
מאגרי קבצים	רשימות קבועות של ערים ותחומי עיסוק המשמשות כבסיס לסינון ובחירה במערכת
מאגר הגדרות מערכת	משתני סביבה רגישים הנדרשות להפעלת השרת והבטחת האבטחה של השרת

מסד הנתונים

טבלת המשתמשים

זוהי הטבלה המרכזית במערכת. היא שומרת את כל המשתמשים שנרשמו (בין אם הם לקוחות ובין אם הם בעלי מקצוע), שומרת את פרטי ההזדהות שלהם קובעת את סוג ההרשאה שלהם ושומרת בה את כל הפרטים הנחוצים.

שם השדה	טיפוס נתונים	מפתח ראשי (Primary Key)	דוגמה לערך
id	Integer	✓	1
full_name	Text		John Doe
email	Text		example.email@gmail.com
password_hash	Text		a665a45920422f9d417e48 67efdc4fb8a04a1f3fff1fa07e 998e86f7f7a27ae3
user_type	Text		Freelancer/Client
salt	Text		random_salt_xyz
created_at	DateTime		10:00:00 2026-05-01

טבלת בעלי מקצוע

טבלה זו נועדה להרחיב את המידע על משתמשים שהוגדרו כבעלי מקצוע. היא שומרת את כל הפרטים העסקיים הדרושים כדי להציג אותם בחיפוש.

שם השדה	טיפוס נתונים	מפתח ראשי (Primary Key)	דוגמה לערך
id	Integer	✓	10
user_id	Integer		1 (מקשר לטבלת המשתמשים)
profession	Text		Cyber Security Analyst
service_cities	Text		Kfar Saba, Ra'anana
description	Text		Expert PC fixer
years_experience	Integer		5
avg_job_duration	Text		01:00
start_time	Text		08:00
end_time	Text		17:00

250		Integer	hour_price
-----	--	---------	------------

טבלת תורים

טבל זו מנהלת את הקישור בין הלקוח ובעל המקצוע. היא מתעדת את פרטי התורים כדי להציג ללקוח ולבעל המקצוע בצורה מסודרת

שם השדה	טיפוס נתונים	מפתח ראשי (Primary Key)	דוגמה לערך
id	Integer	✓	501
professional_id (קישור לטבלת המשתמשים)	Integer		1
customer_id (קישור לטבלת המשתמשים)	Integer		2
date	Date		2026-05-22
start_time	Text		08:00
end_time	Text		10:00
address	Text		הדחליל 5
status	Text		Pending/Accepted/Declined
details	Text		Hey, need help urgently

טבלת המשתמשים בהמתנה

טבלה שמשמשת את המערכת בזמן תהליך הרישום. המידע על המשתמש ועל תהליך האימות נשמר כאן עד שהמשתמש מזין את קוד האימות שנשלח אליו. לאחר האימות, המידע עובר לטבלאות הקבועות (טבלת המשתמשים ובעלי המקצוע) והשורה בטבלה זו נמחקת.

שם השדה	טיפוס נתונים	מפתח ראשי (Primary Key)	דוגמה לערך
id	Integer	✓	1
full_name	Text		John Doe
email	Text		example.email@gmail.com
password_hash	Text		a665a45920422f9d417e48 67efdc4fb8a04a1f3fff1fa07 e998e86f7f7a27ae3
salt	Text		random_salt_xyz
user_type	Text		Freelancer/Client
profession	Text		Cyber Security Analyst
cities	Text		Kfar Saba, Ra'anana
description	Text		Expert PC fixer
years	Integer		5
avg_job_duration	Text		01:00
start_working	Text		08:00
finish_working	Text		17:00
hour_prices	Integer		250
verification_code	Integer		123456
expires_at	Text		2026-05-01 12:25:03.1111 (ISO 8601)

טבלת התראות

טבלה זו משמשת לשמירת ההודעות שהמערכת מעבירה למשתמשים. היא מאפשרת עדכון של המשתמשים בצורה נוחה.

שם השדה	טיפוס נתונים	מפתח ראשי (Primary Key)	דוגמה לערך
id	Integer	✓	1031
user_id	Integer		1
from_id	Integer		2
message	Text		You have a new pending appointment
is_read	Boolean		1/0
created_at	DateTime		12:25:00 2026-05-01

טבלת שינוי סיסמה

טבלה זו משמשת לתהליך שחזור הסיסמה. היא שומרת את כתובת האימייל של משתמשים שהתחילו את תהליך איפוס הסיסמה, ומאפשרת למערכת לעקוב אחרי הבקשה ולוודא שהיא מתבצעת מול המשתמש הנכון בצורה מאובטחת.

שם השדה	טיפוס נתונים	מפתח ראשי (Primary Key)	דוגמה לערך
Email	Text	✓	example.email@gmail.com

חולשות ואיומים

שכבת האפליקציה

שכבת האפליקציה זו השכבה שפוגשת את המשתמש, כאן רץ הקוד של האתר, כאן נמצא הטפסים שהמשתמש ממלא, וכאן מתבצעת הלוגיקה של האתר.

מבחינת אבטחת מידע, זו השכבה הכי חשופה, מכיוון וכל אחד יכול לגשת לאתר ולהתחיל להזין נתונים. לכן, רוב ההתקפות מכוונות לניצול חולשות בקוד או בדרך שבה האפליקציה ניגשת למסד הנתונים.

SQL Injection

האיום - ניסיון של תוקף להחדיר פקודות SQL זדוניות דרך שדות הקלט באתר. אם הקוד לא קורא את הקלט בצורה חכמה, התוקף יכול לעקוף את מנגנון ההתחברות, להשיג גישה למסד הנתונים ולעשות כרצונו.

הפתרון במערכת - שימוש ב-**Parameterized Queries**. במקום להכניס למסד הנתונים מחרוזות טקסט ישירות, המערכת משתמשת בספריות שמבצעות סינון של פקודות ה-SQL וקוראות את כל הקלט ומתייחסות אליו כמחרוזת בלבד ולא כקוד בעת ההכנסה למסד הנתונים.

תהליך ה-Login

האיום - גניבת זהות של משתמשים על ידי פריצה למסד הנתונים או על ידי ניחוש סיסמא באמצעות Brute Force.

הפתרון במערכת

Hashing & Salt - כפי שמופיע במבנה הטבלאות (password_hash ו-salt), המערכת לעולם לא שומרת סיסמאות בטקסט גלוי. כל סיסמה עוברת תהליך ערבול חד-כיווני עם Salt ייחודי לכל משתמש ו-Pepper סודי שרק השרת מודע אליו, מה שהופך פריצה למאגר הנתונים לבלתי יעילה עבור התוקף.

קוד אימות - שימוש בטבלת pending_users וקוד אימות במייל מבטיח שרק בעלי גישה לתיבת המייל יכולים להשלים את הרישום, מה שמונע פתיחת חשבונות פיקטיביים על ידי בוטים.

DoS/DDoS

האיום - הצפת האתר בכמויות אדירות של בקשות HTTP מזויפות כדי לגרום לקריסת השרת ולמנוע מלקוחות אמיתיים לקבוע תורים.

הפתרון במערכת - הגדרת **Rate Limiting** ברמת השרת. המערכת מזהה אם כתובת IP מסוימת מבצעת מספר חריג של בקשות בזמן קצר וחוסמת אותה זמנית, ובכך שומרת על זמינות השירות.

מניעת הזרקת קוד (XSS - Cross-Site Scripting)

האיום - תוקף מנסה להזין קוד JavaScript זדוני בתוך שדות טקסט שמוצגים למשתמשים אחרים. אם הקלט לא ינוקה, הדפדפן של המשתמש שיצפה בפרופיל יריץ את הסקריפט, מה שעלול להוביל לפרצות אבטחה.

הפתרון במערכת - יישום מנגנון **Sanitization**. המערכת סורקת כל מחרוזת טקסט שנשלחת מהמשתמש וצפויה להיות מוצגת לאחרים. המנגנון מסנן תגיות HTML וממיר תווים מיוחדים לייצוג טקסטואלי בלבד. בדרך זו, המערכת מבטיחה שגם אם הוכנס קוד זדוני, הוא יוצג כטקסט ולא יורץ.

שכבת התעבורה

שכבת התעבורה אחראית על העברת הנתונים בצורה אמינה ומאובטחת. שכבת התעבורה דואגת שהמידע יגיע ליעדה בשלמותה ובהצפנה מלאה.

MITM

האיום: תוקף שמצטט ונמצא על החיבור של משתמש עם השרת עלול לנסות להפיל חבילות המידע. ללא הגנה, מידע רגיש כמו סיסמאות או פרטי לקוחות עובר כטקסט גלוי וניתן לקריאה.

הפתרון במערכת - TLS. המערכת עושה שימוש בפרוטוקול HTTPS. בשכבת התעבורה, פרוטוקול זה מפעיל הצפנת TLS המבצע אימות של זהות השרת בעזרת Certificate, ודואגת להחלפת מפתחות בטוחה והצפנת נתונים רציפה בכל זמן שהלקוח והשרת מחוברים (לכן גם מגן מפני מתקפת **Session Hijacking** מכיוון וה-Token לא נשלח כטקסט גלוי).

הצפנה אסימטרית - להחלפת מפתחות מאובטחת בתחילת הקשר.

הצפנה סימטרית - להצפנת גוף הנתונים במהירות גבוהה במהלך התקשורת.

TCP Handshake

האיום - אובדן חבילות מידע/שינוי סדר ההגעה שלהן עלול לגרום לשגיאות לוגיות.

הפתרון במערכת - המערכת משתמשת בפרוטוקול TCP המבטיח העברה אמינה דרך מנגנון לחיצת יד משולשת. תהליך ה-SYN, SYN-ACK, ACK מבטיח שהקשר הוקם בהצלחה לפני שליחת המידע, וכל חבילה שלא תגיע ליעדה תישלח מחדש באופן אוטומטי.

DoS

האיום - ניצול משאבי השרת על ידי פתיחת כמות אדירה של חיבורי TCP ריקים מכתובת אחת, מה שמונע ממשתמשים ליצור קשר עם השרת.

הפתרון במערכת - כפי שיושם במערכת, הוגדרה מגבלה של עד 5 חיבורים מאותה הכתובת IP. מנגנון זה חוסם את היכולת של תוקף בודד להעמיס על משאבי התקשורת של השרת.

פגיעה בשלמות המידע

האיום - ניסיון של תוקף לשנות את תוכן חבילות המידע בזמן שהן עוברות ברשת

הפתרון במערכת - שימוש ב-Message Authentication Code כחלק מפרוטוקול ה-TLS. המערכת מוודאת שכל חבילה שהגיעה אכן זהה לחבילה שנשלחה, ודוחה חבילות שעברו שינוי כלשהו בדרך.

מימוש הפרויקט

סקירת מודולים מיובאים בשרת

הסקריפט המרכזי
Pro_Find.py

שם המודול	למה הוא משמש
socket	יצירת אובייקט התקשורת של השרת שיכול לשלוח/לקבל הודעות.
threading	ניהול מספר חיבורים במקביל, מאפשר לשרת לטפל בלקוחות בו זמנית
datetime	משמש ליצירת אובייקט של הזמן הנוכחי בצורת טקסט לצורך השוואה ומחיקה של הודעות ותורים ישנים
timedelta	משמש לחישוב טווחי זמן כדי לבדוק מה אפשר למחוק ממסד הנתונים
collections (deque)	משמש כשומר זמנים של הודעות שנשלחו לצורך מימוש הגנת Rate Limiting
time	משמש למדוד זמן בבדיקת כמות חיבורים כדי לוודא שהחיבור אפשרי ובנוסף עוזר בחישוב זמנים במנגנון Rate Limiting-ה
sqlite3	משמש לביצוע פעולות תחזוקה במסד הנתונים (מחיקת הודעות לפני מספר ימים ותורים שעברו)

הסקריפט שיוצר ובודק את ה-Token token_handler.py

שם המודול	למה הוא משמש
json (loads,dumps)	הופך אובייקטים של Python לטקסט לצורך יצירת token
hashlib(sha256)	משמש ליצירת החתימה הדיגיטלית
base64(b64encode,b64decode)	משמש לקידוד נתוני ה-token והחתימה לפורמט טקסטואלי
sqlite3	משמש לשליפת פרטי המשתמש ממסד הנתונים כדי להשתמש בהם בעת יצירת ה-token
datetime	לוודא שתוקף ה-token עדיין לא פג
timedelta	משמש להגדרת אורך החיים של ה-token

הסקריפט ששומר את כל סוגי ההודעות שניתן לשלוח
message_types.py

שם המודול	למה הוא משמש
enum (Enum)	משמש ליצירת קבוצות של שמות המייצגים סוגי הודעות

הסקריפט שבו אני מממש את פרוטוקול WebSocket
My_WebSocket.py

שם המודול	למה הוא משמש
socket	משמש ליצירת ה-Server Socket וקבלת חיבורים מהלקוחות
ssl	מימוש שכבת האבטחה (הצפנות)
hashlib	משמש בתהליך חישוב ה-Accept Key בלחיצת יד של ה-WebSocket
base64	קידוד ופענוח נתונים בינאריים למחרוזת טקסט (לדוגמה המפתח בלחיצת היד של ה-WebSocket)
json	המרת אובייקט למחרוזת כדי להעביר ולקבל מהלקוח
enum (Enum)	הגדרת שמות קבועים עבור ערכים של סוגי הודעות שונות של הפרוטוקול

הסקריפט שבו אני מטפל בכל הבקשות של הלקוח
MessageHandler.py

שם המודול	למה הוא משמש
sqlite3	ניהול כל התקשורת מול בסיס הנתונים
collections (defaultdict)	משמר לאסוף את המידע על בעלי המקצוע בצורה נקייה
hashlib	משמש לביצוע Hashing לסיסמאות של משתמשים לפני הכנסתם למסד הנתונים
bleach	משמש ל-Sanitation מקלטים של משתמש כדי למנוע התקפת XSS
smtplib	ניהול התחברות לשרת ה-SMTP של גוגל לביצוע שליחות מיילים
email.mime	בניית אימייל שמשלב טקסט ועיצוב של HTML
random	משמש להפקת מספר אקראי בן 6 ספרות שנשלח למשתמש בהרשמה
datetime, timedelta	חישוב חלונות זמן פנויים לתורים, בדיקת התנגשויות ובדיקת פקיעות התוקף של כל האימות שנשלח למשתמש
abc	משמש להגדרת מחלקות בסיס עבור ה-Handlers וה-Validators
re	שימוש כדי לוודא שפורמט האימייל הגיוני
dotenv	טעינת נתונים רגישים מקבצי .env

os	גישה למערכת הקבצים ושימוש ב-getenv למשיכת משתני סביבה וליצירת salt
----	---

סקירת מודולים מובאים ללקוח

הסקריפט שעוקב אחר תנועת המשתמש באתר ושומר את הדף האחרון שבו ביקר.
RouteTracker.jsx

שם המודול	למה הוא משמש
useEffect	מאפשר בדיקה חוזרת של איפה נמצאים כל פעם שהמיקום משתנה
useLocation	משמש למצוא איפה המשתמש נמצא באתר באותו רגע

הסקריפט שעוטף את כל האפליקציה main.tsx

שם המודול	למה הוא משמש
StrictMode	כלי עזר של React המיועד לאיתור בעיות בקוד
CreateRoot	מהתחל את עץ הרכיבים של React ומחבר אותו לאלמנט ה-HTML בדפדפן
Provider	מאתחל את ה-Store של Redux לאפליקציה
BrowserRouter	מאפשר את יכולת הניתוב בין דפים בעת הרצת האתר
PersistGate	דואג שלפני התנתקות או טעינה מחדש של הדף הכל נשמר (מבחינת Redux)

הסקריפט אחראי על ניהול הניווט App.jsx

שם המודול	למה הוא משמש
Routes	מכיל את ההגדרות של כל הנתבים באפליקציה
Route	מגדיר קישור בין נתיב ספציפי לבין הדף שאמור להיות מוצג באפליקציה
useNavigate	מבצע מעבר בין דפים באופן תכנותי אם לקוח סגר את הדפדפן ופתח מחדש
useEffect	מבצע את הבדיקה של לאן לנווט את הלקוח עם טעינת האפליקציה
useSelector	כעוזר בבדיקה אם משתמש מחובר למערכת כבר

הסקריפטים שיוצרים את ה-Store professionalSlice.js, authSlice.js, appointmentsSlice.js

שם המודול	למה הוא משמש
-----------	--------------

createSlice	משמש להגדרת הלוגיקה של ה-Store ועדכון הנתונים
-------------	---

**הסקריפט שמגדיר את ה-Store
store.js**

שם המודול	למה הוא משמש
configureStore	משמש להקמת החנות
combineReducers	מאחד את כל ה-Reducers לתוך אובייקט אחד שמנהל את האפליקציה
persistStore/persistReducer	מאפשר ל-Redux לשמור מידע בזיכרון של הדפדפן localStorage
storage	מגדיר את ה-localStorage כמיקום שבו המידע ישמר
FLUSH, REHYDRATE...	פעולות פנימיות שדואגות שהמצב החנות ישמר גם בריענון הדפדפן

**הסקריפט מייצר את התשתית לתקשורת עם השרת
SocketContext.jsx**

שם המודול	למה הוא משמש
createContext	מייצר את האובייקט שמחובר אל השרת והופך אותו נגיש לכל האפליקציה
useContext	מאפשר לקומפוננטס אחרים להשתמש באובייקט במחובר לשרת
useState	שומר את ה-WebSocket בתוך ה-State של ה-Provider (כלומר מאפשר לנו לשמור את המצב הנוכחי של האובייקט ולהשתמש בו)
useEffect	מאתחל את החיבור לשרת ברגע שהאפליקציה עולה

**הסקריפט שמסנכרן שמנתב את התשובות לטיפול בעת צפייה בעמוד בית
UserInfo.js**

שם המודול	למה הוא משמש
useEffect	מריץ את הקוד מחדש ברגע שמשהו משתנה בקלט המשתמש

הסקריפט משמש כספריית פונקציות עזר
SignUpModule.jsx

שם המודול	למה הוא משמש
Link	משמש כדי לנווט בחזרה לעמודים אחרים
Select	משמש בשביל ה-Dropdowns
useRef	משמש בשביל האימות אימייל בשביל לנווט בין התיבות של הקוד
useNavigate	מעביר אחרי האימות לעמוד שצריך להעביר
useState	עוזר לניהול מידע זמני בחלק מהקומפוננטס

הסקריפט שמנהל את ההרשמה
SignUp.jsx

שם המודול	למה הוא משמש
useState	שומר על ה-State של כל המידע שהמשתמש מזין בטופס
useEffect	משמש לטעינת רשימת המקצועות וערים לעדכון פורמט שעות העבודה ולקבלת הודעות
useNavigate	מאפשר להעביר את המשתמש לדף ההתחברות

הסקריפט שאחראי על תהליך החיפוש
search.jsx

שם המודול	למה הוא משמש
useSelector	משיג מה-Store את פרטי המשתמש
useDispatch	מאפשר להפעיל את פעולת ה-Logout
useRef	משמש לזיהוי לחיצות מחוץ לתפריט החיפוש
lucide-react	מספק אייקונים גרפים
useNavigate	מאפשר ניווט בין הדפים (מעבר לעמוד בעל המקצוע)
useEffect	מנהל את מחזור החיים של הרכיב
useState	מנהל את המצבים של המשתנים בדף

הסקריפט שאחראי על העברה בין דפים
routerPrint.jsx

שם המודול	למה הוא משמש
Link	מאפשר מעבר בין דפים ללא רענון
useNavigate	פונקציה המשמשת העברת משתמש לדף ההתחברות באופן אוטומטי לאחר התנתקות
useDispatch	מפאשר לבצע את פעולת ה-Logout

הסקריפט שאחראי שמשתמש לא יעבור בין עמודים לפני התחברות
protectedRoutes.jsx

שם המודול	למה הוא משמש
useSelector	לבדוק אם המשתמש היה מחובר
Navigate	מנווט את המשתמש לעמוד ההתחברות אם מנסה לננווט לאנשהו אחר בכוח
Outlet	אם המשתמש כבר מחובר אז מנווט אותו לאן שרצה

הסקריפט שמנהל את כל תהליך ההתחברות
login.jsx

שם המודול	למה הוא משמש
useState	מנהל את כל המצבים של המשתנים ששומרים את הקלט
useEffect	משמש כמאזין לתקשורת מהשרת
useNavigate	עוזר בניווט בין דפים
useDispatch	מעדכן את החנות ברגע של ההתחברות
Link	מעביר בין הדף ההתחברות לדף ההרשמה

הסקריפט שמנווט לעמוד הבית הנכון
homePage.jsx

שם המודול	למה הוא משמש
useSelector	משמש לשליפת מידע המשתמש
useDispatch	מועבר כפרמטר לפעולה useUserSync כדי לעדכן את החנות לפי תשובת השרת
useNavigate	מאפשר ניווט תוכנית, מועבר ל-useUserSync כדי לנתב את המשתמש במקרה של בעיה
useParams	מחלץ את ה-id של בעל המקצוע משורת החיפוש בדפדפן
useLocation	לאפשר גישה לאובייקט ששומר ב-state שבעזרתו ניתן לדעת באיזה פרופיל צופים
useState	מנהל את המידע המקומי של הדף
useEffect	מאזין לשינויים ב-ID או במיקום הניווט

הסקריפט משמש כספרייה עם פעולות עזר
ViewModule.jsx

שם המודול	למה משמש המודול
useEffect	רץ בפקודת האישור לדחיית תורים ומתריע אם קביעת תורים לא חוקית קוראת
useState	משמש לשמירת המצב של משתנים
lucide-react	משמש לאייקונים גרפים בעיצוב הדף

הסקריפט משמש ללקוח לצפות לעצמו בעמוד
UserView.jsx

שם המודול	למה משמש המודול
useSelector	משמש לשליפת מידע מכל מיני מקומות שונות בחנות
Link	משמש כפתור ניווט מהיר לעמוד החיפוש

עמוד הטעינה
LoadingScreen.jsx

שם המודול	למה המודול משמש
useState	משנה את משתנה ה"נקודות"
useEffect	מריץ את הפעולה בעת טעינת העמוד שמשנה את הנקודות לאפקט ההמתנה

הסקריפט שמראה את עמוד הבית של בעל המקצוע (למשתמש ולבעל המקצוע)
FreelancerView.jsx

שם המודול	למה משמש המודול
useSelector	שולף את כל המידע החיוני להצגת עמוד בעל המקצוע
useRef	משמש לכפתור במרכז המסך המאפשר לגלול אל החלק התחתון של העמוד
useState	מנהל את מצב הממשק ואת כל המשתנים בו
useEffect	מבצע קריאות לשרת
lucide-react	מספק את האייקון הגרפי

מחלקות ופעולות ממומשות בשרת

MessageTypes

המחלקה מגדירה מילון קבוע של סוגי הודעות המוכרים למערכת. היא משמשת כפרוטוקול תקשורת בו הלקוח שולח הודעה עם קוד מסוים, והשרת יודע בדיוק לאיזה Dispatcher להעביר את הבקשה לפי הקוד הזה. השרת והלקוח משתמשים באותם מילים בדיוק להודעות. בשרת זה ממומש באמצעות Enum ובלקוח זה ממומש באמצעות Object.freeze(...) (אובייקט שלא ניתן לשנות)

● כללי

- **BROAD** - הודעת שגיאה כללית

● קטגוריית אימות

- **LOGIN**: זיהוי כניסה למערכת.
- **USER_SIGNUP**: רישום לקוח חדש.
- **FREELANCER_SIGNUP**: רישום פריילנסר חדש.
- **VERIFICATION**: שלב אימות המייל (2FA).
- **FORGOT_PASSWORD_REQUEST / AUTHENTICATION**: ניהול תהליך שחזור סיסמה.
- **CHANGE_PASS**: שינוי סיסמה קיימת.

● קטגוריית דף הבית וניהול

- **GET_USER_INFO**: שליפת נתוני המשתמש המחובר.
- **MARK_READ_NOTIFICATION**: עדכון התראות כ"נקראו".
- **UPDATE_APPOINTMENTS_STATUS**: שינוי סטטוס תור (למשל אישור או ביטול).
- **GET_APPOINTMENT_TIMES**: בדיקת חלונות זמן פנויים.
- **MAKE_APPOINTMENT**: יצירת תור חדש בבסיס הנתונים.

● קטגוריית חיפוש ומידע ציבורי

- **GET_PUBLIC_PROFILE_INFO**: הצגת הפרופיל המלא של פריילנסר לכלל המשתמשים.
- **GET_MINIMAL_FREELANCER_INFO**: שליפת "כרטיסי ביקור" תמציתיים לתוצאות החיפוש.
- **GET_JOBS**: שליפת רשימת המקצועות הקיימים במערכת.
- **GET_CITIES_BY_JOB**: סינון ערים שבהן ניתן שירות עבור מקצוע מסוים

StatusMessage

במחלקה מוגדר כל קודי הסטטוס שהשרת שולח בחזרה לאפליקציית ה-React. תפקידה לוודא שה-Frontend יוכל להבין אם פעולה הצליחה או נכשלה. זהו חלק קריטי בניהול השגיאות וחויית המשתמש במערכת "תאם-לי". הלקוח והשרת לשניהם אותם הודעות סטטוס כדי שיהיה להם שפה משותפת ושלוכותב הקוד יהיה קל לעקוב. השרת ממשמש את זה בעזרת Enum והלקוח באמצעות Object.freeze(...).

● כללי

- **TOKEN_BAD** - מציין שה-Token של המשתמש פג תוקף או לא תקין.

● אימות

- **LOGGED_IN / FAILED_LOG_IN**: סטטוס בתהליך ההתחברות
- **SIGNING_UP / FAILED_SIGN_UP**: סטטוס ברישום משתמש
- **VERIFICATION_GOOD / VERIFICATION_BAD**: האם קוד האימות שהוזן תקין או שגוי.
- **CHANGE_PASSWORD_GOOD / BAD**: סטטוס שינוי הסיסמה

● ניהול ותורים

- **GOT_USER_INFO / FAILED_TO_GET_USER_INFO**: סטטוס שליפת נתוני הפרופיל האישי.
- **...UPDATED_APP_STATUS / FAILED**: האם סטטוס התור עודכן בהצלחה (למשל, פרילנסר שאישר תור).
- **...BOOKED_APPOINTMENT / FAILED**: האם קביעת התור החדש בוצעה בהצלחה בבסיס הנתונים.

● חיפוש

- **GOT_JOBS / GOT_CITIES**: אישור שרשימות המקצועות והערים נשלפו בהצלחה.
- **GOT_MINIMAL_INFO**: אישור שתוצאות החיפוש הושגו.
- **GOT_FREELANCER_PUBLIC_INFO**: אישור שכל פרטי הפרופיל הציבורי של פרילנסר נשלפו בהצלחה.

קובץ *token_handler*

המודול אחראי על יצירה, ניהול ואימות של Tokens עבור משתמשי המערכת. הוא משמש כמנגנון אימות שמוודא שכל בקשה שנשלחת לשרת מגיעה ממשתמש מורשה.

שם הפעולה	טענת כניסה (פרמטרים)	טענת יציאה (מה היא מחזירה)	מטרת הפעולה
decode	token	את פרטי המשתמש שבונים את ה-token	לפרק את ה-token למידע שהוא מכיל
is_token_valid	token	boolean	לבדוק אם ה-token בפורמט הנכון ושהוא תקף
_encode	tkn_obj	token's Checksum	ליצור את ה-Checksum כדי לוודא שה-token לא נהרס
create_token	user_email	token	ליצור את ה-token

קובץ *Pro_Find.py*

הקובץ אחראי על הרצת הלולאה המרכזית של השרת ולשלוח כל תהליכון לטפל בבקשות שונות של הלקוחות בצורה היעילה ביותר.

מספר פעולה	שם הפעולה	טענת כניסה (פרמטרים)	טענת יציאה (מה היא מחזירה)	מטרת הפעולה
1	can_accept_connection	ip	boolean	מנגנון הגנה נגד מתקפות DoS בכך שאני מגביל כמות חיבורים לפי כתובת IP בזמן נתון.
2	cleanup_table		None	תהליך רקע למחיקת נתונים פגי תוקף ממסד הנתונים.
3	handle_client	socket	None	ניהול התקשורת מול לקוח ספציפי - קבלת הודעות, בדיקת קצב שליחה וניתוב ל-Dispatcher המרכזי.
4	main_thread	server_socket	None	ניהול לולאת ההאזנה המרכזית, וקבלת לקוחות חדשים.

WebSocketOpcodes

המחלקה מגדירה קודי הפעולה של פרוטוקול WebSocket. מחלקה זו אחראית על ניהול קודי הפעולה של הפרוטוקול. היא מאפשרת לשרת וללקוח להבין האם המידע שמגיע הוא טקסט, מידע בינארי, או פקודת בקרה לניהול הקשר.

- **Continuation** - קוד שמידע על זה שהפקטה הזאת היא חלק מהודעה
- **Text** - קוד שמידע שהפקטה היא מסוג טקסט
- **Binary** - קוד שמידע שהפקטה היא מסוג בינארי
- **Close_Connection** - קוד שמידע שהחיבור נסגר
- **Ping_Message** - קוד בקשת "Proof of Life" בו הלקוח\שרת מבקש לראות שהחיבור חי
- **Pong_Message** - קוד תגובה ל-Ping

WebSocketFactory

אחראית על בניית חבילות מידע לפי פרוטוקול WebSocket. המחלקה מבצעת "עטיפה" של הנתונים בפורמט בינארי כדי שהלקוח יוכל לפענח את ההודעה בצורה תקינה שעוקבת אחרי פרוטוקול WebSocket

שם התכונה	תפקידה	שימושה
opcode	מה סוג ההודעה?	מגדיר את סוג ההודעה כחלק מכותרת ה-Frame.
to_split	האם לפצל את ההודעה למספר פקטות	קובע האם מדובר בהודעה מפוצלת. מגדיר את ביט ה-FIN בראש החבילה.
payload	המידע שבעצם אמור להישלח	ההודעה העצמה שהלקוח אמור לקבל וחלק
message_to_send	כל ההודעה לפי פרוטוקול WebSocket	התוצאה הסופית: רצף בתים מוכן לשליחה על גבי ה-Socket.

פעולות

שם הפעולה	טענת כניסה (פרמטרים)	טענת יציאה (מה היא מחזירה)	מטרת הפעולה
__init__	self, opcode, to_split, payload	אובייקט WebSocketFactory	לבנות את השדות לאובייקט WebSocketFactory
_build_websocket_message	self	רצף בינארי של ההודעה	לבנות את הודעת ה-WebSocket לשליחה
build_pong_message	self	רצף בינארי של ההודעה	לבנות את הודעת ה-WebSocket (מסוג pong) לשליחה

HttpMessage

המחלקה משמשת כאובייקט הודעת HTTP המכיל את כל השדות הרלוונטיים מתוך הודעת ה-HTTP המגיעה מהלקוח. תפקידה הוא לארגן את המידע הדרוש לביצוע לחיצת היד מול הלקוח, כדי לאשר את המעבר לפרוטוקול WebSocket.

שם התכונה	תפקידה	שימושה
http_method	מגדירה את סוג הבקשה הראשונית.	חייבת להיות GET. היא פותחת את תהליך ההתקשרות מול השרת.
protocol_version	מציינת את גרסת ה-HTTP הנלווית.	שימוש ב-HTTP/1.1. זהו הבסיס שעליו נבנה שדרוג החיבור ל-WebSocket.
host	מגדירה את כתובת השרת אליו פונים.	מציינת ל-Socket לאן לשלוח את חבילות המידע (למשל ה-IP של השרת שלך)
upgrade	מודיעה לשרת על רצון להחליף פרוטוקול.	מכילה את הערך websocket. זהו ה"טריגר" שאומר לשרת להפסיק להתנהג כשרת אינטרנט רגיל.
connection	מגדירה את סוג התחזוקה של החיבור.	מכילה את הערך Upgrade. היא מורה לשרת לשמור את הצינור פתוח ולא לסגור אותו אחרי שליחת מידע.
websocket_key	מחרוזת אקראית הנוצרת על ידי הלקוח.	משמשת לאבטחה ואימות. השרת מקבל את המפתח, "מערבב" אותו ומחזיר תשובה כדי להוכיח שהוא אכן תומך ב-WebSocket.

HttpMessageParser

המחלקה אחראית על פיענוח של בקשות HTTP המגיעות מהלקוח. היא מפרקת את טקסט הבקשה לשדות נפרדים ויוצרת מהם אובייקט מסוג HttpMessage, מה שמאפשר למערכת לעבוד עם הנתונים בצורה נוחה ומאורגנת.

שם הפעולה	טענת כניסה (פרמטרים)	טענת יציאה (מה היא מחזירה)	מטרת הפעולה
parse_http_message	msg	אובייקט HttpMessage	פירוק מחרוזת ה-HTTP לשדות המרכיבים אותה והחזרתם כיישנות לוגית אחת.

HttpUpgrade

המחלקה אחראית על הלוגיקה של אישור בקשת השדרוג מ-HTTP ל-WebSocket. היא בוחנת את תקינות השדות שנשלחו מהלקוח ומייצרת את הודעת התגובה הדרושה להשלמת התהליך.

שם התכונה	תפקידה	שימושה
http_message	מכילה את כל השדות שנשלחו	התכונה משמשת למידע לבדיקת תנאי השדרוג ולחילוץ ה-WebSocket Key לצורך חישוב החתימה.

המחלקה משתמשת במספר פעולות כדי לוודא הודעת שדרוג לגיטימית ובניית התגובה

שם הפעולה	טענת כניסה (פרמטרים)	טענת יציאה (מה היא מחזירה)	מטרת הפעולה
__init__	self, http_message	None	ליצור את אובייקט ה-HttpUpgrade
to_upgrade_to_WebSocket	self	bool	לבדוק אם ההודעה באמת בפורמט בקשת שדרוג
upgrade_response	self	str	לבנות את הודעת השדרוג

WebSocketFrame

מחלקה מרכזית בפיענוח התקשורת הנכנסת. WebSocketFrame מאפשרת לשרת לפרק ולהבין את חבילות המידע הבינאריות שמגיעות מהדפדפן.

שם התכונה	תפקידה	שימושה
message_is_split	להודיע אם ההודעה מפוצלת או לא	בעזרת השדה הזה ניתן לדעת אם זה סוף ההודעה או האם יש עוד חלקים שצריך לחכות לחבר אליה
opcode	קוד הפעולה שמגדיר את סוג ההודעה	בעזרת הקוד ניתן לדעת איך לטפל במידע ובאיזה הודעה להגיב
masked	האם המידע מוצפן	מודיע אם צריך לפענח את ההודעה
payload_length	אורך ההודעה	מודיע על אורך ההודעה שיש לקרוא גם כדי לוודא שהכל הגיע
payload	ההודעה	המידע עצמו שאותו רוצים לקבל
masking_key	המפתח להצפנה	המפתח שבעזרתו ניתן לפענח את ההצפנה

WebSocketMessageParser

המחלקה אחראית על קבלת הודעות מה- Socket ופיענוחן בהתאם לפרוטוקול ה-WebSocket. תפקידה העיקרי הוא להפריד את "עטיפת" הפרוטוקול מהתוכן הממשי כך שהשרת יוכל לקבל רק את המידע שהמשתמש שלח.

שם הפעולה	טענת כניסה (פרמטרים)	טענת יציאה (מה היא מחזירה)	מטרת הפעולה
parse_one_frame	clt_soc	WebSocketFrame	קריאת חבילה בודדת מהלקוח ותרגומה לאובייקט לוגי המכיל את כותרות ה-Frame ואת המידע המפוענח.
parse_message	clt_soc	bytes	ניהול קבלת הודעה מלאה - הפעולה אוספת ומחברת חבילות מפוצלות עד לקבלת ביט הסיום ומחזירה את תוכן ההודעה המאוחד.

פעולות נוספות בקובץ

שם הפעולה	טענת כניסה (פרמטרים)	טענת יציאה (מה היא מחזירה)	מטרת הפעולה
accept_client	clt_soc	socket	פונקציית מעטפת המנהלת את כל שלבי הקמת החיבור - הצפנת ה-TLS וה-Handshake של ה-WebSocket.
accept_TLS_encryption	clt_soc	socket	שדרוג ה-Socket הרגיל לחיבור מאובטח באמצעות פרוטוקול TLS.
accept_websocket_upgrade_request_from_client	clt_soc	None	לקבל את בקשת השדרוג ל-WebSocket.
build_proper_json_payload	type, payload, token	dict	יצירת מבנה נתונים אחיד בפורמט מילון הכולל את סוג ההודעה, התוכן ו-Token, כהכנה להפיכתו ל-JSON.
send_message	clt_soc, type, token, msg, to_split_message, opcode,	boolean	לשלוח את ההודעה ללקוח
recv_message	clt_soc	dict	לקבל את ההודעה מהלקוח

Message

המחלקה משמשת כמעטפת לנתונים שמגיעים מהלקוח בפורמט JSON. מטרתה היא לבצע ארגון של המידע באובייקט מסודר, מה שמאפשר גישה נוחה ובטוחה יותר לשדות ההודעה לאורך כל תהליך העיבוד ב-Dispatcher וב-Handlers השונים.

שם התכונה	תפקידה	שימושה
type_of	להודיע ללקוח מה סוג ההודעה	עוזר לנתב את ההודעה
data	לשמור את המידע של ההודעה	עוזר לקרוא את כל המידע שהלקוח שלח
token	המזהה של הלקוח	לבדוק שהמשמש אכן הוא ושהוא לא מתחזה למישהו אחר

MessageHandler

המחלקה מגדירה ממשק אחיד לכל מחלקות השירות במערכת. היא משתמשת במודול של פייתון לטיפול במחלקות אבסטרקטיות, ובכך היא אוכפת על כל מחלקה שיורשת ממנה לממש את הלוגיקה הספציפית שלה בתוך פונקציה קבועה מראש

MessageDispatcher

המחלקה אחראית על ניהול זרימת המידע בשרת. היא מקבלת אובייקט מסוג Message בודקת את ה-Token ומנתבת את ההודעה לטיפול במחלקה המתאימה על פי סוג ההודעה

שם התכונה	תפקידה	שימושה
_handlers	מילון הממפה בין סוג הודעה למחלקת הטיפול המתאימה.	שימוש במילון מאפשר שליפה מהירה של ה-Handler הנכון בזמן אמת.
no_token_check (static)	שומרת את הפעולות שלא צריכים לבדוק להם את המזהה	מניעת מצב שבו משתמש צריך Token כדי להתחבר ולקבל Token. הבדיקה מתבצעת לפני תהליך הניתוב.

פעולות מרכזיות

שם הפעולה	טענת כניסה (פרמטרים)	טענת יציאה (מה היא מחזירה)	מטרת הפעולה
__init__	self	None	לבנות את האובייקט המנתב
register	self, msg_type, cls	None	להוסיף ל-handlers עוד מפתח וערך

לנתב את ההודעה אל המחלקה הנכונה לטיפול בה	MessageType, payload	self, msg	dispatch
--	-------------------------	-----------	----------

LoginDispatcher

מחלקה שמקבלת בקשה להתחבר למערכת ואחראית על לבדוק שהפרטים שהמשתמש שלח לשרת נכונים ואם כן להחזיר תשובה מתאימה כדי שיוכל להתחבר לפלטפורמה.

שם הפעולה	טענת כניסה (פרמטרים)	טענת יציאה (מה היא מחזירה)	מטרת הפעולה
handle	self, msg	tuple (MessageType, payload)	שליפת האימייל והסיסמה מההודעה, בדיקתם מול מסד הנתונים, והחזרת תשובת סטטוס

Freelancer\UserSignUpService

מחלקה שמקבלת בקשה מלקוח/בעל מקצוע להירשם למערכת ואחראית על לבדוק שכל המידע שהכניס נכון ותואם לפורמט ולשלוח לו קוד אימות אם עבר את כל הבדיקות. המחלקות דומות במהותם אבל הבדיקות שמבצעות שונות מאוד לכן מופרדות

שם הפעולה	טענת כניסה (פרמטרים)	טענת יציאה (מה היא מחזירה)	מטרת הפעולה
handle	self, msg	tuple (MessageType, payload)	ניהול זרימת הרישום - הפעלת הבדיקות, קריאה להכנסה לטבלה הזמנית, ושליחת קוד אימות למייל.
add_pending_user	self, msg	tuple [bool, dict]	ביצוע הרישום הפיזי בבסיס הנתונים לטבלת ה-pending_users יחד עם קוד האימות שנוצר

VerificationDispatcher

המחלקה מהווה את השלב הסופי בתהליך הרישום. היא אחראית לקבל את קוד האימות שהזין המשתמש, להשוותו לקוד שנשמר במסד הנתונים, ובמידה והקוד תקין לבצע את העברת הנתונים לטבלאות הקבועות של המערכת.

שם הפעולה	טענת כניסה (פרמטרים)	טענת יציאה (מה היא מחזירה)	מטרת הפעולה
handle	self, msg	tuple (MessageType, payload)	אימות הקוד שהתקבל מהלקוח מול מסד הנתונים והפעלת תהליך הרישום הסופי במקרה של הצלחה.
log_new_customer	self, msg	bool	העברת פרטי המשתמש מטבלת ה-pending_users לטבלת ה-users

ForgotPasswordEmailVerficiationDispatcher

המחלקה אחראית על הטיפול בבקשות של משתמשים ששכחו את סיסמתם. תפקידה המרכזי הוא לוודא שהמשתמש אכן קיים במערכת, וליצור עבורו ערוץ מאובטח כדי לאפשר לו להחליף את סיסמתו.

שם הפעולה	טענת כניסה (פרמטרים)	טענת יציאה (מה היא מחזירה)	מטרת הפעולה
handle	self, msg	tuple (MessageType, payload)	בדיקת קיום המייל במערכת ושליחת קוד אימות ייעודי לאיפוס הסיסמה.

ForgotPasswordAuthenticationDispatcher

מחלקה זו מהווה את המחסום האחרון לפני שינוי הסיסמה בפועל. תפקידה הוא לוודא שהקוד שהמשתמש הזין אכן תואם לקוד האימות שנשלח אליו למייל ושומר במסד הנתונים. במידה והקוד נכון, המחלקה מעניקה למשתמש סטטוס מאושר זמני לביצוע פעולת העדכון.

שם הפעולה	טענת כניסה (פרמטרים)	טענת יציאה (מה היא מחזירה)	מטרת הפעולה
handle	self, msg	tuple (MessageType, payload)	אימות קוד השחזור מול הנתונים השמורים במערכת ומתן הרשאה להמשך תהליך שינוי הסיסמה.

UpdatePasswordDispatcher

מחלקה זו אחראית על השלב הביצועי של החלפת הסיסמה במערכת. היא נקראת רק לאחר שהמשתמש עבר בהצלחה את שלבי האימות הקודמים. תפקידה הוא לעדכן את הסיסמה החדשה במסד הנתונים.

שם הפעולה	טענת כניסה (פרמטרים)	טענת יציאה (מה היא מחזירה)	מטרת הפעולה
handle	self, msg	tuple (MessageType, payload)	עדכון הסיסמה של המשתמש במסד הנתונים, במידה והוא רשאי לזה.

EmailVerification

המחלקה אחראית על יצירת קשר מול שרת ה-SMTP לשם שליחת קודי אימות למשתמשים וניהול תוקף הקודים הללו. היא משמשת ככלי עזר עבור ה-Dispatchers השונים כדי לוודא שכתובת המייל שהוזנה אכן שייכת למשתמש.

שם התכונה	תפקידה	שימושה
email	התכונה שקובעת לאן	קובעת את הכתובת שאליה יישלח המייל עם קוד

האימות.	האימייל ישלח	
שמירת הקוד שנשלח למשתמש והשוואתו לערך השמור במסד הנתונים לצורך אימות.	לשמור את הסיסמה שמאמתת את המשתמש	verification_code
משמש לבדיקה אם עבר התוקף של קוד האימות כדי לוודא שאין ניסיון פריצה ושהמשתמש באמת עדיין מחובר	לשמור עד מתי הקוד תקף	expires_at

שם הפעולה	טענת כניסה (פרמטרים)	טענת יציאה (מה היא מחזירה)	מטרת הפעולה
__init__	self, email	None	פעולה בונה שמגדירה את אובייקט ה-EmailVerification שיהיה קל להשתמש בפעולות במחלקה
send_verification_code	self	bool	פעולה שאחראית על לשלוח את קוד האימות אל האימייל של המשתמש
check_verification_code	msg	tuple (bool, dict)	פעולה סטטית שמקבלת הודעת ואחראית על לבדוק אם המידע אכן נכון

Validator

מחלקה אבסטרקטית שמגדירה שעוזרת לשמור על סדר בקוד, יש לה פעולה אחת בשם validate שמקבלת self, msg מחזירה tuple אבל לא ממומשת אלא רק מגדירה איך כל הפעולות היורשות יצטרכו לממש את הפעולה.

Freelancer/UserSignUpValidator

מחלקות היורשות מ-Validator ומטרתן לבצע בדיקות על פרטי הרישום של משתמשים חדשים. הן מוודאות שהמידע שהוזן אינו מהווה סיכון אבטחתי ועומד בפורמט הנדרש על ידי המערכת לפני שהנתונים מועברים להמשך טיפול בשרת.

שם הפעולה	טענת כניסה (פרמטרים)	טענת יציאה (מה היא מחזירה)	מטרת הפעולה
validate	self, msg	tuple (bool, str)	הפונקציה מוודאת שכל שדות הרישום הנדרשים מולאו בפורמט תקין ונתמך.
_check_email_format	self, email	bool	בודק שפורמט האימייל תקין כדי לא לגרום לשגיאה

בודק שהפורמט נכון וגם שכל הערים מוכרות במערכת	bool	self, cities	check_city (רק אצל בעל המקצוע)
---	------	--------------	-----------------------------------

ExistingJobDispatcher

המחלקה אחראית על השגת כל סוגי המקצועות של בעלי המקצוע הרשומים במערכת. תפקידה לאפשר לצד הלקוח להציג למשתמש רשימה מסודרת של אפשרויות חיפוש, מבלי שהלקוח יצטרך להחזיק רשימה סטטית משלו.

שם הפעולה	טענת כניסה (פרמטרים)	טענת יציאה (מה היא מחזירה)	מטרת הפעולה
handle	self, msg	tuple (MessageType, payload)	אחראי על להשיג את כל סוגי המקצועות שרשומים במערכת ולהחזיר ללקוח

CitiesByJobDispatcher

מחלקה זו מהווה שלב מתקדם בתהליך החיפוש. תפקידה להחזיר רשימה של ערים שבהן קיימים בעלי מקצוע פעילים מהסוג שנבחר על ידי המשתמש.

שם הפעולה	טענת כניסה (פרמטרים)	טענת יציאה (מה היא מחזירה)	מטרת הפעולה
handle	self, msg	tuple (MessageType, payload)	אחראי על להוציא ממסד הנתונים את כל הערים בהם יש בעל מקצוע שעוסק בתחום הרצוי של הלקוח

MinimalProfessionalInfo

אחראית על שליחת מידע בסיסי ותמציתי אודות בעלי מקצוע התואמים את הקטגוריות והאזור שנבחרו בחיפוש. המטרה היא לייצר רשימה של כרטיסי ביקור דיגיטליים המאפשרים ללקוח לעיין במגוון אפשרויות ולבחור את בעל המקצוע המתאים לו.

שם הפעולה	טענת כניסה (פרמטרים)	טענת יציאה (מה היא מחזירה)	מטרת הפעולה
handle	self, msg	tuple (MessageType, payload)	משיג ממסד הנתונים מידע בסיסי על כל בעלי המקצוע מהאזור ומהתחום שבהם בחר הלקוח

FreelancerPublicInfo

המחלקה אחראית על שליפת המידע הפומבי המלא על בעל מקצוע ממסד הנתונים. המחלקה הזו נועדה להציג ללקוח את התמונה המלאה בדף הבית של בעל המקצוע, במטרה לסייע ללקוח לקבל החלטה סופית לפני קביעת התור.

שם הפעולה	טענת כניסה (פרמטרים)	טענת יציאה (מה היא מחזירה)	מטרת הפעולה

משיג ממסד הנתונים מידע שבעזרתו יהיה ניתן להציג את דף הבית של בעל המקצוע	tuple (MessageType, payload)	self, msg	handle
---	------------------------------------	-----------	--------

UserInfoDispatcher

מחלקה שאחראית על שליפה של כל הנתונים האישיים של משתמש ממסד הנתונים. היא זאת שמספקת את המידע להצגת עמוד הבית הפרטי של משתמש.

שם הפעולה	טענת כניסה (פרמטרים)	טענת יציאה (מה היא מחזירה)	מטרת הפעולה
handle	self, msg	tuple (MessageType, payload)	הפעולה הראשית שמשיגה את כל המידע על המשתמש ממסד הנתונים ומחזירה אותו בצורה מאורגנת לצד הלקוח.
_get_appointments	self, user_id, cur, role	list	משיג את כל התורים שיש למשתמש
_get_notifications	self, user_id, cur	list	משיג את כל ההודעות שיש למשתמש

MarkNotificationsReadDispatcher

המחלקה אחראית על עדכון מסד הנתונים כאשר המשתמש צופה בהתראות שלו, וסימון כנקראו. פעולה זו חיונית כדי להבטיח שמעקב אחר ההתראות יהיה מדויק ועדכני בכל כניסה של המשתמש לאזור האישי.

שם הפעולה	טענת כניסה (פרמטרים)	טענת יציאה (מה היא מחזירה)	מטרת הפעולה
handle	self, msg	tuple (MessageType, payload)	עדכון טבלת ההתראות במסד הנתונים) כך שכל ההתראות הרלוונטיות למשתמש יסומנו כהתראות שנקראו.

ChangeAppointmentStatusDispatcher

המחלקה אחראית על ניהול סטטוס התורים באפליקציה. תפקידה לאפשר לבעל המקצוע לאשר או לדחות תורים הממתינים לאישור, תוך ביצוע בקרת איכות על לוח הזמנים כדי למנוע כפילויות.

שם הפעולה	טענת כניסה (פרמטרים)	טענת יציאה (מה היא מחזירה)	מטרת הפעולה
handle	self, msg	tuple (MessageType, payload)	עדכון מסד הנתונים בהתאם להחלטת בעל המקצוע ושליחת אישור לצד הלקוח.
_is_time_slot_taken	self, cur, app_id	bool	בדיקה לוגית מול מסד הנתונים אם משבצת הזמן המבוקשת כבר תפוסה על ידי תור אחר שאושר לאותו בעל מקצוע.

AppointmentStartTimeDispatcher

אחראית על שליחת שעות הפעילות הפנויות של בעל המקצוע אל הלקוח. תפקידה המרכזי לבצע הצלבה בין מסגרת שעות העבודה הכללית של בעל המקצוע לבין התורים שכבר שוריינו ביומן, כדי להציג למשתמש רק את האפשרויות שניתן לקבוע בהן תור חדש.

שם הפעולה	טענת כניסה (פרמטרים)	טענת יציאה (מה היא מחזירה)	מטרת הפעולה
handle	self, msg	tuple (MessageType, payload)	מחשב לפי התורים שכבר נקבעו במסד הנתונים את התורים הפנויים של בעל המקצוע ומתי יכול לקבל תורים חדשים

BookAppointmentDispatcher

המחלקה אחראית על קליטת בקשות לקביעת תור חדש מצד הלקוח והכנסתן למסד הנתונים. תפקידה לוודא שהתור המבוקש תקין מבחינה לוגית ולהשלים את הנתונים הטכניים הדרושים לפני שמירתם בטבלת התורים.

שם הפעולה	טענת כניסה (פרמטרים)	טענת יציאה (מה היא מחזירה)	מטרת הפעולה
handle	self, msg	tuple (MessageType, payload)	בודק שהתור שרוצים לקבוע לא מתנגש עם תור קיים אצל בעל המקצוע ואם לא מוסיף אל מסד הנתונים
calculate_end_time	self, start_time_str, duration_minutes	str	חישוב שעת סיום התור על סמך שעת ההתחלה ומשך תור ממוצע, כדי למלא את שדה שעת הסיום ההכרחי במסד הנתונים.

פעולות נוספות בקובץ

חלק מהלוגיקה של המערכת ממומש כפונקציות עצמאיות מחוץ למחלקות ה-Dispatcher. הפרדה זו נועדה למנוע כפילות קוד ולאפשר למחלקות שונות לגשת לאותם כלים חישוביים ואבטחתיים.

שם הפעולה	טענת כניסה (פרמטרים)	טענת יציאה (מה היא מחזירה)	מטרת הפעולה
hash_password	password, salt	str	הצפנת סיסמת המשתמש באמצעות Salt כדי להבטיח אחסון מאובטח של פרטי הזיהוי במסד הנתונים
configure_dispatcher		MessageDispatcher	מאתחל את אובייקט ניתוב ההודעות עם כל המחלקות וסוגי ההודעות הקיימים במערכת.
_calculator_free_day_working_hours	start_time, end_time, job_duration, existing_appointments	list	אלגוריתם המחשב את חלונות הזמן הפנויים של בעל מקצוע ביום ספציפי על סמך אילוצים.

מחלקות ופעולות ממומשות צד לקוח

ריכוז רכיבי ה-React ופעולות העזר בטבלה מאוחדת נועד להציג את הקשר הישיר בין רכיבי הממשק לבין הלוגיקה הפונקציונלית המשרתת אותם. מבנה זה לדעתי עוזר בהבנה מעמיקה של זרימת המידע במערכת

מטרת הפעולה	טענת יציאה (מה היא מחזירה)	טענת כניסה (פרמטרים)	שם הפעולה	הרכיב בלקוח אליו שייכת
מעקב אוטומטי אחר נתיב הגלישה של המשתמש ושמירתו ב- <code>localStorage</code> . מבטיח שרענון העמוד לא ישבש את זרימת העמוד	null	אין	<i>RouteTracker</i>	מערכת הניווט
ניהול זרימת המשתמש באתר. הרכיב אחראי על טעינת הנתיב האחרון שנשמר.	כל הנתיבים ואבטחתם	אין	<i>App</i>	מערכת ניווט
ניהול וריכוז כל הנתונים הגלובליים של האפליקציה במקום אחד. הפעולה דואגת לסנכרון אוטומטי מול ה- <code>localStorage</code> במקרה של טעינת עמוד.	אובייקט Store מוגדר	rootReducer, persistConfig	<i>store</i>	ניהול הנתונים
ניהול של סטטוס החיבור של המשתמש. הרכיב שומר את נתוני המשתמש במהלך הגלישה ומאפשר לעדכן אותם.	אובייקט reducer	initialState	<i>authSlice</i>	ניהול נתונים
ניהול של רשימות התורים באפליקציה. הרכיב שומר את נתוני התורים של המשתמש במהלך הגלישה ומאפשר לעדכן אותם.	אובייקט reducer	initialState	<i>appointmentSlice</i>	ניהול נתונים
יצירת פורמט הודעה אחיד עבור התקשורת עם השרת.	מחרוזת JSON	type, payload, token, type_of_payload	<i>MsgBuild</i>	תקשורת
פענוח הודעות נכנסות מהשרת.	אובייקט עם type, data, token	payload	<i>websocketParser</i>	תקשורת
הגדרת פרוטוקול התקשורת של המערכת	אובייקט מיפוי לקריאה	Object.freeze()	<i>Constants</i>	תקשורת
ביצוע תהליך התחברות למערכת על ידי שליחת.	שולח הודעה	username, password, ws, token	<i>LoginRequest</i>	תקשורת (בקשות)
רישום משתמש חדש במערכת.	שולח הודעה	ws, email, veri_code, role, token	<i>VerificationRequest</i>	תקשורת (בקשות)
אימות חשבון משתמש באמצעות קוד אימות שנשלח במייל להשלמת הרישום.	שולח הודעה	ws, account_info, type_of_user, token	<i>SignUpRequest</i>	תקשורת (בקשות)

בקשה מהשרת לשלוח קוד אימות למשתמש ששכח את סיסמתו.	שולח הודעה	ws, email, token	<i>ForgotPasswordVerificationCodeRequest</i>	תקשורת (בקשות)
אימות קוד שחזור הסיסמה שהוזן על ידי המשתמש.	שולח הודעה	ws, email, code, token	<i>ForgotPasswordAuthenticationRequest</i>	תקשורת (בקשות)
עדכון הסיסמה החדשה במערכת.	שולח הודעה	ws, email, pass, token	<i>ChangePasswordRequest</i>	תקשורת (בקשות)
שליפת פרטי המשתמש המחובר.	שולח הודעה	ws, token	<i>UserInfoRequest</i>	תקשורת (בקשות)
עדכון סטטוס התורים (אישור/ביטול) בבסיס הנתונים.	שולח הודעה	ws, apps, token	<i>UpdateAppointmentStatusRequest</i>	תקשורת (בקשות)
עדכון מערכת ההתראות שכל ההודעות החדשות נקראו על ידי המשתמש.	שולח הודעה	ws, userId, token	<i>MarkReadNotificationsRequest</i>	תקשורת (בקשות)
שליפת חלונות הזמן הפנויים של בעל מקצוע ספציפי לצורך קביעת תור.	שולח הודעה	ws, targetId, token	<i>GetAvailableWorkTimes</i>	תקשורת (בקשות)
יצירת תור חדש במערכת.	שולח הודעה	ws, app, token	<i>BookAppointment</i>	תקשורת (בקשות)
שליפת מידע ציבורי על פרופיל של בעל מקצוע.	שולח הודעה	ws, targetId, token	<i>GetPublicInfoRequest</i>	תקשורת (בקשות)
שליפת מידע מינימלי להצגת כרטיס ביקור	שולח הודעה	ws, token, city, job	<i>GetMinimalFreelancerInfo</i>	תקשורת (בקשות)
שליפת רשימת כל תחומי העיסוק הקיימים במערכת.	שולח הודעה	ws, token	<i>GetAllJobs</i>	תקשורת (בקשות)
שליפת רשימת כל הערים שקיימים עבורם בעלי מקצוע מהסוג שנקלט על ידי המשתמש	שולח הודעה	ws, job, token	<i>GetCitiesByJob</i>	תקשורת (בקשות)
עיבוד תגובת ההתחברות מהשרת.	מערך success, [data]	payload, dispatch	<i>handleLoginResponse</i>	תקשורת (תשובות)
בדיקה האם תהליך ההרשמה עבר בהצלחה או נכשל בשרת	מערך success, [data]	payload	<i>handleSignUpResponse</i>	תקשורת (תשובות)
פענוח תשובת השרת לגבי תקינות קוד האימות שהזין המשתמש.	מערך success, [data]	payload	<i>handleVerificationResponse</i>	תקשורת (תשובות)
אישור מהשרת על שליחת קוד שחזור סיסמה למייל של המשתמש.	מערך success, [data]	payload	<i>handleForgotPasswordVerificationCodeResponse</i>	תקשורת (תשובות)
בדיקה האם קוד שחזור הסיסמה שהוזן תקין	מערך success, [data]	payload	<i>handleForgotPasswordVerifyCodeResponse</i>	תקשורת (תשובות)

תקשורת (תשובות)	<i>handleChangePasswordResponse</i>	payload	מערך success, [data]	עיבוד אישור או דחייה של השרת לשינוי הסיסמה
תקשורת (תשובות)	<i>handleUserInfoResponse</i>	dispatch, payload	מערך success, [data]	פעולה מרכזית: פירוק המידע המלא על המשתמש, הפרדת התורים וההתראות, ועדכון ה-Store בפרטי הפרופיל ובלו"ז.
תקשורת (תשובות)	<i>handleUpdatedAppointmentsResponse</i>	payload	מערך success, [data]	קבלת אישור מהשרת על כך שסטטוס התור עודכן בהצלחה בבסיס הנתונים.
תקשורת (תשובות)	<i>handleMarkedReadNotifications</i>	payload	מערך success, [data]	אישור מהשרת על עדכון ההתראות כנקראו
תקשורת (תשובות)	<i>handleAppointmentTimes</i>	payload	מערך success, [data]	עיבוד רשימת חלונות הזמן הפנויים
תקשורת (תשובות)	<i>handleAppointmentMadeResponse</i>	payload	מערך success, [data]	בדיקה האם פעולת שריון התור הצליחה או נכשלה
תקשורת (תשובות)	<i>handleProfileInfoResponse</i>	payload	מערך success, [data]	עיבוד והכנת המידע הציבורי של בעל מקצוע להצגה בעמוד הפרופיל שלו.
תקשורת (תשובות)	<i>handleJobsResponse</i>	payload	מערך success, [data]	חילוץ רשימת המקצועות
תקשורת (תשובות)	<i>handleCitiesResponse</i>	payload	מערך success, [data]	חילוץ רשימת הערים הרלוונטיות למקצוע מסוים לצורך סינון החיפוש
תקשורת (תשובות)	<i>handleMinimalFreelancerInfoResponse</i>	payload	מערך success, [data]	עיבוד רשימת כרטיסי בעלי המקצוע להצגה בתוצאות החיפוש.
תקשורת	<i>getWebSocket</i>	ip, port	אובייקט Websocket	מימוש תבנית עיצוב מסוג Singleton. הפונקציה מבטיחה שיווצר רק חיבור Socket אחד ויחיד לשרת לאורך כל חיי האפליקציה.
תקשורת	<i>SocketProvider</i>	children	הזרקת אובייקט ה-Socket לכל עץ הרכיבים	ניהול מחזור החיים של החיבור. הרכיב מאתחל את החיבור לשרת (בכתובת omer.ystr) עם עליית האפליקציה ומנגיש את החיבור לכל חלקי האתר באמצעות ה-Context
תקשורת	<i>useSocket</i>	אין	אובייקט ה-Socket הקיים	מאפשר לרכיבים לשלוף את החיבור הקיים בקלות ובמהירות

ניהול נתונים + תקשורת	<i>useUserSync</i>	ws, token, dispatch, navigate, setViewedFreelancer, setAppointmentTimes, target_id, isPublic	null	סנכרון נתונים רציף. ה-Hook דואג שכל הודעה שנכנסת מהשרת תופנה ל-Handler המתאים
ניהול נתונים + תקשורת	<i>handleMessage</i>	event	עדכון State	ניתוב הודעות, פענוח סוג ההודעה והפעלת הלוגיקה המתאימה.
ניהול נתונים + תקשורת	<i>sendRequest</i>	אין	שליחת בקשות ראשוניות לשרת	איתחול נתונים, שליחת בקשות לקבלת מידע על המשתמש המחובר
הרשמה ואימות	<i>SignUpPage</i>	אין	רינדור של שלבי ההרשמה	הרכיב המרכזי ששולט במעבר בין שלבי ההרשמה
הרשמה ואימות	<i>UserSignUp</i>	userInfo, setUserInfo, onSubmit, serverError	טופס הרשמה ללקוחות רגילים	הרכיב המרכזי ששולט במעבר בין
הרשמה ואימות	<i>FreelancerSignUp</i>	freelancerInfo, setFreelancerInfo, onSubmit	טופס הרשמה לבעלי מקצוע	איסוף מידע מקצועי מפורט
הרשמה ואימות	<i>handleMessage</i>	event	עדכון מצב ההרשמה/ליווט	האזנה לתשובות השרת בזמן אמת
הרשמה ואימות	<i>handleSignUpPageSubmit</i>	אין	שליחת בקשת הרשמה	פונקציית עזר השולחת את הנתונים שנאספו.
הרשמה ואימות	<i>freelancerProfessions</i>	אין	list	שליפת רשימת המקצועות מקובץ טקסט
הרשמה ואימות	<i>israeliLocalities</i>	אין	list	שליפת רשימת יישובי ישראל
הרשמה ואימות	<i>RolePopup</i>	onSelect	רכיב בחירה ויזואלי	הצגת חלון קופץ בתחילת ההרשמה שנותן לבחור את סוג ההרשמה
הרשמה ואימות	<i>Single/MultiChoiceDropDown</i>	options, value, onChange, customWidth	תפריט בחירה מעוצב	אספקת ממשק בחירה נוח ומעוצב
הרשמה ואימות	<i>EmailVerification</i>	verificationCode, setVerificationCode, handleVerificationCodeSubmit	ממשק להזנת קוד אימות	הצגת ממשק להזנת קוד בעל 6 ספרות
מערכת חיפוש ואיתור	<i>SearchPage</i>	אין	רינדור של קומפוננט	רכיב העטיפה שמחליט האם להציג את ממשק החיפוש או את רשימת התוצאות בהתאם לבחירת המשתמש

מאפשר למשתמש לבחור סוג שירות מאפשר למשתמש לבחור סוג שירות	ממשק ויזואלי	ws, token, handleSubmit	SearchBar	מערכת חיפוש ואיתור
שולף ומציג מידע מינימלי על בעלי מקצוע	רשימת כרטיסי בעל מקצוע	ws, token, goBack, city, job	FreelancereMenu	מערכת חיפוש ואיתור
מעבד הודעות מהשרת המכילות את רשימת כל המקצועות	עדכון רשימת המקצועות וערים	event	handleMessage	מערכת חיפוש ואיתור
שליחת בקשה לשרת לקבלת המידע המלא על בעל המקצוע ומעבר לדף הפרופיל הציבורי שלו	שליחת בקשה לפרופיל וניווט אליו	id	handleOnClick	מערכת חיפוש ואיתור
הצגת קישורים רלוונטיים למשתמש בהתאם להרשאות שלו ושליטה על תפריט המערכת.	תפריט ניווט צידי	role	Navbar	מערכת ניווט
מחיקת פרטי המשתמש מה-Redux ומה-LocalStorage כדי לסיים את הסשן בצורה מאובטחת.	ניקוי נתונים וניווט לדף ההתחברות	אין	handleLogout	מערכת ניווט
בדיקה האם המשתמש מחובר למערכת לפני מתן גישה לדפים מאובטחים.	רכיב ה-Outlet או Navigate	אין	ProtectedRoutes	מערכת ניווט (ואבטחה)
הרכיב המרכזי ששולט על המעברים בין דפים	רינדור של דפים לפי קלט	payload	Login	מערכת כניסה
מאפשר למשתמש להזין אימייל כדי לקבל קוד אימות לשינוי הסיסמה.	משמק להזנת אימייל	goBack, ws, email, setEmail, errorMessage	ForgotPassword	מערכת כניסה
מאפשר למשתמש להזין סיסמה חדשה	ממשק לקיעל סיסמה חדשה	changePassword Request, password, setPassword	ChangePassword	מערכת כניסה
מעדכן אם הכניסה הצליחה, אם קוד האימות תקין או אם הסיסמה שונתה בהצלחה.	עדכון מצבי הממשק	event	handleMessage	מערכת כניסה
שולח לשרת את הקוד שהתקבל במייל כדי לאשר שהמשתמש מורשה לשנות סיסמה.	שליחת בקשת אימות קוד לשרת	אין	handleVerification CodeSubmit	מערכת כניסה
מחליט איזה ממשק להציג למשתמש על סמך התפקיד שלו	רינדור מותנה של עמוד בית לפי סוג משתמש	isPublic	HomePage	ניהול עמוד בית
מחשב בזמן אמת איזה מבט להציג - לקוח שוראה כרטיס ביקור או בעל מקצוע שרואה תורים לאשר שעתידים לקרות ועוד.	הרכיב הויזואלי המתאים	אין	renderContent	ניהול עמוד בית

ממשק משתמש כללי	<i>LoadingScreen</i>	אין	מסך טעינה במרכז הדף	הצגת מסך המודיע למשתמש שהמידע בטעינה
תצוגה ומידע	<i>NotificationsView</i>	notifications	רכיב ויזואלי של ההודעות	הצגת רשימת פעילויות אחרונות
תצוגה ומידע	<i>UpcomingAppointmentsView</i>	appointments	רכיב ויזואלי של תורים עתידיים	הצגת סיכום תמציתי של פגישות מאושרות קרובות
תצוגה ומידע	<i>FreelancerInfo</i>	פרטי בעל המקצוע	רכיב ויזואלי של כרטיס ביקור	הצגת כרטיס ביקור מעוצב
רכיבי ניהול פגישות	<i>PendingAppointments</i>	appointments, ws, token	רכיב ויזואלי של אישור/דחיית תורים	ניהול בקשות תורים חדשות עבור בעל המקצוע.
רכיבי ניהול פגישות	<i>AppointmentBooking</i>	availability, jobDuration, onConfirm, onCancel, price	רכיב ויזואלי לקביעת תור	ממשק לקביעת תור עבור לקוח.
רכיבי ניהול פגישות	<i>AppointmentDetailModal</i>	appointment, onClose	רכיב ויזואלי התור הנבחר	חלונית תצוגה עם פרטי פגישה מלאים.
דף הבית של המשתמש	<i>UserView</i>	אין	רכיב ויזואלי לעמוד הבית	הצגת תמונת מצב מהירה ללקוח
דף הבית של בעל המקצוע	<i>FreelancerView</i>	user, isPublic, appointments	מעטפת המחליטה איזו תצוגה להציג	מאפשר שימוש חוזר בלוגיקה של בעל מקצוע תוך הפרדה בין תצוגה ציבורית לפרטית.
פרופיל ציבורי	<i>FreelancerPublic</i>	נתוני בעל המקצוע, חיבור Socket וזמני תורים	ממשק צפייה והזמנת תור עבור הלקוח	מאפשר ללקוח לצפות בפרטי בעל המקצוע ולבצע תהליך קביעת תור
דף הבית של בעל המקצוע	<i>FreelancerPrivate</i>	חיבור Socket ו-Token	עמוד לניהול של בעל המקצוע	ריכוז התראות, תורים מאושרים ובקשות חדשות הממתינות לאישור

סקירת בעיות אלגוריתמיות

חיפוש בעל מקצוע

סקירת היכולת

יכולת זו מאפשרת למשתמש למצוא בעלי מקצוע בצורה חכמה על ידי סינון מדורג של סוג השירות והעיר בה הוא מעוניין בשירות. המערכת מבצעת עדכון דינמי של התוצאות מול השרת בזמן אמת, תוך הצגת כרטיסי מידע תמציתיים הכוללים נתונים קריטיים כמו מחיר וניסיון. בסיום התהליך, המשתמש יכול לעבור בלחיצה לפרופיל המלא של בעל המקצוע שנבחר כדי להתחיל בתהליך קביעת התור. היכולת משתמשת בניהול נתונים גלובלי כדי להבטיח שכל בחירה של המשתמש תסונכרן מול השרת ותציג זמינות עדכנית של בעל המקצוע.

הערה לגבי תיעוד המימוש: בחרתי שלא לפרט את מימוש הקוד המלא של צד הלקוח עבור יכולת זו. הסיבה לכך היא שתפקידו המרכזי של ה-Frontend במערכת הוא לשמש כממשק ויזואלי להצגת נתונים וקליטת קלט מהמשתמש בלבד. הלוגיקה העסקית המורכבת, הכוללת את ניהול מסד הנתונים, אלגוריתמי הסינון ואבטחת המידע, מבוצעת כולה בצד השרת. לכן, הקוד שאסקור בעמודים הבאים יתמקד ברכיבי השרת, ששם מנוהל כל הלוגיקה של החלק הזה

הקוד של יכולת חיפוש בעלי מקצוע

```

class ExistingJobsDispatcher(MessageHandler):
    def handle(self, msg: Message) -> tuple:
        try:
            with sqlite3.connect(DATABASE) as conn:
                cur = conn.cursor()
                cur.execute("SELECT DISTINCT profession FROM professional")
                rows = cur.fetchall()
                if not rows:
                    return MessageTypes.GET_JOBS, {
                        StatusMessage.FAILED_TO_GET_JOBS.value: "Our professionals are
still not verified in the system, sorry!"
                    }
                jobs = []
                for row in rows:
                    jobs.append(row[0])
                jobs.sort()
                return MessageTypes.GET_JOBS, {
                    StatusMessage.GOT_JOBS.value: {"jobs": jobs}
                }
        except Exception as e:
            print("Existing job dispatcher error - ", e)
            return MessageTypes.GET_JOBS, {
                StatusMessage.FAILED_TO_GET_JOBS.value: "Failed retrieving info, reload
page"
            }

```

```

class CitiesByJobDispatcher(MessageHandler):
    def handle(self, msg: Message) -> tuple:
        try:
            with sqlite3.connect(DATABASE) as conn:
                cur = conn.cursor()
                profession = msg.data["job"]
                cur.execute(
                    "SELECT DISTINCT service_cities FROM professional WHERE
profession=?",
                    (profession,),
                )
                rows = cur.fetchall()
                if not rows:
                    return MessageTypes.GET_CITIES_BY_JOB, {
                        StatusMessage.FAILED_TO_GET_CITIES.value: "Server error"
                    }
                unique_cities = set()
                for row in rows:
                    city_string = row[0]
                    if city_string:
                        parts = [c.strip() for c in city_string.split(",")]

```

```

        unique_cities.update(parts)

    cities = sorted(list(unique_cities))
    return MessageTypes.GET_CITIES_BY_JOB, {
        StatusMessage.GOT_CITIES.value: {"cities": cities}
    }
except Exception as e:
    print("Cities find by job dispatcher exception - ", e)
    return MessageTypes.GET_CITIES_BY_JOB, {
        StatusMessage.FAILED_TO_GET_CITIES.value: "Couldn't retrieve cities,
refresh page"
    }

class MinimalProfessionalInfo(MessageHandler):
    def handle(self, msg: Message) -> tuple:
        try:
            with sqlite3.connect(DATABASE) as conn:
                cur = conn.cursor()
                city = msg.data["city"]
                job = msg.data["job"]
                search_term = f"%{city}%"
                cur.execute(
                    """SELECT id, description, rating, hour_price FROM professional
                        WHERE profession=? AND (' || REPLACE(service_cities, ' ', ',') || ')
LIKE ? """,
                    (job, search_term),
                )
                rows = cur.fetchall()
                if not rows:
                    return MessageTypes.GET_MINIMAL_FREELANCER_INFO, {
                        StatusMessage.FAILED_TO_GET_MINIMAL_INFO.value: "Error
fetching info"
                    }
                users = defaultdict(dict)
                for row in rows:
                    id, description, rating, hour_price = row
                    users[id] = {
                        "description": description,
                        "rating": rating,
                        "price": hour_price,
                    }
                if users:
                    dict(users)
                    return MessageTypes.GET_MINIMAL_FREELANCER_INFO, {
                        StatusMessage.GOT_MINIMAL_INFO.value: users
                    }
        except Exception as e:
            print("Minimal professional info error - ", e)
            return MessageTypes.GET_MINIMAL_FREELANCER_INFO, {

```

```

    StatusMessage.FAILED_TO_GET_MINIMAL_INFO.value: "Error fetching
info"
}

```

קביעת תור

סקירת היכולת

יכולת קביעת התור מהווה את הליבה של המערכת, היא מאפשרת סנכרון מלא בין הלקוח לבעל המקצוע. המערכת מנהלת תהליך שמתחיל בחישוב של חלונות זמן פנויים שמתבסס על משך השירות ופגישות קיימות, וממשיך בניהול של בקשת תור הכולל סטטוסים של "ממתין", "מאושר" או "בוטל". הקושי המרכזי במימוש יכולת זו נמצא בצד השרת, הנדרש להתמודד עם אתגרים של מניעת התנגשויות בין משתמשים המנסים להזמין את אותו תור, ניהול בדיקות מול מסד הנתונים, ושליחת כדי לשמור על עקביות הנתונים אצל כל הקצוות WebSockets הודעות עדכון סטטוס דרך פרוטוקול בו-זמנית.

הערה לגבי תיעוד המימוש: בדומה ליכולת החיפוש, בחרתי שלא לפרט את מימוש הקוד של צד הלקוח בתהליך זה. בעוד שהלקוח מספק ממשק ויזואלי אינטראקטיבי לבחירת זמנים והזנת פרטים, המוח המבצע את חישובי הזמינות, את אימות התקינות ואת שמירת הנתונים המאובטחת נמצא כולו בצד השרת. כדי להבטיח את אמינות המערכת, כל לוגיקת קבלת ההחלטות והסנכרון בין ובמסד הנתונים, ולכן מצאתי לנכון להתמקד בפירוט טכני של רכיבי Python-המשתמשים מרוכזת ב השרת המהווים את התשתית הקריטית ליכולת זו.

קוד של יכולת קביעת התורים

```

class ChangeAppointmentStatusDispatcher(MessageHandler):

    def handle(self, msg: Message) -> tuple:

        try:

            with sqlite3.connect(DATABASE) as conn:

                cur = conn.cursor()

                apps = msg.data["appointments"]

                worked = []

                failed = []

                first_key = int(list(apps.keys())[0])

                cur.execute(

                    "SELECT u.full_name, a.professional_id FROM appointments a JOIN
users u ON a.professional_id = u.id WHERE a.id = ?",

                    (first_key,),

                )

                res = cur.fetchone()

                if not res:

                    prof_name = "Freelancer"

                    p_id = None

                else:

                    prof_name, p_id = res

                status_map = {"accepted": "Confirmed", "cancelled": "Cancelled"}

                for app_id, status_input in apps.items():

                    try:

                        target_status = status_map.get(status_input)

                        if target_status == "Confirmed":

                            if self._is_time_slot_taken(cur, app_id):

```

```

cur.execute(
    "DELETE FROM appointments WHERE id=?", (app_id,)
)
failed[app_id] = "Slot already taken"
continue

cur.execute(
    "SELECT customer_id FROM appointments WHERE id=?",
(app_id,)
)
row = cur.fetchone()
if not row:
    failed[app_id] = "Appointment not found"
    continue
u_id = row[0]
if target_status == "Confirmed":
    cur.execute(
        "UPDATE appointments SET status=? WHERE id=?",
        (target_status, app_id),
    )
    msg_text = f"{prof_name} has accepted your appointment"
else:
    cur.execute(
        "DELETE FROM appointments WHERE id=?", (app_id,)
    )
    msg_text = f"{prof_name} has declined your appointment"

cur.execute(
    """"INSERT INTO notifications (user_id, message, is_read,
created_at, from_id)

```

```

VALUES (?, ?, 0, ?, ?)""",
    (
        u_id,
        msg_text,
        datetime.now().strftime("%Y-%m-%d %H:%M:%S"),
        p_id,
    ),
)
worked.append(app_id)
except Exception as inner_e:
    failed[app_id] = str(inner_e)
    continue
conn.commit()
return MessageTypes.UPDATE_APPOINTMENTS_STATUS, {
    StatusMessage.UPDATED_APP_STATUS.value: "Batch processed",
    "results": {"processed": worked, "Failed": failed},
}
except Exception as e:
    print(f"Appointment Status change Dispatcher Error: {e}")
    return MessageTypes.UPDATE_APPOINTMENTS_STATUS, {
        StatusMessage.FAILED_TO_UPDATE_APP_STATUS.value: "System
error"
    }
def _is_time_slot_taken(self, cur, app_id):
    cur.execute(
        "SELECT professional_id, date, start_time, end_time FROM appointments
WHERE id=?",
        (app_id,),
    )

```

```

target = cur.fetchone()
if not target:
    return False

p_id, a_date, a_start, a_end = target
query = """SELECT id FROM appointments WHERE professional_id = ? AND
date = ?
AND status = 'Confirmed' AND id != ? AND (? < end_time AND start_time <
?)
"""
cur.execute(query, (p_id, a_date, app_id, a_start, a_end))
return cur.fetchone() is not None

```

```

class AppointmentStartTimesDispatcher(MessageHandler):
    def handle(self, msg: Message) -> tuple:
        try:
            with sqlite3.connect(DATABASE) as conn:
                cur = conn.cursor()
                pro_id = msg.data["id"]
                cur.execute(
                    "SELECT avg_job_duration, start_time, end_time FROM professional
WHERE user_id=?",
                    (pro_id,),
                )
                row = cur.fetchone()
                if not row:
                    return MessageTypes.GET_APPOINTMENT_TIMES, {
                        StatusMessage.FAILED_TO_GET_APPOINTMENT_TIMES.value:
                        "Couldn't find freelancer"
                    }
                job_duration, start_time, end_time = row

```

```

cur.execute(
    """SELECT start_time, date FROM appointments WHERE
professional_id=?

    AND date BETWEEN date('now') AND date('now', '+1 month')

    AND status='Confirmed' ORDER BY date ASC; """
    (pro_id,),
)
existing_appointments = {}
for row in cur.fetchall():
    time, date = row[0], row[1]
    if date not in existing_appointments:
        existing_appointments[date] = []
        existing_appointments[date].append(time)
existing_appointments = dict(existing_appointments)
h, m = map(int, job_duration.split(":"))
duration_minutes = (h * 60) + m
app_times = {}
current_date = datetime.now() + timedelta(days=1)
for _ in range(0, 35):
    date_str = current_date.strftime("%Y-%m-%d")
    booked_slots = existing_appointments.get(date_str, [])
    app_times[date_str] = _calculate_free_day_working_hours(
        start_time, end_time, duration_minutes, booked_slots
    )
    current_date = current_date + timedelta(days=1)
return MessageTypes.GET_APPOINTMENT_TIMES, {
    StatusMessage.GOT_APPOINTMENT_TIMES.value: app_times
}

```

```

    }

except Exception as e:
    print("Appointment time dispatcher error - ", e)
    return MessageTypes.GET_APPOINTMENT_TIMES, {
        StatusMessage.FAILED_TO_GET_APPOINTMENT_TIMES.value: "Issue
fetching the appointment dates and times"
    }

def _calculate_free_day_working_hours(
    start_time, end_time, job_duration, existing_appointments=None
) -> list:
    if existing_appointments is None:
        existing_appointments = []
    format = "%H:%M"
    current_slot = datetime.strptime(start_time, format)
    end_of_day = datetime.strptime(end_time, format)
    available_times = []
    while current_slot + timedelta(minutes=job_duration) <= end_of_day:
        slot_str = current_slot.strftime(format)
        if slot_str not in existing_appointments:
            available_times.append(slot_str)
            current_slot += timedelta(minutes=job_duration)
    return available_times

class BookAppointmentDispatcher(MessageHandler):
    def handle(self, msg: Message) -> tuple:
        try:
            with sqlite3.connect(DATABASE) as conn:
                cur = conn.cursor()

```

```

app = msg.data["app"]

cur.execute(
    "SELECT avg_job_duration FROM professional WHERE user_id = ?",
    (app["prof_id"],),
)
row = cur.fetchone()
if not row:
    return MessageTypes.MAKE_APPOINTMENT, {
        StatusMessage.FAILED_TO_BOOK_APPOINTMENT.value:
"Freelancer not found"
    }
duration_str = row[0]
h, m = map(int, duration_str.split(":"))
duration_minutes = (h * 60) + m
end_time_str = self.calculate_end_time(
    app["start_time"], duration_minutes
)
cur.execute(
    "SELECT start_time FROM appointments WHERE date = ? AND
professional_id = ? AND status = 'Confirmed'",
    (app["date"], app["prof_id"]),
)
start_times_booked = [row[0] for row in cur.fetchall()]
available_slots = _calculate_free_day_working_hours(
    app["start_time"],
    end_time_str,
    duration_minutes,
    start_times_booked,

```

```

)

if app["start_time"] not in available_slots:

    return MessageTypes.MAKE_APPOINTMENT, {
        StatusMessage.FAILED_TO_BOOK_APPOINTMENT.value: "Time
slot is no longer available."
    }

clean_details = bleach.clean(
    app["details"], tags=[], attributes={}, strip=True
)

clean_address = bleach.clean(
    app["address"], tags=[], attributes={}, strip=True
)

cur.execute(
    """
    INSERT INTO appointments (professional_id, customer_id, date,
start_time, end_time, address, details, status)
    VALUES (?, ?, ?, ?, ?, ?, ?, 'Requested')""",
    (
        app["prof_id"],
        app["user_id"],
        app["date"],
        app["start_time"],
        end_time_str,
        clean_address,
        clean_details,
    ),
)

```



```

cur.execute(
    """INSERT INTO notifications (user_id, message, is_read, created_at)
       VALUES (?, ?, 0, ?)""",
    (
        app["prof_id"],
        "You have a pending appointment waiting for you",
        datetime.now().strftime("%Y-%m-%d %H:%M:%S"),
    ),
)
conn.commit()

return MessageTypes.MAKE_APPOINTMENT, {
    StatusMessage.BOOKED_APPOINTMENT.value: "Appointment
requested successfully"
}

except Exception as e:
    print(f"CRITICAL: Appointment booking failure - {e}")
    return MessageTypes.MAKE_APPOINTMENT, {
        StatusMessage.FAILED_TO_BOOK_APPOINTMENT.value: "Server
error, please try again."
    }

def calculate_end_time(self, start_time_str, duration_minutes):
    fmt = "%H:%M"
    start_dt = datetime.strptime(start_time_str, fmt)
    end_dt = start_dt + timedelta(minutes=duration_minutes)
    return end_dt.strftime(fmt)

```

בדיקות

בדיקות הרשמה (User + בעל מקצוע)

1 הזנת כל השדות הנדרשים הצלחה

מטרה: לבדוק שהרשמה תקינה עובדת כשהלקוח מזין נתונים נכונים.

מה בוצע בפועל: ניסיתי להכניס את כל השדות הנדרשים עם התקפות SQLInjection ומתקפת XSS

2 חוסר בשדה חובה אחד

מטרה: לוודא שמתקבלת הודעת שגיאה מתאימה ושאי אפשר להירשם בלי כל השדות.

מה בוצע בפועל: ניסיתי באמת את הבדיקה

3 סיסמה ואימות סיסמה לא תואמות

מטרה: לבדוק שהשרת מגיב בשגיאה לפני שליחת אימייל אימות.

מה בוצע בפועל: מה שכתוב הנ"ל

4 אימייל פורמט לא תקין

מטרה: לבדוק שרק אימיילים תקינים מתקבלים.

מה בוצע בפועל: ניסיתי אימיילים עם פורמטים לא תקינים וגם אימיילים לא קיימים

5 אימייל שכבר רשום במערכת

מטרה: לבדוק שהמערכת מונעת כפילויות ומחזירה שגיאה ברורה.

מה בוצע בפועל: מה שכתוב הנ"ל

6 קבלת קוד אימות למייל הצלחה

מטרה: לבדוק שהשרת שולח קוד ושהקוד תקין.

מה בוצע בפועל: בדקתי שזה באמת עובד

7 ניסיון להזין קוד אימות לא נכון

מטרה: לבדוק מערכת ניסיונות (כמות מוגבלת) ושיש הודעת שגיאה נכונה.

מה בוצע בפועל: ניסיתי להציף את השרת עם ניסיונות אבל הוא חוסם (Rate Limiting)

8 הזנת קוד נכון אחרי טעות ראשונה

מטרה: לבדוק שהמערכת ממשיכה כרגיל לאחר טעות אחת.

מה בוצע בפועל: מה שכתוב הנ"ל

9. להכניס סיסמה ואימות סיסמה בלי לעבור את תהליך האימות

מטרה: לבדוק שהשרת מנהל מי יכול לאפס סיסמה ומי לא

בדיקות התחברות (Login)

1 התחברות עם פרטים נכונים

מטרה: לוודא שהשרת מזהה את המשתמש ושולח את סוג הלקוח + נתוני הפרופיל.

מה בוצע בפועל: מה שכתוב הנ"ל

2 התחברות עם אימייל נכון וסיסמה שגויה

מטרה: לבדוק הודעת שגיאה מתאימה.

מה בוצע בפועל: בדקתי שאכן הודעה גנרית נשלחת אל הלקוח

3 התחברות עם אימייל שלא קיים

מטרה: לבדוק שמידע שגוי לא חושף מידע על משתמשים קיימים.

מה בוצע בפועל: מה שכתוב הנ"ל

4 חסימת לקוח אחרי 5 ניסיונות כושלים

מטרה: לבדוק מנגנון הגנה מ-Brute Force.

מה בוצע בפועל: ניסיתי להספיק את השרת עם הודעות ולראות שהוא חוסם את החיבור שלי

5 התחברות לאחר שנחסם ל-X דקות

מטרה: לוודא שהחסימה זמנית ושמתחררת בזמן הנכון.

מה בוצע בפועל: מה שכתוב הנ"ל

בדיקות שכחתי סיסמה

1 בקשה לשכחתי סיסמה עם אימייל קיים

מטרה: לבדוק שנשלח אימייל עם קוד התאוששות.

מה בוצע בפועל: מה שכתוב הנ"ל

2 בקשה לשחזור עם אימייל שלא קיים

מטרה: לבדוק התנהגות זהירה שלא נחשף מידע מיותר.

מה בוצע בפועל: מה שכתוב הנ"ל

3 הזנת קוד שגוי פעם אחת

מטרה: לבדוק הודעת שגיאה תקינה.

מה בוצע בפועל: בדקתי וראיתי שההודעה חושפת פרטיות של משתמשים אז הפכתי את ההודעה לגנרית

5 הזנת קוד נכון ושינוי סיסמה

מטרה: לבדוק עדכון סיסמה במסד נתונים.

מה בוצע בפועל: מה שכתוב הנ"ל

בדיקות עמוד בית משתמש

1 טעינת כל הפרטים הנכונים של המשתמש

מטרה: לוודא שהשרת מחזיר נתונים בלי חסר/עודף.

מה בוצע בפועל: מה שכתוב הנ"ל

2 הצגת כל התורים לחודש הקרוב

מטרה: לוודא שהלקוח רואה רק את התורים שלו.

מה בוצע בפועל: ניסיתי להציג את התורים אבל ניתקלתי בבעיות, אז דיבגתי עד שהגעתי לשורש הבעיה

3. הצגת כל ההודעות

מטרה: לוודא שהלקוח מקבל את כל ההודעות ששמורות לו במערכת ומוצגות בתצוגה של חדש/ישן.

מה בוצע בפועל: הצגתי את ההודעות וכולן הוצגו בתור ישנות אז דיבגתי עד שגיליתי שהבעיה נבעה מתקלה שנובעת מפיתוח ב-React ב-StrictMode אז מימשתי מחלקה MarkReadMessageDispatcher שתטפל בזה

בדיקות עמוד בית בעל מקצוע

1 טעינת פרטים מקצועיים, שעות זמינות, תורים

מטרה: לוודא שהשרת מחזיר מידע נכון.

מה בוצע בפועל: מה שכתוב הנ"ל

2 תצוגת התראות לבקשות עבודה

מטרה: לוודא שהמערכת מציגה בקשות חדשות לתורים חדשים.

מה בוצע בפועל: באמת הייתה לי כאן בעיה עם תצוגת הודעות חדשות כמו שכתבתי מעל

3. דחיית אישור תור

מטרה: לוודא שהמערכת קולטת את כל ההחלטות של בעל המקצוע

מה בוצע בפועל: מה שכתוב הנ"ל

4. אישור תורים על אותם שעות

מטרה: לבדוק שהשרת מונע מצב של Double Booking

מה בוצע בפועל: בהתחלה נתקלתי בהרבה בעיות עם Double Booking והייתי צריך לשנות מספר דברים בקוד כדי לפתור את זה, בין עם זה הפורמט שאני שומר את שעות העבודה ובין אם זה הבדיקות שעשיתי לפני אישור תור.

מנגנון חיפוש (משתמש בלבד)

2 חיפוש לפי תחום ועיר

מטרה: לוודא שמופיעים רק בעלי מקצוע שזמינים בעיר שנבחרה מהסוג בעל מקצוע שנבחר.

מה בוצע בפועל: מה שכתבתי הנ"ל

3 כניסה לעמוד של בעל מקצוע

מטרה: לוודא שאינפורמציה פרטית לא מוצגת למשתמשים אחרים.

מה בוצע בפועל: מה שכתוב הנ"ל

4 קביעת תור בשעה פנויה

מטרה: לבדוק ששעה פנויה באמת נקבעת.

מה בוצע בפועל: מה שכתוב הנ"ל

5 ניסיון קביעת תור לשעה תפוסה

מטרה: לבדוק שהשרת מונע התנגשות.

מה בוצע בפועל: השרת מלחתכילה לא נותן את האופציה מוצג למשתמש וגם אם איכשהו עוקף את

זה אז השרת עושה בדיקות נגד מסד הנתונים

6 שליחת בקשה שמגיעה להתראות לבעל מקצוע

מטרה: לוודא זרימת נתונים "משתמש -> שרת -> בעל מקצוע".

מה בוצע בפועל: מה שכתוב הנ"ל

7 לבחור עיר בוא אין בעל מקצוע מהסוג שנבחר

מטרה: לוודא שהשרת מחזיר רשימה רק של ערים שבעלי המקצוע מהסוג שנבחר עובדים בהם

מה בוצע בפועל: מה שכתוב הנ"ל

בדיקות תורים (Scheduling)

1 בעל מקצוע מאשר תור

מטרה: לוודא שהמשתמש מקבל הודעה מהמערכת ושהתור מתווסף ליומן של המשתמש ושל בעל המקצוע.

מה בוצע בפועל: מה שכתוב הנ"ל

2 בעל מקצוע דוחה תור

מטרה: לוודא שהמשתמש מקבל הודעה מהמערכת על דחיית התור ועדכון מהמסד הנתונים.

מה בוצע בפועל: מה שכתוב הנ"ל

בדיקות אבטחה בסיסיות (Black-Box)

1 ניסוי לשליחת בקשת API עם נתונים לא מוצפנים

מטרה: לוודא שהשרת מסרב לקלוט מידע כזה.

מה בוצע בפועל: מה שכתוב הנ"ל

2 ניסיון לקבל מידע של משתמש אחר

מטרה: לבדוק שאין חשיפת מידע בין חשבונות.

מה בוצע בפועל: מה שכתוב הנ"ל

3 ניסיון לקבוע תור לבעל מקצוע בלי הרשאות מתאימות

מטרה: לוודא שמשתמש לא יכול להסוות את עצמו כבעל מקצוע.

מה בוצע בפועל: מה שכתוב הנ"ל

4 ניסיון DDOS

מטרה: לנסות להציף את השרת עם בקשות ולראותו שהוא לא מתרסק.

מה בוצע בפועל: מה שכתוב הנ"ל

5 ניסיון הזרקת SQL

מטרה: לראות שהשרת לא נותן להזרקת SQL.

מה בוצע בפועל: מה שכתוב הנ"ל

7 ניסיון לעקוף Rate Limits

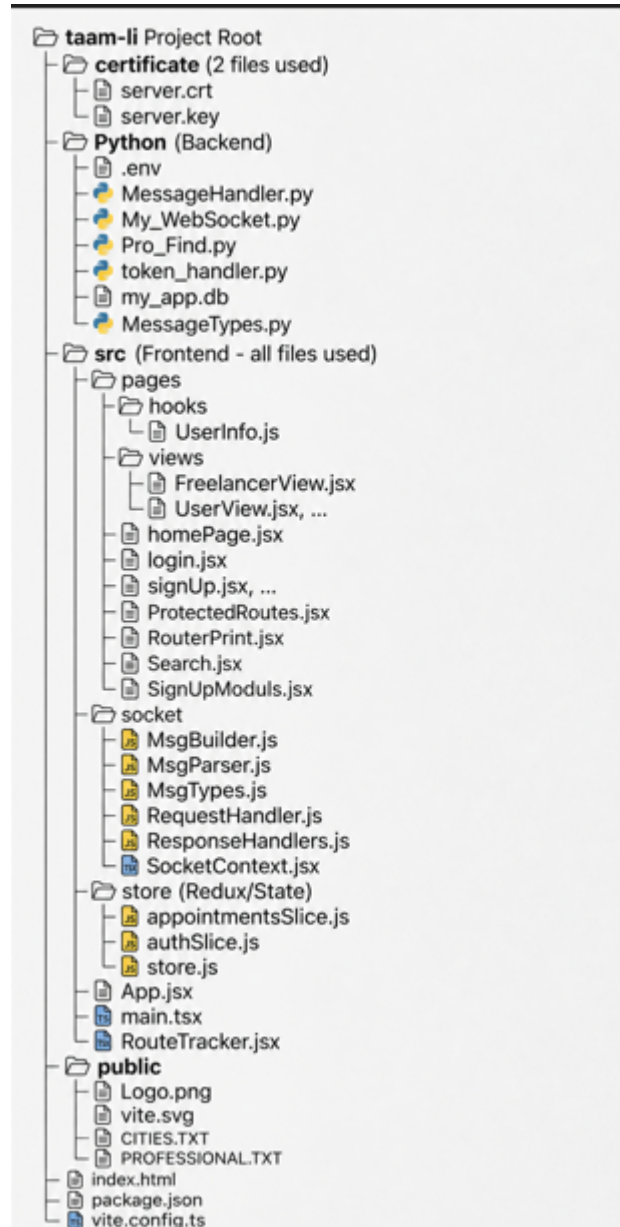
מטרה: לבדוק שלקוח לא יכול להפציץ את השרת בבקשות.

מה בוצע בפועל: מה שכתוב הנ"ל

מדריך למשתמש

מערכת קבצים

המערכת בנויה בארכיטקטורת Server-Client כך שיש הפרדה מוחלטת בין הקבצים של השרת לקבצי הלקוח.



פירוט הסביבה הנדרשת

כדי להריץ את מערכת תאם לי יש לוודא שהכלים הבאים מותקנים

- גרסה +3 של Python - להרצת השרת
- Node.js & npm - לניהול החבילות והרצת הלקוח בדפדפן.
- דפדפן מודרני - Chrome.
- עורך טקסט עם הרשאות מנהל - לצורך עריכת קבצי המערכת (hosts)

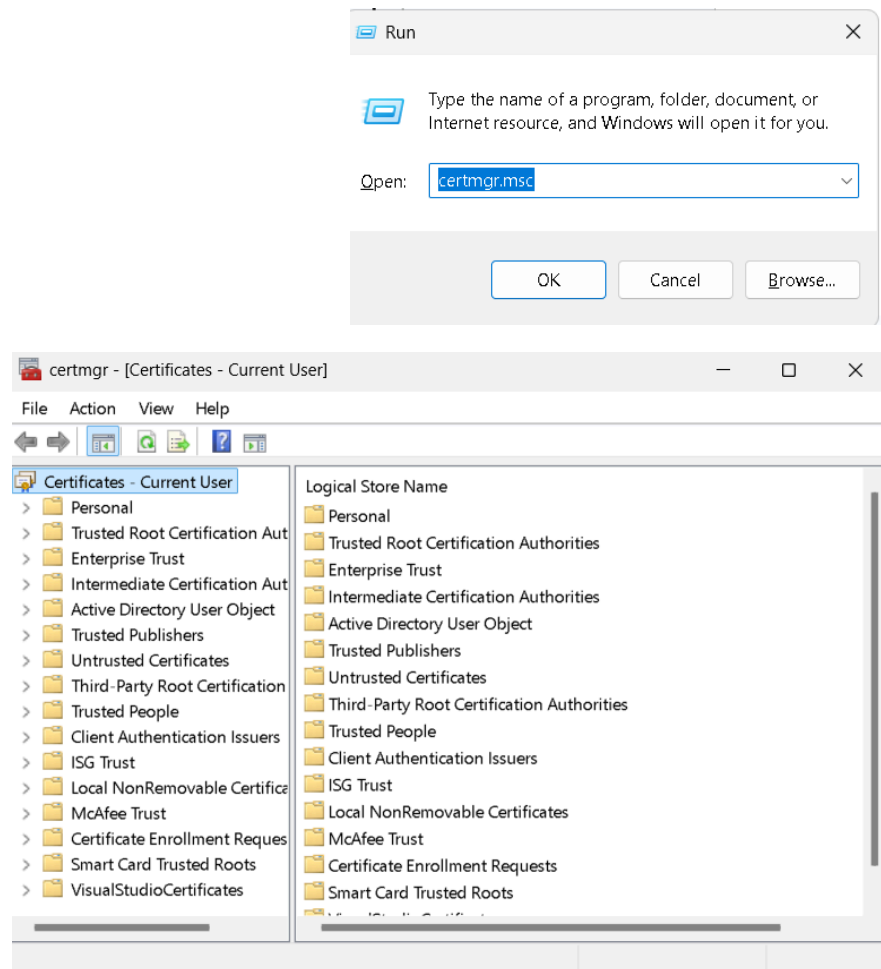
מיקומי קבצים קריטיים

- קובץ הניתוב
- C:\Windows\System32\drivers\etc
- קובץ ניהול תעודות אבטחה שניתן למצוא בהקלדת certmgr.msc ולמצוא את תיקיית Trusted Root Certification Authorities
- קובץ בסיס הנתונים שנמצא בתיקייה עם קוד השרת
- קובץ env. בתיקיית ה-Python
- קובצי ה-cert.pem ו-key.pem הממוקמים בתיקיית השרת לצורך הצפנת התקשורת.

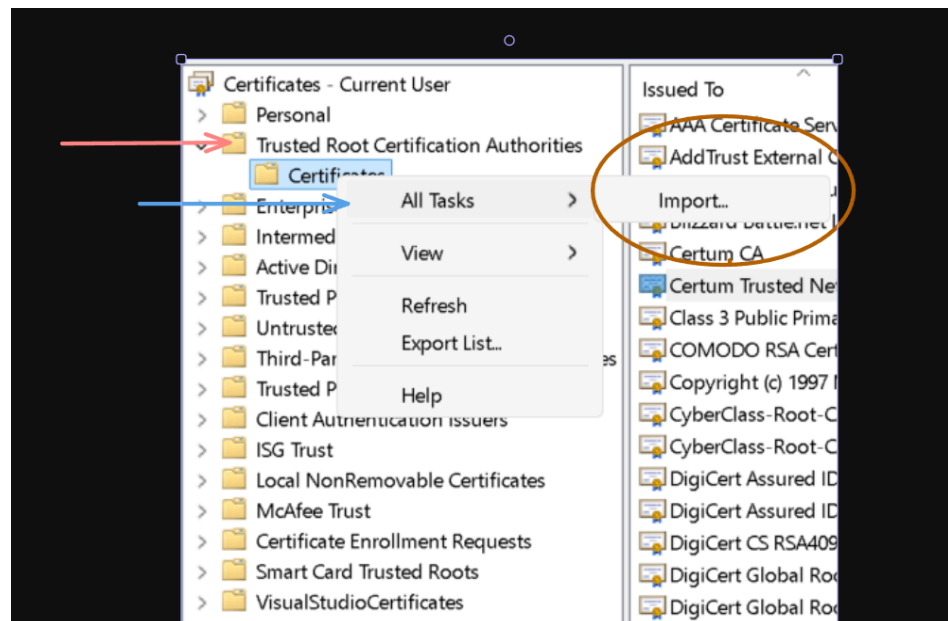
מה לעשות לפני הרצת הפרויקט?

האלעת קובץ חותם מורשה

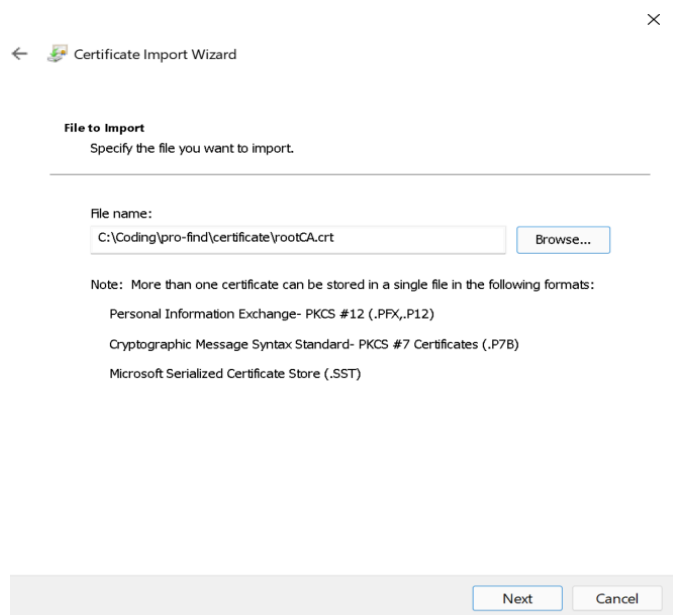
1. פתיחת אפליקציית ה-certmgr.msc

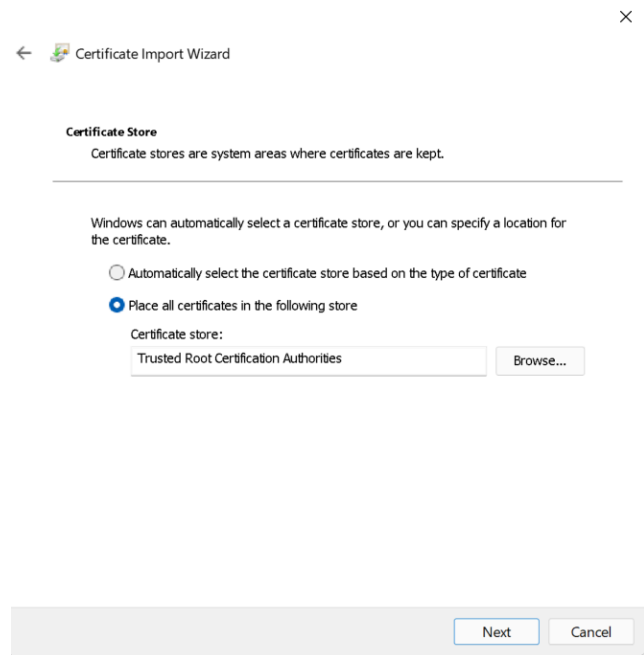


2. לנווט לתיקיית ה-Trusted Root Certification Authorities, שם ללחוץ לחיצה ימנית על Certificates. בpopup שמופיע יש ללחוץ על All Tasks ואז על Import



3. אז יש להמשיך עם המסך שמופיע ולבחור את ה-Path של ה-RootCA (מה שחתם על התעודה של השרת) ברגע שהאפליקציה תבקש ואז להמשיך ולאשר את העמוד הבא ולסיים את התהליך.





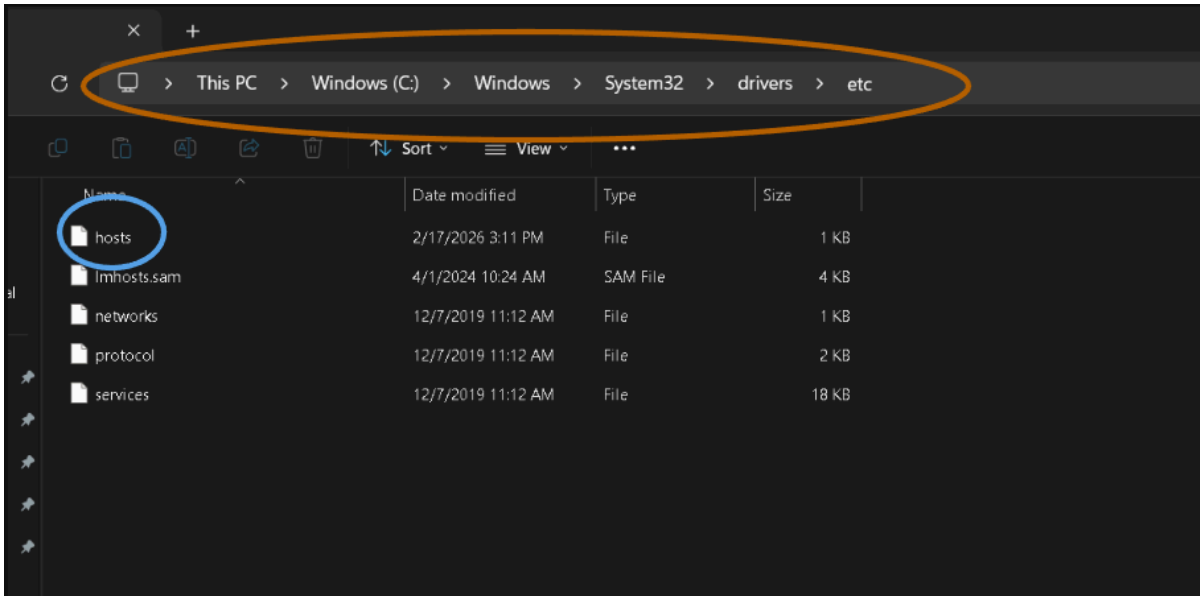
מה שהתהליך הזה בעצם עשה זה שם בספריית ה- Trusted Root Certificate Authorities את הקובץ RootCA. זה הכרחי לאתר שלי מכיוון ובשביל יישום פרוטוקול TLS הלקוח מבקש לראות את התעודה של השרת. כשהלקוח מקבל אותה הוא בודק מי חתם על הקובץ התעודה הזאת ובודק אם החותם נמצא באמת בספרייה הזאת של Trusted Root Certificate Authorities כדי לדעת אם הוא יכול לבטוח בשרת.

עדכון קובץ hosts

1. יש לפתוח את הסייר קבצים ולנווט לתיקייה הזאת:

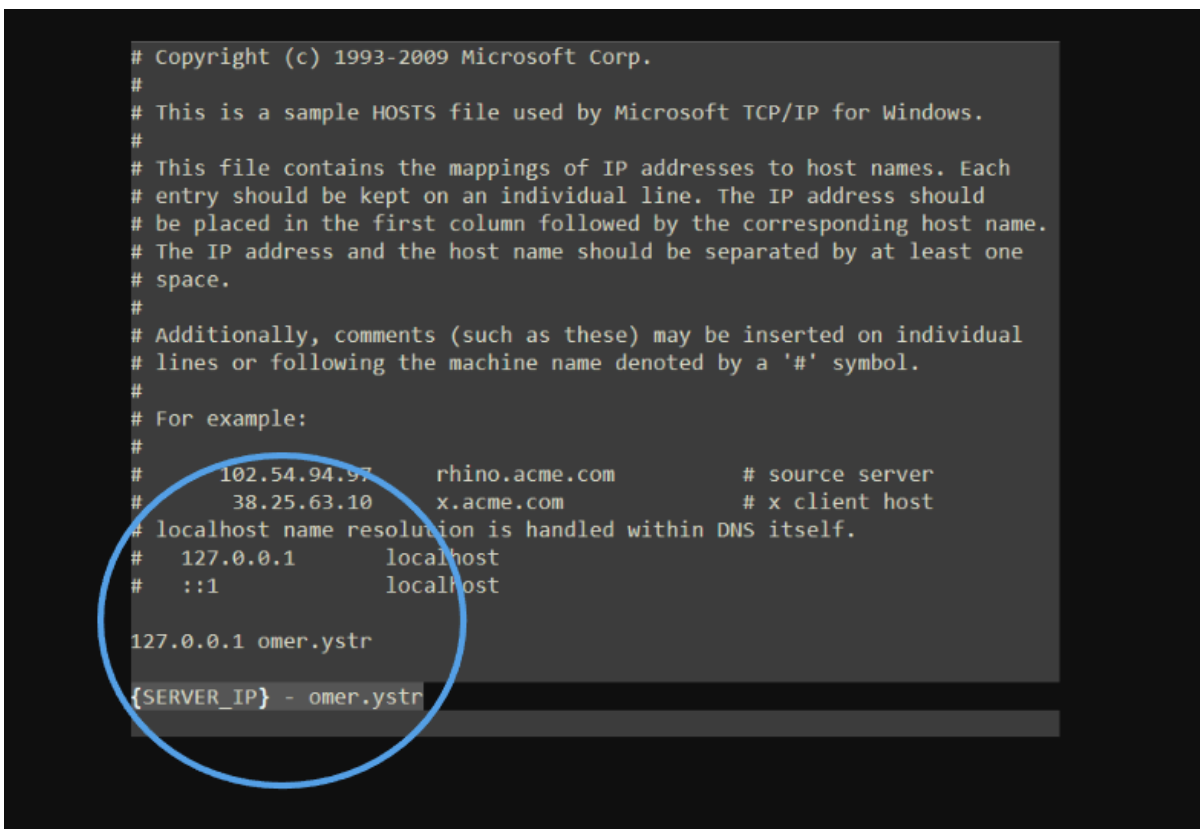
C:\Windows\System32\drivers\etc

ולפתוח את הקובץ על הרשאות מנהל



2. בתוך הקובץ יש לכתוב את ה-IP של השרת ואז רווח ואז את המילה omer.ystr כמו

שהתמונה מראה



מיפוי סטטי - קובץ ה-hosts הוא הראשון בשרשרת של מערכת ההפעלה לתרגום שמות דומיין לכתובות IP. על ידי הוספת השורה, מוגדר למחשב שכל פנייה לכתובת omer.ystr תופנה ישירות לשרת הפרויקט, ללא צורך בשאילתת DNS חיצונית.

תאימות לתעודה שנשלחת בפרוטוקול TLS - בתקשורת מוצפנת, השרת שולח ללקוח תעודה דיגיטלית אחד השדות הקריטיים בתעודה הוא ה-CN או ה-SAN שמגדירים באיזה כתובת ניתן למצוא את השרת. בתעודה של הפרויקט שלי, הישות המורשית מוגדרת כ-omer.ystr. אם הלקוח ינסה להתחבר לשרת דרך כתובת ה-IP (למשל 127.0.0.1), הדפדפן יזהה חוסר התאמה בין הכתובת שהוקלדה לבין הכתובת שרשומה בתעודה, ויחסום את החיבור מחשש להתקפת MITM לכן, השימוש בשם המפורש omer.ystr הוא הדרך היחידה לאמת את זהות השרת ולאפשר את לחיצת היד של ה-TLS.

ארכיטקטורה מינימלית הנדרשת

- **קובץ ה-Hosts:** בנתיב `C:\Windows\System32\drivers\etc\hosts`
- **ניהול תעודות:** ממשק ה-`certmgr.msc` (תחת Trusted Root Certification Authorities)
- **קובץ ה-.env:** ממקם בתיקיית ה-`(Python Server)` ומכיל הגדרות רגישות: `server_email`, `pepper` (להצפנת סיסמאות), ו-`authentication_pas`.
- **תעודות אבטחה:** קובצי ה-`cert.pem` ו-`key.pem` בתיקיית השרת.
- **מסד נתונים:** קובץ ה-`SQLite3` הממוקם בתיקיית השרת.

נתונים התחלתיים

- **משתמשים:** המערכת אינה מגיעה עם משתמש מנהל מובנה מראש. על המשתמש הראשון לבצע תהליך הרשמה דרך ממשק הלקוח כדי להיווצר במסד הנתונים.
- **הגדרות שרת:** יש לוודא שקובץ ה-`env` מעודכן עם פרטי המייל וה-`Pepper` לצורך תפקוד תקין של מנגנוני האימות והאבטחה.

רשת (Network)

- **מיפוי כתובת:** כתובת ה-IP המוגדרת בקובץ ה-`Hosts` תלויה בכתובת ה-IP הנוכחית של השרת ברשת.
- **פורטים (Ports):** המערכת עושה שימוש בפורט 1111 עבור צד השרת וצד הלקוח.
- **פרוטוקול:** התקשורת מתבצעת על גבי `HTTPS` מוצפן (`TLS 1.2/1.3`).

ארכיטקטורה מינימלית נדרשת

המערכת בנויה בארכיטקטורת Client-Server המפרידה בין תצוגת המידע ללוגיקה העסקית:

1. `React`: ממשק המשתמש הפונה לשרת בבקשות API מאובטחות.
2. `Python`: שרת האחראי על עיבוד הנתונים, אימות מול ה-`Client`, ושליחת מיילים מאובטחת.
3. `SQLite3`: שכבת אחסון הנתונים המבוססת על קובץ רלציוני מקומי בשרת.
4. `Security Layer`: שימוש בתעודות `SSL/TLS` וקובץ `Hosts` לצורך אימות זהות השרת והגנה על פרטיות המשתמשים.

רפלקציה

לאחר מימוש הפרויקט הזה אני יכול להגיד שזה היה אתגר. התחלתי בחלום מאוד גדול של לכתוב מערכת שמשלבת מספר טכנולוגיות חזקות בשוק ולהפיק מוצר שיכול להיות משווק ורווחי מאוד בשוק העבודה. התחלתי בתכנון של הפרויקט שבו ממש חלמתי על איך אעשה הכל וחלמתי גבוהה. עם זאת ידעתי שזה עומד להיות מאוד מאתגר בגלל מגבלת הזמן וחוסר הניסיון בטכנולוגיה. אף פעם לא תיכנתתי אתר, לא ידעתי JavaScript ובטח שלא ידעתי איך כותבים בסינטקס JSX. בנוסף היו עוד הרבה טכנולוגיות שלא הכרתי כשלקחתי על עצמי את הפרויקט אבל ידעתי שאני רוצה פרויקט יפה ושימושי שיכול לפתור בעיות שאני נתקל בהם ביום יום אז לקחתי את זה על עצמי.

התחלתי בתכנון של מערכת ענקית שמשלב גם צ'אטים, דירוגים, אפשרויות ביטולי תורים, חסימת ימי עבודה ועוד הרבה פיצ'רים שבטח אחרי שקוראים את תיק הפרויקט הזה מבינים שנשאר רק בחלומות. לאחר שסיימתי את התכנון התחלתי ללמוד על Syntax ב-React ואיך לנהל מסדי נתונים איך לנהל משתנים ב-React, ואיך פרוטוקול WebSocket עובד. התחלתי לממש את הפרוטוקול בצעמי ואחרי שכל כך שמחתי שסיימתי לממש את לחיצת היד וזהו עכשיו אני יכול להתחיל את הפרויקט גיליתי שיש דרך שכל הודעה צריכה להישלח מה שגרר אותי לעוד מחקר ועוד שבוע של פיתוח. אחרי שסיימתי פשוט צללתי למים בצד הלקוח והתחלתי לכתוב פשוט את עמוד ההתחברות. לא הצלחתי לכתוב כלום, כלום לא עבד ולקח לי זמן לקלוט איך זה עובד ומה פתאום לפרמטרים קוראים Props ולפעולות קוראים Components. לאט לאט התחלתי לתפוס תאוצה והצלחתי לסיים את עמוד ההתחברות והוא נראה מאוד יפה

. החלטתי אבל שאני עכשיו לוקח הפסקה מכתובת ה-UI והולך להוסיף את הצפנת ה-TLS אז הוספתי ויצרתי Certificate משלי, הרצתי, לא עבד. הרצתי עוד פעם, לא עבד. התחלתי לחקור כל מה שזה יכול להיות. במשך שבועיים לא כתבתי שורת קוד אלא רק חקרתי איך פותרים וניסיתי כל מיני דברים שונים. בסוף גיליתי שיצירת ה-Certificate שלי הייתה פגומה, שיניתי את זה, הוספתי את ה-Domain שלה לקובץ hosts ופתאום הייתה הצפנה. מכאן התחלתי לכתוב והמשכתי בצורה רציפה, כל פעם מממש Dispatcher בשרת ואז מממש את צד הלקוח.

ניתקלתי במהלך כתיבת הפרויקט במספר דילמות של אבטחה ובמספר StepBacks שגרמו לי לשנות חלקים גדולים של הקוד כדי לסדר אותם, אבל סך הכל הכתיבה הייתה זורמת והצלחתי לכתוב גם בשפה החדשה באופן רציף וגם את השרת שמנהל כל כך הרבה לוגיקה בצורה יעילה. הפרויקט הזה התחיל מחלום להשתמש בטכנולוגיות לא מוכרות ולבצע איתם מוצר מושקע שרץ ועובד וזה מה שקרה. השתמשתי במספר טכנולוגיות חדשות בפרויקט הזה כגון, Certificates, React, Redux, WebSocket, TLS, ועוד.

בפרויקט הזה למדתי המון על עולם המחשבים ובעזרת כל ההתפקות האפשריות שהייתי צריך לחשוב על האתר פיתחתי מאוד את דרך חשיבת ה-"מגן סייבר" שלי. במהלך הפרויקט גם הכרתי את עצמי לתכנת בצורה שהכי עוקבת אחרי עקרונות SOLID שיש. בדרך כלל כשכתבתי קוד הוא היה הופך למסורבל והייתי רק מחכה להיפטר מהפרויקט מרוב הבלאגן אבל בפרויקט הזה אני מרגיש שממש דחפתי את עצמי מחוץ לאזור הנוחות וקודדתי בצורה הכי חכמה ויעילה שיש ואם עכשיו יאמרו לבן אדם זר להמשיך לכתוב את הפרויקט מאיפה שעצרתי אני מאמין שתוך 15 דקות יוכל להבין בדיוק את זרימת המידע ולהמשיך מאיפה שאני עצרתי. בנוסף הייתי תלמיד שבדרך כלל רץ לעזרה כשמשוה לא עובד אבל בפרויקט הזה גם הכרתי את עצמי להתמודד עם בעיות בכוחות עצמי. כשנתקלתי בבעיה ישבתי ודיבגתי עד שמצאתי אותה, לא התקשרתי לחבר ולא ביקשתי עזרה. אני מרגיש שסך הכל יכולות הניתוח ופיתוח מערכות שלי עלו רמה אחרי הפרויקט הזה.

כשאני מסתכל אחורה על תהליך יישום הפרויקט שלי כן הייתי משנה דברים. הייתי מוותר על כל הרקע התאורטי שלמדתי לפני כתיבת צד הלקוח בשפה זרה ופשוט מתחיל עם זה (כי בסופו של דבר כל מה שקראתי לא תרם לי בכלום לפי הרגשתי). בנוסף הייתי קורה יותר לעומק על טכנולוגיות מסוימות שהשתמשתי בפרויקט שהיו מיותרות, כלומר טכנולוגיות שהשקעתי זמן בפיתוח שלהם כדי

שיקלו עלי במימוש האפליקציה אבל בסופו של דבר לא תרמו בהרבה. במידה והיו לי עוד משאבים\זמן הייתי מוסיף מערכת צ'אטים, לוח שנה מסודר למשתמשים, אופציות לשינוי תורים, אופציות לשינוי נתונים אישיים, ובטוח עוד מספר פיצ'רים שאני לא נזכר בהם עכשיו. אני הרגשתי שבפרויקט הזה הייתי במרוץ נגד הזמן. כלומר היה לי את כל הזמן שהייתי צריך לסיים את הפרויקט שעשיתי (וכך באמת עשיתי) אבל הרגשתי שהיו עוד כל כך הרבה דברים שהייתי רוצה לעשות שפשוט לא מצאתי את הזמן לעשות. הייתי רוצה מערכת צ'אטים חיה לתקשר עם בעלי מקצוע ומערכת דירוגים משוכללת שאפשר להשאיר הרבה אפשרויות ואם בודקים בטבלאות ששמורות אצלי בקובץ מסד הנתונים אפשר לראות שבניתי אותם, כלומר ממש חשבתי שאצליח לעשות את זה. אבל בסופו של דבר כמו שלכל טכנולוגיה יש מגבלה ככה גם לכל תלמיד, והמגבלה שלי כאן הייתה הזמן שברשותי.

לאחר מימוש הפרויקט הזה והכרה של כל הטכנולוגיות שיישמתי הייתי האמת רוצה לחקור את כל הטכנולוגיות שלא עשיתי בהם שימוש יותר לעומק. בכל טכנולוגיה שבחרתי בחרתי מכיוון והמתחרות שלהם היו פחות מתאימות לי לפרויקט ולכן פשוט לא נבחרו, אבל מעניין אותי מעבר ללמה לא נבחרו איך הן עובדות ומה הפתרונות שהן מציעות לי. לדוגמה, בפרויקט שלי שימוש ב-Token היה יותר חכם משימוש ב-Session מבחינת הניהול של זה בפרויקט שלי אבל מעניין אותי עכשיו איך Session עובדים יותר לעומק. או לדוגמה איזה פרצות אני יכול למצוא לאתר שלי בדרכים לא קונבנציונליות. כלומר איך אני יכול לשחק עם הדפדפן כדי למצוא עוד פרצות שצריכות להיחסם ברמת האפליקציה.

לסיכום,

אני רוצה להגיד תודה רבה ליוסי זהבי מנהל המגמה שבנה תוכנית לימודים כל כך מושקעת שלימדה אותי ואת כל חבריי למגמה מ-0 לרמה גבוהה מאוד בעולם הסייבר והתכנות לאופיר שביט, המורה שלי בשנתיים האחרונות שתמך בי כשלקחתי על עצמי פרויקטים גדולים ונתן לי מקום לצמוח ולימד אותי כל כך הרבה דברים חדשים בעולם המרתק הזה לאמיל אברמוביץ' שליווה אותי בשנה הראשונה של המגמה ושהוא זה שתפס אותי והכניס אותי לעולם הזה עם תשוקה רבה וסקרנות.

ביליוגרפיה

[SQL Where Clause Optimization](#) - מחקר על הדרך שבה מנועי מסדי נתונים מייעלים שאילתות עם תנאים מרובים.

[Relational Algebra](#) - הבסיס התאורטי של מסדי נתונים

[Indexing Strategies](#) - לימוד על הדרכים לייעל חיפושים במסדי נתונים

[Date and Time Functions](#) - במהלך הפיתוח הסתמכתי על התיעוד הרשמי בנושא ניהול זמנים במסד הנתונים. למדתי איך להשתמש בפונקציות מובנות לחישוב טווחי זמן והשוואת תאריכים בשרת.

[Shceduling in Greedy Algorithms](#) - חקרתי את בעיית ה-Interval Scheduling שהיא בעיה קלאסית באלגוריתמיקה.

[ISO 8601](#) - קראתי על התקן הבינלאומי לייצוג תאריכים וזמנים. הבנתי ששמירת זמנים בפורמט סטנדרטי זה היא קריטית כדי לאפשר למסד הנתונים לבצע מיון וחיפוש בצורה היעילה ביותר.

[Redux a JS library](#) - הסבר על איך בונים חנות ששומרת על המשתנים באתר

נספחים

Redux - טכנולוגיה שבעזרתה ניתן לשמור בצד הלקוח את כל ה-State של הלקוח ואת כל המשתנים שהוגדרו

State - המצב הנוכחי של האפליקציה, כלומר מה כל משתנה מחזיק באיזה עמוד נמצאים, ועוד

Store - המושג הטכני למיכל ששומר על ה-State של כל האפליקציה

נספח קוד

message_types.py

```

0 | from enum import Enum
1 |
2 | class MessageType(Enum):
3 |     #Anything
4 |         BROAD = "BRD"
5 |
6 |     #Authentication
7 |         LOGIN = "LOGIN"
8 |         USER_SIGNUP = "USERSIGNUP"
9 |         FREELANCER_SIGNUP = "FRLNCSIGNUP"
10 |         VERIFICATION = 'VERIFYING'
11 |         FORGOT_PASSWORD_REQUEST = "FRGT_PAS_RQT"
12 |         FORGOT_PASSWORD_AUTHENTICATION = "FRGT_PASS_AUTH"
13 |         CHANGE_PASS = "CHNG_PAS"
14 |
15 |     #Homepage
16 |         GET_USER_INFO = "USR_INFO"
17 |         MARK_READ_NOTIFICATION = "RD_NTFC"
18 |         UPDATE_APPOINTMENTS_STATUS = "UPDT_APP_STAT"
19 |         GET_APPOINTMENT_TIMES = "GET_APP_TIME"
20 |         MAKE_APPOINTMENT = "MK_APP"
21 |
22 |     #Search
23 |         GET_PUBLIC_PROFILE_INFO = "PBL_INFO"
24 |         GET_MINIMAL_FREELANCER_INFO = "MNML_VIEW"
25 |         GET_JOBS = "GET_JBS"
26 |         GET_CITIES_BY_JOB = "GET_CITIS"
27 |
28 |
29 | class StatusMessage(Enum):
30 |     #Anything
31 |         TOKEN_BAD = "BAD_TKN"
32 |
33 |     #Authentication
34 |         LOGGED_IN = "LOGGED"
35 |         FAILED_LOG_IN = "FAILED_LOG"
36 |
37 |         SIGNING_UP = "SIGNED"
38 |         FAILED_SIGN_UP = "FAILED_SIGNED"

```

```
39 |
40 |     VERIFICATION_GOOD = "VERI_GOOD"
41 |     VERIFICATION_BAD = "VERI_BAD"
42 |
43 |     FORGOT_PASSWORD_GOOD = "FRGT_GOOD"
44 |     FORGOT_PASSWORD_BAD = "FRGT_BAD"
45 |
46 |     CHANGE_PASSWORD_GOOD = "CHNG_GOOD"
47 |     CHANGE_PASSWORD_BAD = "CHNG_BAD"
48 |
49 | #Homepage
50 |     GOT_USER_INFO = "GOT_USR_INFO"
51 |     FAILED_TO_GET_USER_INFO = "FLD_USR_INFO"
52 |
53 |     MARKED_READ_NOTIFICATIONS = "READ_NOTIFICATIONS"
54 |     FAILED_TO_MARK_READ_NOTIFICATIONS =
55 | "FAILED_READ_NOTIFICATIONS"
56 |
57 |     UPDATED_APP_STATUS="UPDT_APP"
58 |     FAILED_TO_UPDATE_APP_STATUS="FAILED_UPDT_APP"
59 |
60 |     GOT_APPOINTMENT_TIMES="GOT_APP_TIME"
61 |     FAILED_TO_GET_APPOINTMENT_TIMES = "FLD_APP_TIME"
62 |
63 |     BOOKED_APPOINTMENT="BKD_APP"
64 |     FAILED_TO_BOOK_APPOINTMENT="FLD_BKD_APP"
65 | #Search
66 |     GOT_JOBS="GOT_JBS"
67 |     FAILED_TO_GET_JOBS="FLD_JBS"
68 |
69 |     GOT_CITIES="GOT_CITIS"
70 |     FAILED_TO_GET_CITIES="FLD_CITIS"
71 |
72 |     GOT_MINIMAL_INFO= "GOT_MNML"
73 |     FAILED_TO_GET_MINIMAL_INFO="FLD_MNML"
74 |
75 |
76 |     GOT_FREELANCER_PUBLIC_INFO= "GOT_FREE"
77 |     FAILED_TO_GET_FREELANCER_PUBLIC_INFO="FLD_FREE"
```

MessageHandler.py

```

0 | import sqlite3
1 | import hashlib
2 | import os
3 | from message_types import MessageTypes, StatusMessage
4 | from abc import ABC, abstractmethod
5 | import re
6 | from datetime import datetime, timedelta
7 | import random
8 | import smtplib
9 | from email.mime.text import MIMEText
10| from email.mime.multipart import MIMEMultipart
11| from token_handler import is_token_valid, create_token,
decode
12| from collections import defaultdict
13| from os import getenv
14| from dotenv import load_dotenv
15| import bleach
16|
17| #region Consts
18| DATABASE = r"C:\Coding\pro-find\Python\my_app.db"
19| load_dotenv()
20|
21| EMAIL = getenv("EMAIL")
22| PEPPER = getenv("PEPPER")
23| AUTHENTICATION_PAS = getenv("AUTHENTICATION_PAS")
24|
25|
26| with open (r"C:\Coding\pro-find\Python\professional.txt",
'r') as f:
27|     PROFESSIONS = [line.strip() for line in f]
28| with open(r"C:\Coding\pro-find\Python\cities.txt", 'r') as
29|     LOCALITIES = [line.strip() for line in f]
30|
31|
32| #endregion
33|
34|
35| def hash_password(password:str, salt:str) ->str:
36|
37|     combined_encoded= (salt+password+PEPPER).encode()
38|     return hashlib.sha256(combined_encoded).hexdigest()
39|

```



```

40|
41|
42|
43|
44| class Message:
45|     def __init__(self, payload):
46|         self.type_of = payload["type"]
47|         self.data = payload["data"]
48|         self.token = payload["token"]
49|
50|
51|
52| class MessageHandler(ABC):
53|     @abstractmethod
54|     def handle(self, msg:Message) ->tuple:
55|         pass
56|
57|
58| #region Dispatcher
59| class MessageDispatcher:
60|     no_token_check = {
61|         MessageTypes.LOGIN.value,
62|         MessageTypes.USER_SIGNUP.value,
63|         MessageTypes.FREELANCER_SIGNUP.value,
64|         MessageTypes.VERIFICATION.value,
65|         MessageTypes.FORGOT_PASSWORD_AUTHENTICATION.value,
66|         MessageTypes.FORGOT_PASSWORD_REQUEST.value,
67|         MessageTypes.CHANGE_PASS.value}
68|
69|
70|     def __init__(self):
71|         self._handlers={}
72|
73|
74|     def register(self, msg_type, cls):
75|         self._handlers[msg_type] = cls
76|
77|
78|     def dispatch(self, msg:Message) ->tuple:
79|         if msg.type_of not in self.no_token_check:
80|             if not is_token_valid(msg.token):
81|                 return MessageTypes.BROAD,
82|                 {StatusMessage.TOKEN_BAD.value:"Token format invalid"}

```

```

78 |
79 |         handler_class = self._handlers.get(msg.type_of)
80 |
81 |         if not handler_class:
82 |             raise Exception("No handler registered")
83 |
84 |         handler:MessageHandler = handler_class()
85 |         return handler.handle(msg)
86 |
87 |
88 | def configure_dispatcher() -> MessageDispatcher:
89 |     d = MessageDispatcher()
90 |     d.register(MessageTypes.LOGIN.value, LoginDispatcher)
91 |     d.register(MessageTypes.USER_SIGNUP.value,
UserSignUpService)
92 |     d.register(MessageTypes.FREELANCER_SIGNUP.value,
FreelancerSignUpService)
93 |     d.register(MessageTypes.VERIFICATION.value,
VerificationDispatcher)
94 |     d.register(MessageTypes.FORGOT_PASSWORD_REQUEST.value,
ForgotPasswordEmailVerifciationDispatcher)
95 |
d.register(MessageTypes.FORGOT_PASSWORD_AUTHENTICATION.value,
ForgotPasswordAuthenticationDispatcher)
96 |     d.register(MessageTypes.CHANGE_PASS.value,
UpdatePasswordDispatcher )
97 |     d.register(MessageTypes.GET_USER_INFO.value,
UserInfoDispatcher)
98 |     d.register(MessageTypes.UPDATE_APPOINTMENTS_STATUS.val
ChangeAppointmentStatusDispatcher)
99 |     d.register(MessageTypes.MARK_READ_NOTIFICATION.value,
MarkNotificationsReadDispatcher)
100 |     d.register(MessageTypes.GET_APPOINTMENT_TIMES.value,
AppointmentStartTimesDispatcher)
101 |     d.register(MessageTypes.MAKE_APPOINTMENT.value,
BookAppointmentDispatcher)
102 |     d.register(MessageTypes.GET_JOBS.value,
ExistingJobsDispatcher)
103 |     d.register(MessageTypes.GET_CITIES_BY_JOB.value,
CitiesByJobDispatcher)
104 |
d.register(MessageTypes.GET_MINIMAL_FREELANCER_INFO.value,
MinimalProfessionalInfo)

```

```

105|         d.register(MessageTypes.GET_PUBLIC_PROFILE_INFO.value,
FreelancerPublicInfo)
106|         return d
107|
108|
109|
110|
111| #region authentication
112| class LoginDispatcher(MessageHandler):
113|     def handle(self, msg:Message) -> tuple:
114|         try:
115|
116|             email = msg.data["email"]
117|             password = msg.data["password"]
118|             with sqlite3.connect(DATABASE) as conn:
119|                 cursor = conn.cursor()
120|
121|                 cursor.execute("SELECT password_hash, salt
FROM users WHERE email = ?", (email,))
122|                 data = cursor.fetchone()
123|
124|                 if data:
125|                     db_password, salt = data
126|                     hashed_ps = hash_password(password,
salt)
127|                     if db_password == hashed_ps:
128|                         return MessageTypes.LOGIN,
{StatusMessage.LOGGED_IN.value:create_token(email)}
129|                         return MessageTypes.LOGIN,
{StatusMessage.FAILED_LOG_IN.value:"Email or password incorrec
130|
131|                 except Exception as e:
132|                     print("Login dispatcher - ", e)
133|                     return MessageTypes.LOGIN,
{StatusMessage.FAILED_LOG_IN.value:"Client error"}
134|
135|
136|
137|
138| class UserSignUpService(MessageHandler):
139|     def handle(self, msg:Message) -> tuple:
140|         v = UserSignUpValidator()
141|         info_is_good, message = v.validate(msg)

```

```

142|         if not info_is_good:
143|             return MessageTypes.USER_SIGNUP, message
144|
145|         status, message = self._add_pending_user(msg)
146|         if not status:
147|             return MessageTypes.USER_SIGNUP, message
148|
149|         return MessageTypes.USER_SIGNUP,
150|         {StatusMessage.SIGNING_UP.value:""}
151|
152|
153|     def _add_pending_user(self, msg:Message) ->
154|         tuple[bool,dict]:
155|             try:
156|                 data = msg.data
157|                 with sqlite3.connect(DATABASE) as conn:
158|                     cur = conn.cursor()
159|                     conn.execute("BEGIN")
160|
161|                     cur.execute("SELECT 1 FROM users WHERE
162|email=?", (data["email"],))
163|
164|                     if cur.fetchone():
165|                         conn.rollback()
166|                         return False,
167|                         {StatusMessage.FAILED_SIGN_UP.value:"Email already in use"}
168|
169|                     cur.execute("""SELECT expires_at FROM
170|pending_users WHERE email=?""", (data["email"],))
171|                     find = cur.fetchone()
172|
173|                     if find:
174|                         expires_at = datetime.strptime(find[0]
175|"%Y-%m-%d %H:%M:%S.%f")
176|
177|                         if expires_at<datetime.now():
178|                             cur.execute("""DELETE FROM
179|pending_users WHERE email=?""", (data["email"],))
180|                         else:
181|                             conn.rollback()
182|                             return False,
183|                             {StatusMessage.FAILED_SIGN_UP.value:"Email already in use"}

```

```

177|
178|         salt = os.urandom(16).hex()
179|         password_hash =
hash_password(data["password"], salt)
180|         ver = EmailVerification(data["email"])
181|
182|         cur.execute("""
183|             INSERT INTO pending_users (full_name, email,
password_hash, salt, user_type, verification_code, expires_at)
184|             VALUES (?, ?, ?, ?, ?, ?, ?) """, (data["name"],
data["email"], password_hash, salt, data["role"],
ver.verification_code, ver.expires_at))
185|
186|         if not ver.send_verification_code():
187|             conn.rollback()
188|             return False,
{StatusMessage.FAILED_SIGN_UP.value:"Failed to send verification
email.\nCheck email field"}
189|
190|             conn.commit()
191|             return True,
{MessageTypes.VERIFICATION.value:"Sending verification"}
192|
193|         except sqlite3.IntegrityError:
194|             return False,
{StatusMessage.FAILED_SIGN_UP.value: "Email already in use"}
195|
196|         except Exception as e:
197|             conn.rollback()
198|             print("User sign up service - ", e)
199|             return False,
{StatusMessage.FAILED_SIGN_UP.value: "Not all fields were valid"}
200|
201|
202|
203|
204|
205| class FreelancerSignUpService(MessageHandler):
206|     def handle(self, msg:Message) -> tuple:
207|         v = FreelancerSignUpValidator()
208|         info_is_good, message = v.validate(msg)
209|         if not info_is_good:
210|             return MessageTypes.FREELANCER_SIGNUP, message

```

```

211|
212|         status, message = self._add_pending_freelancer(msg)
213|         if not status:
214|             return MessageTypes.FREELANCER_SIGNUP, message
215|
216|         return MessageTypes.FREELANCER_SIGNUP,
217|         {StatusMessage.SIGNING_UP.value:""}
218|
219|     def _add_pending_freelancer(self, msg:Message) ->
220|     tuple[bool,dict]:
221|         data = msg.data
222|         try:
223|             with sqlite3.connect(DATABASE) as conn:
224|                 cur = conn.cursor()
225|                 conn.execute("BEGIN")
226|                 cur.execute("SELECT 1 FROM users WHERE
227| email=?", (data["email"],))
228|                 if cur.fetchone():
229|                     conn.rollback()
230|                     return False,
231|                     {StatusMessage.FAILED_SIGN_UP.value:"Email already in use"}
232|
233|                 cur.execute("SELECT expires_at FROM
234| pending_users WHERE email=?", (data["email"],))
235|                 row = cur.fetchone()
236|                 if row:
237|                     expires_at = datetime.strptime(row[0]
238| "%Y-%m-%d %H:%M:%S.%f")
239|                     if expires_at<datetime.now():
240|                         cur.execute("""DELETE FROM
241| pending_users WHERE email=?""", (data["email"],))
242|                     else:
243|                         conn.rollback()
244|                         return False,
245|                         {StatusMessage.FAILED_SIGN_UP.value:"Email already in use"}
246|
247|                 salt = os.urandom(16).hex()

```

```

245|         password_hash =
hash_password(data["password"], salt)
246|         ver = EmailVerification(data["email"])
247|         cities = ", ".join(data["cities"])
248|
249|         cur.execute("""INSERT INTO pending_users
(full_name, email, password_hash,
250|                        salt, user_type, profession,
cities, years, job_duration, description, start_working,
251|                        finish_working,
verification_code, expires_at, hour_price) VALUES
(?,?,?,?,?,?,?,?,?,?,?,?,?,?,?,?,?)""",
252|                        (data["name"], data["email"],
password_hash, salt, data["role"], data["profession"],
253|                        cities, data["years"],
data["jobDuration"], data["description"], data["startWorking"],
254|                        data["finishWorking"],
ver.verification_code, ver.expires_at, data["hourPrice"]))
255|
256|         if not ver.send_verification_code():
257|             conn.rollback()
258|             return False,
{StatusMessage.FAILED_SIGN_UP.value: "Failed to send verificat
email. Check email field"}
259|
260|         conn.commit()
261|         return True,
{MessageTypes.VERIFICATION.value: "Sending verification"}
262|
263|     except sqlite3.IntegrityError:
264|         return False,
{StatusMessage.FAILED_SIGN_UP.value: "Email already in use"}
265|
266|     except Exception as e:
267|         conn.rollback()
268|         print("Freelancer sign up service -", e)
269|         return False,
{StatusMessage.FAILED_SIGN_UP.value: "Not all fields were vali
270|
271|
272|
273|
274| class VerificationDispatcher(MessageHandler):

```

```

275|         def handle(self, msg:Message) -> tuple:
276|             verified, message =
EmailVerification.check_verification_code(msg)
277|             if verified:
278|                 if self.log_new_customer(msg):
279|                     return MessageTypes.VERIFICATION,message
280|                 else:
281|                     return MessageTypes.VERIFICATION,
{StatusMessage.VERIFICATION_BAD.value:"Error while logging use
282|
283|             else:
284|                 return MessageTypes.VERIFICATION, message
285|
286|         def log_new_customer(self, msg:Message) -> bool:
287|             try:
288|                 with sqlite3.connect(DATABASE) as conn:
289|                     conn.execute("BEGIN")
290|                     cur = conn.cursor()
291|
292|                     data = msg.data
293|                     email = data["email"]
294|                     role = data["role"]
295|
296|                     if role == "User":
297|                         cur.execute("SELECT full_name,
password_hash, salt, user_type FROM pending_users WHERE email=
(email,))
298|                         row = cur.fetchone()
299|                         if not row:
300|                             conn.rollback()
301|                             return False
302|
303|                         name, password_hash, salt, user_type
row
304|
305|                         cur.execute("""INSERT INTO users
(full_name, email, password_hash, user_type, salt)
306|                                     VALUES (?, ?, ?, ?, ?)""",
(name, email, password_hash, user_type, salt))
307|
308|                     elif role == "Freelancer":

```



```

309|         cur.execute("""SELECT full_name,
password_hash, salt, user_type, profession, cities, years,
job_duration, description,
310|                        start_working,
finish_working, hour_price FROM pending_users WHERE
email=?""", (email,))
311|         row = cur.fetchone()
312|         if not row:
313|             conn.rollback()
314|             return False
315|
316|         (name, password_hash, salt, user_type,
profession, cities,
317|          years, job_duration, description,
start_working, finish_working, hour_price) = row
318|
319|         cur.execute("""INSERT INTO users
(full_name, email, password_hash, user_type, salt)
320|                     VALUES (?, ?, ?, ?, ?)""",
(name, email, password_hash, user_type, salt))
321|         id = cur.lastrowid
322|
323|         cur.execute("""INSERT INTO profession
(user_id, profession, service_cities,
324|                        description,
years_experience, avg_job_duration, start_time, end_time,
hour_price)
325|                     VALUES
(?, ?, ?, ?, ?, ?, ?, ?, ?)""", (id, profession, cities, description,
years, job_duration, start_working, finish_working, hour_price)
326|
327|         cur.execute("DELETE FROM pending_users WHERE
email=?", (email,))
328|         conn.commit()
329|         return True
330|
331|     except sqlite3.IntegrityError:
332|         conn.rollback()
333|         return False
334|
335|     except Exception as e:
336|         conn.rollback()

```

```

337|         print("Verification dispatcher, logging custom
- ", e)
338|         return False
339|
340|
341|
342|
343|
344| class
ForgotPasswordEmailVerifciationDispatcher(MessageHandler):
345|     def handle(self, msg:Message) ->tuple:
346|         try:
347|             with sqlite3.connect(DATABASE) as conn:
348|                 conn.execute("BEGIN")
349|                 email = msg.data["email"]
350|                 cur = conn.cursor()
351|
352|                 cur.execute("SELECT 1 FROM users WHERE
email=?", (email,))
353|                 if not cur.fetchone():
354|                     return
MessageTypes.FORGOT_PASSWORD_REQUEST,
{StatusMessage.FORGOT_PASSWORD_BAD.value:"System error"}
355|
356|
357|                 cur.execute("SELECT expires_at FROM
pending_users WHERE email=?", (email,))
358|
359|                 find =cur.fetchone()
360|                 if find:
361|
362|                     expires_at = datetime.strptime(find[0
"%Y-%m-%d %H:%M:%S.%f")
363|                     if expires_at<datetime.now():
364|                         cur.execute("DELETE FROM
pending_users WHERE email=?", (email,))
365|                     else:
366|                         return
MessageTypes.FORGOT_PASSWORD_REQUEST,
{StatusMessage.FORGOT_PASSWORD_BAD.value:"Expired verification
time"}
367|
368|                 ver = EmailVerification(email)

```

```

369|
370|         if not ver.send_verification_code():
371|             conn.rollback()
372|             return
373|
374|         MessageTypes.FORGOT_PASSWORD_REQUEST,
375|         {StatusMessage.FORGOT_PASSWORD_BAD.value:"Error sending
376|         verification email\ntry again at a later time"}
377|
378|         cur.execute("""
379|             INSERT INTO pending_users
380|             (full_name, email, password_hash, sal
381|             user_type, verification_code, expires_at)
382|             VALUES (?, ?, ?, ?, ?, ?, ?)
383|             """, ("", email, "", "", "User",
384|             ver.verification_code, ver.expires_at))
385|
386|         conn.commit()
387|         return MessageTypes.FORGOT_PASSWORD_REQUEST
388|         {StatusMessage.FORGOT_PASSWORD_GOOD.value:""}
389|
390|     except Exception as e:
391|         conn.rollback()
392|         print("Forgot Password Dispatcher - ",e)
393|         return MessageTypes.FORGOT_PASSWORD_REQUEST,
394|         {StatusMessage.FORGOT_PASSWORD_BAD.value:"Client error"}
395|
396|
397|
398|
399|
400|
401|
402|
403| class
404| ForgotPasswordAuthenticationDispatcher(MessageHandler):
405|     def handle(self, msg:Message) ->tuple:
406|         try:
407|             with sqlite3.connect(DATABASE) as conn:
408|                 conn.execute("BEGIN")
409|                 cur = conn.cursor()
410|                 email = msg.data["email"]
411|                 ver_code = msg.data["veri_code"]
412|
413|                 cur.execute("SELECT verification_code,
414|                 expires_at FROM pending_users WHERE email=?", (email,))

```

```

403|             pending = cur.fetchone()
404|
405|             if not pending:
406|                 return
407|             MessageType.FORGOT_PASSWORD_AUTHENTICATION,
408|             {StatusMessage.FORGOT_PASSWORD_BAD.value:"Email is not
409| verifying"}
410|
411|             cur.execute("SELECT 1 FROM users WHERE
412| email=?", (email,))
413|             user = cur.fetchone()
414|             if not user:
415|                 return
416|             MessageType.FORGOT_PASSWORD_AUTHENTICATION,
417|             {StatusMessage.FORGOT_PASSWORD_BAD.value:"No such user in
418| system"}
419|
420|             expires_at = datetime.strptime(pending[1]
421| "%Y-%m-%d %H:%M:%S.%f")
422|             if expires_at<datetime.now():
423|                 cur.execute("""DELETE FROM pending_us
424| WHERE email=?""", (email,))
425|                 conn.commit()
426|                 return
427|             MessageType.FORGOT_PASSWORD_AUTHENTICATION,
428|             {StatusMessage.FORGOT_PASSWORD_BAD.value:"Verification time
429| finished"}
430|
431|             if ver_code == pending[0]:
432|                 cur.execute("""DELETE FROM pending_us
433| WHERE email=?""", (email,))
434|                 cur.execute("INSERT INTO pass_change
435| (email) VALUES (?)", (email,))
436|                 conn.commit()
437|                 return
438|             MessageType.FORGOT_PASSWORD_AUTHENTICATION,
439|             {StatusMessage.FORGOT_PASSWORD_GOOD.value:""}
440|
441|             else:
442|                 return
443|             MessageType.FORGOT_PASSWORD_AUTHENTICATION,
444|             {StatusMessage.FORGOT_PASSWORD_BAD.value:"Incorrect verificati
445| code"}

```

```

427|
428|
429|
430|
431|         except Exception as e:
432|             conn.rollback()
433|             print("Forgot password authentication dispatch
- ", e)
434|             return
MessageTypes.FORGOT_PASSWORD_AUTHENTICATION,
{StatusMessage.FORGOT_PASSWORD_BAD.value:"Client error"}
435|
436|
437|
438|
439|
440| class UpdatePasswordDispatcher(MessageHandler):
441|     def handle(self, msg:Message) ->tuple:
442|         try:
443|             with sqlite3.connect(DATABASE) as conn:
444|                 conn.execute("BEGIN")
445|                 cur = conn.cursor()
446|                 email = msg.data["email"]
447|                 password = msg.data["password"]
448|
449|                 cur.execute("""SELECT 1 FROM pass_change
WHERE email=?""", (email,))
450|                 user = cur.fetchone()
451|
452|                 if not user:
453|                     return MessageTypes.CHANGE_PASS,
{StatusMessage.CHANGE_PASSWORD_BAD.value:"User not changin
password"}
454|
455|                 salt = os.urandom(16).hex()
456|                 password_hash = hash_password(password,
salt)
457|
458|                 cur.execute(""" UPDATE users SET
password_hash = ?, salt = ? WHERE email = ?""", (password_hash
salt, email))
459|                 cur.execute("""DELETE FROM pass_change WF
email=?""", (email,))

```

```

460 |         conn.commit()
461 |         return MessageTypes.CHANGE_PASS,
462 |         {StatusMessage.CHANGE_PASSWORD_GOOD.value:""}
463 |
464 |     except Exception as e:
465 |         conn.rollback()
466 |         print("Update password dispatcher - ", e)
467 |         return MessageTypes.CHANGE_PASS,
468 |         {StatusMessage.CHANGE_PASSWORD_BAD.value:"Client error"}
469 | #endregion
470 |
471 |
472 |
473 | #region homepage
474 | class UserInfoDispatcher(MessageHandler):
475 |     def handle(self, msg:Message)->tuple:
476 |         try:
477 |             with sqlite3.connect(DATABASE) as conn:
478 |                 actual_email = decode(msg.token)["email"]
479 |                 cur = conn.cursor()
480 |                 email = msg.data["email"]
481 |
482 |                 if email !=actual_email:
483 |                     return MessageTypes.GET_USER_INFO,
484 |                     {StatusMessage.FAILED_TO_GET_USER_INFO.value:"Email not matching
485 | token"}
486 |
487 |                 cur.execute("SELECT id, full_name, user_type
488 | FROM users WHERE email=?", (email,))
489 |                 row = cur.fetchone()
490 |
491 |                 if not row:
492 |                     return MessageTypes.GET_USER_INFO,
493 |                     {StatusMessage.FAILED_TO_GET_USER_INFO.value:"Couldn't find
494 | user"}
495 |
496 |                 id, name, role = row
497 |                 payload_to_send = {"name":name, "role":role}
498 |
499 |                 if role=="Freelancer":

```

```

495|         cur.execute("SELECT profession,
service_cities, description, years_experience, rating,
start_time, end_time, avg_job_duration, hour_price FROM
professional WHERE user_id=?", (id,))
496|         row = cur.fetchone()
497|         if not row:
498|             return MessageTypes.GET_USER_INFO(
{StatusMessage.FAILED_TO_GET_USER_INFO.value:"Couldn't find
user"})
499|         profession, cities, description,
years_experience, rating, start, end, job_duration, hour_price =
row
500|
501|         try:
502|             h, m = map(int,
job_duration.split(':'))
503|
504|             h_str = f"{h}h" if h > 0 else ""
505|             m_str = f"{m}m" if m > 0 else ""
506|
507|             job_duration = f"{h_str}
{m_str}".strip()
508|         except:
509|             pass
510|
511|
payload_to_send.update({"job":profession, "cities":cities,
"description":description, "years":years_experience,
"rating":rating, "start_working":start, "end_working":end,
"job_duration":job_duration, "hour_price":hour_price})
512|
513|
514|         appointments = self._get_appointments(id,
cur, role)
515|
516|         notifications = self._get_notifications(id,
cur)
517|
518|         conn.commit()
519|         payload_to_send["appointments"] =
appointments
520|         payload_to_send["notifications"] =
notifications

```

```

521|
522|         return MessageTypes.GET_USER_INFO,
{StatusMessage.GOT_USER_INFO.value:payload_to_send, "user_id":
id}
523|
524|         except Exception as e:
525|             print(f"User info dispatcher - {e}")
526|             return MessageTypes.GET_USER_INFO,
{StatusMessage.FAILED_TO_GET_USER_INFO.value:"Couldn't find
user"}
527|
528|
529|     def _get_appointments(self, user_id, cur, role) -> li
530|         is_pro = True if role == "Freelancer" else Fa
531|
532|         now = datetime.now()
533|         today = now.strftime("%Y-%m-%d")
534|         time_now = now.strftime("%H:%M")
535|
536|         if is_pro:
537|             query = """
538|                 SELECT u.full_name, a.date, a.id,
a.start_time, a.end_time,
539|                 a.address, a.details, a.status
540|                 FROM appointments a
541|                 JOIN users u ON a.customer_id = u.id
542|                 WHERE a.professional_id = ?
543|             """
544|         else:
545|             query = """
546|                 SELECT u.full_name, a.date, a.id,
a.start_time, a.end_time,
547|                 a.address, a.details, a.status
548|                 FROM appointments a
549|                 JOIN professional p ON a.professional
= p.user_id
550|                 JOIN users u ON p.user_id = u.id
551|                 WHERE a.customer_id = ? AND a.status
'Confirmed'
552|             """
553|
554|             query += """

```



```

555|             AND (a.date > ? OR (a.date = ? AND
a.start_time >= ?))
556|             ORDER BY a.date ASC, a.start_time ASC
557|             """
558|
559|             cur.execute(query, (user_id, today, today,
time_now))
560|             rows = cur.fetchall()
561|
562|             results = []
563|             for r in rows:
564|                 d_parts = str(r[1]).split('-')
565|                 display_date =
f"{d_parts[2]}/{d_parts[1]}/{d_parts[0]}" if len(d_parts) == 3
else r[1]
566|
567|                 results.append({
568|                     "id": r[2],
569|                     "person_name": r[0],
570|                     "display_date": display_date,
571|                     "date": str(r[1]),
572|                     "start_time": str(r[3]),
573|                     "end_time": str(r[4]),
574|                     "address": r[5],
575|                     "details": r[6],
576|                     "status": r[7],
577|                     "timestamp": f"{r[1]} {r[3]}"
578|                 })
579|
580|             return results
581|
582|
583|     def _get_notifications(self, user_id, cur) -> list:
584|         query = """
585|             SELECT n.message, n.created_at, n.is_read,
u.full_name as sender_name
586|             FROM notifications n
587|             LEFT JOIN users u ON n.from_id = u.id
588|             WHERE n.user_id = ?
589|             ORDER BY n.created_at DESC
590|             """
591|         cur.execute(query, (user_id,))
592|         rows = cur.fetchall()

```

```

593|
594|         notifications = []
595|         for r in rows:
596|             message, raw_date, is_read, sender_name = r
597|             try:
598|                 dt_obj = datetime.strptime(raw_date,
599| "%Y-%m-%d %H:%M:%S.%f")
600|                 formatted_date = dt_obj.strftime("%d/%m/%y")
601|             except ValueError:
602|                 try:
603|                     dt_obj = datetime.strptime(raw_date,
604| "%Y-%m-%d %H:%M:%S")
605|                     formatted_date =
606| dt_obj.strftime("%d/%m/%y")
607|                 except Exception:
608|                     formatted_date = str(raw_date)[:10]
609|
610|             notifications.append({
611|                 "message": message,
612|                 "created_at": formatted_date,
613|                 "from_name": sender_name if sender_name else
614| "System",
615|                 "is_read": bool(is_read)
616|             })
617|
618|         return notifications
619|
620|     class MarkNotificationsReadDispatcher(MessageHandler):
621|         def handle(self, msg: Message) -> tuple:
622|             try:
623|                 user_id = msg.data["user_id"]
624|                 with sqlite3.connect(DATABASE) as conn:
625|                     cur = conn.cursor()
626|                     cur.execute("UPDATE notifications SET
627| is_read = 1 WHERE user_id = ? AND is_read = 0", (user_id,))
628|                     conn.commit()
629|                 return MessageTypes.MARK_READ_NOTIFICATION,
630| {StatusMessage.MARKED_READ_NOTIFICATIONS.value: ""}
631|             except Exception as e:
632|                 print("Marked notification dispatcher error -
633| e)

```

```

629|         return MessageTypes.MARK_READ_NOTIFICATION,
630|         {StatusMessage.FAILED_TO_MARK_READ_NOTIFICATIONS.value:"Failed
631| mark messages"}
632|
633| class ChangeAppointmentStatusDispatcher(MessageHandler):
634|     def handle(self, msg: Message) -> tuple:
635|         try:
636|             with sqlite3.connect(DATABASE) as conn:
637|                 cur = conn.cursor()
638|                 apps = msg.data["appointments"]
639|
640|                 worked = []
641|                 failed = []
642|
643|                 first_key = int(list(apps.keys())[0])
644|                 cur.execute("SELECT u.full_name,
645| a.professional_id FROM appointments a JOIN users u ON
646| a.professional_id = u.id WHERE a.id = ?", (first_key,))
647|                 res = cur.fetchone()
648|                 if not res:
649|                     prof_name = "Freelancer"
650|                     p_id = None
651|                 else:
652|                     prof_name, p_id = res
653|
654|                 status_map = {'accepted': "Confirmed",
655| 'cancelled': "Cancelled"}
656|
657|                 for app_id, status_input in apps.items():
658|                     try:
659|                         target_status =
660| status_map.get(status_input)
661|
662|                         if target_status == "Confirmed":
663|                             if self._is_time_slot_taken(c
664| app_id):
665|
666|                                 cur.execute("DELETE FROM
667| appointments WHERE id=?", (app_id,))
668|                                 failed[app_id] = "Slot
669| already taken"
670|
671|                                 continue

```

```

663|
664|             cur.execute("SELECT customer_id FROM
appointments WHERE id=?", (app_id,))
665|             row = cur.fetchone()
666|             if not row:
667|                 failed[app_id] = "Appointment
not found"
668|                 continue
669|
670|             u_id = row[0]
671|
672|             if target_status == "Confirmed":
673|                 cur.execute("UPDATE appointments
SET status=? WHERE id=?", (target_status, app_id))
674|                 msg_text = f"{prof_name} has
accepted your appointment"
675|             else:
676|                 cur.execute("DELETE FROM
appointments WHERE id=?", (app_id,))
677|                 msg_text = f"{prof_name} has
declined your appointment"
678|
679|                 cur.execute("""INSERT INTO
notifications (user_id, message, is_read, created_at, from_id)
680|                             VALUES (?, ?, 0, ?,
?) """,
681|                             (u_id, msg_text,
datetime.now().strftime("%Y-%m-%d %H:%M:%S"), p_id))
682|
683|                 worked.append(app_id)
684|
685|             except Exception as inner_e:
686|                 failed[app_id] = str(inner_e)
687|                 continue
688|
689|             conn.commit()
690|
691|             return
MessageTypes.UPDATE_APPOINTMENTS_STATUS, {
692|                 StatusMessage.UPDATED_APP_STATUS.value,
"Batch processed",
693|                 "results": {"processed":worked,
"Failed":failed}

```

```

694|         }
695|
696|         except Exception as e:
697|             print(f"Appointment Status change Dispatcher
Error: {e}")
698|             return MessageTypes.UPDATE_APPOINTMENTS_STATU
{StatusMessage.FAILED_TO_UPDATE_APP_STATUS.value: "System erro
699|
700|     def _is_time_slot_taken(self, cur, app_id):
701|         cur.execute("SELECT professional_id, date,
start_time, end_time FROM appointments WHERE id=?", (app_id,))
702|         target = cur.fetchone()
703|         if not target:
704|             return False
705|
706|         p_id, a_date, a_start, a_end = target
707|
708|         query = """SELECT id FROM appointments WHERE
professional_id = ? AND date = ?
709|             AND status = 'Confirmed' AND id != ? AND (? <
end_time AND start_time < ?)
710|         """
711|         cur.execute(query, (p_id, a_date, app_id, a_start
a_end))
712|         return cur.fetchone() is not None
713|
714|
715|
716| class AppointmentStartTimesDispatcher(MessageHandler):
717|     def handle(self, msg:Message) ->tuple:
718|         try:
719|             with sqlite3.connect(DATABASE) as conn:
720|                 cur = conn.cursor()
721|                 pro_id = msg.data["id"]
722|
723|                 cur.execute("SELECT avg_job_duration,
start_time, end_time FROM professional WHERE user_id=?",
(pro_id,))
724|                 row = cur.fetchone()
725|                 if not row:
726|                     return
MessageTypes.GET_APPOINTMENT_TIMES,

```

```

{StatusMessage.FAILED_TO_GET_APPOINTMENT_TIMES.value:"Couldn't
find freelancer"}
727|
728|         job_duration, start_time, end_time = row
729|
730|         cur.execute("""SELECT start_time, date FROM
appointments WHERE professional_id=?
731|                     AND date BETWEEN date('now')
date('now', '+1 month')
732|                     AND status='Confirmed' ORDER
date ASC; """, (pro_id,))
733|
734|         existing_appointments = {}
735|         for row in cur.fetchall():
736|             time, date = row[0], row[1]
737|             if date not in existing_appointments:
738|                 existing_appointments[date] = []
739|                 existing_appointments[date].append(ti
740|
741|         existing_appointments =
dict(existing_appointments)
742|
743|         h,m = map(int, job_duration.split(":"))
744|         duration_minutes = (h*60) + m
745|
746|         app_times = {}
747|         current_date = datetime.now() +
timedelta(days=1)
748|
749|         for _ in range(0, 35):
750|             date_str =
current_date.strftime("%Y-%m-%d")
751|             booked_slots =
existing_appointments.get(date_str, [])
752|             app_times[date_str] =
_calculate_free_day_working_hours(start_time, end_time,
duration_minutes, booked_slots)
753|             current_date =
current_date+timedelta(days=1)
754|
755|
756|         return MessageTypes.GET_APPOINTMENT_TIMES
{StatusMessage.GOT_APPOINTMENT_TIMES.value:app_times}

```

```

757|         except Exception as e:
758|             print("Appointment time dispatcher error - ",
759|                 return MessageTypes.GET_APPOINTMENT_TIMES,
760|                 {StatusMessage.FAILED_TO_GET_APPOINTMENT_TIMES.value:"Issue
761| fetching the appointment dates and times"})
762|
763| def _calculate_free_day_working_hours(start_time, end_time,
764| job_duration, existing_appointments=None) ->list:
765|     if existing_appointments is None:
766|         existing_appointments=[]
767|
768|     format = "%H:%M"
769|     current_slot = datetime.strptime(start_time, format)
770|     end_of_day = datetime.strptime(end_time, format)
771|
772|     available_times = []
773|     while current_slot+timedelta(minutes=job_duration) <=
774| end_of_day:
775|         slot_str = current_slot.strftime(format)
776|
777|         if slot_str not in existing_appointments:
778|             available_times.append(slot_str)
779|
780|         current_slot+= timedelta(minutes=job_duration)
781|
782|     return available_times
783|
784| class BookAppointmentDispatcher(MessageHandler):
785|     def handle(self, msg: Message) -> tuple:
786|         try:
787|             with sqlite3.connect(DATABASE) as conn:
788|                 cur = conn.cursor()
789|                 app = msg.data["app"]
790|
791|                 cur.execute("SELECT avg_job_duration FROM
792| professional WHERE user_id = ?", (app["prof_id"],))
793|                 row = cur.fetchone()
794|                 if not row:

```

```

794|         return MessageTypes.MAKE_APPOINTMENT,
{StatusMessage.FAILED_TO_BOOK_APPOINTMENT.value: "Freelancer r
found"}
795|
796|         duration_str = row[0]
797|         h, m = map(int, duration_str.split(':'))
798|         duration_minutes = (h * 60) + m
799|
800|         end_time_str =
self.calculate_end_time(app["start_time"], duration_minutes)
801|
802|         cur.execute("SELECT start_time FROM
appointments WHERE date = ? AND professional_id = ? AND status
'Confirmed'", (app["date"], app["prof_id"]))
803|         start_times_booked = [row[0] for row in
cur.fetchall()]
804|
805|         available_slots =
_calculate_free_day_working_hours(app["start_time"],
end_time_str, duration_minutes, start_times_booked)
806|
807|         if app["start_time"] not in available_slo
808|             return MessageTypes.MAKE_APPOINTMENT,
{StatusMessage.FAILED_TO_BOOK_APPOINTMENT.value: "Time slot is
longer available."}
809|
810|         clean_details = bleach.clean(app["details
tags=[], attributes={}, strip=True)
811|         clean_address = bleach.clean(app["address
tags=[], attributes={}, strip=True)
812|
813|         cur.execute("""
814|             INSERT INTO appointments
(professional_id, customer_id, date, start_time, end_time,
address, details, status)
815|             VALUES (?, ?, ?, ?, ?, ?, ?, ?,
'Requested')""", (app["prof_id"], app["user_id"], app["date"],
app["start_time"], end_time_str, clean_address, clean_details)
816|
817|         cur.execute("""INSERT INTO notifications
(user_id, message, is_read, created_at)

```



```

818|             VALUES (?, ?, 0, ?)""",
(app["prof_id"], "You have a pending appointment waiting for
you", datetime.now().strftime("%Y-%m-%d %H:%M:%S")))
819|
820|             conn.commit()
821|             return MessageTypes.MAKE_APPOINTMENT,
{StatusMessage.BOOKED_APPOINTMENT.value: "Appointment requeste
successfully"}
822|
823|         except Exception as e:
824|             print(f"CRITICAL: Appointment booking failure
{e}")
825|             return MessageTypes.MAKE_APPOINTMENT,
{StatusMessage.FAILED_TO_BOOK_APPOINTMENT.value: "Server error
please try again."}
826|
827|
828|     def calculate_end_time(self, start_time_str,
duration_minutes):
829|         fmt = "%H:%M"
830|         start_dt = datetime.strptime(start_time_str, fmt)
831|         end_dt = start_dt +
timedelta(minutes=duration_minutes)
832|         return end_dt.strftime(fmt)
833|
834| #endregion
835|
836|
837|
838|
839| #region search
840| class ExistingJobsDispatcher(MessageHandler):
841|     def handle(self, msg:Message) -> tuple:
842|         try:
843|             with sqlite3.connect(DATABASE) as conn:
844|                 cur = conn.cursor()
845|
846|                 cur.execute("SELECT DISTINCT profession F
professional")
847|                 rows = cur.fetchall()
848|                 if not rows:

```

```

849|         return MessageTypes.GET_JOBS,
{StatusMessage.FAILED_TO_GET_JOBS.value:"Our professionals are
still not verified in the system, sorry!"}
850|
851|         jobs = []
852|         for row in rows:
853|             jobs.append(row[0])
854|
855|         jobs.sort()
856|         return MessageTypes.GET_JOBS,
{StatusMessage.GOT_JOBS.value:{"jobs":jobs}}
857|     except Exception as e:
858|         print("Existing job dispatcher error - ",e)
859|         return MessageTypes.GET_JOBS,
{StatusMessage.FAILED_TO_GET_JOBS.value:"Failed retrieving info
reload page"}
860|
861|
862| class CitiesByJobDispatcher(MessageHandler):
863|     def handle(self, msg:Message) ->tuple:
864|         try:
865|             with sqlite3.connect(DATABASE) as conn:
866|                 cur = conn.cursor()
867|
868|                 profession = msg.data["job"]
869|
870|                 cur.execute("SELECT DISTINCT service_citi
FROM professional WHERE profession=?", (profession,))
871|                 rows = cur.fetchall()
872|                 if not rows:
873|                     return MessageTypes.GET_CITIES_BY_JOE
{StatusMessage.FAILED_TO_GET_CITIES.value:"Server error"}
874|
875|                 unique_cities = set()
876|
877|                 for row in rows:
878|                     city_string = row[0]
879|                     if city_string:
880|                         parts = [c.strip() for c in
city_string.split(',')]
881|                         unique_cities.update(parts)
882|
883|                 cities = sorted(list(unique_cities))

```

```

884|
885|         return MessageTypes.GET_CITIES_BY_JOB,
    {StatusMessage.GOT_CITIES.value:{"cities":cities}}
886|         except Exception as e:
887|             print("Cities find by job dispatcher exception", e)
888|
    return MessageTypes.GET_CITIES_BY_JOB,
    {StatusMessage.FAILED_TO_GET_CITIES.value:"Couldn't retrieve
cities, refresh page"}
889|
890|
891| class MinimalProfessionalInfo(MessageHandler):
892|     def handle(self, msg:Message) -> tuple:
893|         try:
894|             with sqlite3.connect(DATABASE) as conn:
895|                 cur = conn.cursor()
896|
897|                 city = msg.data["city"]
898|                 job = msg.data["job"]
899|
900|                 search_term = f"%,{city},%"
901|
902|                 cur.execute("""SELECT id, description,
rating, hour_price FROM professional
903|                               WHERE profession=? AND (',' |
REPLACE(service_cities, ', ', ',') || ',') LIKE ? """, (job,
search_term))
904|
905|                 rows = cur.fetchall()
906|                 if not rows:
907|                     return
MessageTypes.GET_MINIMAL_FREELANCER_INFO,
{StatusMessage.FAILED_TO_GET_MINIMAL_INFO.value:"Error fetching
info"}
908|
909|                 users = defaultdict(dict)
910|                 for row in rows:
911|                     id, description, rating, hour_price =
row
912|                     users[id] = {"description":description,
"rating":rating, "price":hour_price}
913|
914|                 if users:

```

```

915|         dict(users)
916|         return
MessageTypes.GET_MINIMAL_FREELANCER_INFO,
{StatusMessage.GOT_MINIMAL_INFO.value:users}
917|
918|
919|         except Exception as e:
920|             print("Minimal professional info error - ",e)
921|             return MessageTypes.GET_MINIMAL_FREELANCER_IN
{StatusMessage.FAILED_TO_GET_MINIMAL_INFO.value:"Error fetchir
info"}
922|
923|
924| class FreelancerPublicInfo(MessageHandler):
925|     def handle(self, msg:Message) ->tuple:
926|         try:
927|             with sqlite3.connect(DATABASE) as conn:
928|                 cur = conn.cursor()
929|
930|                 cur.execute("""
931|                     SELECT u.id, p.profession,
p.service_cities, p.description, p.years_experience,
p.avg_job_duration,
932|                     p.rating, p.hour_price, u.full_name F
professional p INNER JOIN users u ON p.user_id = u.id
933|                     WHERE p.id = ?
934|                     """, (msg.data["id"],))
935|
936|                 row = cur.fetchone()
937|                 if not row:
938|                     return
MessageTypes.GET_PUBLIC_PROFILE_INFO,
{StatusMessage.FAILED_TO_GET_FREELANCER_PUBLIC_INFO.value:"Cou
't get freelancer's info"}
939|
940|                 id, profession, cities, description, year
job_duration, rating, price, name = row
941|                 to_send = {"id":id, "job":profession,
"cities":cities,
942|                             "description":description,
"years":years,
943|                             "job_duration":job_duration,
"rating":rating,

```

```

944 |         "price":price, "username":name}
945 |
946 |         return MessageTypes.GET_PUBLIC_PROFILE_INFO
{StatusMessage.GOT_FREELANCER_PUBLIC_INFO.value:to_send}
947 |
948 |     except Exception as e:
949 |         print("Freelancer public info exception - ",
950 |             return MessageTypes.GET_PUBLIC_PROFILE_INFO,
{StatusMessage.FAILED_TO_GET_FREELANCER_PUBLIC_INFO.value:"Cou
't get freelancer's info"}
951 |
952 | #endregion
953 |
954 |
955 |
956 | #endregion
957 |
958 |
959 |
960 | #region Signup Validators
961 | class Validator(ABC):
962 |     @abstractmethod
963 |     def validate(self, msg:Message) ->tuple[bool,str]:
964 |         pass
965 |
966 |
967 | class UserSignUpValidator(Validator):
968 |     def validate(self, msg:Message) ->tuple[bool, str]:
969 |         #Password I don't check
970 |         #Name I don't check
971 |         try:
972 |             email = msg.data["email"]
973 |             full_name = msg.data["name"]
974 |             if not self._check_email_format(email):
975 |                 return False,
{StatusMessage.FAILED_SIGN_UP.value:"Email format is incorrect
976 |                 if len(full_name)>30:
977 |                     return False,
{StatusMessage.FAILED_SIGN_UP.value:"Name field can't exceed 3
characters"}
978 |             return True, ""
979 |         except Exception as e:
980 |             print("User sign up validator - ",e)

```

```

981|         return False,
982|         {StatusMessage.FAILED_SIGN_UP.value:"Not all fields were sent"}
983|
984|     def _check_email_format(self, email):
985|         pattern = r'^[\w\.-]+@[\w\.-]+\.\w+$'
986|         return re.match(pattern, email) is not None
987|
988|
989| class FreelancerSignUpValidator(Validator):
990|     def validate(self, msg:Message) ->tuple[bool, str]:
991|         #Password I don't check
992|         #Name I don't check
993|         status = False
994|         error_message = StatusMessage.FAILED_SIGN_UP.value
995|
996|
997|         data = msg.data
998|         try:
999|             email = data["email"]
1000|             if not self._check_email_format(email):
1001|                 return status, {error_message:"Email format
is incorrect"}
1002|
1003|                 if len(data["name"]) > 30:
1004|                     return status, {error_message:"Name field
can't exceed 30 characters"}
1005|
1006|                     if data["profession"] not in PROFESSIONS:
1007|                         return status, {error_message:"Job isn't
recognized"}
1008|
1009|
1010|                     if not self._check_city(data["cities"]):
1011|                         return status, {error_message:"City not
recognized"}
1012|
1013|
1014|                     if data["years"] > 40:
1015|                         return status, {error_message:"Experience
seems a bit off..."}
1016|
1017|

```

```

1018|         try:
1019|             start_dt =
datetime.strptime(data["startWorking"], "%H:%M")
1020|             finish_dt =
datetime.strptime(data["finishWorking"], "%H:%M")
1021|             job_duration_dt =
datetime.strptime(data["jobDuration"], "%H:%M")
1022|             job_duration =
timedelta(hours=job_duration_dt.hour,
minutes=job_duration_dt.minute)
1023|         except:
1024|             return status, {error_message:"Work, sta
or finish time bad format"}
1025|
1026|
1027|             if start_dt >= finish_dt:
1028|                 return status, {error_message:"Start wor
time must be before finish work time"}
1029|
1030|
1031|                 if job_duration.total_seconds() <= 0 or
job_duration >= timedelta(days=1):
1032|                     return status, {error_message:"Enter pro
job duration"}
1033|
1034|                     first_job_time = start_dt + job_duration
1035|                     if first_job_time > finish_dt:
1036|                         return status, {error_message:"You won't
complete a single job..."}
1037|
1038|
1039|                     if data["role"] not in ["Freelancer", "User"]
1040|                         return status, {error_message:"No such u
type"}
1041|
1042|                     if type(data["hourPrice"]) != int:
1043|                         return status, {error_message:"price per
hour is not in correct format "}
1044|
1045|                     if data["hourPrice"] > 1000:
1046|                         return status, {error_message: "your wag
is to expensive for this site"}
1047|

```

```

1048|         if len(data["description"]) > 500:
1049|             return status, {error_message:f"Description
must be less than 500 characters, yours is
{len(data['description'])}"}}
1050|
1051|
1052|
1053|             status = True
1054|             return status,
{StatusMessage.SIGNING_UP.value:""}
1055|         except Exception as e:
1056|             print("Freelancer validator - ", e)
1057|             return False,
{StatusMessage.FAILED_SIGN_UP.value:"Not all fields were sent"}
1058|
1059|
1060|     def _check_city(self, cities):
1061|         if type(cities) != list:
1062|             return False
1063|
1064|         for city in cities:
1065|             if city not in LOCALITIES:
1066|                 return False
1067|         return True
1068|
1069|
1070|     def _check_email_format(self, email):
1071|         pattern = r'^[\w\.-]+@[\w\.-]+\.\w+$'
1072|         return re.match(pattern, email) is not None
1073| #endregion
1074|
1075|
1076|
1077|
1078|
1079| class EmailVerification:
1080|     def __init__(self, email):
1081|         self.email = email
1082|         self.verification_code = f"{random.randint(0,
999999):06d}"
1083|         self.expires_at =
datetime.now()+timedelta(minutes=5)
1084|

```



```

1085|
1086| def send_verification_code(self):
1087|     message = MIMEMultipart("alternative")
1088|
1089|     message["Subject"] = "Verification email"
1090|     message["From"] = EMAIL
1091|     message["To"] = self.email
1092|
1093|     html = f""" <html>
1094|                 <body style="font-family: Arial,
1095|                 sans-serif; color: #333;">
1096|                     <div style="max-width: 600px;
1097|                 margin: auto; padding: 20px; border: 1px solid #ddd;
1098|                 border-radius: 10px; background-color: #f9f9f9;">
1099|                         <h2 style="color:
1100|                 #2a9d8f;">Welcome to Pro-Find!</h2>
1101|                         <p>Hi there,</p>
1102|                         <p>Thanks for signing up. Your
1103|                 verification code is:</p>
1104|                         <p style="font-size: 24px;
1105|                 font-weight: bold; color: #e76f51;">{self.verification_code}</p>
1106|                         <p>Please enter this code in
1107|                 the app to verify your account.</p>
1108|                         <hr>
1109|                         <p style="font-size: 12px;
1110|                 color: #888;">If you didn't sign up, you can ignore this
1111|                 email.</p>
1112|                     </div>
1113|                 </body>
1114|             </html>
1115|             """
1116|
1117|     text = f"Hi there!\nYour verification code is
1118| {self.verification_code}\nPlease enter it in the app."
1119|     message.attach(MIMEText(text, "plain"))
1120|     message.attach(MIMEText(html, "html"))
1121|
1122|     try:
1123|         with smtplib.SMTP_SSL("smtp.gmail.com", 465)
1124|         server:
1125|             server.login(EMAIL, AUTHENTICATION_PASSWORD)
1126|             server.sendmail(EMAIL, self.email,
1127|                             message.as_string())

```

```

1116|         print("Email sent successfully!")
1117|         return True
1118|     except Exception as e:
1119|         print(f"Error: {e}")
1120|         return False
1121|
1122|
1123|     @staticmethod
1124|     def check_verification_code(msg:Message) -> tuple
[bool, dict]:
1125|         try:
1126|             email = msg.data["email"]
1127|             veri_code = msg.data["verification_code"]
1128|             with sqlite3.connect(DATABASE) as conn:
1129|                 cur = conn.cursor()
1130|                 cur.execute("SELECT verification_code,
expires_at FROM pending_users WHERE email=?", (email,))
1131|                 data = cur.fetchone()
1132|                 if not data:
1133|                     return False,
{StatusMessage.VERIFICATION_BAD.value:"Email not found"}
1134|
1135|                 else:
1136|                     expires_at = datetime.strptime(data[
"%Y-%m-%d %H:%M:%S.%f")
1137|                     veri_code_fetched = data[0]
1138|
1139|                     if expires_at < datetime.now():
1140|                         cur.execute("""DELETE FROM
pending_users WHERE email=?""", (email,))
1141|                         conn.commit()
1142|                         return False,
{StatusMessage.VERIFICATION_BAD.value:"Verification time expired,
restart the process"}
1143|
1144|                         if veri_code_fetched == veri_code:
1145|                             return True,
{StatusMessage.VERIFICATION_GOOD.value:""}
1146|
1147|                         else:
1148|                             return False,
{StatusMessage.VERIFICATION_BAD.value:"Verification code does
match"}

```

```
1149|  
1150|  
1151|         except Exception as e:  
1152|             print (f"Verification code check - {e}")  
1153|             return False,  
{StatusMessage.VERIFICATION_BAD.value:"Not all fields were ser
```

My_WebSocket.py

```

0 | import socket
1 | import hashlib
2 | import base64
3 | from enum import Enum
4 | import json
5 | from message_types import MessageTypes
6 | import ssl
7 |
8 | #region ---> Handles the WebSocket handshake
9 |
10| def recv_http_handshake_msg(soc:socket.socket) ->bytes:
11|     data=b""
12|     while b'\r\n\r\n' not in data:
13|         chunk=soc.recv(1024)
14|         if not chunk:
15|             return data
16|         data+=chunk
17|     return data
18|
19|
20|
21| class HttpMessage:
22|     def __init__(self, http_method, protocol_version, host,
23| upgrade,connection, websocket_key):
24|         self.http_method = http_method
25|         self.protocol_version = protocol_version
26|         self.host = host
27|         self.upgrade = upgrade
28|         self.connection = connection
29|         self.websocket_key = websocket_key
30|
31|     def __repr__(self):
32|         return (
33|             f"Http method          : {self.http_method}\n"
34|             f"Protocol version          : {self.protocol_version}\n"
35|             f"Server                      : {self.host}\n"
36|             f"Upgrade to                  : {self.upgrade}\n"
37|             f"Connection type            : {self.connection}\n"
38|             f"WebSocket key               : {self.websocket_key}"
39|         )

```

```

40 |
41 |
42 | class HttpMessageParser:
43 |
44 |     @staticmethod
45 |     def parse_http_message(msg: bytes) -> HttpMessage:
46 |         headers = msg.split(b'\r\n\r\n')[0].decode()
47 |         lines = headers.split('\r\n')
48 |         http_method, _, protocol_version = lines[0].split(
49 |             ' ')
50 |
51 |         header_dict = {}
52 |         for line in lines[1:]:
53 |             key, value = line.split(": ", 1)
54 |             header_dict[key] = value
55 |         try:
56 |             return HttpMessage(
57 |                 http_method,
58 |                 protocol_version,
59 |                 host=header_dict["Host"],
60 |                 upgrade=header_dict["Upgrade"],
61 |                 connection=header_dict["Connection"],
62 |                 websocket_key=header_dict["Sec-WebSocket-Key"],
63 |             )
64 |         except Exception as e:
65 |             print(f"Found exception ---- {e}")
66 |             return None
67 |
68 | #Build the Http Upgrade message
69 | class HttpUpgrade:
70 |
71 |     def __init__(self, http_message:HttpMessage):
72 |         self.message = http_message
73 |
74 |     def to_upgrade_to_WebSocket(self) -> bool:
75 |         if self.message.connection != "Upgrade":
76 |             return False
77 |         elif self.message.upgrade != "websocket":
78 |             return False
79 |         else:
80 |             return True

```

```

81|
82|     def upgrade_response(self) -> str:
83|         GUID = "258EAF55-E914-47DA-95CA-C5AB0DC85B11"
84|         contecation:str =
self.message.websocket_key.strip()+GUID
85|         sha1_hash =
hashlib.sha1(contecation.encode()).digest()
86|         accept_text =
base64.b64encode(sha1_hash).decode("ASCII")
87|
88|         version = "HTTP/1.1 "
89|         accept_code = "101 Switching Protocols\r\n"
90|         upgrade = f"Upgrade: {self.message.upgrade}\r\n"
91|         connection = f"Connection:
{self.message.connection}\r\n"
92|         accept_key = f"Sec-WebSocket-Accept:
{accept_text}\r\n"
93|         return_text =
f"{version}{accept_code}{upgrade}{connection}{accept_key}\r\n"
94|         return return_text
95|
96| #endregion
97|
98|
99|
100|
101| class WebSocketOpCodes(Enum):
102|     CONTINUATION = 0x0
103|     TEXT = 0x1
104|     BINARY = 0x2
105|     CLOSE_CONNECTION = 0x8
106|     PING_MESSAGE = 0x9
107|     PONG_MESSAGE = 0xA
108|
109|
110| #Builds WebSocket messages
111| class WebSocketMessageFactory:
112|     def __init__(self, opcode:WebSocketOpCodes,
to_split=None, payload=None): #All types of message except for
pong
113|         self.opcode:WebSocketOpCodes = opcode
114|         self.to_split = to_split
115|         self.payload = payload

```

```

116|         if self.payload ==None:
117|             self.payload = ''
118|
119|         if self.opcode != WebSocketOpcodes.PONG_MESSAGE and
to_split==None:
120|             raise RuntimeError("Expected to receive
'to_split' value but none was given")
121|         else:
122|             self.message_to_send =
self._build_pong_message() if self.opcode ==
WebSocketOpcodes.PONG_MESSAGE else
self._build_websocket_message()
123|
124|
125|     def __repr__(self):
126|         to_print = f"Message content type - {self.opcode}"
127|         if self.to_split != None:
128|             to_print+=f"\nMessage needs to be splitted -
{'True' if self.to_split else 'False'}"
129|         if self.payload:
130|             to_print+=f"\nMessage content - {self.payload}"
131|
132|         to_print+=f"\nMessage to be sent -
{self.message_to_send}"
133|         return to_print
134|
135|
136|     def _build_websocket_message(self) ->bytes:
137|         frame_bytes = bytearray()
138|
139|         fin = 0 if self.to_split else 1
140|         frame_bytes.append((fin<<7) | self.opcode.value)
141|
142|         mask = 0
143|         bytes_payload = self.payload if
type(self.payload)==bytes else self.payload.encode("UTF-8")
144|
145|         def get_length_bits(bytes_payload):
146|             payload_length = len(bytes_payload)
147|
148|             if payload_length<=125:
149|                 length_field = payload_length
150|                 extended_length_bytes = b''

```

```

151|
152|         elif payload_length<=65535:
153|             length_field = 126
154|             extended_length_bytes =
payload_length.to_bytes(2, "big")
155|
156|         else:
157|             length_field = 127
158|             extended_length_bytes =
payload_length.to_bytes(8, "big")
159|
160|             return length_field, extended_length_bytes
161|
162|     length_field, extend_length_bytes =
get_length_bits(bytes_payload)
163|     frame_bytes.append((mask<<7)|length_field)
164|     frame_bytes.extend(extend_length_bytes)
165|     frame_bytes.extend(bytes_payload)
166|     return (bytes(frame_bytes))
167|
168|
169|     def _build_pong_message(self):
170|         frame_bytes = bytearray()
171|         fin = 1
172|
frame_bytes.append((fin<<7)|WebSocketOpcodes.PONG_MESSAGE.valu
173|
174|
175|         mask = 0
176|         if self.payload:
177|             bytes_payload = self.payload if
type(self.payload)==bytes else self.payload.encode("UTF-8")
178|             payload_length = len(bytes_payload)
179|             frame_bytes.append((mask<<7)|payload_length)
180|             frame_bytes.extend(bytes_payload)
181|         else:
182|             payload_length = 0
183|             frame_bytes.append((mask<<7)|payload_length)
184|
185|         return(bytes(frame_bytes))
186|
187|
188|

```



```

189| #region ---> WebSocket message parser
190|
191|
192| class WebSocketFrame:
193|     def __init__(self, fin, opcode, masked, payload_length,
payload, masking_key=None):
194|         self.message_is_split = False if fin == 1 else True
195|         self.opcode = WebSocketOpCodes(opcode)
196|         self.masked = bool(masked)
197|         self.payload_length = payload_length
198|         self.payload = payload
199|         if masking_key:
200|             self.masking_key = masking_key
201|
202|     def __repr__(self):
203|         to_print = ""
204|         if self.message_is_split:
205|             to_print+="Message is splitted"
206|         else:
207|             to_print+="This is the whole message"
208|             to_print+=f"\nMessage type :{self.opcode}"
209|             to_print+=f"\nthis message is {'masked' if
self.masked else 'not masked'}"
210|             to_print+=f"\nThe payload length is
:{self.payload_length}"
211|             to_print+=f"\nPayload :{self.payload}"
212|             if self.masking_key:
213|                 to_print+=f"\nMasking key :{self.masking_key}"
214|             return to_print
215|
216|
217|
218| class WebSocketMessageParser:
219|     @staticmethod
220|     def _parse_one_frame(clt:socket.socket)
->WebSocketFrame:
221|         first_byte = clt.recv(1)[0]
222|         fin = (first_byte>>7) & 1
223|         opcode = first_byte & 0x0F
224|
225|         second_byte = clt.recv(1)[0]
226|         masked = (second_byte>>7) & 1
227|

```

```

228|         payload_len = second_byte & 0x7F
229|         if payload_len == 126:
230|             bytes_of_extended = clt.recv(2)
231|             payload_len = int.from_bytes(bytes_of_extended,
"big")
232|         elif payload_len == 127:
233|             bytes_of_extended = clt.recv(8)
234|             payload_len = int.from_bytes(bytes_of_extended,
"big")
235|
236|         if masked:
237|             mask = clt.recv(4)
238|         else:
239|             mask= None
240|
241|         payload = b""
242|         bytes_to_read = payload_len
243|         while bytes_to_read>0:
244|             chunk = clt.recv(bytes_to_read)
245|             if not chunk:
246|                 raise RuntimeError("Connection closed by
client")
247|             payload+=chunk
248|             bytes_to_read-=len(chunk)
249|
250|         if masked:
251|             payload = bytes(b ^ mask[i % 4] for i, b in
enumerate(payload))
252|
253|         return WebSocketFrame(fin, opcode, masked,
payload_len, payload, masking_key=mask)
254|
255|     @staticmethod
256|     def parse_message(clt:socket.socket) ->bytes:
257|         buffer = b''
258|
259|         while True:
260|             frame =
WebSocketMessageParser._parse_one_frame(clt)
261|
262|             if frame.opcode in
{WebSocketOpcodes.CLOSE_CONNECTION,

```

```

263|
WebSocketOpcodes.PING_MESSAGE,
264|
WebSocketOpcodes.PONG_MESSAGE}:
265|         if frame.payload_length>=125 or
frame.message_is_split:
266|                 return b'FAULTY FRAME'
267|         if frame.opcode
==WebSocketOpcodes.CLOSE_CONNECTION:
268|                 clt.close()
269|                 return b"CLIENT CLOSED"
270|         elif
frame.opcode==WebSocketOpcodes.PING_MESSAGE:
271|                 to_send =
WebSocketMessageFactory(WebSocketOpcodes.PONG_MESSAGE,
payload=frame.payload)
272|                 clt.send(to_send.message_to_send)
273|                 continue
274|
275|         if frame.opcode not in
{WebSocketOpcodes.CONTINUATION,
276|                                     WebSocketOpcodes.TEXT
277|                                     WebSocketOpcodes.BINARY
or (frame.opcode == WebSocketOpcodes.CONTINUATION and buffer =
b'') :
278|                 return b"FAULTY FRAME"
279|
280|                 buffer+=frame.payload
281|
282|                 if not frame.message_is_split:
283|                         break
284|
285|                 return buffer
286|
287| #endregion
288|
289|
290| #region accept client
291| def accept_client(clt_soc):
292|     new_soc = accept_TLS_encryption(clt_soc)
293|     accept_websocket_upgrade_request_from_client(new_soc)
294|     return new_soc
295|

```

```

296|
297| def accept_TLS_encryption(clt_soc):
298|     CERTIFICATE_PATH =
r"C:\Coding\pro-find\certificate\server.crt"
299|     KEY_PATH = r"C:\Coding\pro-find\certificate\server.key"
300|     # CERTIFICATE_PATH
=r"D:\pro-find\certificate\server.crt"
301|     # KEY_PATH = r"D:\pro-find\certificate\server.key"
302|     context = ssl.SSLContext(ssl.PROTOCOL_TLS_SERVER)
303|     context.verify_mode = ssl.CERT_NONE
304|     context.load_cert_chain(certfile=CERTIFICATE_PATH,
keyfile=KEY_PATH)
305|     tls_soc = context.wrap_socket(clt_soc, server_side=True)
306|     return tls_soc
307|
308|
309| def
accept_websocket_upgrade_request_from_client(clt_soc:socket.socket):
310|     info = recv_http_handshake_msg(clt_soc)
311|     http_message =
HttpMessageParser.parse_http_message(info)
312|
313|     accept_WebSocket_upgrade = HttpUpgrade(http_message)
314|     if accept_WebSocket_upgrade.to_upgrade_to_WebSocket():
315|         to_send =
accept_WebSocket_upgrade.upgrade_response()
316|         clt_soc.send(to_send.encode())
317|         print("Connected")
318| #endregion
319|
320|
321|
322|
323| def build_proper_json_payload(type:MessageTypes, payload,
token):
324|     type_of_message = type.value
325|     type_of_data = "JSON"
326|
327|     if isinstance(payload, bytes):
328|         safe_payload =
base64.b64encode(payload).decode("utf-8")
329|         type_of_data = "BYTES"

```

```

330|     else:
331|         try:
332|             json.dumps(payload)
333|             safe_payload = payload
334|         except (TypeError, OverflowError):
335|             safe_payload = str(payload)
336|
337|     return {
338|         "type": type_of_message,
339|         "data": safe_payload,
340|         "data_type": type_of_data,
341|         "token": token
342|     }
343|
344|
345| def send_message(clt: socket.socket, type, token, msg=None,
to_split_message=None,
opcode:WebSocketOpCodes=WebSocketOpCodes.TEXT):
346|     try:
347|         msg_dict = build_proper_json_payload(type, msg,
token) if msg is not None else None
348|
349|         msg_to_send = json.dumps(msg_dict)
350|
351|         message = WebSocketMessageFactory(opcode,
to_split=to_split_message, payload=msg_to_send)
352|         clt.send(message.message_to_send)
353|         return True
354|
355|     except Exception as e:
356|         print("Failed to send message:", e)
357|         return False
358|
359|
360|
361|
362|
363| def recv_message(clt:socket.socket):
364|     try:
365|         parsed_message =
WebSocketMessageParser.parse_message(clt)
366|         if parsed_message == b'FAULTY FRAME' or
parsed_message == b"CLIENT CLOSED":

```

```
367|         return None
368|     else:
369|         return json.loads(parsed_message)
370| except Exception as e:
371|     print(e)
372|     return None
```

```
0 | import socket
1 | import threading
2 | import My_WebSocket
3 | import MessageHandler
4 | from datetime import datetime, timedelta
5 | from collections import deque
6 | import time
7 | import sqlite3
8 |
9 |
10| MAX_MESSAGES = 15
11| TIME_WINDOW = 20
12|
13| dispatcher = MessageHandler.configure_dispatcher()
14| DATABASE = r"C:\Coding\pro-find\Python\my_app.db"
15|
16| MAX_CONNECTIONS_PER_IP = 5
17| IP_TIMEOUT = 60
18|
19| ip_connections = {}
20|
21|
22|
23|
24| def can_accept_connection(ip: str):
25|     now = time.time()
26|     if ip in ip_connections:
27|         ip_connections[ip] = [t for t in ip_connections[ip]
28| if now - t < IP_TIMEOUT]
29|         if len(ip_connections[ip]) >= MAX_CONNECTIONS_PER_IP:
30|             return False
31|
32|         ip_connections[ip].append(now)
33|         return True
34|     else:
35|         ip_connections[ip] = [now]
36|         return True
37|
38|
39|
```

```

40 |
41 | def cleanup_tables():
42 |     while True:
43 |         try:
44 |             now = datetime.now()
45 |             current_date = now.strftime("%Y-%m-%d")
46 |             current_time = now.strftime("%H:%M")
47 |             one_week_ago = (now -
timedelta(days=7)).strftime("%Y-%m-%d %H:%M:%S")
48 |             two_day_ago = (now -
timedelta(days=2)).strftime("%Y-%m-%d %H:%M:%S")
49 |
50 |             with sqlite3.connect(DATABASE) as conn:
51 |                 cur = conn.cursor()
52 |
53 |                 cur.execute("""
54 |                     DELETE FROM pending_users
55 |                     WHERE expires_at IS NOT NULL AND
expires_at <= CURRENT_TIMESTAMP
56 |                     """)
57 |
58 |                 cur.execute("""DELETE FROM appointments WHERE
date < ? OR (date = ? AND start_time < ?)
59 |                 """, (current_date, current_date,
current_time))
60 |
61 |                 cur.execute("""DELETE FROM notifications
WHERE created_at < ? OR (created_at < ? AND is_read = 1)
62 |                 """, (one_week_ago, two_day_ago))
63 |
64 |                 conn.commit()
65 |
66 |         except Exception as e:
67 |             print(f"Cleanup error - {e}")
68 |
69 |             time.sleep(3600)
70 |
71 |
72 |
73 | def handle_client(soc: socket.socket):
74 |     clt_soc = My_WebSocket.accept_client(soc)
75 |
76 |     message_times = deque()

```



```

77|
78|     while True:
79|         try:
80|             while message_times and time.time() -
message_times[0] > TIME_WINDOW:
81|                 message_times.popleft()
82|
83|                 if len(message_times) >= MAX_MESSAGES:
84|                     print("Rate limit exceeded, ignoring
message")
85|                     time.sleep(0.5)
86|                     continue
87|
88|                     payload = My_WebSocket.recv_message(clt_soc)
89|
90|                     message_times.append(time.time())
91|
92|                     msg = MessageHandler.Message(payload)
93| except Exception as e:
94|     print(f"Exception in recieving - {e}")
95|     break
96|
97|     print(f"Got - {msg.type_of}, {msg.data}\n\n")
98|     handler_type, to_send = dispatcher.dispatch(msg)
99|     print(f"Sending - {handler_type}, {to_send}\n\n")
100|     My_WebSocket.send_message(
101|         clt_soc,
102|         handler_type,
103|         msg.token,
104|         to_send,
105|         False
106|     )
107|
108|
109|
110|
111| def main_thread(srv_soc:socket.socket):
112|     while True:
113|         clt, addr = srv_soc.accept()
114|         ip = addr[0]
115|         if not can_accept_connection(ip):
116|             print(f"Too many connections from {ip},
rejecting")

```

```
117|         clt.close()
118|         continue
119|         t = threading.Thread(target=handle_client,
args=(clt,), daemon=True)
120|         t.start()
121|
122|
123|
124|
125| if __name__ == "__main__":
126|     srv = socket.socket()
127|     srv.bind(("0.0.0.0", 1111))
128|     srv.listen(10)
129|
130|     cleanup_thread = threading.Thread(target=cleanup_table,
daemon=True)
131|     cleanup_thread.start()
132|
133|     main_thread(srv)
134|
135|
```

token_handler.py

```
0 | from json import loads, dumps
1 | from hashlib import sha256
2 | from base64 import b64decode, b64encode
3 | import sqlite3
4 | from datetime import datetime, timedelta
5 |
6 | sec = b"CYBERISH"
7 |
8 | DATABASE = r"C:\Coding\pro-find\Python\my_app.db"
9 |
10|
11| def decode(token: str):
12|     token_split = token.split(".")
13|     if len(token_split) != 2:
14|         return None
15|
16|     enc_data, enc_sign = token_split
17|     try:
18|         dec_data = b64decode(enc_data)
19|         data = loads(dec_data)
20|         provided_sign = b64decode(enc_sign)
21|     except:
22|         return None
23|
24|     calc_sign = sha256(sec + dec_data).hexdigest().encode()
25|     if provided_sign != calc_sign:
26|         return None
27|     return data
28|
29|
30|
```

```
31| def is_token_valid(token:str):
32|     dec_tkn = decode(token)
33|
34|     if not dec_tkn:
35|         return False
36|     exp = datetime.fromtimestamp(dec_tkn["exp"])
37|
38|     if exp < datetime.now():
39|         return False
40|     return True
41|
42|
43|
44| def _encode(data):
45|     dec_data = dumps(data).encode()
46|     enc_data = b64encode(dec_data).decode()
47|     dec_sign = sha256(sec + dec_data).hexdigest().encode()
48|     enc_sign = b64encode(dec_sign).decode()
49|     return f"{enc_data}.{enc_sign}"
50|
51|
52|
53| def create_token(user_email:str):
54|     with sqlite3.connect(DATABASE) as conn:
55|         cur = conn.cursor()
56|
57|         cur.execute("SELECT id, full_name, created_at FROM
users WHERE email=?", (user_email,))
58|         row = cur.fetchone()
59|         if not row:
60|             return None
61|         id, name, created_at = row
```

```
62 |         tkn_obj = {"id":id, "email": user_email, "name":name,
    |         "Creation time": created_at,
    |         "exp":int((datetime.now()+timedelta(days=1)).timestamp())}
63 |         return _encode(tkn_obj)
64 |
```

.env file

```
0 | AUTHENTICATION_PAS=dhgSDxvrlemuyup
1 | PEPPER=CYBERISH
2 | EMAIL=x1xprofindx1x@gmail.com
```

RouteTracker

```
0 | import { useEffect } from "react";
1 | import { useLocation } from "react-router-dom";
2 |
3 | export default function RouteTracker() {
4 |     const location = useLocation()
5 |
6 |     useEffect(() => {
7 |         localStorage.setItem("lastRoute", location.pathname)
8 |     }, [location,])
9 |     return null
10| }
```

main.tsx

```

0 | import { StrictMode } from 'react'
1 | import { createRoot } from 'react-dom/client'
2 | import { Provider } from 'react-redux'
3 | import store, { persistor } from './store/store.js'
4 | import { BrowserRouter } from "react-router-dom"
5 | import { SocketProvider } from './socket/SocketContext.js'
6 | import { PersistGate } from
  | "redux-persist/integration/react";
7 | import App from './App.jsx'
8 | import './index.css'
9 |
10 |
11 | const rootElement = document.getElementById('root')!
12 | createRoot(rootElement).render(
13 |   <StrictMode>
14 |     <Provider store={store}>
15 |       <PersistGate loading={null} persistor={persistor}>
16 |         <SocketProvider>
17 |           <BrowserRouter>
18 |             <App />
19 |           </BrowserRouter>
20 |         </SocketProvider>
21 |       </PersistGate>
22 |     </Provider>
23 |   </StrictMode>
24 | )
25 |

```

index.css

```

0 | @import "tailwindcss";
1 |
2 | * {
3 |     scrollbar-width: thin;
4 |     scrollbar-color: #475569 #1e293b;

```

App.jsx

```

0 | import './App.css';
1 | import { Routes, Route, useNavigate } from
  "react-router-dom";
2 | import { useEffect } from "react";
3 | import LogIn from './pages/login.jsx';
4 | import HomePage from './pages/homePage.jsx';
5 | import ProtectedRoutes from './pages/protectedRoutes.jsx';
6 | import SignUpPage from './pages/signUp.jsx';
7 | import Search from './pages/search.jsx';
8 | import RouteTracker from './RouteTracker.jsx';
9 | import { useSelector } from 'react-redux';
10|
11|
12| export default function App() {
13|   const navigate = useNavigate()
14|   const isLoggedIn = useSelector((state) =>
state.auth.loggedIn)
15|
16|   useEffect(() => {
17|     const lastRoute = localStorage.getItem("lastRoute");
18|
19|     const isGuestPage = lastRoute === "/login" || lastRoute
=== "/signup";
20|
21|     if (isLoggedIn && lastRoute && !isGuestPage) {
22|       navigate(lastRoute, { replace: true });
23|     }
24|
25|     else if (isLoggedIn && (isGuestPage || !lastRoute)) {
26|       navigate("/", { replace: true });
27|     }
28|
29|   }, [navigate, isLoggedIn]);
30|
31|
32|   return (
33|     <>
34|       <RouteTracker />
35|       <Routes>
36|         <Route path="/login" element={<LogIn />} />
37|         <Route path="/signup" element={<SignUpPage />} />

```



```
38 |         <Route element={<ProtectedRoutes />}>
39 |             <Route path="/" element={<HomePage
isPublic={false} />} />
40 |             <Route path="/profile/:id" element={<HomePage
isPublic={true} />} />
41 |             <Route path="/search" element={<Search/>} />
42 |         </Route>
43 |     </Routes>
44 | </>
45 | );
46 | }
47 |
```

store.js

```
0 | import { configureStore, combineReducers } from
  | "@reduxjs/toolkit";
1 | import { persistStore, persistReducer, FLUSH, REHYDRATE,
  | PAUSE, PERSIST, PURGE, REGISTER } from "redux-persist";
2 | import storage from "redux-persist/lib/storage"; //
  | localStorage
3 |
4 | import authReducer from './authSlice';
5 | import appointmentReducer from './appointmentsSlice';
6 |
7 | const rootReducer = combineReducers({
8 |   auth: authReducer,
9 |   appointment: appointmentReducer,
10| });
11|
12| const persistConfig = {
13|   key: "root",
14|   storage,
15| };
16|
17| const persistedReducer = persistReducer(persistConfig,
  | rootReducer);
18|
19| export const store = configureStore({
20|   reducer: persistedReducer,
21|   middleware: (getDefaultMiddleware) =>
22|     getDefaultMiddleware({
23|       serializableCheck: {
24|         ignoredActions: [FLUSH, REHYDRATE, PAUSE,
  | PERSIST, PURGE, REGISTER],
25|       },
26|     }),
27| });
28|
29| export const persistor = persistStore(store);
30|
31| export default store;
```

authSlice.js

```
0 | import {createSlice} from '@reduxjs/toolkit'
1 |
2 | const initialState = {
3 |   userToken: "",
4 |   userInfo: {},
5 |   loggedIn: false,
6 |   notifications: []
7 | }
8 |
9 | const authSlice = createSlice({
10 |   name: "auth",
11 |   initialState,
12 |   reducers:{
13 |     login(state, info){
14 |       state.loggedIn = true;
15 |       state.userToken = info.payload;
16 |     },
17 |     setUserInfo(state, info){
18 |       state.userInfo = info.payload.user
19 |       state.notifications = info.payload.notifications
20 |     },
21 |     logout(state){
22 |       state.userInfo = {};
23 |       state.userToken = "";
24 |       state.loggedIn=false;
25 |       state.notifications=[];
26 |     }
27 |   }
28 | })
29 |
30 |
31 |
32 | export const {login, setUserInfo, logout} = authSlice.actions
33 | export default authSlice.reducer
```

appointmentsSlice.js

```

0 | import { createSlice } from '@reduxjs/toolkit'
1 |
2 |
3 | const initialState = {
4 |   userAppointments: [],
5 |   freelancerAppointments: []
6 | }
7 |
8 | const appointmentSlice = createSlice({
9 |   name: "appointments",
10 |   initialState,
11 |   reducers: {
12 |     appointmentsRecvd(state, action) {
13 |       state.userAppointments = action.payload
14 |       state.freelancerAppointments = action.payload
15 |     },
16 |     appointmentAddedForUser(state, action) {
17 |       state.userAppointments.push(action.payload)
18 |     },
19 |     appointmentAddedForFreelancer(state, action) {
20 |       state.freelancerAppointments.push(action.payload)
21 |     },
22 |     appointmentRemovedForUser(state, action) {
23 |       state.userAppointments =
state.userAppointments.filter(a => a !== action.payload)
24 |     },
25 |     appointmentRemovedForFreelancer(state, action) {
26 |       state.freelancerAppointments =
state.freelancerAppointments.filter(a => a !== action.payload)
27 |     }
28 |   }
29 | })
31 | export const {
32 |   appointmentsRecvd,
33 |   appointmentAddedForUser,
34 |   appointmentAddedForFreelancer,
35 |   appointmentRemovedForUser,
36 |   appointmentRemovedForFreelancer
37 | } = appointmentSlice.actions
39 | export default appointmentSlice.reducer

```

SocketContext.jsx

```
0 | import {createContext, useContext, useState, useEffect} fr
  | 'react'
1 |
2 | const SocketContext = createContext(null)
3 | let wsSingleton = null;
4 |
5 | const getWebSocket = (ip, port) =>{
6 |     if (!wsSingleton){
7 |         wsSingleton = new WebSocket(`wss://${ip}:${port}`)
8 |         wsSingleton.onopen = () =>console.log("Socket
Opened");
9 |         wsSingleton.onclose=() =>console.log("Socket
Closed");
10|     }
11|     return wsSingleton;
12| }
13|
14| export const SocketProvider = ({children}) =>{
15|     const [socket, setSocket] = useState(null)
16|
17|     useEffect(() =>{
18|         const IP = "omer.ystr";
19|         const PORT = "1111";
20|         const ws = getWebSocket(IP, PORT)
21|         setSocket(ws)
22|     }, [])
23|
24|     return (
25|         <SocketContext.Provider value={socket}>
26|             {children}
27|         </SocketContext.Provider>
28|     )
29| }
30|
31| export const useSocket = () => useContext(SocketContext)
```

ResponseHandler.js

```

0 | import {MessageTypes, StatusMessage} from "../MsgTypes"
1 | import {login, setUserInfo} from "../store/authSlice"
2 | import { appointmentsRecvd } from
  | "../store/appointmentsSlice";
3 |
4 |
5 | // #region Login
6 | export const handleLoginResponse = (payload, dispatch) =>{
7 |     if (MessageTypes.LOGIN == payload.type)
8 |         if (StatusMessage.LOGGED_IN in payload.data)
9 |             {
10 |
11 | dispatch(login(payload.data[StatusMessage.LOGGED_IN]))
12 |         return [true,
13 | payload.data[StatusMessage.LOGGED_IN]];
14 |     }
15 |     else if (StatusMessage.FAILED_LOG_IN in payload.data)
16 |         return [false,
17 | payload.data[StatusMessage.FAILED_LOG_IN]];
18 |     else
19 |         console.log("Unknown response")
20 |     else
21 |         console.log("Unknown message type")
22 |
23 | }
24 | // #endregion
25 |
26 | // #region SignUp
27 | export const handleSignUpResponse = (payload) =>{
28 |     let signupFailed = StatusMessage.FAILED_SIGN_UP;
29 |     let signupSuccess = StatusMessage.SIGNING_UP;
30 |     if (MessageTypes.USER_SIGNUP === payload.type ||
31 | MessageTypes.FREELANCER_SIGNUP === payload.type){
32 |         if (payload.data && signupFailed in payload.data){
33 |             console.log("Failed")
34 |             return [false, payload.data[signupFailed]]
35 |         }
36 |     }
37 |     else{
38 |         return [true, payload.data[signupSuccess]]

```

```
36|         }
37|     }
38| }
39| //endregion
40|
41|
42|
43| //region Verification
44| export const handleVerificationReponse = (payload) =>{
45|     let data = payload.data
46|     if (StatusMessage.VERIFICATION_BAD in data){
47|         return [false, data[StatusMessage.VERIFICATION_BAD]]
48|     }
49|     else if (StatusMessage.VERIFICATION_GOOD in data)
50|         return [true, data[StatusMessage.VERIFICATION_GOOD]]
51|     return [false, "Server error"]
52| }
53| //endregion
54|
55|
56|
57| //region Forgot password
58| export const handleForgotPasswordVerificationCodeResponse
59| =(payload) =>{
60|     let data = payload.data
61|     if (StatusMessage.FORGOT_PASSWORD_GOOD in data)
62|         return [true,
63| data[StatusMessage.FORGOT_PASSWORD_GOOD]]
64|     else
65|         return [false,
66| data[StatusMessage.FORGOT_PASSWORD_BAD]]
67| }
68|
69| export const handleForgotPasswordVerifyCodeResponse =
70| (payload) =>{
71|     let data = payload.data
72|     if (StatusMessage.FORGOT_PASSWORD_GOOD in data)
73|         return [true, StatusMessage.FORGOT_PASSWORD_GOOD]
74|     else
75|         return [false, StatusMessage.FORGOT_PASSWORD_BAD]
76| }
77|
78| export const handleChangePasswordResponse = (payload) =>{
```

```

75|     let data = payload.data
76|     if (StatusMessage.CHANGE_PASSWORD_BAD in data)
77|         return [false,
data[StatusMessage.CHANGE_PASSWORD_BAD]]
78|     else
79|         return
[true, data[StatusMessage.CHANGE_PASSWORD_GOOD]]
80| }
81|
82| // #endregion
83|
84|
85|
86| // #region Homepage
87| export const handleUserInfoResponse = (dispatch, payload)
88|     let data = payload.data
89|
90|     if (StatusMessage.FAILED_TO_GET_USER_INFO in data) {
91|         return [false, "Issue retrieving info"]
92|     }
93|     else {
94|         const accInfo = data[StatusMessage.GOT_USER_INFO]
95|         let toInsert = {"user": {}}
96|         for (const key in accInfo) {
97|             if (key === "appointments" ||
key === "notifications") continue;
98|
99|             toInsert["user"][key] = accInfo[key]
100|         }
101|         toInsert["user"]["id"] = data["user_id"]
102|         toInsert["notifications"] = accInfo["notification"]
103|         dispatch(setUserInfo(toInsert))
104|         dispatch(appointmentsRecvd(accInfo["appointments"]
105|         return [true, data["user_id"]]
106|     }
107| }
108|
109|
110| export const handleUpdatedAppointmentsResponse = (payload) => {
111|     let data = payload.data
112|     if (StatusMessage.FAILED_TO_UPDATE_APP_STATUS in data)
113|         return [false, "Failed to update status"]

```



```
114|     else if (StatusMessage.UPDATED_APP_STATUS in data)
115|         return [true, ""]
116| }
117|
118|
119| export const handleMarkedReadNotifications = (payload) =>
120|     let data = payload.data
121|     if (StatusMessage.MARKED_READ_NOTIFICATIONS in data) {
122|         return [true, ""]
123|     }
124|     else{
125|         return [false,
data[StatusMessage.FAILED_TO_MARK_READ_NOTIFICATIONS]]
126|     }
127|
128| }
129|
130|
131| export const handleAppointmentTimes = (payload) =>{
132|     let data = payload.data
133|     if (StatusMessage.GOT_APPOINTMENT_TIMES in data)
134|         return [true,
data[StatusMessage.GOT_APPOINTMENT_TIMES]]
135|     else
136|         return [false,
data[StatusMessage.FAILED_TO_GET_APPOINTMENT_TIMES]]
137|
138| }
139|
140|
141| export const handleAppointmentMadeResponse = (payload) =>
142|     let data = payload.data
143|
144|     if (StatusMessage.BOOKED_APPOINTMENT in data)
145|         return [true, ""]
146|     else
147|         return [false,
data[StatusMessage.FAILED_TO_BOOK_APPOINTMENT]]
148| }
149|
150|
151| // #endregion
152|
```

```
153|
154| // #region Search
155| export const handleProfileInfoResponse = (payload) => {
156|     let data = payload.data
157|     if (StatusMessage.GOT_FREELANCER_PUBLIC_INFO in data)
158|         return [true,
data[StatusMessage.GOT_FREELANCER_PUBLIC_INFO]]
159|     }
160|     else{
161|         return [false,
data[StatusMessage.FAILED_TO_GET_FREELANCER_PUBLIC_INFO]]
162|     }
163| }
164|
165|
166| export const handleJobsResponse = (payload) =>{
167|     let data = payload.data
168|     console.log(data)
169|     if (StatusMessage.GOT_JOBS in data){
170|         return [true, data[StatusMessage.GOT_JOBS] ["jobs"]
171|     }
172|     else{
173|         return [false,
data[StatusMessage.FAILED_TO_GET_JOBS]]
174|     }
175| }
176|
177|
178| export const handleCitiesResponse = (payload) =>{
179|     let data = payload.data
180|     if (StatusMessage.GOT_CITIES in data)
181|         return [true,
data[StatusMessage.GOT_CITIES] ["cities"]]
182|     else
183|         return [false,
data[StatusMessage.FAILED_TO_GET_CITIES]]
184| }
185|
186|
187| export const handleMinimalFreelancerInfoResponse = (payload) =>{
188|     let data = payload.data
189|     if (StatusMessage.GOT_MINIMAL_INFO in data){
```

```
190|         return [true, data[StatusMessage.GOT_MINIMAL_INFO]]
191|     }
192|     else
193|         return [false,
194| data[StatusMessage.FAILED_TO_GET_MINIMAL_INFO]]
195| }
196| // #endregion
```

RequestHandler.js

```
0 | import MsgBuild from "../MsgBuilder"
1 | import {useSocket} from '../socket/SocketContext'
2 | import { MessageTypes } from "../MsgTypes"
3 |
4 | // #region Login
5 | export function LoginRequest(username, password, ws,
  token="") {
6 |     const payload = {
7 |         "email": username,
8 |         "password": password
9 |     }
10 |    const msg = MsgBuild(MessageTypes.LOGIN, payload, token,
  "JSON")
11 |    console.log(`Sending - ${msg}`)
12 |    ws.send(msg)
13 | }
14 | // #endregion
15 |
16 |
17 | // #region SignUp
18 | export function SignUpRequest(ws, account_info, type_of_user,
  token="User", token="") {
19 |     let payload = account_info;
20 |     payload.role = type_of_user;
21 |     let msg;
22 |     if (type_of_user === "User") {
23 |         msg = MsgBuild(MessageTypes.USER_SIGNUP, payload,
  token, "JSON")
24 |     }
25 |     else {
26 |         msg = MsgBuild(MessageTypes.FREELANCER_SIGNUP,
  payload, token, "JSON")
27 |     }
28 |     console.log(`Sending - ${msg}`)
29 |     ws.send(msg)
30 | }
31 | // #endregion
32 |
33 |
34 | // #region Verification
```

```
35| export function VerificationRequest(ws, email, veri_code,
    role, token="") {
36|     let payload = {"email":email,
    "verification_code":veri_code, "role":role}
37|     let msg = MsgBuild(MessageTypes.VERIFICATION,
    payload,token, "JSON")
38|     ws.send(msg)
39| }
40| //#endregion
41|
42|
43| //#region Forgot Password
44| export function ForgotPasswordVerificationCodeRequest(ws,
    email, token="") {
45|     let payload = {"email":email}
46|     let msg = MsgBuild(MessageTypes.FORGOT_PASSWORD_REQUEST,
    payload,token, "JSON")
47|     ws.send(msg)
48| }
49|
50|
51| export function ForgotPasswordAuthenticationRequest(ws,
    email, code, token="") {
52|     let payload = {"email":email, "veri_code":code}
53|     let msg =
    MsgBuild(MessageTypes.FORGOT_PASSWORD_AUTHENTICATION,
    payload,token, "JSON")
54|     ws.send(msg)
55| }
56|
57|
58| export function ChangePasswordRequest(ws, email, pass,
    token="") {
59|     let payload = {"email":email,"password":pass }
60|     let msg = MsgBuild(MessageTypes.CHANGE_PASS, payload,
    token, "JSON")
61|     ws.send(msg)
62| }
63| //#endregion
64|
65|
66| //#region HomePage
67| export function UserInfoRequest(ws, token){
```

```

68|     const [encodedData] = token.split('.')
69|     const decodedJson = atob(encodedData)
70|     const data = JSON.parse(decodedJson)
71|     let payload = {"email":data.email}
72|     let msg = MsgBuild(MessageTypes.GET_USER_INFO, payload,
token, "JSON")
73|     ws.send(msg)
74| }
75|
76|
77| export function UpdateAppointmentsStatusRequest(ws, apps,
token) {
78|     let payload = {"appointments":apps}
79|     let msg =
MsgBuild(MessageTypes.UPDATE_APPOINTMENTS_STATUS, payload, tok
"JSON")
80|     console.log(`About to send - ${msg}`)
81|     ws.send(msg)
82| }
83|
84|
85| export function MarkReadNotificationsRequest(ws, userId,
token) {
86|     let payload = {"user_id":userId}
87|     let msg = MsgBuild(MessageTypes.MARK_READ_NOTIFICATION
payload, token, "JSON")
88|     ws.send(msg)
89| }
90|
91|
92| export function GetAvailableWorkTimes(ws, targetId, token)
93|     let payload = {"id":targetId}
94|     let msg = MsgBuild(MessageTypes.GET_APPOINTMENT_TIMES,
payload, token, "JSON")
95|     ws.send(msg)
96| }
97|
98|
99| export function BookAppointment(ws, app, token) {
100|     let payload = {"app":app}
101|     let msg = MsgBuild(MessageTypes.MAKE_APPOINTMENT,
payload, token, "JSON")
102|     console.log(`sending - ${msg}`)

```

```
103|     ws.send(msg)
104| }
105|
106| //endregion
107|
108|
109| //region Search
110| export function GetPublicProfileInfoRequest(ws, targetId,
token) {
111|     let payload = {id:targetId}
112|     let msg = MsgBuild(MessageTypes.GET_PUBLIC_PROFILE_IN
payload, token, "JSON")
113|     ws.send(msg)
114| }
115|
116|
117| export function GetMinimalFreelancerInfo(ws, token, city,
job) {
118|     let payload = {"city":city, "job":job}
119|     let msg =
MsgBuild(MessageTypes.GET_MINIMAL_FREELANCER_INFO, payload,
token, "JSON")
120|     ws.send(msg)
121| }
122|
123| export function GetAllJobs(ws, token)
124| {
125|     let msg = MsgBuild(MessageTypes.GET_JOBS, "", token,
"JSON")
126|     ws.send(msg)
127| }
128|
129| export function GetCitiesByJob(ws, job, token) {
130|     let payload = {"job":job}
131|     let msg = MsgBuild(MessageTypes.GET_CITIES_BY_JOB,
payload, token, "JSON")
132|     ws.send(msg)
133| }
134| }
135|
136| //endregion
```

MsgTypes.js

```

0 | export const MessageTypes = Object.freeze({
1 | //Anything
2 |   BROAD:"BRD",
3 |
4 | //Authentication
5 |   LOGIN: "LOGIN",
6 |   USER_SIGNUP: "USERSIGNUP",
7 |   FREELANCER_SIGNUP:"FRLNCSIGNUP",
8 |   VERIFICATION:"VERIFYING",
9 |   FORGOT_PASSWORD_REQUEST:"FRGT_PAS_RQT",
10|   FORGOT_PASSWORD_AUTHENTICATION:"FRGT_PASS_AUTH",
11|   CHANGE_PASS: "CHNG_PAS",
12|
13| //Homepage
14|   GET_USER_INFO: "USR_INFO",
15|   UPDATE_APPOINTMENTS_STATUS: "UPDT_APP_STAT",
16|   MARK_READ_NOTIFICATION: "RD_NTFC",
17|   GET_APPOINTMENT_TIMES: "GET_APP_TIME",
18|   MAKE_APPOINTMENT:"MK_APP",
19|
20| //Searchg
21|   GET_PUBLIC_PROFILE_INFO: "PBL_INFO",
22|   GET_MINIMAL_FREELANCER_INFO:"MNML_VIEW",
23|   GET_JOBS:"GET_JBS",
24|   GET_CITIES_BY_JOB:"GET_CITIS"
25|
26| });
27|
28|
29| export const StatusMessage = Object.freeze({
30| //Anything
31|   TOKEN_BAD:"BAD_TKN",
32|
33| //Authentication
34|   LOGGED_IN: "LOGGED",
35|   FAILED_LOG_IN: "FAILED_LOG",
36|
37|   SIGNING_UP: "SIGNED",
38|   FAILED_SIGN_UP: "FAILED_SIGNED",
39|
40|   VERIFICATION_GOOD:"VERI_GOOD",

```



```
41 |     VERIFICATION_BAD: "VERI_BAD",
42 |
43 |     FORGOT_PASSWORD_GOOD: "FRGT_GOOD",
44 |     FORGOT_PASSWORD_BAD: "FRGT_BAD",
45 |
46 |     CHANGE_PASSWORD_GOOD: "CHNG_GOOD",
47 |     CHANGE_PASSWORD_BAD: "CHNG_BAD",
48 |
49 | //Homepage
50 |     GOT_USER_INFO: "GOT_USR_INFO",
51 |     FAILED_TO_GET_USER_INFO: "FLD_USR_INFO",
52 |
53 |     UPDATED_APP_STATUS: "UPDT_APP",
54 |     FAILED_TO_UPDATE_APP_STATUS: "FAILED_UPDT_APP",
55 |
56 |     MARKED_READ_NOTIFICATIONS: "READ_NOTIFICATIONS",
57 |     FAILED_TO_MARK_READ_NOTIFICATIONS:
58 |     "FAILED_READ_NOTIFICATIONS",
59 |
60 |     GOT_APPOINTMENT_TIMES: "GOT_APP_TIME",
61 |     FAILED_TO_GET_APPOINTMENT_TIMES: "FLD_APP_TIME",
62 |
63 |     BOOKED_APPOINTMENT: "BKD_APP",
64 |     FAILED_TO_BOOK_APPOINTMENT: "FLD_BKD_APP",
65 | //Search
66 |     GOT_JOBS: "GOT_JBS",
67 |     FAILED_TO_GET_JOBS: "FLD_JBS",
68 |
69 |     GOT_CITIES: "GOT_CITIS",
70 |     FAILED_TO_GET_CITIES: "FLD_CITIS",
71 |
72 |     GOT_MINIMAL_INFO: "GOT_MNML",
73 |     FAILED_TO_GET_MINIMAL_INFO: "FLD_MNML",
74 |
75 |     GOT_FREELANCER_PUBLIC_INFO: "GOT_FREE",
76 |     FAILED_TO_GET_FREELANCER_PUBLIC_INFO: "FLD_FREE",
77 | })
78 |
79 |
```

MsgParser.js

```
0 | //Messages are sent in this format:
1 | //{
2 | //  type: type of message...
3 | //  data: actual payload of the message, if more then one
  | field then sent as a dict
4 | //  data_type: BYTES or JSON
5 | //}
6 |
7 |
8 | export default function websocketParser(payload)
9 | {
10 |     try{
11 |         const msgObj = JSON.parse(payload);
12 |         const {type, data, data_type, token} = msgObj;
13 |         let parsedData = data
14 |         if (data_type == "BYTES") {
15 |             const binaryStr = atob(data);
16 |             const bytes = new Uint8Array(binaryStr.length)
17 |             for (let i=0; i<binaryStr.length; i++) {
18 |                 bytes[i] = binaryStr.charCodeAt(i);
19 |             }
20 |             parsedData = bytes;
21 |         }
22 |
23 |         return {type, data:parsedData, token};
24 |     } catch(err) {
25 |         console.error("Failed to parse message", err);
26 |         return null;
27 |     }
28 | }
29 |
```

MsgBuilder.py

```
0 | export default function MsgBuild(type,  
  | payload,token,type_of_data="JSON") {  
1 |     const my_dict = {  
2 |         "type": type,  
3 |         "data": payload,  
4 |         "data_type":type_of_data,  
5 |         "token":token  
6 |     }  
7 |     return JSON.stringify(my_dict)  
8 | }
```

UserInfo.js

```

0 | // hooks/useUserSync.js
1 | import { useEffect } from "react";
2 | import { UserInfoRequest, MarkReadNotificationsRequest,
  | GetPublicProfileInfoRequest } from "../../socket/RequestHandle
3 | import websocketParser from "../../socket/MsgParser";
4 | import { handleUserInfoResponse, handleProfileInfoResponse,
  | handleUpdatedAppointmentsResponse, handleMarkedReadNotificatio
  | handleAppointmentTimes, handleAppointmentMadeResponse } from
  | "../../socket/ResponseHandlers";
5 | import { logout } from "../../store/authSlice";
6 | import { MessageTypes, StatusMessage } from
  | "../../socket/MsgTypes";
7 |
8 | export default function useUserSync(ws, token, dispatch,
  | navigate, setViewedFreelancer, setAppointmentTimes, target_id,
  | isPublic=false) {
9 |     useEffect(() => {
10 |
11 |         if (!ws || !token) return;
12 |         const sendRequest = () =>{ if (!isPublic)
  | {UserInfoRequest(ws, token);} if (target_id){
  | GetPublicProfileInfoRequest(ws, target_id, token)}}
13 |
14 |
15 |         const handleOpen = () => sendRequest()
16 |         if (ws.readyState === WebSocket.OPEN) {
17 |             sendRequest();
18 |         } else {
19 |             ws.addEventListener("open", handleOpen);
20 |         }
21 |
22 |         const handleMessage = (event) => {
23 |
24 |             let info = websocketParser(event.data);
25 |             console.log("Recvd - ", info)
26 |             if (MessageTypes.GET_USER_INFO === info.type){
27 |                 const [status, msg] =
  | handleUserInfoResponse(dispatch, info);
28 |                 if (!status) {
29 |                     dispatch(logout());
30 |                     navigate("/login");

```

```

31 |         }
32 |         else{
33 |             const userId = info.data.user_id;
34 |             MarkReadNotificationsRequest(ws, userI
token)
35 |         }
36 |     }
37 |
38 |     else if (MessageTypes.GET_PUBLIC_PROFILE_INFO
info.type){
39 |         const [status, data] =
handleProfileInfoResponse(info)
40 |         if (status){
41 |             setViewedFreelancer(data)
42 |         }
43 |         else{
44 |             alert(data)
45 |             navigate("/search")
46 |         }
47 |     }
49 |     else if (MessageTypes.UPDATE_APPOINTMENTS_STATU
=== info.type){
50 |         const [status, data] =
handleUpdatedAppointmentsResponse(info)
51 |         if (status){
52 |             UserInfoRequest(ws, token)
53 |         }
54 |         else{
55 |             console.log("Error occurred ", data)
56 |             UserInfoRequest(ws, token)
57 |         }
58 |     }
60 |     else if (MessageTypes.MARK_READ_NOTIFICATION ==
info.type){
61 |         const [status, data] =
handleMarkedReadNotifications(info)
62 |         if (!status){
63 |             console.log("Didn't manage to mark
notifications as read - ", data)
64 |         }
65 |     }
66 |

```

```

67 |         else if (MessageTypes.GET_APPOINTMENT_TIMES ===
info.type) {
68 |             const [status, msg] =
handleAppointmentTimes (info)
69 |             if (status) {
70 |                 setAppointmentTimes (msg)
71 |                 console.log (msg)
72 |             }
73 |             else
74 |                 setAppointmentTimes (null)
75 |         }
76 |
77 |         else if (MessageTypes.MAKE_APPOINTMENT ===
info.type) {
78 |             const [status, msg] =
handleAppointmentMadeResponse (info);
79 |             if (status) {
80 |                 setAppointmentTimes (null)
81 |                 alert ("Success!");
82 |                 navigate ("/search");
83 |             } else {
84 |                 alert ("Error: " + msg);
85 |             }
86 |         }
87 |
88 |         else if (MessageTypes.BROAD === info.type) {
89 |             if (StatusMessage.TOKEN_BAD in info.data) {
90 |                 alert ("Your session has ended, login
again to get service")
91 |                 dispatch (logout ())
92 |                 navigate ('/login')
93 |             }
94 |         }
95 |     };
96 |
97 |     ws.addEventListener ("message", handleMessage);
98 |
99 |     return () => {
100 |         ws.removeEventListener ("open", handleOpen);
101 |         ws.removeEventListener ("message",
handleMessage);
102 |     };
103 | }, [ws, token, target_id, isPublic]);
104 | }

```

FreelancerView.jsx

```

0 | import { WelcomeBack, WelcomeTo, NotificationsView,
  UpcomingAppointmentsView, PendingAppointments, FreelancerInfo,
  AppointmentBooking } from "../ViewModule"
1 | import { useSelector } from "react-redux"
2 | import { useRef, useState, useEffect } from "react"
3 | import { useSocket } from "../../socket/SocketContext"
4 | import { Calendar } from "lucide-react"
5 | import { BookAppointment, GetAvailableWorkTimes } from
  "../../socket/RequestHandler"
6 | import LoadingScreen from "../LoadingScreen"
7 | import { ErrorMessage } from "../signUpModule"
8 |
9 | function FreelancerPublic({ user, ws, token, appointmentTi
  }){
10 |     const userId = useSelector((state) =>
state.auth.userInfo?.id)
11 |     const profId = user.id
12 |     const username = user.username
13 |     const job = user.job
14 |     const cities = user.cities
15 |     const description = user.description
16 |     const years = user.years
17 |     const jobDuration = user.job_duration
18 |     const rating = user.rating
19 |     const price = user.price
20 |
21 |
22 |     const [makeAppointment, setMakeAppointment] =
useState(false)
23 |
24 |     const [app, setApp] = useState({prof_id:profId,
user_id:userId})
25 |
26 |     useEffect(() => {
27 |         if (Object.keys(app).length=== 2) {return;}
28 |         else{BookAppointment(ws, app, token);}
29 |     }, [app])
30 |
31 |     useEffect(() =>{
32 |         if (makeAppointment){
33 |             GetAvailableWorkTimes(ws, profId, token)

```

```

34 |     }
35 |   }, [makeAppointment])
36 |
37 |
38 |   const handleConfirm = (newAppData) => {
39 |     const fullAppointment = { ...app, ...newAppData };
40 |     BookAppointment(ws, fullAppointment, token);
41 |   };
42 |
43 |   return (<>
44 |
45 |     { !makeAppointment ? (
46 |       <div className="p-8 max-w-6xl mx-auto animate-fadeIn space-y-8">
47 |         <WelcomeTo username={username} />
48 |
49 |         <FreelancerInfo profession={job}
50 | serviceCities={cities} description={description}
51 | years_experience={years} jobDuration={jobDuration}
52 | rating={rating} pricePerHour={price} />
53 |
54 |         <button
55 |           onClick={() =>
56 |             setMakeAppointment(true) }
57 |           className="mt-8 w-full py-4
58 |             bg-gradient-to-r from-blue-600 to-indigo-600 hover:from-blue-5
59 |             hover:to-indigo-500 text-white
60 |             font-bold rounded-2xl shadow-lg shadow-blue-500/20 transition-
61 |             duration-300 transform
62 |             hover:-translate-y-1 active:scale-95 flex items-center
63 |             justify-center gap-3 group">
64 |           <Calendar className="w-5 h-5
65 |             group-hover:animate-pulse" />
66 |           <span>Make Appointment</span>
67 |         </button>
68 |       </div>
69 |     ) : (appointmentTimes ? (
70 |
71 |       <AppointmentBooking
72 |         availability={appointmentTimes}
73 |         jobDuration={jobDuration}
74 |         onConfirm={handleConfirm}

```



```

67 |             onCancel={ () =>
setMakeAppointment (false) }
68 |             price={price}/>
69 |
70 |         ) : (<LoadingScreen/>)
71 |
72 |     )
73 | }
74 | </>
75 | )
76 | }
77 |
78 |
79 |
80 | function FreelancerPrivate ({ws, token}) {
81 |     const acceptAppointmentsRef = useRef (null);
82 |     const scrollToRef = ()
=> (acceptAppointmentsRef.current?.scrollIntoView ({behavior: "smooth"}))
83 |
84 |
85 |     const userInfo = useSelector ((state) =>
state.auth.userInfo)
86 |
87 |     const username = useSelector ((state) =>
state.auth.userInfo?.name)
88 |     const notifications = useSelector ((state) =>
state.auth.notifications || [])
89 |     const appointments = useSelector ((state) =>
state.appointment?.freelancerAppointments || [])
90 |
91 |     const confirmedAppointments = appointments.filter ((app)
=> app.status === "Confirmed")
92 |     const pendingAppointments = appointments.filter ((app)
app.status === "Requested")
93 |
94 |     return (
95 |         <div className="p-8 max-w-6xl mx-auto animate-fade
space-y-8">
96 |             <WelcomeBack username={username}/>
97 |
98 |

```

```

99|         <FreelancerInfo profession={userInfo?.job}
serviceCities={userInfo?.cities}
description={userInfo?.description}
years_experience={userInfo?.years}
jobDuration={userInfo?.job_duration} rating={userInfo?.rating}
pricePerHour={userInfo?.hour_price} />
100|
101|         <div className="grid grid-cols-1 md:grid-cols-2 gap-6">
102|             <div className="bg-slate-800/50 border-l-4 border-purple-500 p-6 rounded-xl shadow-lg backdrop-blur-sm">
103|                 <p className="text-slate-400 text-sm font-medium uppercase tracking-wider">Notifications</p>
104|                 <p className="text-3xl font-bold text-white mt-1">{notifications.length}</p>
105|             </div>
106|
107|             <div className="bg-slate-800/50 border-l-4 border-blue-500 p-6 rounded-xl shadow-lg backdrop-blur-sm">
108|                 <p className="text-slate-400 text-sm font-medium uppercase tracking-wider">Scheduled Appointments</p>
109|                 <p className="text-3xl font-bold text-white mt-1">
110|                     {confirmedAppointments.length}
111|                 </p>
112|             </div>
113|
114|             <div className="bg-slate-800/50 border-l-4 border-emerald-500 p-6 rounded-xl shadow-lg backdrop-blur-sm">
115|                 <p className="text-slate-400 text-sm font-medium uppercase tracking-wider">Pending Appointments</p>
116|                 <p className="text-3xl font-bold text-white mt-1">
117|                     {pendingAppointments.length}
118|                 </p>
119|             </div>
120|         </div>
121|
122|         <div className="bg-gradient-to-br from-blue-600/20 to-purple-600/20 py-16 px-8 rounded-3xl border border-blue-500/20 flex flex-col items-center justify-center shadow-2xl">

```

```

123|         <h3 className="text-2xl font-bold text-wh
mb-8">Quick Actions</h3>
124|         <div className="grid grid-cols-1
sm:grid-cols-1 gap-6 w-full max-w-2xl">
125|             <button onClick={scrollToRef}
className="flex items-center justify-center py-10 bg-slate-800
hover:bg-slate-700 text-white rounded-xl transition-all text-l
font-semibold border border-slate-600 shadow-lg hover:scale-10
126|                 Accept Pending appointments
127|             </button>
128|         </div>
129|     </div>
130|
131|
132|     <div className="grid grid-cols-1 lg:grid-cols
gap-8">
133|         <NotificationsView
notifications={notifications} />
134|
135|         <UpcomingAppointmentsView
appointments={confirmedAppointments} />
136|     </div>
137|
138|     <div ref={acceptAppointmentsRef}>
139|         <PendingAppointments
appointments={pendingAppointments} ws={ws} token={token} />
140|     </div>
141|
142|
143|
144|     </div>
145|
146|
147|
148|
149|     )
150| }
151|
152|
153|
154|
155| export default function FreelancerView({ user, isPublic,
appointmentTimes }) {

```

```
156|     const ws = useSocket()
157|     const token = useSelector((state) =>
state.auth.userToken)
158|
159|     return (
160|         <>
161|             {isPublic ? (
162|                 <FreelancerPublic user={user} ws={ws}
token={token} appointmentTimes={appointmentTimes}/>
163|             ) : (
164|                 <FreelancerPrivate ws={ws} token={token}/>
165|             )
166|         }
167|     </>
168| )
169| }
```

LoadingScreen.jsx

```
0 | import { useState, useEffect } from "react";
1 | import Navbar from "../routerPrint";
2 | export default function LoadingScreen() {
3 |     const [dots, setDots] = useState(".");
4 |
5 |     useEffect(() => {
6 |         const interval = setInterval(() => {
7 |             setDots((prev) => (prev === "... " ? "." : prev
8 |             }, 500);
9 |             return () => clearInterval(interval);
10 |         }, []);
11 |
12 |         return (
13 |             <>
14 |                 <div className="fixed inset-0 bg-slate-900 flex
15 |                 <h1 className="text-white text-5xl
16 |                 font-bold">Loading{dots}</h1>
17 |                 </div>
18 |             </>
19 |         );
20 |     }
```

UserView.jsx

```

0 | import { useSelector } from "react-redux"
1 | import { Link } from "react-router-dom"
2 | import { WelcomeBack, NotificationsView,
  UpcomingAppointmentsView } from "../ViewModule"
3 |
4 |
5 |
6 | export default function UserView() {
7 |     const username = useSelector((state) =>
state.auth.userInfo?.name)
8 |     const notifications = useSelector((state) =>
state.auth.notifications || [])
9 |     const appointments = useSelector((state) =>
state.appointment?.userAppointments || [])
10 |
11 |     return (
12 |         <div className="p-8 max-w-6xl mx-auto animate-fade
space-y-8">
13 |             <WelcomeBack username={username}/>
14 |
15 |             <div className="grid grid-cols-1 md:grid-cols-
gap-6">
16 |                 <div className="bg-slate-800/50 border-l-4
border-purple-500 p-6 rounded-xl shadow-lg backdrop-blur-sm">
17 |                     <p className="text-slate-400 text-sm
font-medium uppercase tracking-wider">Notifications</p>
18 |                     <p className="text-3xl font-bold
text-white mt-1">{notifications.length}</p>
19 |                 </div>
20 |                 <div className="bg-slate-800/50 border-l-4
border-blue-500 p-6 rounded-xl shadow-lg backdrop-blur-sm">
21 |                     <p className="text-slate-400 text-sm
font-medium uppercase tracking-wider">Scheduled Appointments</p>
22 |                     <p className="text-3xl font-bold
text-white mt-1">{appointments.length}</p>
23 |                 </div>
24 |             </div>
25 |
26 |             <div className="bg-gradient-to-br
from-blue-600/20 to-purple-600/20 py-16 px-8 rounded-3xl borde

```

```

border-blue-500/20 flex flex-col items-center justify-center
shadow-2xl">
27|         <h3 className="text-2xl font-bold text-white
mb-8">Quick Actions</h3>
28|         <div className="grid grid-cols-1
sm:grid-cols-1 gap-6 w-full max-w-2xl">
29|             <Link to="/search" className="flex
items-center justify-center py-10 bg-slate-800 hover:bg-slate-
text-white rounded-xl transition-all text-lg font-semibold border
border-slate-600 shadow-lg hover:scale-105">
30|                 Find Freelancers
31|             </Link>
32|             {/* <Link to="/schedule" className="flex
items-center justify-center py-10 bg-slate-800 hover:bg-slate-
text-white rounded-xl transition-all text-lg font-semibold border
border-slate-600 shadow-lg hover:scale-105">
33|                 View Calendar
34|             </Link> */}
35|         </div>
36|     </div>
37|
38|     <div className="grid grid-cols-1 lg:grid-cols-
gap-8">
39|
40|         <NotificationsView
notifications={notifications}/>
41|
42|         <UpcomingAppointmentsView
appointments={appointments}/>
43|     </div>
44| </div>
45| )
46| }

```

ViewModule.jsx

```

0 | import { useEffect, useState } from "react";
1 | import { CalendarIcon, CheckCircle, Info, Check, X,
  Briefcase, MapPin, AlignLeft, Calendar, Star, Clock, FileText,
  User, DollarSign } from "lucide-react"
2 | import { UpdateAppointmentsStatusRequest } from
  "../socket/RequestHandler";
3 | import { ErrorMessage } from "../signUpModule";
4 |
5 |
6 |
7 |
8 | export function WelcomeBack({ username }) {
9 |     return (<>
10 |         <header>
11 |             <h1 className="text-4xl font-extrabold
text-white tracking-tight">
12 |                 Home Page
13 |             </h1>
14 |             <p className="text-lg text-slate-400 mt-2">
15 |                 Welcome back <span
className="text-blue-400 font-semibold">{username}</span>
16 |             </p>
17 |         </header>
18 |     </>)
19 | }
20 |
21 |
22 | export function WelcomeTo({ username }) {
23 |     return (
24 |         <>
25 |             <header>
26 |                 <h1 className="text-4xl font-extrabold text-white
tracking-tight">
27 |                     Freelancer Page
28 |                 </h1>
29 |                 <p className="text-lg text-slate-400 mt-2">
30 |                     Welcome to <span className="text-blue-400
font-semibold">{username}</span>'s page
31 |                 </p>
32 |             </header>
33 |         </>

```



```

34|   )
35| }
36|
37|
38| export function NotificationsView({ notifications }) {
39|   return (
40|     <div className="bg-slate-800/40 p-6 rounded-2xl border
border-slate-700 flex flex-col h-[500px]">
41|       <h3 className="text-xl font-semibold text-white mk
text-left">Recent Activity</h3>
42|       <div className="space-y-4 overflow-y-auto pr-2
custom-scrollbar flex-grow">
43|         {notifications.length > 0 ? (
44|           notifications.map((note, idx) => (
45|             <div key={idx} className="flex flex-co
items-start text-left text-slate-300 text-sm py-3 border-b
border-slate-700/50 last:border-0 hover:bg-slate-700/20
transition-colors rounded-lg px-2">
46|               <div className="flex items-center
justify-between w-full mb-1">
47|                 <div className="flex
items-center">
48|                   {!note.is_read ? (
49|                     <>
50|                       <span className="w-2 h
rounded-full bg-blue-500 mr-3"></span>
51|                       <span
className="text-blue-400 font-semibold text-xs uppercase
tracking-wider">
52|                         From:
{note.from_name}
53|                       </span>
54|                     </>
55|                   ) : (
56|                     <span
className="text-gray-400 font-semibold text-xs uppercase
tracking-wider">
57|                       From:
{note.from_name}
58|                     </span>
59|                   ) }
60|                 </div>

```

```

61|         <span className="text-[10px]
text-slate-500 font-mono italic">
62|             {note.created_at}
63|         </span>
64|     </div>
65|     <p className="ml-5 text-slate-200
leading-relaxed text-left">
66|         {note.message}
67|     </p>
68| </div>
69|     ))
70|   ) : (
71|     <div className="flex flex-col items-start
justify-center h-full">
72|       <p className="text-slate-500 italic
text-sm">No new activity found.</p>
73|     </div>
74|   )}
75| </div>
76| </div>)
77| }
78|
79|
80| export function UpcomingAppointmentsView({ appointments })
81|   return (
82|     <div className="bg-slate-800/40 p-6 rounded-2xl border
border-slate-700 flex flex-col h-[500px]">
83|       <h3 className="text-xl font-semibold text-white mk
text-left">Upcoming Appointments</h3>
84|       <div className="space-y-4 overflow-y-auto pr-2
custom-scrollbar flex-grow">
85|         {appointments.length > 0 ? (
86|           //Here I can control if I want to show a
limited amount of appointments
87|           appointments.map((app, idx) => (
88|             <div key={idx} className="bg-slate-700
p-4 rounded-xl text-slate-300 text-sm border border-slate-600/
shadow-inner flex flex-col items-start">
89|               <div className="flex flex-col
space-y-2 w-full">
90|                 <p className="flex items-start
justify-start">

```

```

91 |                 <span
className="text-blue-400 font-semibold uppercase text-[10px]
tracking-wider w-24 shrink-0 mt-1">Date:</span>
92 |                 <span className="text-white
font-medium">{app.display_date}</span>
93 |                 </p>
94 |
95 |             <p className="flex items-start
justify-start">
96 |                 <span
className="text-blue-400 font-semibold uppercase text-[10px]
tracking-wider w-24 shrink-0 mt-1">Start Time:</span>
97 |                 <span className="text-white
font-medium">{app.start_time}</span>
98 |                 </p>
99 |
100 |            <p className="flex items-start
justify-start min-w-0 w-full">
101 |                <span
className="text-blue-400 font-semibold uppercase text-[10px]
tracking-wider w-24 shrink-0 mt-1">
102 |                    Description:
103 |                </span>
104 |                <span
title={app.details}
className="text-white
font-medium break-words text-left flex-1 min-w-0, truncate"
107 |                >
108 |                    {app.details}
109 |                </span>
110 |            </p>
111 |        </div>
112 |    </div>
113 |    ))
114 |    ) : (
115 |        <div className="flex flex-col items-start
justify-center h-full">
116 |            <p className="text-slate-500 italic
text-sm">No upcoming appointments found.</p>
117 |        </div>
118 |    ) }
119 | </div>
120 | </div>

```

```

121| }
122|
123|
124| export function PendingAppointments({ appointments = [],
    token }) {
125|   const [decisions, setDecisions] = useState({});
126|   const [selectedApp, setSelectedApp] = useState(null);
127|
128|   const totalDecisions = Object.keys(decisions).length;
129|   const gridLayout = "grid
    grid-cols-[1fr_1.2fr_0.5fr_0.5fr_1fr_2fr_0.4fr_1fr]";
130|
131|   const [allowed, setAllowed] = useState(true)
132|
133|   const actualDecisionsCount =
    Object.values(decisions).filter(status => status !==
    null).length;
134|
135|   const handleDecision = (appId, status) => {
136|     setDecisions((prev) => ({ ...prev, [appId]: status }))
137|   };
138|
139|   const handleFinalSubmit = () => {
140|     UpdateAppointmentsStatusRequest(ws, decisions, token)
141|     setDecisions({})
142|   };
143|
144|   useEffect(() => {
145|
146|     const seen = new Set();
147|     let conflictFound = false;
148|
149|     const acceptedIds = Object.keys(decisions).filter(
150|       (id) => decisions[id] === "accepted"
151|     );
152|
153|     for (const appId of acceptedIds) {
154|       const appData = appointments.find((a) =>
    String(a.id) === String(appId))
155|       if (appData) {
156|         const slotKey =
    `${appData.date}-${appData.start_time}`

```

```

157|         if (seen.has(slotKey)) { conflictFound=true;
break;}
158|         seen.add(slotKey)
159|     }}
160|     setAllowed(!conflictFound)
161|
162| }, [decisions, appointments])
163|
164|     return (
165|
166|         <>
167|             <div className="w-full bg-slate-900 text-white
rounded-xl shadow-2xl overflow-hidden mt-8 border
border-slate-800">
168|
169|                 <div className={` ${gridLayout} gap-4 bg-slate-800
p-4 font-bold border-b border-slate-700 text-[10px] uppercase
tracking-wider`} >
170|                     <div>Date</div>
171|                     <div>Client</div>
172|                     <div>Start</div>
173|                     <div>End</div>
174|                     <div>Address</div>
175|                     <div>Details</div>
176|                     <div className="text-center">Info</div>
177|                     <div className="text-center">Action</div>
178|                 </div>
179|
180|                 <div className="divide-y divide-slate-800">
181|                     {appointments.map((app) => (
182|                         <div key={app.id} className={` ${gridLayout}
gap-4 p-4 items-center transition-all ${
183|                             decisions[app.id] === 'accepted' ?
'bg-emerald-900/10' :
184|                             decisions[app.id] === 'cancelled' ?
'bg-rose-900/10' : 'hover:bg-slate-800/30'
185|                         } `}>
186|                             <div className="text-slate-400
text-xs">{app.display_date}</div>
187|                             <div className="font-semibold truncate
text-sm">{app.person_name}</div>
188|                             <div className="text-slate-300
text-xs">{app.start_time}</div>

```

```

189|         <div className="text-slate-300
text-xs">{app.end_time}</div>
190|         <div className="truncate text-xs
text-slate-500" title={app.address}>{app.address}</div>
191|         <div className="text-sm text-slate-300
truncate" title={app.details}>{app.details}</div>
192|
193|         <div className="flex justify-center">
194|             <button
195|                 onClick={ () => setSelectedApp(app) }
196|                 className="p-2 text-blue-400
hover:text-blue-300 hover:bg-blue-400/10 rounded-lg
transition-all"
197|             >
198|                 <Info size={18} />
199|             </button>
200|         </div>
201|
202|         <div className="flex justify-center gap-2">
203|             <button
204|                 onClick={ () => handleDecision(app.id,
'accepted') }
205|                 className={ `p-2 rounded-full
transition-all ${decisions[app.id] === 'accepted' ?
'bg-emerald-500 text-white' : 'bg-slate-800 text-slate-500
hover:bg-emerald-600 hover:text-white' } ` }
206|             >
207|                 <Check size={16} />
208|             </button>
209|             <button
210|                 onClick={ () => handleDecision(app.id,
'cancelled') }
211|                 className={ `p-2 rounded-full
transition-all ${decisions[app.id] === 'cancelled' ? 'bg-rose-
text-white' : 'bg-slate-800 text-slate-500 hover:bg-rose-600
hover:text-white' } ` }
212|             >
213|                 <X size={16} />
214|             </button>
215|         </div>
216|     </div>
217| ))}
218| </div>

```

```

219|
220|         <div className="p-4 bg-slate-800/80 border-t
border-slate-700 flex items-center justify-between">
221|             <div className="text-sm text-slate-400">
222|                 {totalDecisions > 0 ? (
223|                     <span>You have marked <b
className="text-white">{totalDecisions}</b> appointments.</spa
224|                 ) : (
225|                     <span>Select appointments to accept or
decline.</span>
226|                 )}
227|             </div>
228|             {!allowed && (<ErrorMessage message={"Selected
appointments with the same date and starting time"} />)}
229|
230|             <button
231|                 onClick={handleFinalSubmit}
232|                 disabled={actualDecisionsCount === 0 ||
!allowed}
233|                 className={`flex items-center gap-2 px-6 py-2
rounded-lg font-bold transition-all transform active:scale-95
234|                     actualDecisionsCount > 0 && allowed
235|                     ? 'bg-blue-600 hover:bg-blue-500
shadow-[0_0_15px_rgba(37,99,235,0.4)] text-white'
236|                     : 'bg-slate-700 text-slate-500
cursor-not-allowed'
237|                 }`>
238|                 >
239|                 Confirm  Save Changes
240|             </button>
241|         </div>
242|
243|     </div>
244|     <AppointmentDetailModal
245|         appointment={selectedApp}
246|         onClose={() => setSelectedApp(null)}
247|     />
248| </>
249| );
250| }
251|
252|

```

```

253| export function FreelancerInfo({ profession, serviceCity,
description, years_experience, jobDuration, rating, pricePerHour }) {
254|     return (
255|         <div className="bg-slate-800/40 border
border-slate-700 p-8 rounded-3xl shadow-2xl backdrop-blur-md
animate-fadeIn">
256|             <div className="flex flex-col md:flex-row
md:items-center justify-between gap-4 mb-8">
257|                 <div>
258|                     <h2 className="text-3xl font-extrabold
text-white tracking-tight">{profession}</h2>
259|                     <div className="flex items-center gap-4
mt-2 text-yellow-400">
260|                         <Star className="w-5 h-5
fill-current" />
261|                         <span className="font-bold
text-lg">{rating} || "New"</span>
262|                         <span className="text-slate-500
text-sm font-normal">Rating</span>
263|                     </div>
264|                 </div>
265|
266|                 <div className="bg-blue-600/20 border
border-blue-500/30 px-5 py-2 rounded-2xl">
267|                     <p className="text-blue-400 text-sm
uppercase font-bold tracking-widest">Experience</p>
268|                     <p className="text-white text-xl
font-bold">{years_experience} Years</p>
269|                 </div>
270|
271|                 {pricePerHour > 0 && (
272|                     <div className="bg-green-600/20
border border-green-500/30 px-5 py-2 rounded-2xl flex
items-center gap-3">
273|                         <DollarSign className="w-8 h-8
text-green-400" />
274|                         <div>
275|                             <p className="text-green-400
text-sm uppercase font-bold tracking-widest">Rate</p>
276|                             <p className="text-white
text-xl font-bold">${pricePerHour}/hr</p>
277|                         </div>

```



```

278 |                                     </div>
279 |                                 ) }
280 |
281 |                             </div>
282 |
283 |                             <hr className="border-slate-700/50 mb-8" />
284 |
285 |                             <div className="grid grid-cols-1 md:grid-cols-2 gap-8">
286 |                                 <div className="space-y-4">
287 |                                     <div className="flex items-center gap-4 text-slate-400">
288 |                                         <FileText className="w-5 h-5" />
289 |                                         <h4 className="font-semibold uppercase text-xs tracking-wider">About Me</h4>
290 |                                     </div>
291 |                                     <p className="text-slate-300 leading-relaxed italic">
292 |                                         "{description}"
293 |                                     </p>
294 |                                 </div>
295 |
296 |                                 <div className="grid grid-cols-1 gap-4">
297 |                                     <div className="flex items-start gap-4 p-4 bg-slate-900/40 rounded-xl border border-slate-700/30">
298 |                                         <MapPin className="w-6 h-6 text-purple-500 mt-1" />
299 |                                         <div>
300 |                                             <p className="text-slate-500 text-xs font-bold uppercase">Service Cities</p>
301 |                                             <p
302 |                                                 className="text-slate-200">{serviceCities || 'Remote'}</p>
303 |                                             </div>
304 |                                         </div>
305 |                                     <div className="flex items-start gap-4 p-4 bg-slate-900/40 rounded-xl border border-slate-700/30">
306 |                                         <Clock className="w-6 h-6 text-blue-500 mt-1" />
307 |                                         <div>
308 |                                             <p className="text-slate-500 text-xs font-bold uppercase">Avg. Project Duration</p>

```

```

309|         <p
className="text-slate-200">{jobDuration}</p>
310|         </div>
311|     </div>
312| </div>
313| </div>
314| </div>
315|     );
316| }
317|
318|
319| export function AppointmentDetailModal({ appointment,
onClose }) {
320|     if (!appointment) return null;
321|
322|     return (
323|         <div className="fixed inset-0 z-50 flex items-center
justify-center p-4 bg-[#020617]/90 backdrop-blur-md">
324|             <div className="bg-[#0f172a] border-l-4
border-blue-500 w-full max-w-lg rounded-br-3xl rounded-tr-xl
rounded-bl-xl shadow-[20px_20px_60px_rgba(0,0,0,0.5)]
overflow-hidden transform -rotate-1 md:rotate-0">
325|
326|                 <div className="pt-8 px-8 pb-4 flex justify-between
items-start">
327|                     <div>
328|                         <h3 className="text-4xl font-black text-white
tracking-tighter flex items-baseline gap-3">
329|                             {appointment.person_name}
330|                             <span className="w-2 h-2 bg-green-500
rounded-full animate-pulse" title="Active Appointment" />
331|                         </h3>
332|                     </div>
333|                     <button
334|                         onClick={onClose}
335|                         className="p-2 bg-slate-800/50
hover:bg-rose-500/20 hover:text-rose-500 text-slate-500
rounded-full transition-all"
336|                         >
337|                             <X size={20} />
338|                     </button>
339|                 </div>
340|

```

```

341 |         <div className="p-8 pt-4 space-y-8">
342 |             <div className="flex flex-wrap gap-3">
343 |                 <div className="bg-slate-800/30 px-4 py-2
rounded-lg border-b-2 border-slate-700">
344 |                     <p className="text-[10px] font-bold
text-slate-500 uppercase tracking-widest flex items-center
gap-2">
345 |                         <Calendar size={10}/> Scheduled
346 |                     </p>
347 |                     <p className="text-blue-100 font-mono
text-sm">{appointment.display_date}</p>
348 |                 </div>
349 |                 <div className="bg-slate-800/30 px-4 py-2
rounded-lg border-b-2 border-slate-700">
350 |                     <p className="text-[10px] font-bold
text-slate-500 uppercase tracking-widest flex items-center
gap-2">
351 |                         <Clock size={10}/> Window
352 |                     </p>
353 |                     <p className="text-blue-100 font-mono
text-sm">{appointment.start_time} – {appointment.end_time}</p>
354 |                 </div>
355 |             </div>
356 |
357 |             <div className="relative group">
358 |                 <p className="text-xs font-black text-slate-5
uppercase tracking-tighter mb-2 flex items-center gap-2">
359 |                     <MapPin size={14} className="text-blue-500"
Location
360 |                 </p>
361 |                 <div className="bg-white/[0.03]
hover:bg-white/[0.05] p-5 rounded-2xl border border-white/10
transition-colors">
362 |                     <p className="text-slate-300 font-medium
leading-snug">
363 |                         {appointment.address}
364 |                     </p>
365 |                 </div>
366 |             </div>
367 |
368 |             <div className="space-y-3 relative">
369 |                 <div className="absolute -left-8 top-0 bottc
w-1 bg-slate-800/50" />

```

```

370|         <p className="text-xs font-black text-slate-
uppercase tracking-tighter flex items-center gap-2">
371|             <AlignLeft size={14} className="text-blue-5
/> Briefing
372|         </p>
373|         <div className="text-slate-400 leading-relaxe
text-sm font-medium selection:bg-blue-500 selection:text-white
374|             {appointment.details ? (
375|                 <div className="space-y-2">
376|
377| {appointment.details.split('\n').map((line, i) => (
378|                 <p key={i} className="border-b
border-white/5 pb-1">{line}</p>
379|                 </div>
380|             ) : (
381|                 <span className="italic opacity-30">No
special instructions provided...</span>
382|             )}
383|             </div>
384|         </div>
385|     </div>
386|
387|     <div className="p-8 bg-white/[0.02] flex
justify-between items-center">
388|         <div className="hidden sm:block">
389|             <p className="text-[9px] text-slate-600
font-mono uppercase">Appointment ID: {appointment.id ||
'N/A'}</p>
390|         </div>
391|         <button
392|             onClick={onClose}
393|             className="px-10 py-4 bg-white text-black
hover:bg-blue-600 hover:text-white rounded-xl font-black text-
uppercase tracking-[0.2em] transition-all shadow-lg
active:scale-95"
394|             >
395|             Close Record
396|         </button>
397|     </div>
398| </div>
399| </div>
400| );

```

```

401| }
402|
403|
404| export function AppointmentBooking({ availability,
jobDuration, onConfirm, onCancel, price }) {
405|     const [selectedDate, setSelectedDate] = useState('');
406|     const [selectedTime, setSelectedTime] = useState('');
407|     const [formData, setFormData] = useState({
408|         address: '',
409|         details: ''
410|     });
411|
412|
413|     const parseDurationToHours = (durationStr) => {
414|         if (!durationStr || !durationStr.includes(':'))
return 0;
415|         const [hours, mins] =
durationStr.split(':').map(Number);
416|         return hours + (mins / 60);
417|     };
418|
419|     const calculateEndTime = (startTime, durationStr) =>
420|         if (!startTime || !durationStr) return '';
421|
422|         const [startH, startM] =
startTime.split(':').map(Number);
423|         const [durH, durM] =
durationStr.split(':').map(Number);
424|
425|         const totalMinutes = (startH * 60) + startM + (dur
* 60) + durM;
426|
427|         const endH = Math.floor(totalMinutes / 60) % 24;
428|         const endM = totalMinutes % 60;
429|
430|         return `${endH.toString().padStart(2,
'0')}:${endM.toString().padStart(2, '0')}`;
431|     };
432|
433|     const availableDates = Object.keys(availability);
434|     const timeSlots = selectedDate ?
availability[selectedDate] : [];
435|

```

```

436|     const handleConfirm = () => {
437|         if (!selectedDate || !selectedTime ||
!formData.address) {
438|             alert("Please fill in all required fields");
439|             return;
440|         }
441|
442|         const endTime = calculateEndTime(selectedTime,
jobDuration);
443|
444|         onConfirm({
445|             date: selectedDate,
446|             start_time: selectedTime,
447|             end_time: endTime,
448|             address: formData.address,
449|             details: formData.details,
450|         });
451|     };
452|
453|     return (
454|         <div className="bg-slate-800/60 border
border-slate-700 p-8 rounded-3xl shadow-2xl backdrop-blur-xl
animate-fadeIn space-y-8 max-w-2xl mx-auto">
455|             <h2 className="text-2xl font-bold text-white fl
items-center gap-3">
456|                 <CalendarIcon className="text-blue-500" />
Book Appointment
457|             </h2>
458|
459|             <div className="space-y-4">
460|                 <label className="text-slate-400 text-sm
font-bold uppercase tracking-wider block text-center mb-2">
461|                     Select Date
462|                 </label>
463|
464|                 <div className="grid grid-cols-4 sm:grid-cols
gap-2">
465|                     {availableDates.map(date => (
466|                         <button
467|                             key={date}
468|                             onClick={() => {
setSelectedDate(date); setSelectedTime(''); }}

```

```

469|             className={`py-3 px-1 rounded-xl
border text-sm font-medium transition-all ${
470|                 selectedDate === date
471|                 ? 'bg-blue-600 border-blue-400
text-white shadow-lg shadow-blue-500/20'
472|                 : 'bg-slate-900/40
border-slate-700 text-slate-300 hover:border-slate-500'
473|             }}
474|         >
475|             <span className="block text-[10px]
uppercase opacity-60">
476|                 {new
Date(date).toLocaleDateString(undefined, { month: 'short' })}
477|             </span>
478|             <span className="text-base">
479|                 {new
Date(date).toLocaleDateString(undefined, { day: 'numeric' })}
480|             </span>
481|             </button>
482|         )))}
483|     </div>
484| </div>
485|
486|     {selectedDate && (
487|         <div className="space-y-4 animate-fadeIn">
488|             <label className="text-slate-400 text-sm
font-bold uppercase tracking-wider flex justify-between">
489|                 Available Times
490|                 <span className="text-blue-400
lowercase font-normal italic">Duration: {jobDuration}</span>
491|             </label>
492|             <div className="grid grid-cols-3
sm:grid-cols-4 gap-2">
493|                 {timeSlots.map(time => (
494|                     <button
495|                         key={time}
496|                         onClick={() =>
setSelectedTime(time)}
497|                         className={`py-2 rounded-lg
text-sm font-medium transition-all ${
498|                             selectedTime === time
499|                             ? 'bg-indigo-600
text-white ring-2 ring-indigo-400'

```

```

500|                                     : 'bg-slate-700/50
text-slate-300 hover:bg-slate-600'
501|                                     }` }
502|                                     >
503|                                     {time}
504|                                     </button>
505|                                 ) ) }
506|                             </div>
507|                         </div>
508|                 ) }
509|
510|                 <div className="grid grid-cols-1 gap-6">
511|                     <div className="space-y-2">
512|                         <label className="text-slate-400 text-s
font-bold uppercase tracking-wider flex items-center gap-2">
513|                             <MapPin className="w-4 h-4" /> Serv
Address
514|                             </label>
515|                             <input
516|                                 type="text"
517|                                 placeholder="123 Street Name, City"
518|                                 className="w-full bg-slate-900/50
border border-slate-700 rounded-xl px-4 py-3 text-white
focus:outline-none focus:ring-2 focus:ring-blue-500/50"
519|                                 value={formData.address}
520|                                 onChange={ (e) =>
setFormData ({...formData, address: e.target.value}) }
521|                             />
522|                         </div>
523|
524|                         <div className="space-y-2">
525|                             <label className="text-slate-400 text-s
font-bold uppercase tracking-wider flex items-center gap-2">
526|                                 <AlignLeft className="w-4 h-4" />
Additional Details
527|                                 </label>
528|                                 <textarea
529|                                     rows="3"
530|                                     placeholder="Gate codes, specific
requirements..."
531|                                     className="w-full bg-slate-900/50
border border-slate-700 rounded-xl px-4 py-3 text-white
focus:outline-none focus:ring-2 focus:ring-blue-500/50"

```



```

532 |         value={formData.details}
533 |         onChange={(e) =>
setFormData({...formData, details: e.target.value})}
534 |     ></textarea>
535 | </div>
536 | </div>
537 |
538 |     {selectedDate && selectedTime && formData.address
(
539 |         <div className="bg-slate-900/90 border-l-4
border-blue-500 rounded-2xl p-6 space-y-4 animate-fadeIn
shadow-xl">
540 |             <div className="flex justify-between
items-center border-b border-slate-800 pb-3">
541 |                 <h3 className="text-blue-400 text-xs
font-bold uppercase tracking-widest">Appointment Summary</h3>
542 |                 <span className="bg-blue-500/10
text-blue-400 text-[10px] px-2 py-1 rounded-md border
border-blue-500/20">
543 |                     {parseDurationToHours(jobDuration)} h
Service
544 |                 </span>
545 |             </div>
546 |
547 |             <div className="space-y-4">
548 |                 <div className="flex items-center gap-3">
549 |                     <div className="p-2 bg-slate-800
rounded-lg">
550 |                         <Clock className="w-5 h-5
text-blue-400" />
551 |                     </div>
552 |                     <div>
553 |                         <p className="text-white font-bol
text-lg">
554 |                             {selectedTime} –
{calculateEndTime(selectedTime, jobDuration)}
555 |                         </p>
556 |                         <p className="text-xs
text-slate-500">
557 |                             {new
Date(selectedDate).toLocaleDateString(undefined, {
558 |                                 weekday: 'long',
559 |                                 month: 'short',

```

```

560 |         day: 'numeric'
561 |     }} }
562 |     </p>
563 |   </div>
564 | </div>
565 |
566 |   <div className="flex items-center gap-3">
567 |     <div className="p-2 bg-slate-800
rounded-lg">
568 |       <MapPin className="w-5 h-5
text-red-400" />
569 |     </div>
570 |     <div>
571 |       <p className="text-white font-mec
text-sm leading-tight">
572 |         {formData.address}
573 |       </p>
574 |       <p className="text-[10px]
text-slate-500 uppercase tracking-tighter">
575 |         Service Location
576 |       </p>
577 |     </div>
578 |   </div>
579 |
580 |   <div className="flex items-center gap-3">
581 |     <div className="p-2 bg-slate-800
rounded-lg">
582 |       <DollarSign className="w-5 h-5
text-green-400" />
583 |     </div>
584 |     <div>
585 |       <p className="text-green-400
font-bold text-lg">
586 |         ${ (parseFloat(price) *
parseDurationToHours(jobDuration)).toFixed(2) }
587 |       </p>
588 |       <p className="text-[10px]
text-slate-500 uppercase tracking-tighter">
589 |         Total Estimate (${price}/hr ×
{parseDurationToHours(jobDuration)} hrs)
590 |       </p>
591 |     </div>
592 |   </div>

```

```

593|         </div>
594|     </div>
595|     ) }
596|
597|         <div className="space-y-4">
598|             <div className="flex items-start gap-3 p-4
bg-blue-500/5 border border-blue-500/20 rounded-2xl">
599|                 <CheckCircle className="w-4 h-4
text-blue-400 mt-1" />
600|                 <p className="text-slate-400 text-sm
leading-relaxed">
601|                     By confirming, you agree to the hou
fee of <span className="text-white font-bold">${price}</span>.
602|                     <br/>By confirming you also agree t
knowledge that<br/>Pro-Find is not accountable to any price
changes the freelancer might impose
603|                 </p>
604|             </div>
605|
606|             <div className="flex gap-4">
607|                 <button
608|                     onClick={onCancel}
609|                     className="flex-1 py-4 border
border-slate-700 text-slate-400 font-bold rounded-2xl
hover:bg-slate-700/30 transition-all"
610|                 >
611|                     Cancel
612|                 </button>
613|
614|                 <button
615|                     disabled={!selectedDate || !selectedTime
|| !formData.address}
616|                     onClick={handleConfirm}
617|                     className={`flex-[2] py-4 font-bold
rounded-2xl shadow-lg flex items-center justify-center gap-2
transition-all
618|                         ${(!selectedDate || !selectedTime
!formData.address)
619|                             ? "bg-slate-700 text-slate-500
cursor-not-allowed opacity-50"
620|                             : "bg-gradient-to-r
from-green-600 to-emerald-600 hover:from-green-500
hover:to-emerald-500 text-white transform hover:-translate-y-1

```

```
621|         }`}  
622|     >  
623|         <CheckCircle className="w-5 h-5" />  
Confirm Booking  
624|         </button>  
625|     </div>  
626| </div>  
627| </div>  
628| );  
629| }  
630|
```

homePag.jsx

```

0 | import { useSelector, useDispatch } from "react-redux";
1 | import { useNavigate, useParams, useLocation } from
  "react-router-dom";
2 | import { useState, useEffect } from "react";
3 | import { useSocket } from "../socket/SocketContext";
4 |
5 | import LoadingScreen from "../views/LoadingScreen";
6 | import Navbar from "../routerPrint";
7 | import FreelancerView from "../views/FreelancerView";
8 | import UserView from "../views/UserView";
9 | import useUserSync from "../hooks/UserInfo";
10|
11|
12| export default function HomePage({isPublic=false}) {
13|     const ws = useSocket();
14|     const dispatch = useDispatch();
15|     const navigate = useNavigate();
16|     const location = useLocation();
17|
18|     const authUser = useSelector((state) =>
state.auth.userInfo);
19|     const token = useSelector((state) =>
state.auth.userToken);
20|
21|     const { id } = useParams();
22|
23|     const [targetId, setTargetId] = useState(id ||
location.state?.freelancerId || null)
24|
25|     const [appointmentTimes, setAppointmentTimes] =
useState(null)
26|
27|     const [viewedFreelancer, setViewedFreelancer] =
useState(null)
28|
29|     useUserSync(ws, token, dispatch, navigate,
setViewedFreelancer, setAppointmentTimes, targetId, isPublic |
(targetId !== null));
30|
31|     const isLoading = authUser && Object.keys(authUser).ler
> 0;

```

```
32 |  
33 |     useEffect(() =>{  
34 |         const currentUserId = id ||  
location.state?.freelancerId || null  
35 |         if (currentUserId === null){  
36 |             setViewedFreelancer(null)  
37 |             setAppointmentTimes(null)  
38 |             return  
39 |         }  
40 |     }, [id, location.state])  
41 |  
42 |  
43 |     if (!isLoading) return <LoadingScreen />;  
44 |  
45 |  
46 |  
47 |     const renderContent = () => {  
48 |         if (authUser.role === "User" && viewedFreelancer)  
49 |             return (  
50 |                 <FreelancerView  
51 |                     user={viewedFreelancer}  
52 |                     isPublic={true}  
53 |                     appointmentTimes = {appointmentTimes}  
54 |                 />  
55 |             );  
56 |         }  
57 |  
58 |         if (authUser.role === "User") {  
59 |             return (  
60 |                 <UserView/>  
61 |             );  
62 |         }  
63 |  
64 |         return <FreelancerView user={authUser}  
isPublic={false} />;  
65 |     };  
66 |  
67 |     return (  
68 |         <div className="flex fixed inset-0 bg-slate-900  
w-full min-h-screen">  
69 |             <Navbar role={authUser.role} />  
70 |
```

```
71 |         <div className="ml-60 pt-8 pl-4 w-full  
overflow-y-auto">  
72 |             <div className="max-w-7xl mx-auto px-4">  
73 |                 {renderContent() }  
74 |             </div>  
75 |         </div>  
76 |     </div>  
77 | );  
78 | }
```

```

0 | // #region imports
1 | import {Link, useNavigate} from 'react-router-dom'
2 | import {useState, useEffect} from 'react'
3 | import {LoginRequest, ForgotPasswordVerificationCodeRequest,
ForgotPasswordAuthenticationRequest, ChangePasswordRequest} from
'../socket/RequestHandler'
4 | import {useSocket} from "../socket/SocketContext"
5 | import { handleLoginResponse, handleChangePasswordResponse,
handleForgotPasswordVerificationCodeResponse,
handleForgotPasswordVerifyCodeResponse } from
"../socket/ResponseHandlers"
6 | import { useDispatch } from "react-redux"
7 | import websocketParser from "../socket/MsgParser"
8 | import {InputField, ErrorMessage, EmailVerification} from
"./signUpModule"
9 | import { MessageType } from '../socket/MsgTypes'
10 | // #endregion
11 |
12 |
13 | function ForgotPassword({goBack, ws, email, setEmail,
errorMessage}){
14 |
15 |     const [attempted, setAttempted] = useState(false)
16 |
17 |
18 |     return(
19 |         <div className="fixed inset-0 flex items-center
justify-center bg-slate-900 w-full min-h-screen">
20 |             <div className="bg-slate-700 p-10 rounded-xl
shadow-md w-96 text-center">
21 |                 <h2 className="text-2xl font-bold
text-gray-300 mb-4">
22 |                     Forgot Password
23 |                 </h2>
24 |                 <p className="text-gray-300 text-sm mb-6">
25 |                     Enter your email and we will send a
verification code so you can reset your password.
26 |                 </p>
27 |                 <form className="flex flex-col items-center
space-y-4" onSubmit={ (e) => {e.preventDefault(); setAttempted(true);
ForgotPasswordVerificationCodeRequest(ws, email); } }

```



```

28 |         >
29 |             <div className="w-full">
30 |                 <InputField type="email" name="email"
value={email} onChange={ (e) => setEmail (e.target.value) }
placeholder="Email"/>
31 |             </div>
32 |
33 |             {errorMessage && <ErrorMessage
message={errorMessage} />}
34 |             <button
35 |                 type="submit"
36 |                 className="w-full p-2 bg-blue-500
text-white rounded-md hover:bg-blue-600 transition"
37 |             >
38 |                 Send Reset Email
39 |             </button>
40 |         </form>
41 |
42 |         <button
43 |             type = "button"
44 |             className="w-full p-2 mt-4 bg-blue-500
text-white rounded-md hover:bg-blue-600 transition"
45 |             onClick={goBack}
46 |         >
47 |             Back to login
48 |         </button>
49 |
50 |     </div>
51 | </div>
52 | )
53 | }
54 |
55 |
56 | function ChangePassword({changePasswordRequest, password,
setPassword }) {
57 |     const [confirmPassword, setConfirmPassword] =
useState("");
58 |     const [errorMessage, setErrorMessage] = useState("");
59 |
60 |     const handleSubmit = (e) => {
61 |         e.preventDefault();
62 |
63 |         if (password !== confirmPassword) {

```

```

64 |         setErrorMessage("Passwords do not match");
65 |         return;
66 |     }
67 |     else
68 |         changePasswordRequest()
69 |     };
70 |
71 |     return (
72 |         <div className="fixed inset-0 flex items-center
justify-center bg-slate-900 w-full min-h-screen">
73 |             <div className="bg-slate-700 p-10 rounded-xl
shadow-md w-96 text-center">
74 |                 <h2 className="text-2xl font-bold
text-gray-300 mb-4">
75 |                     Change Password
76 |                 </h2>
77 |                 <p className="text-gray-300 text-sm mb-6">
78 |                     Enter your new password and confirm it
79 |                 </p>
80 |
81 |                 <form className="flex flex-col items-center
space-y-4" onSubmit={handleSubmit}>
82 |                     <div className="w-full">
83 |                         <InputField
84 |                             type="password"
85 |                             name="password"
86 |                             value={password}
87 |                             onChange={ (e) =>
setPassword(e.target.value) }
88 |                             placeholder="New Password"
89 |                         />
90 |                     </div>
91 |
92 |                     <div className="w-full">
93 |                         <InputField type="password"
name="confirmPassword" value={confirmPassword} onChange={ (e) =>
setConfirmPassword(e.target.value) } placeholder="Confirm
Password"/>
94 |                     </div>
95 |
96 |                     {errorMessage && <ErrorMessage
message={errorMessage} />}
97 |

```

```

98|         <button
99|             type="submit"
100|             className="w-full p-2 bg-blue-500
text-white rounded-md hover:bg-blue-600 transition"
101|         >
102|             Change Password
103|         </button>
104|     </form>
105| </div>
106| </div>
107| );
108| }
109|
110|
111| export default function LogIn(payload) {
112|     const ws = useSocket();
113|     const navigate = useNavigate()
114|     const dispatch = useDispatch()
115|
116|     const [username, setUsername] = useState("");
117|     const [password, setPassword] = useState("");
118|
119|     const [forgotPassword, setForgotPassword] =
useState(false)
120|
121|     const [verificationCode, setVerificationCode] =
useState("")
122|
123|     const [email, setEmail] = useState("")
124|
125|     const [serverError, setServerError] = useState("")
126|
127|     const [showVerification, setShowVerification] =
useState(false)
128|
129|     const [showChangePass, setShowChangePass] =
useState(false)
130|
131|     const [pass, setPass] = useState("")
132|
133|     const handleVerificationCodeSubmit = () =>
134|     {

```

```

135|         ForgotPasswordAuthenticationRequest(ws, email,
verificationCode)
136|     }
137|
138|     const handleChangePassword = () =>{
139|         ChangePasswordRequest(ws, email, pass);
140|     }
141|
142|     useEffect(() =>{
143|         if (!ws) return;
144|         const handleMessage = (event) => {
145|             setServerError("")
146|             console.log(`Recvd - ${event.data}`)
147|             const info = websocketParser(event.data)
148|             console.log(`type - ${info.type}\ndata -
${info.data}`)
149|             if (info.type == MessageTypes.LOGIN){
150|                 const [loggedIn, message] =
handleLoginResponse(info, dispatch);
151|                 if (!loggedIn){
152|                     setServerError(message)
153|                 }
154|
155|                 else if(info.type ==
MessageTypes.FORGOT_PASSWORD_REQUEST){
156|                     const [veri_sent, message] =
handleForgotPasswordVerificationCodeResponse(info)
157|                     if (veri_sent){setShowVerification(true);
setForgotPassword(false);
158|                     }else setServerError(message)}
159|
160|                     else if(info.type ==
MessageTypes.FORGOT_PASSWORD_AUTHENTICATION){
161|                         const [veri_sent,message] =
handleForgotPasswordVerifyCodeResponse(info)
162|                         if (veri_sent){setShowVerification(false)
setShowChangePass(true)}
163|                         else setServerError(message)
164|                     }
165|
166|                     else if (info.type ==MessageTypes.CHANGE_PASS
167|                         const [pass_accepted, message] =
handleChangePasswordResponse(info)

```

```

168|         if (pass_accepted) {
169|             setForgotPassword(false)
170|             setShowChangePass(false)
171|         }
172|         else setServerError(message)
173|     }
174| }
175| ws.addEventListener("message", handleMessage);
176|
177| return () =>{
178|     ws.removeEventListener("message", handleMessage)
179| }
180| }, [ws, dispatch, navigate])
181|
182|
183| if (forgotPassword) {
184|     return <ForgotPassword errorMessage={serverError}
goBack={() => setForgotPassword(false)} ws={ws} email={email}
setEmail={email} => setEmail(email)} />
185| }
186|
187| if (showVerification)
188|     return <EmailVerification
verificationCode={verificationCode}
setVerificationCode={setVerificationCode}
handleVerificationCodeSubmit={handleVerificationCodeSubmit}
serverError={serverError} />
189|
190| if (showChangePass)
191|     return <ChangePassword
changePasswordRequest={handleChangePassword} password={pass}
setPassword={setPass} />
192|     return (
193|         <div className="fixed inset-0 flex items-center
justify-center bg-slate-900 w-full min-h-screen">
194|             <div className="bg-slate-700 p-10 rounded-xl
shadow-md w-96 text-center">
195|                 <h2 className="text-2xl font-bold text-gray-300
mb-4">Login</h2>
196|
197|                 <form className="flex flex-col items-center
space-y-4" onSubmit={(e) => {
198|                     e.preventDefault();

```

```

199|         LoginRequest(username, password, ws);
200|     }}>
201|         <div className='w-full'>
202|             <InputField type={"email"} name={"email"}
value={username} onChange={ (e) => setUsername(e.target.value)}
placeholder={"Email"} />
203|         </div>
204|
205|         <div className='w-full'>
206|             <InputField type={"password"}
name={"password"} value={password} onChange={ (e) =>
setPassword(e.target.value)} placeholder={"Password"} />
207|         </div>
208|
209|         {serverError && (
210|             <ErrorMessage message={serverError} />
211|         )}
212|
213|         <button
214|             type="submit"
215|             className="w-full p-2 bg-blue-500 text-white
rounded-md hover:bg-blue-600 transition"
216|         >
217|             Login
218|         </button>
219|
220|     </form>
221|
222|     <Link to="/signup" className="block mt-3
text-blue-600 hover:underline">
223|         Sign up
224|     </Link>
225|
226|     <button
227|         type="button"
228|         onClick={ () => setForgotPassword(true) }
229|         className="mt-3 text-blue-600 hover:underline
text-sm"
230|     >
231|         Forgot Password?
232|     </button>
233|
234| </div>

```

```
235|      </div>  
236|  ) };
```

protectedRoutes.jsx

```
0 | import { useSelector } from "react-redux";
1 | import { Navigate, Outlet } from "react-router-dom";
2 |
3 | export default function ProtectedRoutes() {
4 |     const isLoggedIn = useSelector((state) =>
state.auth.loggedIn);
5 |
6 |     if (!isLoggedIn) {
7 |         return <Navigate to="/login" replace />;
8 |     }
9 |
10 |     return <Outlet />;
11 | }
12 |
```



```

0 | import { Link, useNavigate } from "react-router-dom";
1 | import { useDispatch } from "react-redux";
2 | import { logout } from "../store/authSlice";
3 |
4 | function Navbar({ role }) {
5 |   const dispatch = useDispatch();
6 |   const navigate = useNavigate();
7 |
8 |   const handleLogout = () => {
9 |     dispatch(logout());
10 |    localStorage.clear();
11 |    navigate("/login");
12 |  };
13 |
14 |   return (
15 |     <nav className="fixed left-0 top-0 h-full w-60
bg-slate-800 flex flex-col py-8 px-4 shadow-lg">
16 |
17 |       <div className="flex flex-col gap-2">
18 |         <Link
19 |           to="/"
20 |           className="text-gray-200 px-4 py-2 rounded-md
hover:bg-slate-700 transition flex items-center gap-3
justify-start"
21 |         >
22 |           <span>🏠</span> <span>Home Page</span>
23 |         </Link>
24 |
25 |         {role === "User" && (
26 |           <Link
27 |             to="/search"
28 |             className="text-gray-200 px-4 py-2 rounded-md
hover:bg-slate-700 transition flex items-center gap-3
justify-start"
29 |           >
30 |             <span>🔍</span> <span>Find Freelancers</span>
31 |           </Link>
32 |         ) }
33 |       </div>
34 |

```

```
35 |         <div className="mt-auto border-t border-slate-700
36 |             <button
37 |                 onClick={handleLogout}
38 |                 className="w-full flex items-center gap-3
39 |                 justify-start text-red-500 font-semibold px-4 py-2 rounded-md
40 |                 hover:bg-red-950/30 transition duration-200"
41 |             >
42 |                 <span className="text-xl"> 🚪 </span>
43 |                 <span>Logout</span>
44 |             </button>
45 |         </div>
46 |     </nav>
47 | );
48 | }
49 | export default Navbar;
```

search.jsx

```

0 | import { useState, useEffect, useRef } from "react";
1 | import { useSelector, useDispatch } from "react-redux";
2 | import Navbar from "../routerPrint";
3 | import { MapPin, Briefcase, Search, ChevronRight, Ellipse,
  User, Star, CreditCard, ArrowLeft } from "lucide-react";
4 | import { GetAllJobs, GetCitiesByJob, GetMinimalFreelancerInfo,
  GetPublicProfileInfoRequest } from "../socket/RequestHandler";
5 | import { useSocket } from "../socket/SocketContext";
6 | import websocketParser from "../socket/MsgParser";
7 | import { MessageTypes, StatusMessage } from
  "../socket/MsgTypes";
8 | import { handleJobsResponse, handleCitiesResponse,
  handleMinimalFreelancerInfoResponse } from
  "../socket/ResponseHandlers"
9 | import LoadingScreen from "../views/LoadingScreen";
10 | import { useNavigate } from "react-router-dom";
11 | import { logout } from "../store/authSlice";
12 |
13 | function SearchBar({ ws, token, handleSubmit }) {
14 |     const navigate = useNavigate()
15 |     const authUser = useSelector((state) =>
state.auth.userInfo);
16 |     const dispatch = useDispatch()
17 |
18 |
19 |     const [allJobTypes, setAllJobTypes] = useState([]);
20 |     const [jobCitiesFromServer, setJobCitiesFromServer] =
useState([]);
21 |
22 |     const [searchJob, setSearchJob] = useState("");
23 |     const [selectedJob, setSelectedJob] = useState(null);
24 |     const [isJobOpen, setIsJobOpen] = useState(false);
25 |
26 |     const [searchCity, setSearchCity] = useState("");
27 |     const [selectedCity, setSelectedCity] = useState(null)
28 |     const [isCityOpen, setIsCityOpen] = useState(false);
29 |
30 |     const jobRef = useRef(null);
31 |     const cityRef = useRef(null);
32 |
33 |     useEffect(() => {

```

```

34|         const handleClickOutside = (event) => {
35|             if (jobRef.current &&
!jobRef.current.contains(event.target)) setIsJobOpen(false);
36|             if (cityRef.current &&
!cityRef.current.contains(event.target)) setIsCityOpen(false);
37|         };
38|         document.addEventListener("mousedown",
handleClickOutside);
39|         return () =>
document.removeEventListener("mousedown", handleClickOutside);
40|     }, []);
41|
42|     useEffect(() => {
43|         if (!ws) return;
44|         const handleMessage = (event) => {
45|             let info = websocketParser(event.data);
46|
47|             if (MessageTypes.BROAD == info.type) {
48|                 if (StatusMessage.TOKEN_BAD in info.data) {
49|                     alert("Your session has ended, login
again to get service")
50|                     dispatch(logout())
51|                     navigate('/login')
52|                 }
53|             }
54|             else if (info.type == MessageTypes.GET_JOBS) {
55|                 const [status, message] =
handleJobsResponse(info);
56|                 if (status) setAllJobTypes(message);
57|
58|             } else if (info.type ==
MessageTypes.GET_CITIES_BY_JOB) {
59|                 const [status, message] =
handleCitiesResponse(info);
60|                 if (status) setJobCitiesFromServer(message);
61|             }
62|         };
63|
64|         const setup = () => {
65|             ws.addEventListener("message", handleMessage);
66|             GetAllJobs(ws, token);
67|         };
68|

```

```

69|         if (ws.readyState === WebSocket.OPEN) setup();
70|         else ws.addEventListener("open", setup, { once: true });
71|     });
72|     return () => {
73|         ws.removeEventListener("message", handleMessage);
74|     };
75| }, [ws, token]);
76|
77| const resetSearch = () => {
78|     setSelectedJob(null)
79|     setSearchJob("")
80|     setSelectedCity(null)
81|     setSearchCity("")
82|     setJobCitiesFromServer([]);
83| };
84|
85| const isLocked = selectedJob && selectedCity;
86| return (
87|     <div className="fixed inset-0 bg-[#020617] flex
overflow-hidden font-sans antialiased text-slate-200">
88|         <Navbar role={authUser?.role} />
89|
90|         <div className="absolute inset-0 overflow-hidden
pointer-events-none select-none">
91|             <div className="absolute -top-24 -right-24 w-1/2
aspect-square bg-blue-600/10 blur-[120px] rounded-full" />
92|             <div className="absolute -bottom-24 -left-24 w-1/3
aspect-square bg-indigo-600/10 blur-[100px] rounded-full" />
93|         </div>
94|
95|         <main className="flex-1 ml-60 h-full relative flex
flex-col items-center justify-center p-8 lg:p-16 z-10">
96|
97|             <div className="w-full max-w-5xl space-y-12">
98|
99|                 <header>
100|                     <h1 className="text-7xl md:text-8xl
lg:text-9xl font-black text-white tracking-tight leading-[0.9]" />
101|                     Hire <span
className="text-blue-500">Quality</span><br />

```

```

102|                                     <span className="opacity-10
italic">Freelancers</span>
103|                                     </h1>
104|                                     </header>
105|
106|                                     <div className="grid grid-cols-1
lg:grid-cols-2 gap-6">
107|
108|                                     <div className="relative group"
ref={jobRef}>
109|                                     <Briefcase className={`absolute
left-7 top-1/2 -translate-y-1/2 w-7 h-7 z-20 transition-colors
${selectedJob ? 'text-blue-400' : 'text-slate-500'}`} />
110|
111|                                     <input
112|                                         type="text"
113|                                         readOnly={!selectedJob}
114|                                         placeholder={selectedJob ? ""
"Service"}
115|                                         className={`w-full
bg-slate-900/40 border-2 text-white text-3xl py-10 pl-20 pr-32
rounded-3xl outline-none transition-all backdrop-blur-md
116|                                             ${selectedJob ?
'border-blue-500/40' : 'border-slate-800 focus:border-blue-500
hover:border-slate-700'}`}
117|                                         value={searchJob}
118|                                         onFocus={() => !selectedJob &
setIsJobOpen(true)}
119|                                         onChange={(e) =>
setSearchJob(e.target.value)}
120|                                     />
121|
122|                                     {selectedJob && (
123|                                         <button
124|                                             onClick={resetSearch}
125|                                             className="absolute right
top-1/2 -translate-y-1/2 z-30 px-5 py-2 rounded-xl text-xs
font-bold tracking-widest bg-blue-500/10 text-blue-400 border
border-blue-500/20 hover:bg-blue-500/20 transition-colors
uppercase"
126|                                         >
127|                                             Change
128|                                         </button>

```

```

129|         ) }
130|
131|         {isJobOpen && (
132|             <div className="absolute z-50
w-full mt-3 bg-slate-900 border border-slate-800 rounded-2xl
shadow-2xl max-h-80 overflow-y-auto ring-1 ring-white/5">
133|                 {allJobTypes.filter(j =>
j.toLowerCase().includes(searchJob.toLowerCase()))}.map(job =>
134|                     <button
135|                         key={job}
136|                         onClick={() => {
setSelectedJob(job); setSearchJob(job); setIsJobOpen(false);
GetCitiesByJob(ws, job, token); }}
137|                         className="w-full
text-left px-8 py-6 text-xl text-slate-300 hover:bg-blue-600
hover:text-white flex justify-between items-center
transition-colors group/item"
138|                     >
139|                         {job}
140|                         <ChevronRight
className="w-5 h-5 opacity-0 group-hover/item:opacity-100
transition-opacity" />
141|                     </button>
142|                 ) ) }
143|             </div>
144|         ) }
145|     </div>
146|
147|     <div className={`relative transition-
duration-500 ${!selectedJob ? 'opacity-40 grayscale
pointer-events-none' : 'opacity-100'}`} ref={cityRef}>
148|         <MapPin className={`absolute left
top-1/2 -translate-y-1/2 w-7 h-7 z-20 ${selectedCity ?
'text-blue-400' : 'text-slate-500'}`} />
149|         <input
150|             type="text"
151|             readOnly={!selectedCity}
152|             placeholder={selectedCity ? '
"In which city?"}
153|             className={`w-full
bg-slate-900/40 border-2 text-white text-3xl py-10 pl-20 pr-32
rounded-3xl outline-none transition-all backdrop-blur-md

```

```

154|             ${selectedCity ?
'border-blue-500/40' : 'border-slate-800 focus:border-blue-500
hover:border-slate-700'} `}
155|             value={searchCity}
156|             onFocus={() => !selectedCity
setIsCityOpen(true)}
157|             onChange={(e) =>
setSearchCity(e.target.value)}
158|             />
159|
160|             {selectedCity && (
161|                 <button
162|                     onClick={() => {
setSelectedCity(null); setSearchCity(""); }}
163|                     className="absolute right
top-1/2 -translate-y-1/2 z-30 px-5 py-2 rounded-xl text-xs
font-bold tracking-widest bg-blue-500/10 text-blue-400 border
border-blue-500/20 hover:bg-blue-500/20 transition-colors
uppercase"
164|                 >
165|                     Change
166|                 </button>
167|             ) }
168|
169|             {isCityOpen && (
170|                 <div className="absolute z-50
w-full mt-3 bg-slate-900 border border-slate-800 rounded-2xl
shadow-2xl max-h-80 overflow-y-auto ring-1 ring-white/5">
171|                 {jobCitiesFromServer.filter(c =>
c.toLowerCase().includes(searchCity.toLowerCase())) .map(city =>
172|                     <button
173|                         key={city}
174|                         onClick={() => {
setSelectedCity(city); setSearchCity(city); setIsCityOpen(false)
}}
175|                         className="w-full
text-left px-8 py-6 text-xl text-slate-300 hover:bg-blue-600
hover:text-white transition-colors"
176|                     >
177|                         {city}
178|                     </button>
179|                 )) }

```



```

180|                                     </div>
181|                                     ) }
182|                                     </div>
183|                                 </div>
184|
185|                                 <div className={`transition-all duration-
delay-100 ${isLocked ? 'translate-y-0 opacity-100' :
'translate-y-8 opacity-0 pointer-events-none'}}`>
186|                                     <button
187|                                         onClick={() =>
handleSubmit(selectedCity, selectedJob)}
188|                                         className="w-full relative group
p-[1px] rounded-3xl overflow-hidden bg-slate-800
hover:bg-blue-500 transition-colors duration-300"
189|                                     >
190|                                         <div className="bg-white text-blue-
py-10 rounded-[calc(1.5rem-1px)] font-black text-4xl flex
items-center justify-center gap-6 group-hover:bg-transparent
group-hover:text-white transition-all">
191|                                             <Search className="w-10 h-10
stroke-[3px]" />
192|                                             <span>SEARCH NOW</span>
193|                                         </div>
194|                                     </button>
195|
196|                                     <p className="mt-6 text-center
text-slate-500 text-base">
197|                                         Searching for <span
className="text-blue-400">{selectedJob}</span> in <span
className="text-blue-400">{selectedCity}</span>
198|                                     </p>
199|                                 </div>
200|
201|                             </div>
202|                         </main>
203|                 </div>
204| )
205| }
206|
207|
208| function FreelancersMenu({ws, token, goBack, city, job}){
209|     const dispatch = useDispatch()
210|     const navigate = useNavigate()

```

```

211|
212|     const authUser = useSelector((state) =>
state.auth.userInfo)
213|     const [gotFreelancersInfo, setGotFreelancersInfo] =
useState(false)
214|     const [freelancersInfo, setFreelancersInfo] =
useState(null)
215|
216|     const hasRequested = useRef(false)
217|
218|     const handleClick = (id) =>{
219|         GetPublicProfileInfoRequest(ws, id, token)
220|         navigate(`/profile/${id}`, { state: { freelancerId
id } });
221|     }
222|
223|
224|     useEffect(() => {
225|         const handleMessage = (event) =>{
226|             if (!ws) return;
227|
228|             const info = websocketParser(event.data)
229|             if (info.type == MessageTypes.BROAD){
230|                 if (StatusMessage.TOKEN_BAD in info.data)
231|                     alert("Invalid token, ending session.
Log in to keep searching");
232|                     dispatch(logout())
233|                     navigate("/login")
234|                 }
235|             }
236|
237|             else if(info.type ===
MessageTypes.GET_MINIMAL_FREELANCER_INFO){
238|                 const [status, msg] =
handleMinimalFreelancerInfoResponse(info)
239|                 if (status){
240|                     console.log(msg)
241|                     setFreelancersInfo(msg)
242|                     setGotFreelancersInfo(true);
243|                 }
244|                 else{
245|                     alert(msg)
246|                     goBack()

```

```

247|         }
248|     }
249| }
250|
251|
252|     ws.addEventListener("message", handleMessage)
253|
254|     if (!hasRequested.current) {
255|         GetMinimalFreelancerInfo(ws, token, city, job)
256|         hasRequested.current = true
257|     }
258|     return () => ws.removeEventListener("message",
handleMessage)
259|
260|     }, [ws, token, city, job])
261|
262|     if (!gotFreelancersInfo) return <LoadingScreen/>
263|
264|     return (
265|         <div className="fixed inset-0 bg-[#020617] flex
overflow-hidden font-sans antialiased text-slate-200">
266|             <Navbar role={authUser?.role} />
267|
268|             <div className="absolute inset-0 overflow-hidden
pointer-events-none select-none">
269|                 <div className="absolute top-[-20%]
left-[10%] w-[800px] h-[800px] bg-blue-600/5 blur-[120px]
rounded-full" />
270|             </div>
271|
272|             <main className="flex-1 ml-60 h-full relative
flex flex-col p-8 lg:p-16 overflow-y-auto scrollbar-hide">
273|
274|                 <div className="w-full max-w-6xl mx-auto
z-10">
275|
276|                     <header className="flex flex-col
lg:flex-row lg:items-end justify-between gap-10 mb-20">
277|                         <div className="space-y-6">
278|                             <div className="flex
items-center gap-4 text-blue-500 font-bold tracking-widest
text-xs uppercase">

```

```

279|         <span className="w-8 h-p>
bg-blue-500/50"></span>
280|         Available Professionals
281|     </div>
282|     <h1 className="text-6xl
md:text-7xl font-black text-white tracking-tighter leading-nor
283|         {job} <br />
284|         <span
className="text-slate-600">in</span> {city}
285|     </h1>
286| </div>
287|
288|     <button
289|         onClick={goBack}
290|         className="group flex
items-center gap-3 bg-slate-900 border border-slate-800
hover:border-blue-500 px-6 py-4 rounded-2xl text-slate-400
hover:text-white transition-all duration-300 shadow-xl"
291|     >
292|         <ArrowLeft className="w-5 h-5
group-hover:-translate-x-1 transition-transform" />
293|         <span className="font-bold
text-sm uppercase tracking-widest">Change Search</span>
294|     </button>
295| </header>
296|
297|     <div className="grid grid-cols-1
md:grid-cols-2 xl:grid-cols-3 gap-8">
298|     {Object.entries(freelancersInfo).map(([pro_id, data]) => (
299|         <div
300|             key={pro_id}
301|             className="group relative
bg-slate-900/30 border border-slate-800 hover:border-blue-600/
rounded-[2rem] p-8 transition-all duration-500
hover:shadow-[0_20px_50px_rgba(8,_112,_184,_0.1)] flex flex-cc
302|         >
303|             <div className="flex
justify-between items-start mb-10">
304|                 <div className="flex
items-center gap-2 px-3 py-1.5 bg-blue-500/10 border
border-blue-500/20 rounded-lg text-blue-400">

```

```

305|                                     <Star
className="w-3.5 h-3.5 fill-blue-400" />
306|                                     <span
className="font-black text-xs uppercase tracking-tighter">
307|                                     {data.rating
`${data.rating} Rating` : "New"}
308|                                     </span>
309|                                 </div>
310|                             <div
className="text-right">
311|                                     <span
className="block text-[10px] font-bold text-slate-500 uppercase
tracking-widest mb-1">Hourly Rate</span>
312|                                     <span
className="text-3xl font-black text-white
tracking-tight">${data.price}</span>
313|                                     </div>
314|                                 </div>
315|
316|                             <div className="flex-1
space-y-4 mb-12">
317|                                     <div className="flex
items-center gap-4">
318|                                     <div className="w
h-12 bg-slate-800 rounded-xl flex items-center justify-center
border border-slate-700 group-hover:scale-110
transition-transform duration-500">
319|                                     <User
className="text-slate-500 group-hover:text-blue-400 w-6 h-6" /
320|                                     </div>
321|                                     <div>
322|                                     <h3
className="font-black text-lg text-white
group-hover:text-blue-400 transition-colors">Verified
Professional</h3>
323|                                     <p
className="text-[10px] text-slate-500 font-bold uppercase
tracking-widest leading-none">Pro-Find Member</p>
324|                                     </div>
325|                             </div>
326|

```

```

327|                                     <p
className="text-slate-400 leading-relaxed text-base italic
line-clamp-4 pt-2">
328|                                     {data.description
`"${data.description}"` : "This professional hasn't provided a
bio yet, but their credentials meet our high platform
standards."}
329|                                     </p>
330|                                     </div>
331|
332|                                     <button
333|                                         onClick={ () =>
handleOnClick(pro_id) }
334|                                         className="w-full
bg-white text-black py-5 rounded-2xl font-black text-sm
tracking-widest flex items-center justify-center gap-2
group-hover:bg-blue-600 group-hover:text-white transition-all
duration-300 active:scale-95"
335|                                     >
336|                                         <span>VIEW
PROFILE</span>
337|                                         <ChevronRight
className="w-4 h-4" />
338|                                     </button>
339|                                     </div>
340|                                     ) }
341|                                 </div>
342|
343|                                     {Object.keys(freelancersInfo).length
0 && (
344|                                     <div className="flex flex-col
items-center justify-center py-32 border border-dashed
border-slate-800 rounded-[3rem] bg-slate-900/10">
345|                                         <div className="w-16 h-16
bg-slate-800 rounded-full flex items-center justify-center mb-
346|                                         <Search
className="text-slate-500 w-8 h-8" />
347|                                         </div>
348|                                         <h3 className="text-white
text-2xl font-black tracking-tight mb-2">No matches found</h3>
349|                                         <p className="text-slate-500
text-lg max-w-sm text-center">

```

```

350|           Try broadening your search
criteria or checking a neighboring city.
351|           </p>
352|       </div>
353|   })
354| </div>
355| </main>
356| </div>
357| )
358| }
359|
360|
361|
362|
363|
364| export default function SearchPage({}) {
365|     const ws = useSocket()
366|     const token = useSelector((state) =>
state.auth.userToken)
367|
368|     const [serching, setSearching] = useState(true)
369|     const [gotFreelancers, setGotFreelancers] =
useState(false)
370|
371|     const [serchFields, setSearchFields] =
useState({city:null, job:null})
372|
373|     const handleSearch = (city, job) =>{
374|         setSearchFields({city, job})
375|         setSearching(false)
376|     }
377|
378|     return (
379|         <>
380|             {serching ? (
381|                 <SearchBar ws={ws} token={token}
handleSubmit={handleSearch} />
382|             ) : ( <FreelancersMenu ws={ws} token={tok
goBack={() => setSearching(true)} city={serchFields.city}
job={serchFields.job} />
383|             ) }
384|         ) }
385|     </>

```

386 |)
 387 | }

signUp.jsx

```

0 | import { useState, useEffect } from "react";
1 | import { useNavigate } from "react-router-dom";
2 | import { BackToLogin, freelancerProfessions, RolePopup,
  | InputField, ErrorMessage, SingleChoiceDropDownMenu,
  | MultiChoiceDropDownMenu, israeliLocalities, EmailVerification }
  | from "../signUpModule";
3 | import { useSocket } from "../socket/SocketContext"
4 | import { SignUpRequest, VerificationRequest } from
  | "../socket/RequestHandler";
5 | import websocketParser from "../socket/MsgParser";
6 | import { handleSignUpResponse, handleVerificationResponse }
  | from "../socket/ResponseHandlers";
7 | import { MessageTypes } from "../socket/MsgTypes";
8 |
9 | function UserSignUp({ userInfo, setUserInfo, onSubmit,
  | serverError }) {
10 |   const [confirmPassword, setConfirmPassword] = useState('')
11 |   const [passwordMismatch, setPasswordMismatch] =
  | useState(false)
12 |   const [infoEntered, setInfoEntered] = useState(true)
13 |
14 |   const handleChange = (e) => {
15 |     const { name, value } = e.target;
16 |     setUserInfo(prev => ({
17 |       ...prev,
18 |       [name]: value
19 |     }))
20 |   }
21 |
22 |
23 |
24 |   const handleSubmit = (e) => {
25 |     console.log(`User name - ${userInfo.name}\nUser email
  | ${userInfo.email}\nUser password - ${userInfo.password}`)
26 |     e.preventDefault()
27 |     const hasEmptyFields = Object.values(userInfo).some((va
  | => !value)
28 |     if (!hasEmptyFields && confirmPassword !== "") {
29 |       setInfoEntered(true)
30 |
31 |       if (userInfo.password !== confirmPassword) {

```

```

32 |         setPasswordMismatch(true)
33 |         setConfirmPassword("");}
34 |
35 |         else{
36 |             setPasswordMismatch(false)
37 |             onSubmit()
38 |         }}
39 |
40 |         else{
41 |             setPasswordMismatch(false)
42 |             setInfoEntered(false)
43 |         }
44 |     }
45 |
46 |     return (
47 |         <div className="fixed inset-0 flex items-center
justify-center bg-slate-900 w-full min-h-screen">
48 |             <BackToLogin/>
49 |             <div className="bg-slate-700 p-12 rounded-xl shadow-
w-120 text-center">
50 |                 <h1 className="text-gray-300 text-2xl font-bold
mb-4">
51 |                     Please enter your information below
52 |                 </h1>
53 |                 <form className="flex flex-col gap-4"
onSubmit={handleSubmit}>
54 |                     <InputField name={"name"} value={userInfo.name}
onChange={handleChange} placeholder={"Username"} />
55 |
56 |                     <InputField name={"email"} value={userInfo.email}
onChange={handleChange} placeholder={"Email"} type={"email"} />
57 |
58 |                     <InputField name={"password"}
value={userInfo.password} onChange={handleChange}
placeholder={"password"} type={"password"} />
59 |
60 |                     <InputField name={"password2"}
value={confirmPassword} onChange ={(e) =>
setConfirmPassword(e.target.value)} placeholder={"Confirm
password"} type={"password"} />
61 |
62 |                     {passwordMismatch && (

```

```

63|         <ErrorMessage message={"Passwords do not
match"}/>
64|     ) }
65|
66|     {!infoEntered && (
67|         <ErrorMessage message={"Not all fields were
filled"}/>
68|     ) }
69|
70|     {serverError && (
71|         <ErrorMessage message={serverError}/>
72|     ) }
73|
74|     <button
75|         type="submit"
76|         className="bg-blue-400 hover:bg-blue-500
text-white font-semibold py-2 rounded-lg shadow-md mt-2
transition-all"
77|     >
78|         Sign Up
79|     </button>
80| </form>
81| </div>
82| </div>
83| );
84| }
85|
86|
87|
88| function FreelancerSignUp({ freelancerInfo,
setFreelancerInfo, onSubmit, serverError }) {
89|     const [confirmPassword, setConfirmPassword] = useState('')
90|     const [passwordMismatch, setPasswordMismatch] =
useState(false);
91|     const [infoEntered, setInfoEntered] = useState(true);
92|
93|
94|     const [startHour, setStartHour] = useState('');
95|     const [startMinute, setStartMinute] = useState('')
96|     const [endHour, setEndHour] = useState('')
97|     const [endMinute, setEndMinute] = useState('')
98|
99|     const [jobDurationHour, setJobDurationHour] = useState('')

```

```

100|   const [jobDurationMinute, setJobDurationMinute] =
    useState("")
101|
102|   const [professions, setProfessions] = useState([]);
103|   const [cities, setCities] = useState([]);
104|
105|   let widthForBigDropDowns = "480px";
106|   let widthForSmallDropDowns = "90px";
107|   let timeWidth = '50px';
108|
109|
110|   useEffect(() => {
111|     if (startHour !== "" && startMinute !== "") {
112|       setFreelancerInfo(prev => ({
113|         ...prev,
114|         startWorking: `${startHour}:${startMinute}`
115|       }));
116|     }
117|   }, [startHour, startMinute])
118|
119|   useEffect(() => {
120|     if (endHour !== "" && endMinute !== "") {
121|       setFreelancerInfo(prev => ({
122|         ...prev,
123|         finishWorking: `${endHour}:${endMinute}`
124|       }));
125|     }
126|   }, [endHour, endMinute])
127|
128|   useEffect(() => {
129|     if (jobDurationHour !== "" && jobDurationMinute !== "")
130|     {
131|       setFreelancerInfo(prev => ({
132|         ...prev, jobDuration:
133|         `${jobDurationHour}:${jobDurationMinute}`
134|       }));
135|     }
136|   }, [jobDurationHour, jobDurationMinute])
137|
138|   useEffect(() => {
139|     const loadProfessions = async () => {
140|       const profs = await freelancerProfessions();
141|       setProfessions(profs);

```

```

141|     };
142|     loadProfessions();
143|
144|     const loadCities = async () => {
145|         const cityList = await israeliLocalities();
146|         setCities(cityList);
147|     };
148|     loadCities();
149| }, []);
150|
151| const handleChange = (e) =>{
152|     const {name, value} = e.target;
153|     setFreelancerInfo(prev =>({
154|         ...prev,
155|         [name]:value
156|     })
157| )
158| }
159|
160|
161| const handleSubmit = (e) =>{
162|     console.log(`
163| User name - ${freelancerInfo.name}
164| User email - ${freelancerInfo.email}
165| User password - ${freelancerInfo.password}
166| Profession - ${freelancerInfo.profession}
167| Cities - ${freelancerInfo.cities}
168| Years of expertee - ${freelancerInfo.years}
169| Working hours -
170| ${freelancerInfo.startWorking}-${freelancerInfo.finishWorking}
171| Avg job duration - ${freelancerInfo.jobDuration}
172| Self description - ${freelancerInfo.description}
173| `)
174|     e.preventDefault()
175|     const hasEmptyFields =
176| Object.entries(freelancerInfo).some(([key, value]) => {
177|         if (Array.isArray(value)) return value.length ===
178|         if (typeof value === "string") return value.trim()
179|         if (typeof value === "number") return value === 0
180|         return !value;
181|     });

```

```

181|
182|         if (!hasEmptyFields && confirmPassword !== "") {
183|             setInfoEntered(true)
184|             if(freelancerInfo.password!==confirmPassword) {
185|                 setPasswordMismatch(true)
186|                 setConfirmPassword(""); }
187|             else{
188|                 setPasswordMismatch(false)
189|                 onSubmit()
190|             }}
191|         else{
192|             setPasswordMismatch(false)
193|             setInfoEntered(false)
194|         }
195|
196|
197|     }
198|
199|
200|     return (
201|         <div className="fixed inset-0 flex items-center
justify-center bg-slate-900 w-full min-h-screen">
202|             <BackToLogin/>
203|             <div className="bg-slate-600 p-8 rounded-xl
shadow-lg w-140 max-h-[90vh] overflow-y-auto text-center
scrollbar-hide">
204|                 <h1 className="text-gray-300 text-xl font-bold
mb-4">
205|                     Please enter your information below
206|                 </h1>
207|                 <form className="text-gray-300 flex flex-col
gap-4" onSubmit={handleSubmit}>
208|                     <InputField name={"name"}
value={freelancerInfo.name} onChange={handleChange}
placeholder={"Username"} />
209|
210|                     <InputField name={"email"}
value={freelancerInfo.email} onChange={handleChange}
placeholder={"Email"} type={"email"} />
211|
212|                     <InputField name={"password"}
value={freelancerInfo.password} onChange={handleChange}
placeholder={"password"} type={"password"} />

```

```

213 |
214 |         <InputField name={ "password2" }
value={ confirmPassword } onChange = { (e) =>
setConfirmPassword(e.target.value) } placeholder={ "Confirm
password" } type={ "password" } />
215 |         <SingleChoiceDropDownMenu
216 |         placeholder = { "Profession" }
217 |         name={ "profession" }
218 |         options={ professions }
219 |         value={ freelancerInfo.profession }
220 |         onChange= { (professionPicked) =>
setFreelancerInfo( prev =>
({ ...prev, profession: professionPicked }) ) }
221 |         customWidth = { widthForBigDropDowns }
222 |         />
223 |
224 |         <MultiChoiceDropDownMenu
225 |         name={ "cities" }
226 |         options={ cities }
227 |         value={ freelancerInfo.cities }
228 |         onChange= { (selectedCities) =>
setFreelancerInfo( prev => ({ ...prev, cities: selectedCities }) ) }
229 |         placeholder= "Select your areas"
230 |         customWidth= { widthForBigDropDowns }
231 |         />
232 |
233 |         <SingleChoiceDropDownMenu
234 |         name={ "years" }
235 |         options= { Array.from( { length: 41 }, ( _, i ) => i + 1 ) }
236 |         value = { freelancerInfo.years }
237 |         onChange= { (yearsPicked) =>
setFreelancerInfo( prev => ( { ...prev, years: yearsPicked } ) ) }
238 |         placeholder= "Select years of expertise"
239 |         customWidth= { widthForBigDropDowns }
240 |         />
241 |
242 |
243 |         <div className= "flex items-center gap-2" >
244 |         <span className= "text-m font-bold w-50" > Start
time</span>
245 |         <SingleChoiceDropDownMenu
246 |         name= { "startTime" }

```

```

247 |             options={Array.from({length:24},
    |             (_,i)=>String(i).padStart(2,"0"))}
248 |             value={startHour}
249 |             onChange={ (startHourPicked) =>
    |             setStartHour(startHourPicked)}
250 |             customWidth={widthForSmallDropDowns}
251 |             placeholder=""
252 |         />
253 |
254 |         <span className="text-2xl font-bold">:</spa
255 |
256 |         <SingleChoiceDropDownMenu
257 |             name={ "startMinute" }
258 |             options={Array.from({length:60},
    |             (_,i)=>String(i).padStart(2,"0"))}
259 |             value={startMinute}
260 |             onChange={ (startMinutePicked) =>
    |             setStartMinute(startMinutePicked)}
261 |             customWidth={widthForSmallDropDowns}
262 |             placeholder=""
263 |         />
264 |     </div>
265 |
266 |
267 |     <div className="flex items-center gap-2">
268 |         <span className="text-m font-bold w-50">Enc
time</span>
269 |         <SingleChoiceDropDownMenu
270 |             name={ "endTime" }
271 |             options={Array.from({length:24},
    |             (_,i)=>String(i).padStart(2,"0"))}
272 |             value={endHour}
273 |             onChange={ (endHourPicked) =>
    |             setEndHour(endHourPicked)}
274 |             customWidth={widthForSmallDropDowns}
275 |             placeholder=""
276 |         />
277 |
278 |         <span className="text-2xl font-bold">:</spa
279 |
280 |         <SingleChoiceDropDownMenu
281 |             name={ "endMinute" }

```



```

282 |             options={Array.from({length:60},
    |             (_,i)=>String(i).padStart(2,"0"))}
283 |             value={endTime}
284 |             onChange={ (endTimePicked) =>
    |             setEndTime(endMinutePicked) }
285 |             customWidth={widthForSmallDropDowns}
286 |             placeholder=""
287 |         />
288 |     </div>
289 |
290 |
291 |     <div className="flex items-center gap-2">
292 |         <span className="text-m font-bold w-50">Job
    | duration time</span>
293 |         <SingleChoiceDropDownMenu
294 |             name={ "jobStart" }
295 |             options={Array.from({length:24},
    |             (_,i)=>String(i).padStart(2,"0"))}
296 |             value={jobDurationHour}
297 |             onChange={ (jobDurationHourPicked) =>
    |             setJobDurationHour(jobDurationHourPicked) }
298 |             customWidth={widthForSmallDropDowns}
299 |             placeholder=""
300 |         />
301 |
302 |         <span className="text-2xl font-bold">:</span>
303 |
304 |         <SingleChoiceDropDownMenu
305 |             name={ "jobEnd" }
306 |             options={Array.from({length:60},
    |             (_,i)=>String(i).padStart(2,"0"))}
307 |             value={jobDurationMinute}
308 |             onChange={ (jobDurationMinutePicked) =>
    |             setJobDurationMinute(jobDurationMinutePicked) }
309 |             customWidth={widthForSmallDropDowns}
310 |             placeholder=""
311 |         />
312 |     </div>
313 |
314 |
315 |     <InputField name={ "description" }
    | value={freelancerInfo.description} onChange={handleChange}
    | placeholder={ "Description" } />

```

```

316 |
317 |         <SingleChoiceDropDownMenu
318 |             name={"hourPrice"}
319 |             options={Array.from({length:1001}, (_,i) => i
320 |             value={freelancerInfo.hourPrice}
321 |             onChange={ (pricePicked) =>
322 |             setFreelancerInfo(prev => ({
323 |                 ...prev,
324 |                 hourPrice: pricePicked
325 |             }))) }
326 |             placeholder={"Price per hour $$$$"}
327 |             customWidth={widthForBigDropDowns}/>
328 |
329 |             {passwordMismatch && (
330 |                 <ErrorMessage message={"Passwords do not
match"}/>
331 |             ) }
332 |
333 |             {!infoEntered && (
334 |                 <ErrorMessage message={"Not all fields we
entered"}/>
335 |             ) }
336 |
337 |             {serverError && (
338 |                 <ErrorMessage message={serverError}/>
339 |             ) }
340 |
341 |         <button
342 |             type="submit"
343 |             className="bg-blue-400 hover:bg-blue-500
text-white font-semibold py-2 rounded-lg shadow-md mt-2
transition-all"
344 |             >
345 |             Sign Up
346 |         </button>
347 |     </form>
348 | </div>
349 | </div>
350 | );
351 | }
352 |
353 |

```

```
354|
355|
356|
357| export default function SignUpPage() {
358|   const ws = useSocket()
359|   const navigate = useNavigate();
360|
361|   const [role, setRole] = useState(null);
362|   const [userInfo, setUserInfo] = useState({ name:"",
email:"", password:""});
363|   const [freelancerInfo, setFreelancerInfo] = useState({
364|     name:"",
365|     email:"",
366|     password:"",
367|     profession:"",
368|     cities: [],
369|     years: 0,
370|     startWorking: "",
371|     finishWorking:"",
372|     jobDuration:0,
373|     description: "",
374|     hourPrice:""
375|   });
376|
377|   const [serverError, setServerError] = useState("");
378|
379|   const [showVerification, setShowVerification] =
useState(false);
380|   const [verificationCode, setVerificationCode] =
useState("");
381|
382|
383|   const handleSignUpPageSubmit = () => {
384|     setServerError("")
385|     console.log(role)
386|     if (role === "User")
387|       SignUpRequest(ws, userInfo, role);
388|     else if (role === "Freelancer")
389|       SignUpRequest(ws, freelancerInfo, role);
390|   }
391|
392|
393|   const handleVerificationCodeSubmit = () =>{
```

```
394|     setServerError("");
395|     if (role==="User")
396|         VerificationRequest(ws, userInfo.email,
verificationCode, role);
397|     else if (role === "Freelancer")
398|         VerificationRequest(ws, freelancerInfo.email,
verificationCode, role);
399|     }
400|
401|
402|     useEffect(() =>{
403|         if (!ws) return;
404|         const handleMessage = (event) =>{
405|             console.log(`Recvd - ${event.data}`)
406|             const info = websocketParser(event.data)
407|
408|             if (info.type === MessageTypes.VERIFICATION) {
409|                 const [infoGood, serverMessage] =
handleVerificationReponse(info);
410|                 if (infoGood) {
411|                     navigate("/login");
412|                 }
413|                 else{
414|                     setServerError(serverMessage)
415|                 }
416|
417|             }
418|             else{
419|                 const [infoGood, errorMessage]=
handleSignUpResponse(info)
420|
421|                 if (infoGood) {
422|                     setServerError("")
423|                     setShowVerification(true);
424|                 }
425|                 else{
426|                     setServerError(errorMessage)
427|                 }
428|             }
429|         }
430|
431|         ws.addEventListener("message", handleMessage);
432|     }
```

```

433|         return () =>{
434|             ws.removeEventListener("message", handleMessage);
435|         }
436|     }, [ws])
437|
438|
439|     return (
440|         <>
441|         { showVerification ? (
442|             <EmailVerification
443|             verificationCode={verificationCode}
444|             setVerificationCode={setVerificationCode}
445|             handleVerificationCodeSubmit={handleVerificationCodeSubmit}
446|             serverError={serverError} />
447|         ) : !role ? (
448|             <RolePopup onSelect={ setRole } />
449|         ) : role === "User" ? (
450|             <UserSignUp userInfo={userInfo}
451|             setUserInfo={setUserInfo} serverError={serverError}
452|             onSubmit={handleSignUpPageSubmit} />
453|         ) : (
454|             <FreelancerSignUp
455|             freelancerInfo={freelancerInfo}
456|             setFreelancerInfo={setFreelancerInfo} serverError={serverError}
457|             onSubmit={handleSignUpPageSubmit} />
458|         ) }
459|     )
460| }
461|

```

signUpModule.jsx

```

0 | import {Link} from "react-router-dom"
1 | import Select from "react-select"
2 | import {useRef, useState} from "react"
3 | import { StatusMessage } from "../socket/MsgTypes";
4 | import {useNavigate} from 'react-router-dom'
5 | import {VerificationRequest} from "../socket/RequestHandle
6 |
7 | // #region lists
8 |
9 | export const freelancerProfessions = async () => {
10 |   try {
11 |     const response = await fetch("/professional.txt");
12 |     const text = await response.text();
13 |     return text
14 |       .split("\n")
15 |       .map(line => line.trim())
16 |       .filter(Boolean);
17 |   } catch (error) {
18 |     console.error("Error loading professions:", error);
19 |     return [];
20 |   }
21 | };
22 |
23 | export const israeliLocalities = async () => {
24 |   try {
25 |     const response = await fetch("/cities.txt");
26 |     const text = await response.text();
27 |     return text
28 |       .split("\n")
29 |       .map(line => line.trim())
30 |       .filter(Boolean);
31 |   } catch (error) {
32 |     console.error("Error loading cities:", error);
33 |     return [];
34 |   }
35 | };
36 | // #endregion
37 |
38 |
39 | // #region Popups and buttons (HTML code)
40 | export function BackToLogin()

```

```

41| {
42|   return(
43|     <Link
44|       to="/login"
45|       className="absolute top-4 left-4 bg-slate-900
text-blue-500 px-4 py-2 rounded-lg shadow hover:bg-slate-600
transition"
46|     >
47|       ← Back to Login
48|     </Link>
49|   );
50| }
51|
52|
53| export function ErrorMessage({message}){
54|   return (
55|     <p className="text-red-600 font-bold text-base mt-
56|       {message}
57|     </p>
58|   )
59| }
60|
61|
62| export function InputField({ label, type="text", name, val
onChange, placeholder }) {
63|   return (
64|     <div className="flex flex-col text-left">
65|       {label} && <label className="mb-1
font-medium">{label}</label>
66|       <input
67|         type={type}
68|         required
69|         name={name}
70|         value={value || ""}
71|         onChange={onChange}
72|         placeholder={placeholder}
73|         className="border text-gray-300 bg-slate-700
border-blue-400 rounded-lg px-4 py-2 focus:outline-none
focus:ring-2 focus:ring-blue-400 placeholder-gray-400"
74|       />
75|     </div>
76|   );
77| }

```

```

78|
79|
80| export function SingleChoiceDropDownMenu({label, name,
options, value, onChange, customWidth, placeholder = "Select a
option"}){
81|   let orderedOptions;
82|   try{
83|     orderedOptions = options.slice().sort((a,b) =>
a.localeCompare(b))
84|   } catch(error){
85|     orderedOptions = options
86|   }
87|
88|   const formattedOptions = orderedOptions.map(option =>
({value:option, label:option}));
89|
90|   return (
91|     <div className="flex flex-col text-left">
92|       {label} && <label className="mb-1
font-medium">{label}</label>
93|       <Select
94|         options = {formattedOptions}
95|         value = {value ? {value:value, label:value}:null}
96|         onChange ={(selected) => onChange(selected.value)}
97|         placeholder={placeholder}
98|         classNamePrefix = 'react-select'
99|         menuPlacement = "auto"
100|         maxMenuHeight={200}
101|         styles={{
102|           container: (base) => ({ ...base, width:
customWidth || '100%' }),
103|           control: (base, state) => ({
104|             ...base,
105|             backgroundColor: '#374151',
106|             borderColor: '#60a5fa',
107|             boxShadow: state.isFocused ? '0 0 0 2px
#60a5fa' : 'none',
108|             color: 'ffffff',
109|             '&:hover': { borderColor: '#3b82f6' },
110|           }),
111|           menu: (base) => ({ ...base, backgroundCol
'#374151', color: 'ffffff' }),
112|           option: (base, state) => ({

```



```

113|         ...base,
114|         backgroundColor: state.isFocused ?
115|         '#1e293b' : '#374151',
116|         color: '#ffffff',
117|     )),
118|     singleValue: (base) => ({ ...base, color:
119|     '#ffffff' }),
120|     placeholder: (base) => ({ ...base, color:
121|     '#cbd5e1' }),
122|     input: (base) => ({ ...base, color:
123|     '#d1d5db' })
124|     })
125| }
126|
127| export function MultiChoiceDropDownMenu({label, name,
128| options, value, onChange, customWidth, placeholder = "Select
129| options"}) {
130|     const sortedOptions = options.slice().sort((a,b) =>
131|     (a.localeCompare(b)))
132|
133|     const orderedOptions = sortedOptions.map(v => ({value:v
134|     label:v}))
135|     const formattedValue = value?.map(v => ({value:v
136|     ,label:v})) || []
137|
138|     return (
139|         <div className="flex flex-col text-left">
140|             {label && <label className="mb-1
141|             font-medium">{label}</label>}
142|             <Select
143|                 options = {orderedOptions}
144|                 value = {formattedValue}
145|                 onChange ={(selected) => {const selectedValue =
146|                 selected ? selected.map(s => s.value) : [];
147|                 onChange(selectedValue)}}
148|                 placeholder={placeholder}
149|                 classNamePrefix = 'react-select'
150|                 menuPlacement = "auto"
151|                 maxMenuHeight={200}

```

```

144|         isMulti ={true}
145|         styles={ {
146|             container: (base) => ({ ...base, width:
customWidth || '100%' } ),
147|             control: (base, state) => ({
148|                 ...base,
149|                 backgroundColor: '#374151',
150|                 borderColor: '#60a5fa',
151|                 boxShadow: state.isFocused
152|                     ? '0 0 0 2px #60a5fa'
153|                     : 'none',
154|                 color: '#ffffff',
155|                 '&:hover': {
156|                     borderColor: '#3b82f6',
157|                 },
158|             } ),
159|             menu: (base) => ({
160|                 ...base,
161|                 backgroundColor: '#374151',
162|                 color: '#ffffff',
163|             } ),
164|             option: (base, state) => ({
165|                 ...base,
166|                 backgroundColor: state.isFocused ?
'#1e293b' : '#374151',
167|                 color: '#ffffff',
168|             } ),
169|             multiValue: (base) => ({
170|                 ...base,
171|                 backgroundColor: '#1e293b',
172|                 color: '#ffffff',
173|             } ),
174|             multiValueLabel: (base) => ({
175|                 ...base,
176|                 color: '#ffffff',
177|             } ),
178|             multiValueRemove: (base) => ({
179|                 ...base,
180|                 color: '#ffffff',
181|                 ':hover': {
182|                     backgroundColor: '#3b82f6',
183|                     color: '#ffffff',
184|                 },

```

```

185|         }},
186|         singleValue: (base) => ({ ...base, color:
187|         placeholder: (base) => ({ ...base, color:
188|         }}
189|     />
190| </div>
191| )
192| }
193| //endregion
194|
195|
196| //region page components
197| export function RolePopup({ onSelect }) {
198|     return (
199|         <div className="fixed inset-0 flex items-center
200|         justify-center bg-slate-900 w-full min-h-screen">
201|             <BackToLogin/>
202|             <div className="bg-slate-700 p-8 rounded-xl shadow-
203|             w-80 text-center">
204|                 <h2 className="text-lg font-semibold text-gray-30
205|                 mb-6">
206|                     Hey! Please pick how you would like to be
207|                     presented in Pro-Find
208|                 </h2>
209|                 <div className="flex justify-between">
210|                     <button
211|                         className="flex-1 bg-blue-400 hover:bg-blue-5
212|                         text-white py-2 rounded mr-2 transition"
213|                         onClick={() => onSelect("User")}
214|                     >
215|                         User
216|                     </button>
217|                     <button
218|                         className="flex-1 bg-blue-400 hover:bg-blue-5
219|                         text-white py-2 rounded ml-2 transition"
220|                         onClick={() => onSelect("Freelancer")}
221|                     >
222|                         Freelancer
223|                     </button>

```

```

220|         </div>
221|     </div>
222| </div>
223| );
224| }
225|
226| export function EmailVerification({ verificationCode,
setVerificationCode, handleVerificationCodeSubmit, serverError
{
227|     const navigate = useNavigate()
228|     const inputsRef = useRef([])
229|     const [error, setError] = useState("")
230|
231|     const handleChange = (value, index) => {
232|         const codeArray = verificationCode.split("")
233|         codeArray[index] = value
234|         const newCode = codeArray.join("")
235|         setVerificationCode(newCode)
236|
237|         if (value && index < 5) {
238|             inputsRef.current[index + 1].focus()
239|         }
240|     }
241|
242|     const handleKeyDown = (e, index) => {
243|         if (e.key === "Backspace") {
244|             const codeArray = verificationCode.split("")
245|             codeArray[index] = ""
246|             setVerificationCode(codeArray.join(""))
247|
248|             if (index > 0) {
249|                 inputsRef.current[index - 1].focus()
250|             }
251|         }
252|     }
253|
254|     const handleSubmit = () => {
255|         if (verificationCode.length < 6) {
256|             setError("Bad format: 6 digits required")
257|             return
258|         }
259|
260|         if (!/^\d{6}$/.test(verificationCode)) {

```

```

261|         setError("Bad format: Only numbers allowed")
262|         return
263|     }
264|
265|     setError("")
266|     handleVerificationCodeSubmit()
267| }
268|
269| return (
270|     <div className="fixed inset-0 flex items-center
justify-center bg-slate-900 w-full min-h-screen p-4">
271|
272|         <div className="bg-slate-700 p-10 rounded-xl shadow
w-full max-w-md text-center">
273|
274|             <h2 className="text-4xl font-semibold text-gray-3
mb-2">
275|                 Insert Verification Code
276|             </h2>
277|
278|             <p className="text-gray-400 mb-6">
279|                 A verification code was sent to your email.
280|             </p>
281|
282|             <div className="flex justify-center gap-4 mb-6">
283|                 {[0,1,2,3,4,5].map((i) => (
284|                     <input
285|                         key={i}
286|                         ref={(el) => (inputsRef.current[i] = el)}
287|                         type="text"
288|                         maxLength="1"
289|                         value={verificationCode[i] || ""}
290|                         onChange={(e) => handleChange(e.target.valu
i)}
291|                         onKeyDown={(e) => handleKeyDown(e, i)}
292|                         className="w-14 h-16 text-center text-white
text-3xl border-b-4 border-blue-400 focus:border-blue-500
outline-none rounded-sm flex items-center justify-center"
293|                     />
294|                 )]}
295|             </div>
296|
297|             {error && <ErrorMessage message={error} />}

```

```
298 |  
299 |  
300 |         {serverError && <ErrorMessage  
message={serverError} />}  
301 |  
302 |  
303 |         <button  
304 |             onClick={handleSubmit}  
305 |             className="mt-6 bg-blue-400 hover:bg-blue-500  
text-white font-semibold py-2 px-6 rounded transition text-xl  
w-60 h-15"  
306 |         >  
307 |             Submit  
308 |         </button>  
309 |  
310 |     </div>  
311 |  
312 | </div>  
313 | )  
314 | }  
315 | // #endregion
```